1 Appendix U: Agent Semantics

U.1. Scope, dependency position, and platform status

This appendix defines the semantics of agents used in the search, platform, proof-management, proof-compression, problem-interface, and audit layers of AK-HPDST v20.0.0. It belongs to the Search / Platform Appendices. It does not enlarge the Core mathematical package of Chapters 1–8 or the Foundation Appendices. Its function is to govern how human, AI, tool, proof-assistant, platform, and hybrid agents may generate, transform, retrieve, verify, replay, store, or adjudicate objects without silently changing their mathematical status.

The central rule of this appendix is:

Agent output is not proof unless verified by the declared verifier-side protocol.

For an invoked degree n, no human proposal, AI-generated answer, search score, confidence value, ranking, heuristic, proof sketch, generated schema, platform artifact, stored object, replay result, or consensus record may replace any required component of the v20 chain, including:

$$
\begin{aligned}
&B\text{-Gate}^{+}, \qquad B_{n,W}(F) := \mathsf{W}_W \mathbf{P}_n(F),\\
&\mathrm{Eval}_{n,W,\tau}(F) = T_\tau^{\mathrm{bar}}\big(B_{n,W}(F)\big), \qquad \text{a complete detector certificate when invoked,}\\
&C_{\tau,W}F, \qquad \text{a standard degree-}n\text{ eligible realization and edge record when invoked,}\\
&\phi_{i,W,\tau} : \mathrm{ColimProfile}_{i,W,\tau}(F_\bullet) \longrightarrow \mathrm{ApexProfile}_{i,W,\tau}(F_\infty),\\
&(\mu_{\mathrm{Collapse}},\ u_{\mathrm{Collapse}}) = (0,0) \quad \text{on a valid invoked terminal scope,}\\
&\mathrm{val}_B(\mathfrak{d}_{\mathrm{met}}) < \mathrm{gap}_\tau(\mathfrak{B}), \qquad \text{a finite-to-infinite reduction when required,}\\
&\text{and a separately proved Return theorem for any external conclusion.}
\end{aligned}
$$

Remark 1.1 (Search / Platform status)**.** This appendix describes who or what may generate, transform, inspect, verify, store, replay, or audit objects in the AK workflow. Its propositions are operational consequences of the v20 typed-verification architecture. They do not create a new persistence, higher-Ext, Type IV, arithmetic, PDE, geometric, categorical, or external-domain theorem.

Remark 1.2 (Relation to Core sovereignty)**.** Chapter 7 established Core sovereignty. The same rule governs this appendix: agents may propose, navigate, annotate, package, formalize, criticize, or replay data, but only verifier-side conditions determine the corresponding atomic verdict. An agent may verify a condition only inside its declared task-specific authority scope and environment.

Remark 1.3 (Why agent semantics are needed)**.** The v20 framework is search-heavy, artifact-heavy, and bridge-sensitive. It may involve finite detector certificates, higher obstruction edges, actual tower morphisms, transient-defect profiles, finite-to-infinite reductions, source-statement bindings, Known-Theorem Recovery tracks, proof assistants, and semantic replay. Without explicit agent semantics, a finite observation can be inflated into a finite proof, a stored path into a theorem, or a benchmark conclusion into a realization input. This appendix prevents those category errors.

Remark 1.4 (Dependency and registration boundary)**.** This appendix consumes the v20 Main Chapters 1–8, Foundation Appendices A–G, and the CR-v20 baseline register at their canonical strengths. CM-v20, TB-v20, Technical Extension Appendices H–N, Problem Interface Part IV, and this Part V are consumed only at their frozen or reviewed local strengths and immutable artifact identities while final CR promotion remains pending. A local label is not a canonical Claim UID, and registration is not proof.

U.1bis. v20.0 agent sovereignty and verifier-side boundary

Definition 1.5 (Agent-sovereignty boundary)**.** The *agent-sovereignty boundary* is the rule that no agent-side object, action, score, artifact, approval, storage event, or generated proof text changes a Core verdict

unless it is converted into declared verifier-side data and passes the applicable verifier-side protocol. The boundary applies uniformly to human agents, AI agents, tool agents, proof assistants, platform services, and hybrid workflows.

Definition 1.6 (Verifier-side conversion record)**.** A *verifier-side conversion record* converts one declared agent-side output into one declared verifier-side target schema. It must declare:

(i) the immutable identity of the source output, its producer, role, action, and workflow status;

(ii) the target verifier-side schema and exactly which mathematical or operational predicate is to be checked;

(iii) the source and target mathematical scopes, monitored degrees, windows, thresholds, endpoint conventions, and profile stages;

(iv) the document role, evidence status, invocation role, conformance status, workflow lifecycle status, and proposed Chapter 7 atomic status, without identifying any two axes;

(v) the task-specific authority, authority environment, and verification mode allowed to check the conversion;

(vi) all hypotheses, dependencies, source bindings, artifacts, model/tool/environment versions, and comparison modes;

(vii) for a detector target, exactly one source stage among

windowed_prethreshold, repaired_postthreshold, kernel_defect, cokernel_defect,

together with separate certificate status and mathematical value;

(viii) for a higher-edge target, the standard degree-n eligibility, realization, edge-extractor, H^{-n}-identification, naturality, and artifact package;

(ix) for a Type IV or transient target, the actual morphism, semantic source authority, terminal-tameness data, full defect profiles, and the exact terminal, full-interior, and long-transient scopes consumed;

(x) any finite-to-infinite reduction and Return theorem required by the target conclusion;

(xi) for a Known-Theorem Recovery target, the source-statement binding, permitted precursor package, benchmark quarantine, non-circularity record, and prohibited conclusion-side inputs;

(xii) the resulting atomic status: pass, reject, undefined, or a valid scope-excluded not_invoked;

(xiii) the typed failure route for every missing, false, ill-typed, stale, circular, contaminated, or unsupported condition.

The conversion is target-schema specific. Passing one conversion does not convert the same agent output into every other verifier-side schema.

Definition 1.7 (Agent-side default status)**.** Every agent output is proposer-side and non-verdict by default until its identity, status axes, task-specific authority, source and dependency bindings, artifact adequacy, and target-specific verifier-side conversion record justify a stronger role. This default applies even if the output is written in theorem style, generated by a strong model, approved by a human, checked by several agents, stored in the platform, replayed successfully, or produced by a deterministic tool.

Proposition 1.8 (No agent sovereignty override)**.** *No agent-side object or action overrides the Core verifier-side requirements.*

Proof. The v20 verifier consumes repaired evaluation records, finite detector certificates, representatives, degree-n eligibility records, actual tower artifacts, terminal and transient diagnostics, comparison records, reductions, Return records, ledgers, gate verdicts, claim-use records, and manifests at their declared types. Agent-side objects have provenance and workflow semantics, not verifier-side status by default. Without a verifier-side conversion record, they are outside the verdict function. Therefore they cannot override the Core requirements. □

Proposition 1.9 (Agent success is non-compensating)**.** *A successful agent action, including generation, replay, retrieval, ranking, formal checking, platform storage, or human approval, cannot compensate for a failed or missing non-scalar obligation.*

Proof. Non-scalar obligations such as constructibility, detector completeness, actual morphisms, representative existence, degree-n eligibility, terminal tameness, finite-to-infinite reduction, Return, source authority, interpretation, manifest identity, semantic replay, and artifact adequacy have their own typed requirements. Agent success in another role does not supply those requirements. Since the v20 obligation calculus is non-compensating, no favorable agent-side result can repair a missing non-scalar obligation. □

Remark 1.10 (Agent-semantics scope)**.** This appendix governs workflow semantics only. It does not rank agents by epistemic authority and does not assert that any class of agent is inherently reliable or unreliable. Reliability is established only relative to a declared task, authority scope, environment, and verifier-side evidence.

U.2. Agent types

Definition 1.11 (Agent)**.** An *agent* is any human, AI system, automated tool, proof assistant component, search module, platform service, or hybrid workflow component that performs an action in the AK search, platform, proof-management, or audit pipeline.

Definition 1.12 (Human agent)**.** A *human agent* is a person who proposes, reviews, edits, verifies, audits, approves, rejects, or interprets AK-related objects or claims.

Definition 1.13 (AI agent)**.** An *AI agent* is a machine-learning, language-model, symbolic, neuro-symbolic, or automated reasoning component that proposes, transforms, summarizes, searches, ranks, critiques, or assists with AK-related objects or claims.

Definition 1.14 (Tool agent)**.** A *tool agent* is a deterministic or semi-deterministic software component, such as a persistence-computation engine, proof assistant, type checker, parser, build system, replay engine, artifact validator, dependency resolver, or certificate checker.

Definition 1.15 (Platform agent)**.** A *platform agent* is a service that stores, versions, indexes, retrieves, displays, routes, or synchronizes AK artifacts, proof objects, logs, manifests, or certificate records.

Definition 1.16 (Hybrid agent)**.** A *hybrid agent* is a workflow in which human, AI, tool, and platform agents jointly produce, transform, inspect, verify, or audit an object.

Remark 1.17 (Agent type does not determine validity)**.** The type of agent that produced an object does not determine the object's proof status. A human-written claim may be false. An AI-generated lemma may be correct. A tool-generated artifact may be misconfigured. A stored artifact may be obsolete. Validity depends on verifier-side checks, not on authorship.

U.3. Agent roles

Definition 1.18 (Proposer)**.** A *proposer* is an agent that suggests a candidate object, lemma, bridge, window, threshold, realization, proof route, diagnostic interpretation, certificate record, or proof object field.

Definition 1.19 (Searcher)**.** A *searcher* is an agent that explores a search space and returns candidate regions, structures, windows, bridges, proof fragments, counterexamples, or artifact references.

Definition 1.20 (Mapper)**.** A *mapper* is an agent that organizes candidate objects into maps, diagrams, dependency graphs, validity maps, bridge maps, or certificate DAGs.

Definition 1.21 (Lifter)**.** A *lifter* is an agent that attempts to turn a candidate, advisory object, search artifact, or platform artifact into a Core-readable object, interface object, formal fragment, or certificate candidate.

Definition 1.22 (Verifier)**.** A *verifier* is an agent or procedure that checks whether a declared object, formal fragment, bridge, certificate record, gate, diagnostic, ledger, or proof object satisfies the relevant verifier-side criteria.

Definition 1.23 (Auditor)**.** An *auditor* is an agent that checks whether the verification process, artifact references, manifest entries, proof object schema, provenance records, and replay data are sufficient for independent reconstruction.

Definition 1.24 (Curator)**.** A *curator* is an agent that stores, labels, versions, indexes, maintains, or retires proof objects, artifacts, status labels, and platform records.

Definition 1.25 (Adjudicator)**.** An *adjudicator* is an agent that resolves status conflicts among outputs, for example when one agent labels an object as a candidate while another reports a failure. Adjudication does not establish mathematical validity unless backed by verifier-side data.

Remark 1.26 (Roles may overlap)**.** A single agent may act in multiple roles. For example, an AI system may propose a bridge, a human may edit it, and a proof assistant may verify a formal fragment. However, the semantic role of each action must remain distinguishable.

Definition 1.27 (Role-separation record)**.** A *role-separation record* identifies, for each action on an object, whether the acting agent is proposer, searcher, mapper, lifter, verifier, auditor, curator, adjudicator, source binder, or Return interpreter. When one physical or computational agent performs multiple roles, each action receives a distinct record and authority scope.

Proposition 1.28 (Self-review is not independent verification)**.** *An agent's successful self-critique, self-rating, self-consistency check, or second-pass generation does not constitute independent verification unless the declared verification protocol proves that the second action has the required independence and authority.*

Proof. The same agent may reuse the same prompt assumptions, hidden state, retrieved corpus, model defects, or unsupported premise. A second action may improve an output, but independence is a separate property requiring its own record. □

U.4. Proposer–verifier separation

Definition 1.29 (Proposer-side data)**.** *Proposer-side data* are outputs generated before verifier-side validation. They include:

(i) candidate lemmas;

(ii) candidate proof routes;

(iii) advisory indicators;

(iv) search rankings;

(v) AI confidence values;

(vi) validity maps;

(vii) bridge program sketches;

(viii) informal explanations;

(ix) draft certificate records;

(x) candidate normalization or realization routes;

(xi) candidate [Spec] bridges;

(xii) proposed proof object fields.

Definition 1.30 (Verifier-side data)**.** *Verifier-side data* are typed records actually consumed to determine a declared mathematical, interface, bridge, audit, or support status. They include, as invoked:

(i) declared source objects, coefficients, source semantics, purpose profiles, and immutable source-statement bindings;

(ii) declared monitored degrees, right-open windows W, thresholds τ, endpoint conventions, and the fixed window-first policy;

(iii) exact pre-threshold and repaired evaluation records

$$B_{i,W}(F) = \mathsf{W}_W \mathbf{P}_i(F), \qquad \mathrm{Eval}_{i,W,\tau}(F) = T_\tau^{\mathrm{bar}}\big(B_{i,W}(F)\big);$$

(iv) detector records with exactly one source stage, separate certificate status and mathematical value, and every invoked soundness, zero-completeness, coverage, family, endpoint, non-adaptivity, and excess-bound obligation;

(v) admissible representative records $C_{\tau,W}F$ when invoked;

(vi) standard degree-n eligibility and higher-edge records, including an actual profile identification and artifact-backed H^{-n} interpretation, when Ext^n is invoked;

(vii) declared actual tower comparison artifacts

$$\phi_{i,W,\tau} : \mathrm{ColimProfile}_{i,W,\tau}(F_\bullet) \longrightarrow \mathrm{ApexProfile}_{i,W,\tau}(F_\infty)$$

when Type IV diagnostics are invoked;

(viii) separately typed terminal Type IV, full-interior, and long-transient records, including full kernel and cokernel defect profiles when transient claims are consumed;

(ix) exact, metric-gap-certified, or valid not-compared mode records, with the two-sided threshold-gap and $2\varepsilon \le \tau$ zero-transport obligations whenever metric inference crosses hard deletion;

(x) a declared commutative-quantale ledger, scalarization, actual profile-discrepancy support, and no-double-counting relation to the Appendix M catalog;

(xi) finite-to-infinite reduction, apex-agreement, and Return records when required by the target conclusion;

(xii) source/precursor/benchmark quarantine and non-circularity records for Known-Theorem Recovery;

(xiii) claim-use preflight, typed failure routes, gate verdicts, immutable artifact references, semantic preservation records, and proof objects satisfying the v20 manifest discipline;

(xiv) formal or independently reconstructible checks of every consumed theorem claim.

No item becomes verifier-side merely because an agent labels, serializes, stores, or displays it as such.

Theorem 1.31 (Proposer–verifier separation)**.** *Core pass/reject is determined only by verifier-side data. Proposer-side data may guide, motivate, or prioritize verification, but they do not determine the verdict.*

Proof. By Chapter 7 and Appendix G, every verdict is a typed function of the verifier-side records actually declared for the invoked clause. The v20 additions—finite detector certification, standard higher edges, transient-defect separation, finite-to-infinite reduction, Return, and source quarantine—add obligations; they do not transfer verdict authority to proposer-side objects. Therefore proposer-side data cannot determine Core pass/reject or any external conclusion. □

Remark 1.32 (The central safety boundary)**.** This theorem is the central safety boundary of Part V. Search may be aggressive. Agents may be creative. The verifier remains conservative.

U.4bis. Promotion, adjudication, and claim-use preflight

Definition 1.33 (Agent promotion certificate)**.** An *agent promotion certificate* is the evidence package required to promote an agent output to a stronger status. It contains the source output, prior status, target status, verifier-side conversion record, claim-use preflight result, dependency closure, artifact identity, and failure-route mapping.

Definition 1.34 (Agent claim-use preflight)**.** An *agent claim-use preflight* checks whether an agent output may be consumed at one proposed role. It verifies immutable identity and source locator; document role, evidence status, invocation role, conformance status, workflow lifecycle, and atomic verifier status; authority scope; dependency closure; comparison mode; detector stage and completeness; degree-n eligibility; actual-morphism and defect scope; reduction and Return requirements; source/precursor/benchmark quarantine; artifact adequacy and freshness; semantic preservation; failure-route compatibility; and prohibited stronger readings. For downstream material not yet promoted by the final CR-v20, the preflight records frozen or reviewed local strength and immutable local labels rather than fabricating canonical Claim UIDs. Missing required data yield undefined, not pass, zero, or not_invoked.

Definition 1.35 (Adjudication record)**.** An *adjudication record* resolves a conflict among agent outputs, status labels, or review results. It records the conflicting claims, the adjudication rule, the selected status, the losing statuses, the evidence actually consumed, and whether the resolution is operational, verifier-side, or non-verdict.

Proposition 1.36 (Adjudication is not verification)**.** *An adjudication record does not establish mathematical validity unless it includes the verifier-side evidence required for the adjudicated claim.*

Proof. Adjudication resolves a workflow conflict. Verification checks a typed mathematical or certificate condition. Resolving which status label is currently assigned does not by itself prove the underlying mathematical condition. Thus adjudication is not verification unless it carries the required verifier-side evidence. □

Proposition 1.37 (Promotion without preflight is invalid)**.** *A status promotion from candidate, draft, advisory, search artifact, proof sketch, bridge draft, or platform artifact to theorem evidence, bridge theorem, Core verified, or audit-ready status is invalid without a successful claim-use preflight and the required promotion certificate.*

Proof. Higher statuses impose additional evidence, scope, artifact, and role requirements. If preflight is missing, those requirements are not known to be satisfied. By the weaker-safe rule, the promotion cannot be consumed. □

Remark 1.38 (Human approval and adjudication)**.** Human approval may be part of adjudication or governance. It remains scoped to the approved purpose and does not itself become the proof of a mathematical claim.

U.5. AI confidence, ranking, and plausibility

Definition 1.39 (AI confidence value)**.** An *AI confidence value* is any scalar, categorical, probabilistic, linguistic, or model-generated estimate of plausibility, correctness, priority, or relevance associated with an AI-generated output.

Definition 1.40 (Agent ranking)**.** An *agent ranking* is an ordering of candidate objects, regions, lemmas, proof routes, bridge statements, or artifacts produced by an agent.

Definition 1.41 (Plausibility score)**.** A *plausibility score* is any non-verifier-side score estimating whether a candidate is worth checking. It may be human-assigned, AI-assigned, tool-assisted, or platform-derived.

Proposition 1.42 (AI confidence is not proof)**.** *An AI confidence value is not a proof, not a Core gate verdict, not a monitored diagnostic, and not a budget comparison.*

Proof. A proof or Core verdict must be reconstructible through declared verifier-side conditions. An AI confidence value is a model-side estimate. It does not establish B-Gate$^+$, repaired evaluation vanishing, eligibility, Type IV cleanliness, tower artifact validity, or ledger admissibility. □

Proposition 1.43 (Ranking does not imply validity)**.** *The rank assigned to a candidate by an agent does not imply that the candidate is valid.*

Proof. Ranking is a search or prioritization operation. Validity requires verification. A highly ranked candidate may fail a gate, diagnostic, ledger, eligibility, artifact, or bridge condition. □

Proposition 1.44 (Plausibility does not imply verifiability)**.** *A plausible candidate is not necessarily verifiable.*

Proof. Plausibility concerns apparent promise. Verifiability requires typed objects, declared criteria, artifact support, and reconstructible checks. A candidate may appear plausible while lacking the data needed for verification. □

Remark 1.45 (Correct use of confidence)**.** AI confidence, rankings, and plausibility scores may be useful for triage. They may determine what is checked first. They may not determine what is accepted.

U.5bis. Retrieval, consensus, and model-context boundaries

Definition 1.46 (Retrieval context record)**.** A *retrieval context record* stores the query, corpus identity, corpus version, trust class, retrieved artifact identifiers, exact locator ranges, retrieval and ranking policies, extraction method, freshness policy, authoritative-source relation, local source-statement binding, permitted precursor relation, benchmark-quarantine status, and any untrusted-content flags. It is a provenance and source-navigation record, not a proof record.

Definition 1.47 (Model-context boundary)**.** The *model-context boundary* is the distinction between a model's generated statement and the artifact-backed statements available in the declared context. A generated statement is not source evidence unless it is tied to an immutable source locator or a verifier-side proof object.

Definition 1.48 (Agent consensus record)**.** An *agent consensus record* records agreement among multiple agents, models, tools, reviewers, or search procedures. It must identify the agents, prompts or tasks, model and tool families, shared corpora, shared dependencies, independence assumptions, compared outputs, agreement predicate, conflicts of interest, correlated-failure risks, and unresolved disagreements.

Proposition 1.49 (Retrieval is not verification)**.** *Retrieving a document, passage, theorem statement, citation, or artifact does not verify the mathematical claim for which it is retrieved.*

Proof. Retrieval supplies access or evidence candidates. Verification checks whether the retrieved item has the correct statement, hypotheses, scope, proof, status, and permitted use for the target obligation. The retrieval action alone performs none of those checks. □

Proposition 1.50 (Agent consensus is not proof)**.** *Agreement among agents, including agreement among multiple AI systems or human reviewers, is not proof and does not determine a Core verdict.*

Proof. Consensus is a proposer-side or governance-side correlation among outputs. All agreeing outputs may share the same unsupported premise, stale source, prompt error, or scope mistake. The Core verdict depends on verifier-side records, not on the number of agreeing agents. □

Proposition 1.51 (Context inclusion is not claim adoption)**.** *A statement included in an agent's prompt, retrieved context, memory, RAG result, note, source summary, or platform page is not adopted as a theorem claim unless it is independently source-bound and consumed at an admissible role and evidence status.*

Proof. Context inclusion affects what an agent may read or generate. Claim adoption requires role, evidence status, invocation role, source binding, and permitted use. Therefore context inclusion cannot itself promote a statement. □

Remark 1.52 (Stale or partial context)**.** A retrieved or remembered context may be stale, partial, or superseded. The current source artifact and claim-governance record control over the agent's context window.

U.6. Agent outputs and status labels

Definition 1.53 (Agent output)**.** An *agent output* is any object produced by an agent, including text, code, theorem statements, diagrams, proof sketches, formal fragments, data files, certificate candidates, maps, rankings, annotations, logs, or status updates.

Definition 1.54 (Agent-output lifecycle label)**.** Every agent output used in the AK workflow must carry a workflow lifecycle label chosen from a declared schema, including as applicable:

(i) *search artifact*, *candidate*, *draft*, or *advisory*;

(ii) *formal fragment*, *verified formal fragment*, or *proof sketch*;

(iii) *bridge draft*, *bridge program*, or *bridge theorem candidate*;

(iv) *Core certificate candidate*, *conversion pending*, *verification pending*, or *Core verified*;

(v) *audit-ready*, *replayable*, or *semantically replayed*;

(vi) *incomplete*, *blocked*, *failed*, *rejected*, *deprecated*, *superseded*, or *stale*;

(vii) *non-claim* or *scope-excluded*.

These are lifecycle labels, not substitutes for document role, evidence status, invocation role, conformance status, detector value, Type IV status, or Chapter 7 atomic status. Terms such as *operational policy*, *interface specification*, *Spec*, and *bridge theorem* remain document or evidence classifications and must be recorded on their own axes rather than inferred from the lifecycle label.

Definition 1.55 (Unlabeled agent output)**.** An *unlabeled agent output* is an agent output lacking an explicit status label.

Definition 1.56 (Status promotion)**.** *Status promotion* is an operation that moves an output from a lower-status class to a higher-status class, for example:

$$\text{candidate} \longrightarrow \text{Core certificate candidate} \longrightarrow \text{Core verified}.$$

Definition 1.57 (Status demotion)**.** *Status demotion* is an operation that moves an output to a lower or safer status class, for example:

$$\text{Core certificate candidate} \longrightarrow \text{incomplete} \quad \text{or} \quad \text{verified formal fragment} \longrightarrow \text{deprecated}.$$

Definition 1.58 (Agent status-axis record)**.** An *agent status-axis record* contains independent fields for:

(i) document role;

(ii) evidence status;

(iii) invocation role;

(iv) conformance status;

(v) workflow lifecycle status;

(vi) the Chapter 7 atomic verifier status for the exact checked clause;

(vii) any specialized detector, Type IV, transient, reduction, Return, replay, or storage status.

No field is computed by copying another field.

Definition 1.59 (Immutable scope-exclusion policy)**.** An *immutable scope-exclusion policy* identifies a clause that is outside the declared run scope and explains why its omission is semantically legitimate. Only such a policy permits the specialized status not_invoked; otherwise missing theorem-facing evidence is undefined.

Proposition 1.60 (Status-axis collapse is invalid)**.** *An agent may not infer mathematical pass from workflow completion, storage acceptance, replay success, human approval, or conformance status, nor infer* not_invoked *from a blank or absent field.*

Proof. The listed statuses answer different typed questions. The v20 verifier evaluates each axis and specialized status independently, and missing theorem-facing evidence is governed by the weaker-safe rule. □

Proposition 1.61 (Unlabeled outputs are non-verdict by default)**.** *An unlabeled agent output is non-verdict data by default.*

Proof. Without a status label, an auditor cannot determine whether the output is intended as a search artifact, theorem, proof fragment, policy, bridge, certificate, or non-claim. Therefore it cannot affect Core verdicts. □

Proposition 1.62 (Promotion requires evidence)**.** *An agent output may be promoted to a higher status only if the required evidence for that higher status is supplied.*

Proof. Higher status labels impose stronger semantic requirements. For example, Core verified status requires verifier-side data, and bridge theorem status requires a proved bridge. Without such evidence, the promotion is unsupported. □

Remark 1.63 (Labeling prevents claim inflation)**.** Status labels prevent a search artifact from being misread as a theorem, or a draft from being misread as a verified proof.

U.7. Agent action semantics

Definition 1.64 (Generate)**.** An agent action *generate* produces a new candidate object or artifact. Generation does not imply verification.

Definition 1.65 (Transform)**.** An agent action *transform* modifies an existing object into another form, such as translating, normalizing, refactoring, formalizing, compressing, expanding, or summarizing it. Transformation does not preserve validity unless preservation is proved or checked.

Definition 1.66 (Rank)**.** An agent action *rank* orders candidates by a declared priority function, heuristic, score, or policy. Ranking does not imply proof.

Definition 1.67 (Annotate)**.** An agent action *annotate* adds metadata, comments, explanations, warnings, or status labels to an object. Annotation does not change the mathematical content unless explicitly declared.

Definition 1.68 (Verify)**.** An agent action *verify* checks a claim, object, formal fragment, bridge, or certificate against declared verifier-side criteria. Verification must specify the criteria being checked.

Definition 1.69 (Audit)**.** An agent action *audit* checks whether the verification record, artifacts, manifest, provenance, replay data, and proof object schema allow independent reconstruction.

Definition 1.70 (Replay)**.** An agent action *replay* re-executes, recomputes, or reconstructs a declared workflow from stored artifacts and manifests. Replay checks reproducibility, not necessarily mathematical truth beyond the replayed criteria.

Proposition 1.71 (Action semantics are non-interchangeable)**.** *Generation, transformation, ranking, annotation, verification, audit, and replay are distinct actions and cannot be substituted for one another without explicit justification.*

Proof. Each action has a different semantic effect. Generation creates candidates. Ranking prioritizes. Verification checks validity. Audit checks reconstructibility. Replay checks reproducibility. Since their outputs answer different questions, they cannot be interchanged. □

Remark 1.72 (Transformation caution)**.** A transformed proof, transformed definition, or transformed certificate may no longer be equivalent to the source object. Equivalence requires a preservation check or proof.

U.7bis. Transformation preservation and semantic diff

Definition 1.73 (Transformation preservation certificate)**.** A *transformation preservation certificate* proves or verifies that a transformation preserves the declared mathematical content, status label, hypotheses, dependency links, artifact identities, and prohibited stronger readings. It is required for transformations such as rewriting, summarizing, translating, refactoring, formalizing, normalizing, compressing, code generation, schema generation, serialization, regeneration, model replacement, or migration when the transformed object is consumed as equivalent to the source.

Definition 1.74 (Semantic diff record)**.** A *semantic diff record* compares a source object and a transformed object at the level of definitions, theorem statements, hypotheses, quantifiers, scopes, symbols, references, claims, and status labels. It distinguishes text-only changes from mathematical, interface, governance, or artifact changes.

Proposition 1.75 (Transformation without preservation is non-equivalent)**.** *A transformed object cannot be consumed as equivalent to its source without a transformation preservation certificate or a successful semantic diff showing that no consumed semantic field changed.*

Proof. Transformations may alter hypotheses, scope, quantifiers, notation, references, or artifact identity. Equivalence is therefore an additional claim. Without a preservation certificate or semantic diff, that claim is unsupported. □

Proposition 1.76 (Formalization is a transformation, not theorem promotion)**.** *Translating an informal statement into proof-assistant syntax is a transformation and does not promote the informal statement to theorem status unless the formal statement is verified and an interpretation theorem connects it back to the intended claim.*

Proof. Formalization may change the statement or omit intended hypotheses. A proof assistant verifies only the formal statement in its environment. The connection to the informal claim requires a separate interpretation theorem. □

Remark 1.77 (Summary and translation caution)**.** Summaries, translations, and refactorings are useful audit tools, but a shorter or clearer text may be weaker, stronger, or differently scoped than the original. When theorem evidence is at stake, the original source binding or preservation certificate controls.

U.8. Verification authority

Definition 1.78 (Verification authority)**.** A *verification authority* is a declared procedure or agent whose output is allowed to establish a specific verifier-side status, such as verified formal fragment, passed gate, passed diagnostic, passed budget check, passed manifest check, or replay success.

Definition 1.79 (Authority scope)**.** The *authority scope* of a verification authority is the class of statements, objects, or checks it is authorized to verify.

Definition 1.80 (Authority environment)**.** The *authority environment* is the declared versioned environment in which a verification authority operates, including tool versions, libraries, assumptions, configuration, input artifacts, and accepted axioms.

Proposition 1.81 (Verification authority is scoped)**.** *A verification authority verifies only claims within its declared authority scope and declared environment.*

Proof. Different tools and agents check different properties. A type checker may verify type correctness but not mathematical soundness of an informal bridge. A persistence engine may compute barcodes but not prove a PDE regularity theorem. A proof assistant verifies only relative to its formal environment. Thus authority must be scoped and environment-relative. □

Remark 1.82 (Tool output requires interpretation)**.** Even deterministic tool output requires interpretation. A successful run proves only what the tool is specified, configured, and authorized to check.

U.8bis. Verification modes and tool-scope discipline

Definition 1.83 (Verification mode)**.** A *verification mode* declares what kind of check has been performed. Typical modes include:

(i) syntax_checked;

(ii) compiled;

(iii) type_checked;

(iv) computed;

(v) computationally_replayed;

(vi) semantically_replayed;

(vii) artifact_validated;

(viii) formally_verified;

(ix) interpretation_proved;

(x) human_reviewed;

(xi) not_checked;

(xii) failed.

The mode determines only the checked property, not every stronger property one might desire.

Definition 1.84 (Authority-to-mode compatibility)**.** An *authority-to-mode compatibility record* states which verification modes a declared authority may establish in a declared environment. A build system may establish compiled; a type checker may establish type_checked; a proof assistant may establish formally_verified for encoded statements; none of these modes automatically establishes an external-domain theorem.

Proposition 1.85 (Tool success is mode-scoped)**.** *A successful tool run establishes only the verification mode and scope declared for that tool run.*

Proof. Tools operate under specifications and environments. The output of a parser, compiler, persistence engine, proof assistant, or renderer has the meaning assigned by that tool and its configuration. No stronger theorem follows unless a further theorem or authority record supplies it. □

Proposition 1.86 (Build success is not mathematical verification)**.** *A successful build, render, serialization, replay, or platform validation does not establish the mathematical truth of a theorem statement unless the verified mode explicitly includes the relevant mathematical proof obligation.*

Proof. Build and render systems check syntactic, packaging, or reproducibility properties. Mathematical truth requires proof of the statement under the stated hypotheses. Those are different modes. □

Remark 1.87 (Mode labels prevent tool inflation)**.** Verification mode labels prevent a successful local check from being read as a stronger global theorem, bridge theorem, or Core certificate.

U.9. Human review

Definition 1.88 (Human review)**.** *Human review* is the process by which a human agent reads, checks, critiques, edits, approves, rejects, or interprets an AK-related object.

Definition 1.89 (Human approval)**.** *Human approval* is an explicit statement that a human agent accepts an object for a declared purpose.

Definition 1.90 (Human mathematical judgment)**.** *Human mathematical judgment* is a human assessment of correctness, plausibility, rigor, usefulness, or risk. It may guide verification, but it is not identical to verifier-side proof unless supported by proof data.

Proposition 1.91 (Human approval is scoped)**.** *Human approval establishes only the approved status within the declared scope. It does not automatically establish Core validity.*

Proof. A human may approve style, structure, plausibility, publication readiness, or readiness for verification. Core validity requires Core verifier-side checks. Therefore human approval is scoped to its declared purpose. □

Remark 1.92 (Human review remains important)**.** Human review is crucial for detecting conceptual gaps, claim inflation, poor definitions, missing hypotheses, and misaligned interfaces. However, review and verification should remain semantically distinct.

U.10. Agent provenance

Definition 1.93 (Agent provenance)**.** *Agent provenance* is metadata recording which agents produced, transformed, verified, audited, approved, rejected, deprecated, or superseded an object.

Definition 1.94 (Provenance record)**.** A *provenance record* contains, as applicable:

(i) the agent identifier or class and semantic role;

(ii) the exact task, action, authority scope, and verification mode;

(iii) immutable input and output artifact identities;

(iv) prompt, system instruction, context bundle, retrieval corpus, locator set, and post-processing identity;

(v) model, tool, library, proof-assistant, platform, and environment versions;

(vi) random seed, temperature, sampling policy, nondeterminism policy, and retry policy when relevant;

(vii) timestamp or release reference, while recognizing that time alone is not artifact identity;

(viii) status axes before and after the action;

(ix) source, precursor, benchmark, claim-use, comparison, detector, reduction, Return, and replay links when invoked;

(x) conflicts of interest, independence assumptions, contamination flags, audit notes, and failure routes.

Definition 1.95 (Versioned provenance)**.** *Versioned provenance* is provenance tied to immutable or version-controlled artifact identifiers.

Proposition 1.96 (Provenance supports audit but does not prove validity)**.** *Agent provenance supports auditability, but it does not by itself prove mathematical validity.*

Proof. Provenance records who did what. Validity requires that the resulting object satisfies the relevant mathematical or verifier-side conditions. The former supports reconstruction, but does not replace verification. □

Remark 1.97 (Why provenance matters)**.** Provenance is important for reproducibility, accountability, debugging, and version control. It also helps detect when an output has moved from search artifact to certificate candidate to verified object.

U.10bis. Artifact identity, drift, and immutable replay

Definition 1.98 (Immutable artifact identity)**.** An *immutable artifact identity* is a content-addressed or version-pinned identifier sufficient to distinguish an artifact from every changed, regenerated, repaired, superseded, or environment-dependent variant.

Definition 1.99 (Agent artifact-drift record)**.** An *agent artifact-drift record* is opened when an input, output, source statement, locator, dependency, tool environment, retrieval corpus, prompt, system instruction, model, seed policy, proof-assistant library, schema, or platform object differs from the identity referenced by a certificate or proof object. It records whether the drift is irrelevant, requires computational replay, semantic replay, source re-binding, semantic diff, demotion, supersession, or a new claim.

Definition 1.100 (Replay boundary)**.** The *replay boundary* is the exact collection of inputs, environments, code, models, artifacts, settings, random seeds, and external dependencies that a replay action reconstructs. A replay success is meaningful only inside this boundary.

Proposition 1.101 (Artifact drift blocks silent reuse)**.** *If a consumed artifact has drifted outside its declared identity or replay boundary, it cannot be silently reused as current verifier-side evidence.*

Proof. The consumed evidence was tied to a specific artifact identity and environment. A drifted artifact may have different mathematical content, code, data, or dependencies. Silent reuse would erase the distinction between the old and new object. Therefore replay, semantic diff, or demotion is required before reuse. □

Proposition 1.102 (Replay success is not semantic identity)**.** *Replay success proves reconstruction of the declared workflow output under the replay boundary; it does not by itself prove semantic identity with a different artifact, statement, or environment.*

Proof. Replay checks reproducibility of the recorded process. Semantic identity with another artifact is a separate comparison claim between contents and scopes. That comparison requires a semantic diff or preservation theorem. □

Remark 1.103 (New artifact, new claim when semantics change)**.** If an artifact change alters a theorem statement, hypothesis, comparison mode, diagnostic artifact, ledger, manifest field, or external-domain implication, the repaired object should be treated as a new claim or new revision rather than a silent update.

U.11. Agent failure modes

Definition 1.104 (Agent failure mode)**.** An *agent failure mode* is a way in which an agent output may be misleading, invalid, incomplete, unsupported, stale, or misclassified.

Definition 1.105 (Hallucinated claim)**.** A *hallucinated claim* is a claim generated by an agent without adequate support, proof, citation, artifact, or verification.

Definition 1.106 (Claim inflation)**.** *Claim inflation* occurs when a lower-status output, such as a search artifact, advisory indicator, draft, confidence score, or [Spec] bridge, is described or used as if it were a theorem or verified certificate.

Definition 1.107 (Scope overrun)**.** *Scope overrun* occurs when an agent or tool output is used beyond its declared authority scope.

Definition 1.108 (Artifact drift)**.** *Artifact drift* occurs when an artifact changes, is replaced, becomes stale, or becomes inconsistent with the certificate or proof object that references it.

Definition 1.109 (Bridge inflation)**.** *Bridge inflation* occurs when a bridge draft, bridge program, or [Spec] bridge is treated as a proved bridge theorem.

Definition 1.110 (Formalization mismatch)**.** It occurs when a verified formal statement is read as a stronger or different informal claim.

Definition 1.111 (Detector-stage mutation)**.** *Detector-stage mutation* occurs when an agent changes, guesses, or reuses a detector source stage without constructing a new detector record and proving applicability on the new source object.

Definition 1.112 (Benchmark leakage)**.** *Benchmark leakage* occurs when a target theorem conclusion, target truth value, benchmark-supplied witness, stabilization index, regularity conclusion, or equivalent conclusion-side datum is inserted into the realization, finite certificate, search oracle, reduction, or Return input intended to recover that theorem.

Definition 1.113 (Fabricated source authority)**.** *Fabricated source authority* occurs when an agent invents or misstates a source, theorem locator, quotation, DOI, artifact identity, dependency, or semantic-authority relation.

Definition 1.114 (Untrusted-content injection)**.** *Untrusted-content injection* occurs when retrieved text, a prompt, source file, comment, generated schema, or external instruction is treated as governing proof or platform policy without passing the declared trust, source-binding, and authority checks.

Definition 1.115 (Circular self-citation)**.** *Circular self-citation* occurs when an agent-generated summary, register entry, DAG node, proof-store record, replay output, or prior model answer is used as independent evidence for the same underlying claim without an independent source or proof path.

Proposition 1.116 (Agent contamination is non-compensating)**.** *Benchmark leakage, detector-stage mutation, fabricated source authority, untrusted-content injection, and circular self-citation cannot be repaired by confidence, consensus, metric slack, successful replay, or manifest completeness.*

Proof. Each defect invalidates the semantic origin, source object, independence, or admissibility of the consumed evidence. The listed favorable signals address different predicates and do not reconstruct the missing semantic arrow. □

Proposition 1.117 (Agent failure modes require status downgrade)**.** *If an agent failure mode is detected in an object, the object must be downgraded to an appropriate non-verified status until repaired and rechecked.*

Proof. A detected failure mode undermines the current status of the object. Until the defect is repaired and verification or audit is repeated, the previous verified or candidate status is not justified. □

Remark 1.118 (Failure modes are expected)**.** Agent failure modes are not exceptional in a search-heavy workflow. The platform must assume they will occur and must provide status labels, provenance, and audit mechanisms to contain them.

U.12. Agent outputs and Core claims

Definition 1.119 (Core claim by agent)**.** A *Core claim by agent* is any assertion by an agent that a Core condition, theorem, gate, diagnostic, budget comparison, representative condition, eligibility condition, artifact condition, or certificate status holds.

Definition 1.120 (Admissible agent Core claim)**.** An agent Core claim is *admissible* only if it is supported by verifier-side data sufficient to reconstruct the claimed Core status.

Proposition 1.121 (Agent assertion does not establish Core claim)**.** *An agent assertion that a Core claim holds does not establish that claim unless the supporting verifier-side data are present.*

Proof. Core claims require verification. An assertion is a statement about the claim, not the verification itself. Without verifier-side data, the assertion is unsupported. □

Remark 1.122 (This includes AI and human assertions)**.** This rule applies equally to AI agents and human agents. Neither confidence nor authority substitutes for the certificate record.

U.13. Agent interaction with proof assistants

Definition 1.123 (Proof assistant interaction)**.** A *proof assistant interaction* is an action in which an agent submits, modifies, checks, or reads formal code or proof objects in a proof assistant.

Definition 1.124 (Formal fragment)**.** A *formal fragment* is a piece of formal code, theorem statement, definition, lemma, tactic script, or proof script intended for proof assistant checking.

Definition 1.125 (Verified formal fragment)**.** A *verified formal fragment* is a formal fragment that has been successfully checked by the declared proof assistant under the declared environment.

Definition 1.126 (Formal interpretation theorem)**.** A *formal interpretation theorem* is a theorem connecting a verified formal statement to an intended informal mathematical statement or interface claim.

Proposition 1.127 (Verified formal fragment is scoped to its formal statement)**.** *A verified formal fragment establishes only the formal statement that was checked in the declared environment.*

Proof. A proof assistant checks formal syntax, typing, and proof obligations relative to its environment. It does not automatically verify informal interpretations, external mathematical claims, or problem-specific meanings not encoded in the formal statement. □

Proposition 1.128 (Formal interpretation requires a bridge)**.** *A verified formal fragment may support an informal or problem-specific claim only if an interpretation theorem connects the formal statement to that claim.*

Proof. The formal statement and the intended informal claim may differ in hypotheses, semantics, or scope. A connecting theorem is required to justify the interpretation. □

Remark 1.129 (Formalization boundary)**.** A formal proof of a formal statement may still require an interpretation theorem connecting that statement to the intended mathematical claim. This is especially important for problem interfaces.

U.14. Agent manifest entries

Definition 1.130 (Agent manifest entry)**.** An *agent manifest entry* records:

(i) the agent identity or class, semantic role, task, and action;

(ii) immutable input and output artifacts and dependency closure;

(iii) prompt/context/retrieval, model/tool/environment, nondeterminism, and post-processing identities when relevant;

(iv) every independent status axis and the exact target schema of any verifier-side conversion;

(v) monitored degree, window, threshold, endpoint, detector stage, comparison mode, and ledger relation when invoked;

(vi) source-statement, precursor, benchmark-quarantine, finite-to-infinite, Return, and interpretation links when invoked;

(vii) confidence, score, plausibility, or ranking only as proposer-side metadata;

(viii) authority scope, verification mode, source authority, artifact adequacy, semantic-preservation, and replay records;

(ix) contamination, circularity, drift, conflict-of-interest, demotion, supersession, audit, and failure-route notes.

Definition 1.131 (Agent action log)**.** An *agent action log* is an ordered list of agent manifest entries recording the lifecycle of a search artifact, proof object, certificate candidate, formal fragment, bridge statement, or verified object.

Definition 1.132 (Agent lifecycle)**.** The *agent lifecycle* of an object is the sequence:

$$\text{generated} \longrightarrow \text{transformed} \longrightarrow \text{labeled} \longrightarrow \text{checked} \longrightarrow \text{audited} \longrightarrow \text{curated},$$

with stages omitted or repeated as appropriate.

Proposition 1.133 (Agent manifest entries support reproducibility)**.** *Agent manifest entries support reproducibility by recording how objects were generated, transformed, verified, audited, and curated.*

Proof. Reproducibility requires reconstruction of the workflow. Agent manifest entries record the relevant actions, inputs, outputs, roles, status labels, environments, and artifact references. Thus they support reconstruction. □

Remark 1.134 (Manifest is not verdict)**.** An agent manifest entry records actions and status. It does not itself establish mathematical truth unless it contains or references the required verifier-side evidence.

U.15. Agent interaction with bridges and proof objects

Definition 1.135 (Agent-generated bridge draft)**.** An *agent-generated bridge draft* is a proposed implication connecting two layers of the AK framework, such as:

$$\text{Core-visible condition} \Longrightarrow \text{problem-specific conclusion},$$

generated by an agent before proof.

Definition 1.136 (Agent-generated proof object)**.** An *agent-generated proof object* is a structured proof object or proof-object candidate generated, filled, transformed, or assembled by an agent.

Definition 1.137 (Bridge verification requirement)**.** A bridge draft may be promoted to a bridge theorem only after the complete invoked chain is supplied and verified, including:

(i) exact source and target semantics, direction, source class, hypotheses, and prohibited converse;

(ii) immutable source-statement binding and purpose-faithful realization;

(iii) the Core-readable profile and direct proof or complete finite detector certificate;

(iv) representative and standard higher-edge packages when invoked;

(v) actual Type IV morphism and separately typed transient/full-interior records when invoked;

(vi) a finite-to-infinite reduction when a finite witness supports an infinite target;

(vii) a separately proved Return theorem for the external conclusion;

(viii) comparison, ledger, claim-use, artifact, non-circularity, and typed failure-route records;

(ix) verification by an authority whose scope covers each consumed arrow.

A path, program, generated proof object, or completed schema is not a substitute for any missing arrow.

Proposition 1.138 (Agent-generated bridge draft is not bridge theorem)**.** *An agent-generated bridge draft is not a bridge theorem.*

Proof. A bridge theorem requires a precise statement, declared hypotheses, semantic compatibility, and proof. A draft supplies only a candidate bridge. Therefore it is not theorem-level until verified. □

Proposition 1.139 (Agent-generated proof object is not proof by default)**.** *An agent-generated proof object is not a proof by default.*

Proof. A proof object becomes proof-level only after its required fields, claims, formal fragments, bridges, artifacts, and verifier-side checks are supplied and verified. Generation alone does not establish those conditions. □

Remark 1.140 (Bridge drafts are useful)**.** Bridge drafts are useful because they expose what must be proved. Their value lies in structuring the proof search, not in establishing theorem-level validity.

U.16. Agent interaction with Core repaired objects

Definition 1.141 (Agent-produced repaired evaluation record)**.** An *agent-produced repaired evaluation record* is an output in which an agent claims to have produced or computed:

$$\mathrm{Eval}_{i,W,\tau}(F) = T_\tau^{\mathrm{bar}}\big(\mathsf{W}_W \mathbf{P}_i(F)\big).$$

Definition 1.142 (Agent-produced representative record)**.** An *agent-produced representative record* is an output in which an agent claims to have produced or checked the admissible representative $C_{\tau,W}F$.

Definition 1.143 (Agent-produced tower artifact)**.** An *agent-produced tower artifact* is an output in which an agent claims to have produced or checked:

$$\phi_{i,W,\tau} : \mathrm{ColimProfile}_{i,W,\tau}(F_\bullet) \to \mathrm{ApexProfile}_{i,W,\tau}(F_\infty).$$

Proposition 1.144 (Agent-produced Core records require verification)**.** *Agent-produced repaired evaluation records, representative records, and tower artifacts require verification before they may support Core certification.*

Proof. These records are verifier-side data only when they are well-typed, artifact-supported, and checked under the declared Core protocol. Agent production alone does not establish those conditions. □

Remark 1.145 (Core object sensitivity)**.** Small changes in windows, thresholds, barcode policies, representative choices, or tower artifacts may change the verdict. Therefore agent-produced Core records must be treated as candidate records until verified.

U.16bis. Agent-produced metric, budget, and Type IV records

Definition 1.146 (Agent-produced comparison record)**.** An *agent-produced comparison record* is an agent output claiming an exact comparison, a metric-gap-certified comparison, or a non-comparison status between declared profiles or artifacts. It must declare exactly one runtime mode:

$$\text{exact}, \qquad \text{metric\_gap\_certified}, \qquad \text{not\_compared}.$$

The mode not_compared cannot discharge an invoked comparison obligation.

Definition 1.147 (Agent-produced threshold-gap record)**.** An *agent-produced threshold-gap record* is an agent output claiming that a metric comparison is protected through hard threshold deletion. It must identify the actual windowed pre-threshold profiles, bottleneck discrepancy evidence, protected profile family $\mathfrak{B}$, scalarization val_B, and strict inequality

$$\mathrm{val}_B(\mathfrak{d}_{\mathrm{met}}) < \mathrm{gap}_\tau(\mathfrak{B}).$$

It must also record the specialized threshold-gap status

$$\text{gap\_certified}, \qquad \text{gap\_failed}, \qquad \text{gap\_pending},$$

with the canonical mapping to pass, reject, and undefined in the audit status calculus.

Definition 1.148 (Agent-produced Type IV diagnostic record)**.** An *agent-produced Type IV diagnostic record* is an agent output claiming exactly one of

$$\text{not\_invoked}, \quad \text{undefined}, \quad \text{certified\_zero}, \quad \text{obstructed}$$

for one immutable monitored Type IV scope. If invoked, it must bind an actual persistence morphism $\phi_{i,W,\tau}$, the construction and semantic authority of its source and target, transition and colimit data, apex

and apex-agreement status, terminal-tameness evidence, and the full kernel and cokernel defect profiles. Numerical diagnostics are computed only from

$$\mu_{i,W,\tau} := \mathrm{tgdim}_W(\ker \phi_{i,W,\tau}), \qquad u_{i,W,\tau} := \mathrm{tgdim}_W(\mathrm{coker}\, \phi_{i,W,\tau})$$

on the same valid terminal scope. For one fixed (W, τ) and one declared finite monitored degree set I_{mon}, the aggregate is

$$\mu_{\mathrm{Collapse}} := \sum_{i \in I_{\mathrm{mon}}} \mu_{i,W,\tau}, \qquad u_{\mathrm{Collapse}} := \sum_{i \in I_{\mathrm{mon}}} u_{i,W,\tau}.$$

No aggregation is performed across different windows or thresholds. No numerical pair is assigned when the status is not_invoked or undefined. Terminal Type IV status does not encode full-interior or long-transient status.

Proposition 1.149 (Agent comparison mode does not self-certify)**.** *An agent's declaration of comparison mode does not establish that the declared mode is valid.*

Proof. Mode validity requires the corresponding evidence: an exact equality or profile isomorphism for exact mode, a bottleneck bound plus two-sided threshold-gap record for metric mode, or proof that no comparison obligation is consumed for not-compared mode. The declaration names the intended mode but does not supply the evidence. □

Proposition 1.150 (Exact output is not robustness)**.** *An agent-produced exact equality or profile isomorphism is not a positive-radius robustness certificate unless a separate metric-gap-certified record is supplied.*

Proof. Exact comparison and metric robustness are different v20 comparison modes. A specific equality may hold even when the threshold clearance is zero. Robustness through hard deletion requires a bottleneck discrepancy bound and two-sided threshold-gap protection. □

Proposition 1.151 (Agent Type IV zero requires actual artifact)**.** *An agent-produced claim that Type IV diagnostics are zero is unsupported unless the declared tower comparison artifact, terminal-tameness evidence, monitored scope, and terminal-generic defect computation are present.*

Proof. Type IV diagnostics are defined from the kernel and cokernel defect profiles of an actual comparison artifact. Without that artifact and terminal-tameness evidence, the diagnostic status is undefined or not_invoked, not certified zero. □

Remark 1.152 (Agent-produced zero values)**.** A displayed zero, empty list, passed test, green dashboard, or natural-language phrase such as "no defect found" is not by itself a certified zero diagnostic, exact comparison, or zero metric defect. The typed record determines the meaning of zero.

U.16ter. Agent-produced finite detector records

Definition 1.153 (Agent-produced detector record)**.** An *agent-produced detector record* contains:

(i) one immutable source artifact and exactly one source stage among

windowed_prethreshold, repaired_postthreshold, kernel_defect, cokernel_defect;

(ii) a detector certificate status separate from its mathematical value;

(iii) the monitored degree, window, threshold, endpoint or germ convention, and family membership;

(iv) the soundness theorem for any nonzero reading;

(v) the zero-completeness or characterization theorem for any zero reading;

(vi) birth, germ, stencil, mesh, or atlas coverage and any non-adaptivity or holdout condition;

(vii) the source and provenance of every uniform excess or coverage bound;

(viii) comparison and threshold-gap transport records when the detector conclusion is transferred between profiles;

(ix) immutable computation, proof, and source artifacts and typed failure routes.

The four stage tokens are a typing vocabulary, not a blanket existence theorem. The standard v20 finite detector is proved at stage windowed_prethreshold. Any use at repaired_postthreshold, kernel_defect, or cokernel_defect requires its own stage-specific detector theorem or an explicit stage-compatible transport theorem carrying every invoked soundness, completeness, coverage, and artifact field. Without such evidence, the detector certificate status is undefined.

Definition 1.154 (Detector certificate/value separation)**.** A detector record has two independent outputs:

(i) a certificate status describing whether the detector theorem and coverage obligations are discharged;

(ii) a mathematical value describing the detector output on its declared source object.

A numerical or symbolic zero with an incomplete certificate is not a certified zero.

Proposition 1.155 (Finite agent output is not finite proof)**.** *A finite list of values, ranks, sampled maps, critical parameters, bars, mesh points, or generated witness fields is not a finite proof certificate merely because it is finite.*

Proof. Finite proof requires the declared theorem connecting the finite data to the universal target predicate, together with applicability, coverage, scope, and immutable artifact evidence. Finiteness of the output supplies none of those implications by itself. □

Proposition 1.156 (Detector stage cannot be inherited or mutated)**.** *An agent may not relabel a detector from one source stage to another, reuse its completeness theorem on a different profile class, or infer a defect-profile conclusion from a topological-profile detector without a new typed detector record and theorem.*

Proof. The four stages have different source objects and semantics. Applicability and completeness are theorem-relative to the exact source class, so changing the stage changes the proposition being certified. □

Proposition 1.157 (Soundness does not supply zero completeness)**.** *An agent-produced nonzero-sound detector theorem does not license a zero verdict when all sampled outputs vanish.*

Proof. Nonzero soundness proves a one-way implication from an observed nonzero event. A zero verdict requires the converse-style coverage or characterization theorem on the full declared family. □

U.16quater. Agent-produced higher-obstruction records

Definition 1.158 (Agent-produced standard degree-n edge record)**.** An *agent-produced standard degree-n edge record* binds, for one invoked n:

(i) the exact repaired degree-n profile and admissible representative;

(ii) an actual profile isomorphism at the stated strength;

(iii) a genuine realization or separately proved admissible replacement in the declared realization domain;

(iv) amplitude, admissibility, and a fixed edge extractor E_n with $E_n(0) = 0$;

(v) an artifact-backed identification of H^{-n} with the extractor;

(vi) naturality scope and compatibility under representative or realization change;

(vii) claim status, artifact identity, and non-circularity evidence.

Its default theorem direction is

$$\mathrm{Eval}_{n,W,\tau}(F) = 0 \Longrightarrow \mathrm{Ext}^n(A_n(F), k) = 0.$$

Proposition 1.159 (Agent-produced higher edge does not self-promote)**.** *A generated derived object, shifted detector value, synthetic duality, or formally type-correct* Ext^n *expression does not establish the standard degree-n bridge package.*

Proof. The standard package requires a purpose-appropriate realization, actual profile identification, edge interpretation, naturality, and artifact support. A synthetic expression can satisfy an algebraic identity while lacking those semantic arrows. □

Proposition 1.160 (Reverse higher-edge inference requires separate reflection)**.** *An agent may not infer* $\mathrm{Eval}_{n,W,\tau}(F) = 0$ *from* $\mathrm{Ext}^n(A_n(F), k) = 0$ *unless a separately proved natural zero-reflection or faithfulness theorem is bound to the same realization and profile class.*

Proof. The v20 default bridge is one-way. The reverse implication is a distinct theorem and cannot be generated by notation, duality language, or agent confidence. □

U.16quinquies. Terminal, transient, and finite-to-infinite semantics

Definition 1.161 (Agent-produced transient-defect record)**.** An *agent-produced transient-defect record* is a new detector record whose source stage is kernel_defect or cokernel_defect and whose source object is a full interior defect profile of the actual comparison morphism. It separately records the full-interior status

not_checked, certified_absent, certified_present

and, when invoked, the long-transient status

not_invoked, undefined, certified_absent, certified_present.

Proposition 1.162 (Terminal zero is not full-defect zero)**.** *An agent-produced terminal Type IV certified-zero record does not establish vanishing of the full kernel or cokernel profiles, absence of finite interior or long-transient defects, isomorphism of the comparison morphism, apex agreement, or external information preservation.*

Proof. Terminal-generic dimensions retain only terminal information on the declared scope. Each stronger statement requires a different source object or theorem. □

Definition 1.163 (Agent-produced finite-to-infinite reduction record)**.** An *agent-produced finite-to-infinite reduction record* identifies the exact infinite target, finite source evidence, transition system, reduction theorem, hypotheses, actual colimit or completion construction, apex object, apex-agreement artifact, and failure boundary. It distinguishes finite observation, finite certificate, finite prefix, numerical convergence, Cauchy behavior, eventual constancy, colimit or homotopy colimit, completion, apex, and Return.

Proposition 1.164 (Finite plateau and replay do not imply reduction)**.** *A finite plateau, repeated finite prefix, numerical convergence, graph path, stored apex, or successful replay does not establish eventual constancy, a colimit identification, apex agreement, or a finite-to-infinite reduction.*

Proof. Each listed observation is compatible with later change or with a mismatched infinite object. The universal or limiting conclusion requires its own theorem and actual transition/apex data. □

Definition 1.165 (Agent-produced Return record)**.** An *agent-produced Return record* binds the exact AK-side certified predicate to an external-domain conclusion through a proved theorem with declared source and target semantics, direction, hypotheses, source class, artifact identity, and prohibited stronger readings. It is distinct from realization, Core certification, reduction, formal interpretation, and independent validation.

Proposition 1.166 (Agent-generated chain is not Return)**.** *A generated path, completed DAG, Bridge Program, proof-store package, manifest, replay, or natural-language explanation does not establish an external conclusion when a required realization, reduction, higher edge, or Return theorem is absent.*

Proof. The external conclusion is licensed only by the composition of proved arrows at their declared strengths. Infrastructure records may organize those arrows but do not create a missing implication. □

U.16sexies. Known-Theorem Recovery and source quarantine

Definition 1.167 (Agent-managed Known-Theorem Recovery track)**.** An *agent-managed Known-Theorem Recovery track* is an operational record for reconstructing a frozen known theorem or theorem clause through a declared Interface–Core–Reduction–Return chain. It records the exact benchmark statement, permitted precursor package, source and target classes, realization, finite certificate, reduction, Return, independent validation, and prohibited stronger readings. Its existence is not the recovered theorem.

Definition 1.168 (Source-statement and precursor binding)**.** A *source-statement and precursor binding* distinguishes:

(i) the authoritative bibliographic source and exact theorem locator;

(ii) a local immutable statement-binding artifact;

(iii) the permitted precursor statements and their locators;

(iv) the benchmark conclusion reserved for final validation;

(v) any source-to-interface specialization that remains separate.

Definition 1.169 (Benchmark-quarantine record)**.** A *benchmark-quarantine record* proves by dependency and input inspection that no benchmark proof, known truth value, answer table, target constant, exceptional bound, conclusion-derived witness, or equivalent oracle entered the realization, Core certificate, finite-to-infinite reduction, higher-edge proof, or Return premise. The immutable benchmark statement and locator may identify the exact target and the consequent of the proposed Return theorem; they do not supply derivational evidence.

Definition 1.170 (Target-separating realization record)**.** A *target-separating realization record* establishes, on the declared source class and for the target predicate, that source objects with different target truth values are not silently collapsed into indistinguishable Core inputs in a way that invalidates the proposed Return direction. It is predicate-relative and does not assert full categorical faithfulness.

Proposition 1.171 (Benchmark leakage invalidates recovery evidence)**.** *If benchmark leakage is detected, the affected recovery conversion is rejected or left undefined according to the typed failure route; storage, consensus, replay, or an exact-looking final equality cannot repair it.*

Proof. The recovery would no longer demonstrate that the target conclusion follows from the permitted inputs. It would encode part of the conclusion into its own premise, creating circular evidence. □

Proposition 1.172 (Recovery infrastructure is not recovered mathematics)**.** *The existence of a recovery track, source registry, DAG, proof-store bundle, replay package, or multi-agent agreement does not by itself establish exact-strength recovery.*

Proof. Exact-strength recovery is a theorem-level property of the complete non-circular chain and its source/Return semantics. Infrastructure establishes organization, identity, or reproducibility only at its declared mode. □

Remark 1.173 (v20 reference tracks)**.** The following frozen local tracks may be managed as v20 operational examples:

$$\text{IW-KTR-GROWTH} \qquad \text{and} \qquad \text{NS-KTR-SERRIN-L6}.$$

For the former, only the Iwasawa 1959 Theorem 4 characteristic-growth clause is recovered relative to its registered precursor package; no main conjecture, class-number specialization, or general information-preservation claim follows. For the latter, only the one-way fixed-exponent strict-subcritical Serrin criterion is recovered; no new analytic proof or unconditional three-dimensional regularity result follows. Their final CR promotion remains a separate governance act.

U.16septies. Adversarial content, independence, and semantic replay

Definition 1.174 (Agent trust-boundary record)**.** An *agent trust-boundary record* classifies prompts, retrieved passages, uploaded files, generated code, comments, platform metadata, tool output, and external instructions as trusted governing policy, authoritative source, untrusted data, or quarantined content. Untrusted data cannot redefine the verifier, source hierarchy, or claim status.

Definition 1.175 (Agent independence record)**.** An *agent independence record* states whether compared agents or reviews share models, prompts, training lineage, retrieved corpora, source summaries, proof scripts, human editors, or cached outputs. It identifies correlated failure and conflict-of-interest risks.

Proposition 1.176 (Majority vote and consensus are proposer-side)**.** *A majority vote, self-consistency sample, ensemble score, debate outcome, or multi-agent consensus remains proposer-side unless every consumed theorem obligation is independently verified.*

Proof. Agreement aggregates outputs, not proofs. Correlated agents may reproduce the same unsupported premise, source error, or benchmark leakage. □

Definition 1.177 (Computational and semantic replay)**.** A *computational replay* reconstructs declared bytes, logs, computations, or formal checks inside a replay boundary. A *semantic replay* additionally verifies that the replayed artifacts still instantiate the same statements, scopes, source bindings, comparison modes, detector stages, status axes, and interpretation arrows consumed by the proof object.

Proposition 1.178 (Computational replay is not semantic replay)**.** *A byte-identical or numerically identical replay does not establish semantic replay when statements, source authority, dependencies, interpretations, or scopes have changed or were never verified.*

Proof. Computational equality concerns the reproduced process or output. Semantic identity is a separate relation among meanings, scopes, and authority bindings. □

Proposition 1.179 (Generated structured data are candidate records)**.** *Agent-generated JSON, YAML, CSV, manifests, source registries, claim tables, hashes, or proof-object fields are candidate records until schema validation, artifact adequacy, semantic consistency, and target-specific verifier conversion are complete.*

Proof. Serialization can preserve or organize asserted fields without establishing that the fields are true, complete, non-circular, or semantically authorized. □

U.16octies. Ledger discipline and final CR deferral

Definition 1.180 (Agent-produced ledger relation record)**.** An *agent-produced ledger relation record* declares whether each local discrepancy name is an alias, refinement, aggregate, or disjoint component relative to the Appendix M catalog. A component enters the metric ledger only when a scope-matched theorem or artifact proves that it upper-bounds an actual profile discrepancy consumed by the proof.

Proposition 1.181 (Agent labels do not create metric components)**.** *Search error, confidence error, ranking error, prompt drift, storage drift, environment mismatch, detector incompleteness, missing Return, or semantic replay failure does not become a numerical bottleneck component merely because an agent assigns it a scalar name.*

Proof. Metric admissibility requires an actual profile pair and a proved discrepancy bound in the declared quantale and scalarization. The listed defects are ordinarily non-scalar obligations or governance statuses. □

Proposition 1.182 (No agent-side double counting)**.** *An agent may not count one primitive discrepancy separately under search, lift, bridge, map, storage, execution, or replay names unless the records prove that the components are genuinely disjoint or sequentially composed on the same proof path.*

Proof. Renaming a primitive discrepancy does not create additional independent error. The ledger relation record controls aggregation. □

Definition 1.183 (Final-CR deferral record)**.** A *final-CR deferral record* states that Part V and Companion claims retain stable local labels and immutable artifact identities while canonical Claim UID assignment, final CR promotion, JSON/CSV release registration, and promotion receipts are deferred until the full appendix sequence and Companion are frozen and cross-audited.

Proposition 1.184 (Agent cannot fabricate provisional promotion)**.** *An agent may inventory local statements and dependencies during Part V construction, but it may not represent a provisional UID, local register, generated JSON/CSV table, or draft receipt as canonical CR-v20 promotion.*

Proof. Canonical promotion is a later governance act over frozen artifacts and their final dependency closure. A construction-stage record lacks that closure and authority. □

U.17. Summary of agent semantics

Theorem 1.185 (Appendix U v20 agent-semantics package)**.** *Within the Search / Platform layer, the following operational package is available.*

(i) Agent type, authorship, confidence, consensus, storage, and replay do not determine mathematical validity.

(ii) Proposer, verifier, auditor, curator, adjudicator, source-binder, and Return-interpreter actions remain semantically distinct, even when performed by one agent.

(iii) Every theorem-facing consumption requires a target-schema-specific verifier-side conversion record.

(iv) Status axes and specialized statuses are independent. Missing evidence is undefined*; scope exclusion is required for* not_invoked*.*

(v) Finite observation is not finite proof; detector certificate status and detector mathematical value are distinct.

(vi) Detector stage cannot be guessed, inherited, relabeled, or transported without a new typed theorem and record.

(vii) *Soundness does not imply zero completeness.*

(viii) *A standard higher-n edge requires the complete eligibility, realization, extractor, H^{-n}, naturality, artifact, and non-circularity package; the reverse direction requires separate zero reflection.*

(ix) *Type IV requires an actual comparison morphism; terminal, full-interior, and long-transient statuses remain separate.*

(x) *Finite plateau, replay, or stored apex does not establish finite-to-infinite reduction or apex agreement.*

(xi) *An external conclusion requires the complete realization–Core–reduction–higher-edge–Return chain actually invoked.*

(xii) *Known-Theorem Recovery requires source/precursor binding, benchmark quarantine, target-sensitive semantics, and non-circularity.*

(xiii) *Retrieval, RAG, memory, context inclusion, generated summaries, and source registries do not become source evidence by inclusion.*

(xiv) *Transformation, translation, summary, refactoring, formalization, code generation, and migration require semantic preservation or semantic diff when equivalence is consumed.*

(xv) *Proof-assistant success establishes only the encoded statement in its environment; informal or external interpretation requires an interpretation theorem.*

(xvi) *Prompt, system instruction, model, tool, corpus, environment, seed, sampling, and post-processing identities belong to provenance when relevant.*

(xvii) *Adversarial instructions, untrusted retrieved content, fabricated locators, benchmark leakage, circular self-citation, and detector-stage mutation are quarantined failure modes.*

(xviii) *Computational replay and semantic replay are distinct.*

(xix) *Generated JSON, YAML, CSV, manifests, hashes, and DAGs are candidate records, not verifier evidence by generation alone.*

(xx) *Metric ledger inclusion requires an actual discrepancy theorem, quantale/scalarization support, and no-double-counting relation.*

(xxi) *Local labels remain stable while final downstream CR promotion is deferred; an agent cannot fabricate provisional canonical promotion.*

Proof. Items (i)–(iv) follow from the sovereignty, role, conversion, preflight, and status-axis provisions above. Items (v)–(vii) follow from the finite-detector definitions and Propositions 1.155, 1.156, and 1.157. Item $(viii)$ follows from Definition 1.158 and Propositions 1.159 and 1.160. Items (ix)–(xi) follow from the Type IV, transient, reduction, and Return provisions. Item (xii) follows from the Known-Theorem Recovery and benchmark-quarantine provisions. Items $(xiii)$–$(xvii)$ follow from the retrieval, transformation, formalization, provenance, trust-boundary, and contamination provisions. Items $(xviii)$–(xix) follow from Definition 1.177 and Proposition 1.179. Item (xx) follows from the ledger relation and no-double-counting provisions. Item (xxi) follows from Definition 1.183 and Proposition 1.184. □

Theorem 1.186 (Appendix U v20.0 hardening and compatibility package)**.** *The v20 reconstruction conservatively preserves every valid v19 agent-sovereignty, proposer–verifier, promotion, retrieval, transformation, authority, artifact, comparison, and Type IV boundary, and strengthens them as follows:*

(i) *degree-one wording is replaced by invoked degree-n semantics and the standard n-eligible edge package;*

(ii) detector records acquire the four-stage discipline and certificate/value separation;

(iii) Type IV acquires explicit actual-morphism provenance and terminal/transient/full-interior separation;

(iv) finite-to-infinite reduction and Return become independent conversion obligations;

(v) source, precursor, benchmark-quarantine, and target-separation records govern recovery tracks;

(vi) semantic replay, trust boundaries, nondeterministic provenance, and contamination control supplement computational replay;

(vii) status-axis separation, typed failure routing, quantale ledger support, and final-CR deferral prevent governance inflation.

No inherited label is reused for an unrelated claim.

Proof. The inherited statements remain available at no stronger than their displayed v19 meanings after the global v20 substitutions. The new definitions and propositions add independent obligations and explicit prohibitions; they do not remove a v19 hypothesis or promote an operational record to theorem evidence. Thus the reconstruction is a conservative governance strengthening. □

U.18. Explicit exclusions

The following are not asserted in Appendix U.

- No AI output, human assertion, self-evaluation, majority vote, debate outcome, or multi-agent consensus is proof by itself.
- No confidence value, ranking, plausibility score, dashboard state, search hit, or validity-map path is a Core verdict.
- No finite list, sampled zero, empty output, finite prefix, plateau, or generated witness field is a finite proof without the governing completeness or reduction theorem.
- No detector stage is guessed, relabeled, inherited, or reused across topological, repaired, kernel-defect, and cokernel-defect objects.
- No nonzero-sound detector is read as zero-complete.
- No agent-generated synthetic shift or derived expression is a natural higher-n bridge by itself.
- No reverse Ext^n-to-repaired-zero implication is used without a separate natural zero-reflection theorem.
- No object equality, barcode similarity, bottleneck matching, Betti equality, stored representation, or agent assertion manufactures the actual morphism required by Type IV.
- No Type IV undefined or not_invoked record receives a numerical pair.
- No terminal Type IV zero is interpreted as full kernel/cokernel zero, transient absence, morphism isomorphism, apex agreement, or external information preservation.
- No finite plateau, numerical convergence, graph path, stored apex, build, or replay establishes eventual constancy, colimit identification, completion, or apex agreement.
- No generated bridge chain, Bridge Program, DAG, manifest, proof-store bundle, or replay substitutes for realization, finite certification, reduction, higher edge, or Return.

- No Known-Theorem Recovery uses the benchmark conclusion, target truth value, conclusion-derived tail witness, regularity conclusion, or equivalent oracle as a realization or certificate input.
- No recovery-track metadata, source registry, storage event, or consensus regenerates the mathematical exactness of IW-KTR-GROWTH or NS-KTR-SERRIN-L6.
- No Iwasawa main conjecture, Iwasawa Theorem 4 full AK-only reproof, class-number theorem, Mazur arithmetic specialization, arbitrary tower faithfulness, new Serrin proof, unconditional Navier–Stokes regularity, or Clay-level conclusion is asserted here.
- No concrete Fukaya/HMS external bridge is invoked or promoted by this appendix.
- No retrieval hit, memory item, RAG result, context inclusion, AI summary, generated citation, or platform page is source evidence by inclusion alone.
- No fabricated locator, source, DOI, hash, theorem statement, or semantic authority is tolerated as evidence.
- No prompt injection, source injection, untrusted file instruction, or generated comment may redefine the verifier or source hierarchy.
- No transformation, translation, summary, refactoring, formalization, code generation, serialization, or migration is semantics-preserving without a preservation certificate or successful semantic diff when equivalence is consumed.
- No proof-assistant check proves a stronger informal or external claim without an interpretation theorem.
- No successful parse, compile, render, serialization, hash match, artifact validation, computational replay, or storage event is mathematical verification by itself.
- No byte-identical replay is semantic replay by itself.
- No generated JSON, YAML, CSV, register, manifest, Claim UID, promotion receipt, or release table is canonical merely because it is well-formed.
- No workflow label, conformance label, approval, curation, sign-off, or storage status is copied into mathematical pass.
- No absent field becomes not_invoked without an immutable scope-exclusion policy.
- No missing evidence is converted into zero, pass, or a mathematical obstruction; it remains undefined unless an evidenced false condition triggers its typed reject route.
- No exact comparison is treated as positive-radius robustness without metric-gap certification.
- No confidence, search, execution, storage, environment, semantic, Return, or governance defect enters the metric ledger without an actual profile-discrepancy bound and Appendix M relation.
- No primitive discrepancy is double-counted under multiple agent-layer names.
- No stale source, drifted dependency, changed model/tool/environment, or superseded artifact is silently reused.
- No local Part V label, provisional inventory, or generated sidecar is represented as final CR-v20 promotion before Part V and the Companion are frozen and cross-audited.

U.19. Provenance and status

The material in this appendix depends on:

(i) the v20 claim taxonomy, three-axis claim typing, finite-certificate contract, and Problem-Bridge/Return discipline of Chapters 1 and 8;

(ii) the Core sovereignty, atomic status, failure-route, and manifest disciplines of Chapter 7 and Appendix G;

(iii) the CR-v20 baseline register for canonical Main/Foundation governance, while downstream final promotion remains pending;

(iv) Appendix A for window-first repaired evaluation, hard deletion, exact transport, and two-sided threshold-gap safety;

(v) Appendix B for representatives, detector stages, completeness, and stage transport;

(vi) Appendix C for the standard degree-n eligible realization and higher-edge package;

(vii) Appendix D for actual-morphism Type IV, terminal, transient, full-interior, and tower finite-to-infinite discipline;

(viii) Appendix E for coverage, atlas, overlap, Restart, higher coherence, and local-to-global transport when invoked;

(ix) Appendix F for atomic status, typed failure routing, and noncompensation;

(x) CM-v20 for reviewed Core micro-theorems at their displayed local strengths;

(xi) the toy-boundary and counterexample discipline of TB-v20;

(xii) the frozen Technical Extension Appendices H–N, including Appendix M ledger semantics, at their reviewed local strengths;

(xiii) the frozen Problem Interface Part IV, including source binding, target-separating realization, Known-Theorem Recovery, and Return quarantine, at its reviewed local strength;

(xiv) the Part V construction policy that stable local labels and immutable artifacts precede final CR extraction.

Its role is to define how agents may participate in search, proof compression, bridge construction, recovery, execution, storage, and audit without contaminating Core or external verdicts.

Status labels for Appendix U. *Search / Platform definitions:* Definitions 1.11, 1.12, 1.13, 1.14, 1.15, 1.16, 1.18, 1.19, 1.20, 1.21, 1.22, 1.23, 1.24, 1.25, 1.29, 1.30, 1.39, 1.40, 1.41, 1.53, 1.54, 1.55, 1.56, 1.57, 1.64, 1.65, 1.66, 1.67, 1.68, 1.69, 1.70, 1.78, 1.79, 1.80, 1.88, 1.89, 1.90, 1.93, 1.94, 1.95, 1.104, 1.105, 1.106, 1.107, 1.108, 1.109, 1.110, 1.119, 1.120, 1.123, 1.124, 1.125, 1.126, 1.130, 1.131, 1.132, 1.135, 1.136, 1.137, 1.141, 1.142, 1.143.

Search / Platform propositions/theorems: Theorems 1.31, 1.185, Propositions 1.42, 1.43, 1.44, 1.61, 1.62, 1.71, 1.81, 1.91, 1.96, 1.117, 1.121, 1.127, 1.128, 1.133, 1.138, 1.139, 1.144.

Operational cautions: Remarks 1.1, 1.2, 1.3, 1.17, 1.26, 1.32, 1.45, 1.63, 1.72, 1.82, 1.92, 1.97, 1.118, 1.122, 1.129, 1.134, 1.140, 1.145.

Inherited and v20-updated hardening definitions: Definitions 1.5, 1.6, 1.7, 1.33, 1.34, 1.35, 1.46, 1.47, 1.48, 1.73, 1.74, 1.83, 1.84, 1.98, 1.99, 1.100, 1.146, 1.147, 1.148.

Inherited and v20-updated hardening propositions/theorems: Theorem 1.186, Propositions 1.8, 1.9, 1.36, 1.37, 1.49, 1.50, 1.51, 1.75, 1.76, 1.85, 1.86, 1.101, 1.102, 1.149, 1.150, 1.151.

Inherited and v20-updated hardening remarks: Remarks 1.10, 1.38, 1.52, 1.77, 1.87, 1.103, 1.152.

v20 extension definitions: Definitions 1.27, 1.58, 1.59, 1.111, 1.112, 1.113, 1.114, 1.115, 1.153, 1.154, 1.158, 1.161, 1.163, 1.165, 1.167, 1.168, 1.169, 1.170, 1.174, 1.175, 1.177, 1.180, 1.183.

v20 extension propositions: Propositions 1.28, 1.60, 1.116, 1.155, 1.156, 1.157, 1.159, 1.160, 1.162, 1.164, 1.166, 1.171, 1.172, 1.176, 1.178, 1.179, 1.181, 1.182, 1.184.

v20 extension remarks: Remarks 1.4, 1.173.

Explicit exclusions: Section U.18.

2 Appendix V: Hunter / Mapper / Lifter Search Protocols

V.1. Scope, dependency position, and search status

This appendix defines the Hunter / Mapper / Lifter search protocols of AK-HPDST v20.0.0. It belongs to the Search / Platform Appendices and does not enlarge the Core mathematical package of Chapters 1–8 or the Foundation Appendices. Its role is to organize discovery, counterexample search, finite-certificate search, map construction, candidate lifting, and verifier-side submission without converting search success into proof.

The central rule of this appendix is:

Search protocols generate typed candidates; they do not certify them.

Fix a monitored degree i for any invoked Type IV scope and, independently, a degree n for any invoked higher obstruction edge. No search hit, mapped structure, lifted candidate, priority score, agent recommendation, finite sample, generated detector, search-map edge, bridge draft, replay result, or stored package may replace any required component of the v20 chain, including:

$B\text{-Gate}^{+}$, $B_{n,W}(F) := \mathsf{W}_W \mathbf{P}_n(F)$,

$\mathrm{Eval}_{n,W,\tau}(F) = T_\tau^{\mathrm{bar}}\big(B_{n,W}(F)\big)$, a complete stage-correct detector certificate when invoked,

$C_{\tau,W}F$, the complete degree-n eligibility and edge package when invoked,

$\phi_{i,W,\tau} : \mathrm{ColimProfile}_{i,W,\tau}(F_\bullet) \longrightarrow \mathrm{ApexProfile}_{i,W,\tau}(F_\infty)$,

$(\mu_{\mathrm{Collapse}},\ u_{\mathrm{Collapse}}) = (0,0)$ only on a valid fixed Type IV scope,

$\mathrm{val}_B(\mathfrak{d}_{\mathrm{met}}) < \mathrm{gap}_\tau(\mathfrak{B})$, finite-to-infinite reduction and Return records when required.

Remark 2.1 (Search / Platform status)**.** The protocols in this appendix organize discovery, mapping, prioritization, counterexample search, and candidate preparation. They create no external-domain theorem and no new Core theorem.

Remark 2.2 (Relation to Appendix U)**.** Appendix U defines the governing agent semantics. The present appendix specializes those semantics to the roles

$$\text{Hunter}, \qquad \text{Mapper}, \qquad \text{Lifter}.$$

The role of an action, rather than the human, AI, tool, platform, or hybrid identity of its producer, determines how the output may be consumed.

Remark 2.3 (Relation to Core re-entry)**.** The Hunter / Mapper / Lifter workflow prepares candidates for verifier-side re-entry. A re-entry package remains a candidate until every invoked verifier-side obligation has been checked at its own type. Search, lifting, packaging, and manifest completeness are not certification.

Remark 2.4 (v20 dependency and registration boundary)**.** This appendix consumes the v20 Main and Foundation Appendices at their canonical strengths, Appendix U at its frozen local strength, and CM-v20, TB-v20, Technical Extension H–N, and Problem Interface Part IV only at their reviewed or frozen local strengths and immutable artifact identities. The current CR-v20 baseline does not yet promote Part V. Accordingly, the local labels in this appendix are stable construction identifiers, not canonical Claim UIDs, and no provisional Claim UID or promotion receipt is generated here.

V.1bis. Search sovereignty and verifier-side conversion

Definition 2.5 (Search sovereignty)**.** *Search sovereignty* is the rule that Hunter, Mapper, Lifter, ranking, search-map, and platform outputs remain proposer-side or workflow-side data until converted into verifier-side records by a declared protocol. A search-side object may guide the creation of a verifier-side record, but it is not itself a Core verdict, bridge theorem, Type IV diagnostic, metric budget certificate, admissible representative, or proof object.

Definition 2.6 (Search-to-verifier conversion record)**.** A *search-to-verifier conversion record* documents a transition from a search-side object to a verifier-side candidate. It is a Search / Platform specialization of the verifier-side conversion record discipline of Appendix U. It must declare:

(i) the source search object and its status;

(ii) the target verifier-side record type;

(iii) the conversion rule;

(iv) all hypotheses and non-scalar obligations;

(v) the exact profile stage, detector certificate status, and detector mathematical value when a detector is involved;

(vi) the claim document role, evidence status, invocation role, and conformance status;

(vii) every metric, nonmetric, and non-scalar effect introduced by the conversion, kept in separate fields;

(viii) artifact identifiers, source bindings, and semantic-diff records before and after conversion;

(ix) the verification authority, authority scope, environment, and comparison mode;

(x) the result status, chosen from *pass*, *reject*, or *undefined*;

(xi) the typed failure route when the result status is *reject* or *undefined*.

Definition 2.7 (Search claim inflation)**.** *Search claim inflation* occurs when a Hunter output, Mapper edge, Lifter output, score, search map, bridge draft, or re-entry package is described or consumed as if it already established a verifier-side Core condition, bridge theorem, or external-domain theorem.

Proposition 2.8 (No search sovereignty override)**.** *No search protocol, search lifecycle status, score, map edge, lift output, or re-entry package overrides verifier-side Core requirements.*

Proof. Search-side records belong to the proposer and workflow layers. Core verdicts are determined by verifier-side repaired evaluations, representatives, eligibility data, tower artifacts, diagnostics, ledgers, gates, and manifests. Therefore a search-side object can at most propose or prepare verifier-side evidence; it cannot override or replace it. □

Proposition 2.9 (Conversion is not achieved by relabeling)**.** *Relabeling a search artifact as* verified*,* Core-ready*,* theorem-like*, or* certificate-like *does not convert it into verifier-side data.*

Proof. Verifier-side status depends on typed construction, artifact support, and verification under the declared protocol. A label change supplies none of these data. Hence conversion requires a search-to-verifier conversion record and the corresponding checks. □

Remark 2.10 (Search remains proposer-side until conversion)**.** The search layer may be aggressive, heuristic, automated, or AI-assisted. That is acceptable precisely because the transition to Core evidence is guarded by conversion, verification, and audit records.

V.1ter. Claim axes, search objectives, and weaker-safe consumption

Definition 2.11 (Typed search objective)**.** A *typed search objective* records the target document role, intended evidence status, intended invocation role, source and target semantics, monitored degrees, windows, thresholds, detector stage, comparison mode, required artifacts, prohibited stronger readings, and success predicate. A phrase such as "find a proof" is not a typed objective until these fields are resolved.

Definition 2.12 (Search claim-axis record)**.** A *search claim-axis record* keeps distinct:

(i) document role;

(ii) evidence status;

(iii) invocation role;

(iv) conformance status;

(v) workflow lifecycle status.

A favorable lifecycle status does not improve any theorem-facing axis.

Proposition 2.13 (Search cannot collapse claim axes)**.** *No Hunter score, Mapper edge, Lifter status, consensus result, or re-entry readiness label may be used to infer a stronger document role, evidence status, invocation role, or conformance status.*

Proof. The axes answer different questions: what an item is, what evidence supports it, how it is used, whether it conforms, and where it sits in the workflow. Search progress supplies only workflow information unless separate evidence discharges the theorem-facing axes. □

Proposition 2.14 (Weaker-safe search consumption)**.** *If a search object has ambiguous scope, stage, status, authority, source binding, or permitted use, it is consumed only at the weaker safe interpretation; missing required data yield* undefined*, not pass, zero, not invoked, or a numbered mathematical obstruction.*

Proof. An ambiguity prevents reconstruction of the exact proposition submitted to the verifier. The v20 governance rule therefore withholds the stronger interpretation rather than imputing evidence that is not present. □

V.2. Search objects

Definition 2.15 (Search object)**.** A *search object* is any object produced, selected, transformed, ranked, mapped, or lifted during search. Examples include:

(i) candidate windows;

(ii) candidate thresholds;

(iii) candidate stable bands;

(iv) candidate bridge statements;

(v) candidate proof routes;

(vi) advisory indicators;

(vii) persistence profiles;

(viii) candidate filtered objects;

(ix) candidate tower diagrams;

(x) candidate normalization routes;

(xi) candidate problem-interface realizations;

(xii) candidate repaired evaluation records;

(xiii) candidate admissible representative records;

(xiv) candidate tower comparison artifacts;

(xv) draft certificate records;

(xvi) proof object fragments;

(xvii) finite detector candidates with declared source stages;

(xviii) finite-to-infinite reduction candidates;

(xix) higher degree eligibility and edge candidates;

(xx) Problem-Bridge and Return-chain candidates;

(xxi) source-statement and precursor-binding candidates;

(xxii) counterexamples, no-go witnesses, and rejected-route artifacts.

Definition 2.16 (Search artifact)**.** A *search artifact* is a stored or referenced output of a search process. It may include text, code, diagrams, rankings, maps, datasets, proof sketches, logs, generated candidates, or structured records.

Definition 2.17 (Candidate)**.** A *candidate* is a search object that has been selected for possible further development, mapping, lifting, verification, formalization, or audit.

Definition 2.18 (Candidate status)**.** A candidate has one of the following statuses:

(i) *raw*;

(ii) *ranked*;

(iii) *mapped*;

(iv) *lift-attempted*;

(v) *lifted*;

(vi) *verification-ready*;

(vii) *blocked*;

(viii) *rejected*;

(ix) *incomplete*;

(x) *deprecated*;

(xi) *superseded*.

Proposition 2.19 (Search artifacts are non-verdict by default)**.** *A search artifact does not affect Core pass/reject unless it is converted into verifier-side data and checked under the Core protocol.*

Proof. Search artifacts are proposer-side data. By Appendix U, proposer-side data do not determine Core verdicts. Only verifier-side data determine Core pass/reject. □

Remark 2.20 (Search artifact value)**.** Search artifacts may still be highly valuable. They support discovery, comparison, reproducibility, prioritization, debugging, and proof planning. Their limitation is logical, not operational.

V.3. Hunter protocol

Definition 2.21 (Hunter)**.** A *Hunter* is an agent role whose task is to find candidate objects, regions, lemmas, bridges, failure modes, proof routes, or artifact references in a declared search space.

Definition 2.22 (Hunt space)**.** A *hunt space* is a declared space of possible search targets. Examples include:

(i) window families;

(ii) threshold bands;

(iii) tower configurations;

(iv) arithmetic interface routes;

(v) PDE observable families;

(vi) normalization choices;

(vii) bridge lemma templates;

(viii) candidate proof programs;

(ix) formalization fragments;

(x) certificate DAG fragments;

(xi) known failure-mode neighborhoods.

Definition 2.23 (Hunt objective)**.** A *hunt objective* is the declared goal of a Hunter protocol, such as finding a stable band, locating a bridge candidate, identifying a Type IV proxy, searching for a counterexample, or discovering a Core-readable realization route.

Definition 2.24 (Hunt output)**.** A *hunt output* is a list, set, graph, table, or ranked family of candidates produced by a Hunter, together with any scores, rankings, reasons, constraints, or artifact references.

Definition 2.25 (Hunter protocol)**.** A *Hunter protocol* specifies:

(i) the hunt space;

(ii) the hunt objective;

(iii) the search constraints;

(iv) the candidate selection rule;

(v) the ranking or scoring method, if any;

(vi) stopping or continuation conditions, if any;

(vii) the output status labels;

(viii) artifact references.

Proposition 2.26 (Hunter output is not certificate)**.** *A Hunter output is not a Core certificate.*

Proof. Hunter output is a candidate-generating search artifact. A Core certificate requires verifier-side data satisfying the Minimal Manifest Axiom, repaired evaluation records, gates, diagnostics, ledgers, tower artifacts when invoked, and artifact requirements. Thus Hunter output is not a certificate. □

Remark 2.27 (Hunter success)**.** Hunter success means that promising candidates were found. It does not mean that the candidates are valid.

V.3bis. Finite-certificate, counterexample, and no-go hunting

Definition 2.28 (Finite-detector hunt objective)**.** A *finite-detector hunt objective* seeks a candidate finite certificate for a declared predicate on exactly one source stage among

windowed_prethreshold, repaired_postthreshold, kernel_defect, cokernel_defect.

It records the endpoint convention, family, audit range, candidate sampling or stencil rule, proposed soundness direction, proposed completeness direction, and the source of every coverage or excess bound. The four stage tokens are a typing vocabulary, not a claim that one detector theorem is available on every stage. The standard v20 finite detector is sourced at windowed_prethreshold; any other stage requires its own stage-specific theorem or an explicit stage-compatible transport theorem. Absent that theorem, the candidate remains undefined for theorem-facing zero use.

Definition 2.29 (Detector hunt output)**.** A *detector hunt output* separates:

(i) the detector algorithm or stencil candidate;

(ii) detector certificate status;

(iii) detector mathematical value;

(iv) nonzero-soundness evidence;

(v) zero-completeness evidence;

(vi) stage applicability;

(vii) family membership and coverage evidence;

(viii) immutable computation and source artifacts.

Definition 2.30 (Counterexample and no-go hunt)**.** A *counterexample and no-go hunt* searches for a typed witness falsifying an overstrong candidate, or for a theorem showing that a requested uniform finite certificate cannot exist on the declared scope. Negative search is successful only when the counterexample or no-go theorem is itself checked; failure to find a witness is not evidence of truth.

Proposition 2.31 (Finite observation is not finite certificate)**.** *A finite list of samples, ranks, critical values, agent observations, or computed zeros is not a finite detector certificate unless the required stage, soundness, completeness, coverage, family, endpoint, excess-bound, and artifact obligations are discharged.*

Proof. Finiteness of the output does not establish that the finite data characterize the target predicate. The missing characterization obligations are exactly what turns observation into proof compression. □

Proposition 2.32 (Hunter may not mutate detector stage)**.** *A detector candidate found for one source stage may not be relabeled for another stage without a separately proved stage-transport theorem that supplies the required soundness and completeness at the target stage.*

Proof. The four stages have different source objects and predicates. Renaming a record does not transport the theorem that justified its readings. □

Proposition 2.33 (No-hit result is non-verdict)**.** *Failure of a Hunter to find a retained bar, counterexample, obstruction, bridge, or proof route is non-verdict unless a complete exhaustive-search or reduction theorem identifies the searched space with the full mathematical scope.*

Proof. A search procedure may omit candidates because of finite resources, ranking, representation, corpus, or stopping policy. Only an exhaustiveness theorem closes that gap. □

Remark 2.34 (Right-unbounded no-go boundary)**.** On a right-unbounded window with unbounded possible birth positions, a Hunter must display the stabilization, finite-birth, global-bound, or finite-to-infinite reduction theorem that defeats the v20 finite-detector no-go boundary. A finite search horizon is not such a theorem.

V.4. Mapper protocol

Definition 2.35 (Mapper)**.** A *Mapper* is an agent role whose task is to organize search objects into maps, dependency structures, diagrams, classifications, validity landscapes, certificate DAG fragments, or bridge-route graphs.

Definition 2.36 (Search map)**.** A *search map* is a structured representation of relations among search objects. It may include:

(i) dependency arrows;

(ii) obstruction labels;

(iii) candidate bridge routes;

(iv) window overlap relations;

(v) tower comparison relations;

(vi) status labels;

(vii) search priorities;

(viii) known gaps;

(ix) failure-mode labels;

(x) artifact references.

Definition 2.37 (Map edge semantics)**.** A *map edge semantics* specifies what an edge in a search map means. Allowed edge types include:

(i) dependency;

(ii) analogy;

(iii) candidate implication;

(iv) verified implication;

(v) obstruction;

(vi) refinement;

(vii) replacement;

(viii) contradiction;

(ix) artifact reference;

(x) status transition.

Definition 2.38 (Mapper output)**.** A *Mapper output* is a search map, validity map fragment, dependency diagram, certificate DAG fragment, bridge graph, obstruction map, or classification table produced by a Mapper.

Definition 2.39 (Mapper protocol)**.** A *Mapper protocol* specifies:

(i) the objects to be mapped;

(ii) the relation types;

(iii) the edge semantics;

(iv) the status labels;

(v) the artifact references;

(vi) the update policy;

(vii) the conflict-resolution policy.

Proposition 2.40 (Mapper output is navigation, not theorem)**.** *A Mapper output is a navigation artifact and does not establish theorems by itself.*

Proof. A map records relations, candidates, priorities, dependencies, or gaps. The truth of a theorem requires proof or verification of the relevant statements. A map may point to such proof objects, but it does not replace them. □

Remark 2.41 (Map edges must be typed)**.** Edges in a search map must be typed. An edge may mean dependency, analogy, conjectural implication, verified implication, blocked route, or artifact reference. Untyped edges invite claim inflation.

V.4bis. Map-edge verification discipline

Definition 2.42 (Map-edge verification status)**.** A map edge has exactly one declared verification status:

(i) *untyped*;

(ii) *proposed*;

(iii) *artifact-referenced*;

(iv) *theorem-backed*;

(v) *verified*;

(vi) *failed*;

(vii) *deprecated*;

(viii) *superseded*.

A *verified* map edge must name the exact theorem, proof object, formal fragment, artifact, or verifier-side record that establishes the edge semantics.

Definition 2.43 (Map-edge promotion certificate)**.** A *map-edge promotion certificate* is the record required to promote a map edge from proposed or artifact-referenced status to theorem-backed or verified status. It contains the edge type, source and target objects, proof or artifact reference, authority scope, dependencies, and prohibited stronger readings.

Proposition 2.44 (Candidate implication edge is not implication)**.** *A search-map edge labeled as a candidate implication does not establish the corresponding implication.*

Proof. A candidate implication edge records a proposed route in the search map. The truth of an implication requires proof or a verified bridge. Since the edge by itself supplies only map structure, it does not establish the implication. □

Proposition 2.45 (Verified edge requires evidence)**.** *A map edge may be assigned verified status only when its declared semantics are supported by verifier-side evidence or by a proved theorem at the stated strength.*

Proof. Map edges can represent dependency, analogy, refinement, replacement, contradiction, artifact reference, or implication. Each edge type has different evidentiary requirements. A verified label is therefore admissible only after the required evidence for that specific edge type is supplied. □

Remark 2.46 (Map semantics are not inherited from drawing conventions)**.** Arrow direction, graph layout, color, proximity, clustering, or visual prominence does not determine logical implication. Only the declared edge semantics and supporting evidence determine how a map edge may be consumed.

V.4ter. Logical-path, dependency, and certificate-DAG discipline

Definition 2.47 (Search path semantics)**.** A *search path* is a composable sequence of typed map edges. Its displayed path does not establish a composite implication unless every implication edge is theorem-backed, the source and target scopes match, all intermediate hypotheses are discharged, and the composition rule is valid.

Definition 2.48 (Negative and missing map edges)**.** A Mapper distinguishes:

(i) a verified contradiction or refutation edge;

(ii) an evidenced failed obligation;

(iii) a missing edge;

(iv) an unexplored edge;

(v) an edge excluded by immutable scope policy.

Only the first two are evidenced failures; missing or unexplored edges remain undefined.

Proposition 2.49 (Graph reachability is not theorem composition)**.** *Reachability in a search map, validity landscape, or candidate DAG does not establish the corresponding mathematical implication or certificate closure.*

Proof. A graph may contain analogy, dependency, artifact, status, and candidate edges in addition to proved implications. Reachability forgets these distinctions unless the path is type-checked and theorem-backed edge by edge. □

Proposition 2.50 (Pairwise map agreement is not higher coherence)**.** *Pairwise agreement of overlapping search-map nodes or edges does not establish higher Čech coherence, descent, global certificate consistency, or a common realized object.*

Proof. Pairwise compatibility does not supply triple and higher compatibility cells, descent data, or an identification of the glued object. Those are separate obligations. □

Remark 2.51 (Mapper must preserve failure-route types)**.** A Mapper may visualize failure routes but may not turn undefined, non-verdict, transient-defect, Type III, Type IV, Return-missing, and governance failures into one undifferentiated red edge. The machine-readable edge type controls over visual presentation.

V.5. Lifter protocol

Definition 2.52 (Lifter)**.** A *Lifter* is an agent role whose task is to transform a search object into a Core-readable object candidate, bridge candidate, certificate candidate, formal fragment candidate, or proof object candidate.

Definition 2.53 (Lift input)**.** A *lift input* is a candidate produced by a Hunter or organized by a Mapper and selected for possible conversion into verifier-side or verifier-adjacent data.

Definition 2.54 (Lift target)**.** A *lift target* is the declared type into which the candidate is intended to be transformed. Examples include:

$$\mathsf{FiltCh}(k), \qquad \mathsf{Pers}_k^{\mathrm{cons}}, \qquad \mathrm{Eval}_{i,W,\tau}(F), \qquad C_{\tau,W}F, \qquad \phi_{i,W,\tau},$$

a ledger entry, a bridge lemma candidate, a formal fragment, or a certificate record candidate.

Definition 2.55 (Lift output)**.** A *lift output* is one of:

(i) a Core-readable filtered object candidate;

(ii) a constructible persistence object candidate;

(iii) a repaired evaluation record candidate

$$\mathrm{Eval}_{i,W,\tau}(F);$$

(iv) an admissible representative candidate

$$C_{\tau,W}F;$$

(v) an eligible realized object candidate;

(vi) a tower comparison artifact candidate

$$\phi_{i,W,\tau};$$

(vii) a tower diagnostic candidate;

(viii) a ledger entry candidate;

(ix) a bridge lemma candidate;

(x) a certificate record candidate;

(xi) a formal fragment candidate.

Definition 2.56 (Lifter protocol)**.** A *Lifter protocol* specifies:

(i) the lift input;

(ii) the lift target;

(iii) the transformation rule;

(iv) the required declarations;

(v) constructibility requirements;

(vi) representative and eligibility requirements, if invoked;

(vii) tower artifact requirements, if Type IV diagnostics are invoked;

(viii) error or defect accounting;

(ix) status labels before and after lifting;

(x) artifact references.

Proposition 2.57 (Lifted candidate is not verified by lifting)**.** *A lifted candidate is not verified merely because it has been lifted.*

Proof. Lifting changes the type or representation of a candidate. Verification checks whether the resulting object satisfies declared verifier-side conditions. These are distinct actions by Appendix U. □

Proposition 2.58 (Core-looking lift is not Core verified)**.** *A lift output that syntactically resembles a Core object, such as*

$$\mathrm{Eval}_{i,W,\tau}(F), \qquad C_{\tau,W}F, \qquad \phi_{i,W,\tau},$$

is not Core verified until checked under the declared Core protocol.

Proof. A Core-looking object may be malformed, unsupported, nonconstructible, incorrectly parameterized, or inconsistent with artifacts. Core verification requires typed declarations, artifact support, and verifier-side checks. Therefore syntactic form alone is insufficient. □

Remark 2.59 (Lifting is the re-entry preparation step)**.** Lifting is the natural preparation step before Core re-entry. It attempts to turn search artifacts into objects that the Core verifier can actually inspect.

V.5bis. Lift preservation and defect accounting

Definition 2.60 (Lift preservation record)**.** A *lift preservation record* states which properties of the lift input are preserved by the lift output. It must declare the preserved feature, source semantics, target semantics, preservation direction, hypotheses, artifact evidence, and failure mode.

Definition 2.61 (Search effect namespace)**.** The local Search / Platform effect names are

$$\delta^{\mathrm{hunt}}, \quad \delta^{\mathrm{map}}, \quad \delta^{\mathrm{lift}}, \quad \delta^{\mathrm{score}}, \quad \delta^{\mathrm{reentry}}, \quad \Delta_{\phi,\mathrm{search}}.$$

Each occurrence is classified separately as:

(i) a metric component supported by an actual scope-matched profile-discrepancy upper bound;

(ii) a nonmetric controlled effect;

(iii) a non-scalar obligation or status field;

(iv) not invoked.

The name alone does not make an effect metric. Every consumed local name declares its relation to Appendix M and overlapping interface namespaces as exactly one of

$$\text{alias}, \quad \text{refinement}, \quad \text{aggregate}, \quad \text{disjoint}.$$

Definition 2.62 (Lift equivalence claim)**.** A *lift equivalence claim* asserts that the lifted output preserves the source object up to a declared equivalence, exact equality, interleaving, bottleneck bound, bridge implication, or problem-interface semantics. Such a claim is [Spec] by default unless supported by a lift preservation record and the required verifier-side evidence.

Proposition 2.63 (Lift is not preservation by default)**.** *A lift operation does not preserve mathematical content, Core-readability, exact comparison, metric safety, or theorem status by default.*

Proof. Lifting transforms a search-side object into a target representation or candidate record. Transformation may discard information, change parameters, alter artifacts, or introduce defects. Therefore any preservation claim requires a separate preservation record. □

Proposition 2.64 (Lift defects must be typed before budget use)**.** *An effect introduced by hunting, mapping, lifting, scoring, re-entry, or search-produced tower-artifact transport may enter a Core metric budget only when it is proved to upper-bound an actual scope-matched pre-threshold profile discrepancy, placed on the selected comparison path, related to every overlapping namespace, and scalarized by the declared Appendix-M-compatible map.*

Proof. The Core budget consumes supported metric-ledger components, not names for operational effort, uncertainty, status, or theorem obligations. Path placement and namespace relations are required to prevent omission and double counting. Unsupported or nonmetric effects remain outside the numerical aggregate. □

Remark 2.65 (Lift defects and global ledger registration)**.** The symbols in Definition 2.61 are local Search / Platform notation until the global ledger catalog registers them as aliases, refinements, aggregates, or disjoint contributions relative to Appendix M and any interface-layer defect namespace consumed by the same certificate.

V.5ter. Stage-aware, higher-edge, and problem-bridge lifting

Definition 2.66 (Stage-aware detector lift)**.** A *stage-aware detector lift* converts a detector search object into a detector-certificate candidate while preserving exactly one declared source stage. It must not infer certificate status from mathematical value or mathematical value from certificate status.

Definition 2.67 (Higher-edge lift candidate)**.** A *higher-edge lift candidate* for degree n records the exact repaired degree-n profile, admissible representative, actual profile isomorphism, genuine realization or proved admissible replacement, realization-domain membership, amplitude, fixed edge extractor E_n, the condition $E_n(0) = 0$, artifact-backed H^{-n}-identification, naturality scope, change compatibility, claim status, and artifact identity. Its default target direction is

$$\mathrm{Eval}_{n,W,\tau}(F) = 0 \Longrightarrow \mathrm{Ext}^n(A_n(F), k) = 0.$$

Definition 2.68 (Problem-Bridge lift candidate)**.** A *Problem-Bridge lift candidate* is a typed package proposing one or more arrows in

$$\begin{aligned}\text{external source} &\longrightarrow \text{purpose-faithful realization} \longrightarrow \text{Core-readable profile}\\ &\longrightarrow \text{finite Core certificate} \longrightarrow \text{reduction} \longrightarrow \text{higher edge}\\ &\longrightarrow \text{Return} \longrightarrow \text{external conclusion}.\end{aligned}$$

Every non-invoked arrow is explicitly marked; every invoked arrow requires its own theorem or certificate.

Proposition 2.69 (Higher-edge lifting is not higher-edge proof)**.** *A syntactically complete higher-edge lift candidate does not establish the degree-n discharge until every eligibility, realization, edge, naturality, and artifact obligation is verified.*

Proof. The candidate records a proposed typed package. The conclusion follows only from the verified hypotheses of the higher-edge theorem, not from the package shape. □

Proposition 2.70 (Synthetic shift does not create natural bridge)**.** *A Lifter may construct a synthetic or toy shifted object, but that construction does not by itself produce a natural PH_n-to-Ext^n bridge, purpose-faithful realization, or representative-independent interpretation.*

Proof. A formal shift can force an algebraic identity in an engineered model while omitting the external source semantics, naturality, and realization theorem required by the intended bridge. □

Proposition 2.71 (Lifted Return arrow is not Return theorem)**.** *A generated or lifted Return arrow does not license an external conclusion unless a separately proved Return theorem binds the exact audited AK-side predicate to the exact external predicate.*

Proof. The external conclusion lies outside Core sovereignty. A diagram label or generated function cannot replace the cross-layer mathematical theorem that carries the conclusion back. □

Remark 2.72 (Target-separating realization)**.** A Lifter must distinguish a realization that merely encodes the source from one that separates the target predicate without importing the target conclusion. Purpose-faithfulness is feature-relative and direction-relative; it is not global faithfulness by name.

V.6. Search lifecycle

Definition 2.73 (Search lifecycle)**.** The *search lifecycle* is the ordered process:

$$\text{hunt} \longrightarrow \text{map} \longrightarrow \text{lift} \longrightarrow \text{verify} \longrightarrow \text{audit}.$$

Definition 2.74 (Lifecycle status)**.** A search object may have one of the following lifecycle statuses:

(i) *found*;

(ii) *mapped*;

(iii) *prioritized*;

(iv) *lift-attempted*;

(v) *lifted*;

(vi) *verification-ready*;

(vii) *verified*;

(viii) *audited*;

(ix) *rejected*;

(x) *blocked*;

(xi) *incomplete*;

(xii) *deprecated*;

(xiii) *superseded*.

Definition 2.75 (Lifecycle transition)**.** A *lifecycle transition* is a status change in a search object. Every transition must record the agent, action, evidence, and artifact references supporting the transition.

Remark 2.76 (Lifecycle status and Appendix U labels)**.** The lifecycle status of Definition 2.74 is a workflow field and does not replace the agent-output status label required by Appendix U. Likewise, the candidate status of Definition 2.18 is a pre-verification specialization of the lifecycle field, not a separate proof-status algebra. Operationally, *raw* corresponds to initial *found* status, *ranked* corresponds to *prioritized*, and *mapped*, *lift-attempted*, *lifted*, *verification-ready*, *blocked*, *rejected*, *incomplete*, *deprecated*, and *superseded* retain the same workflow meanings. Every output consumed by the AK workflow still carries an Appendix U status label, and any conflict is resolved by the weaker safe interpretation.

Proposition 2.77 (Lifecycle advancement is not validity monotonicity)**.** *Advancing through the search lifecycle does not monotonically increase mathematical validity.*

Proof. A candidate may be found, mapped, and lifted, yet fail verification. Lifecycle stages record workflow progress, not mathematical truth. Validity is established only at the appropriate verification stage. □

Remark 2.78 (Why lifecycle status matters)**.** Lifecycle status helps manage large search spaces. It prevents premature conversion of found or mapped objects into verified claims.

V.6bis. Typed failure routing and noncompensation

Definition 2.79 (Search failure-route record)**.** A *search failure-route record* maps an evidenced failure or missing obligation to its governing route, including as applicable

$$\text{FR-MISSING}, \quad \text{FR-DETECTOR-NONZERO},$$
$$\text{FR-DETECTOR-CERTIFICATION}, \quad \text{FR-HIGHER-EDGE},$$
$$\text{FR-T1}, \ldots, \text{FR-T5}, \quad \text{FR-TRANSIENT},$$
$$\text{FR-RETURN-MISSING}, \quad \text{FR-NONVERDICT}.$$

It records the evidence actually supporting the route and does not infer a numbered obstruction from absence alone.

Definition 2.80 (Search contamination record)**.** A *search contamination record* is opened when a target conclusion, benchmark answer, prohibited theorem conclusion, stale claim, or unverified generated statement enters the candidate-generation, mapping, lifting, scoring, or validation inputs in a way that could make the output circular.

Proposition 2.81 (Missing evidence remains undefined)**.** *Missing constructibility, detector completeness, eligibility, actual morphism, reduction, Return, source binding, permission, or artifact identity is routed to* undefined*; it is not converted by search into pass, zero,* not_invoked*, or a mathematical counterexample.*

Proof. Absence of evidence establishes neither the governing predicate nor its negation. A mathematical reject route requires evidence of the corresponding false or nonzero condition. □

Proposition 2.82 (Search success is noncompensating)**.** *A favorable Hunter score, Mapper closure, Lifter success, replay result, metric reserve, or agent consensus cannot compensate for a failed or missing non-scalar obligation.*

Proof. The favorable result and the missing obligation inhabit different typed fields. No arithmetic or order operation on a search score or metric reserve manufactures existence, theorem status, source authority, or identity evidence. □

Proposition 2.83 (Contaminated search cannot validate recovery)**.** *A Known-Theorem Recovery candidate whose realization, detector, reduction, or Return inputs consume the quarantined benchmark conclusion cannot be validated as an independent recovery by a successful search or replay.*

Proof. The output may merely reproduce information imported from the conclusion being recovered. This violates non-circularity independently of operational success. □

V.7. Search scores

Definition 2.84 (Search score)**.** A *search score* is a scalar, vector, categorical, or ordered value assigned to a search object for prioritization. Examples include:

(i) plausibility score;

(ii) structural alignment score;

(iii) advisory stability score;

(iv) bridge-likelihood score;

(v) defect-risk score;

(vi) novelty score;

(vii) proof-readiness score;

(viii) agent confidence value;

(ix) re-entry-readiness score;

(x) failure-risk score.

Definition 2.85 (Score semantics)**.** A *score semantics* specifies what a search score measures, how it is computed, and what it does not measure.

Definition 2.86 (Proof-readiness score)**.** A *proof-readiness score* estimates whether a candidate appears ready for formalization, verifier-side checking, or proof-object assembly. It does not establish proof.

Proposition 2.87 (Search score is not proof)**.** *A search score is not proof and does not determine Core pass/reject.*

Proof. A search score is proposer-side prioritization data. By Appendix U, proposer-side data do not determine Core verdicts. □

Remark 2.88 (Score discipline)**.** Scores may be useful for selecting what to inspect. They should not be used to accept or reject mathematical claims without verification.

V.7bis. Score calibration and consensus discipline

Definition 2.89 (Score calibration record)**.** A *score calibration record* declares the input scope, score generator, normalization, codomain, interpretation, prohibited stronger readings, and any theorem or verified computation connecting the score to a verifier-side quantity.

Definition 2.90 (Score-to-budget conversion)**.** A *score-to-budget conversion* is a certified conversion from a search score to a metric-ledger component. It must supply actual pre-threshold profiles, a bottleneck discrepancy upper bound, all crop and window-edge contributions, a declared scalarization, and the strict threshold-gap inequality on the protected family.

Definition 2.91 (Agent consensus)**.** *Agent consensus* is agreement among multiple human, AI, tool, platform, or hybrid agents about a candidate, ranking, map edge, lift output, bridge draft, or proof route.

Proposition 2.92 (Score is not budget without conversion)**.** *A search score, even a zero risk score or high proof-readiness score, is not a metric-ledger component unless a certified score-to-budget conversion is supplied.*

Proof. Scores and metric-ledger components live in different typed spaces. The Core budget consumes scalarized ledger components that upper-bound actual bottleneck discrepancies. A score has no such authority unless converted by a certified record. □

Proposition 2.93 (Consensus is not verification)**.** *Agent consensus does not establish proof, Core pass, bridge-theorem status, or external-domain validity.*

Proof. Consensus records agreement among agents. Verification checks declared mathematical or verifier-side conditions. Agreement may guide prioritization, but it is not the checked evidence required for validity. □

Remark 2.94 (Consensus may guide search ordering)**.** Consensus, ensemble ranking, or repeated agent agreement may be operationally useful for prioritization. It remains proposer-side unless converted into a verified record of the appropriate type.

V.8. Search-to-Core re-entry

Definition 2.95 (Search-to-Core re-entry)**.** *Search-to-Core re-entry* is the process by which a lifted candidate is submitted to the Core verifier. It requires:

(i) declared objects;

(ii) declared windows W;

(iii) threshold and collapse policies;

(iv) repaired evaluation records
$$\mathrm{Eval}_{i,W,\tau}(F) = T_\tau^{\mathrm{bar}}\big(\mathsf{W}_W \mathbf{P}_i(F)\big);$$

(v) admissible representative records
$$C_{\tau,W} F$$
when invoked;

(vi) eligibility and edge records when a degree-n Ext^n-clause is invoked;

(vii) declared tower comparison artifacts
$$\phi_{i,W,\tau}$$
when Type IV diagnostics are invoked;

(viii) monitored diagnostics;

(ix) gate verdicts;

(x) ledger policy and budget comparison;

(xi) artifact references;

(xii) proof object or certificate record schema;

(xiii) status labels for all imported search artifacts.

Definition 2.96 (Re-entry candidate)**.** A *re-entry candidate* is a lifted object prepared for Core verification but not yet verified.

Definition 2.97 (Re-entry package)**.** A *re-entry package* is the collection of artifacts, declarations, labels, records, and candidate verifier-side data submitted together for Core verification.

Definition 2.98 (Re-entry failure)**.** A *re-entry failure* occurs when a lifted candidate cannot satisfy the required manifest, typing, constructibility, repaired evaluation, representative, eligibility, diagnostic, tower artifact, ledger, or gate preconditions needed for Core verification.

Proposition 2.99 (Re-entry candidate is not Core certificate)**.** *A re-entry candidate is not a Core certificate until it passes the required verifier-side checks.*

Proof. A re-entry candidate is prepared for verification. A Core certificate has passed verification according to the declared Core conditions. Preparation is not verification. □

Remark 2.100 (Re-entry is the semantic boundary)**.** Before re-entry, data are search-side. After successful verification, data may become Core-certified. This boundary must remain explicit.

V.8bis. Re-entry modes and scalarized search budget

Definition 2.101 (Re-entry verification mode)**.** A re-entry package has exactly one verification mode:

(i) *not-submitted*;

(ii) *candidate*;

(iii) *manifest-complete*;

(iv) *checked*;

(v) *Core verified*;

(vi) *failed*;

(vii) *undefined*.

The mode *manifest-complete* is not proof; it only states that required fields are present for checking.

Definition 2.102 (Search-adjusted metric budget)**.** Let $(\mathcal{V}, \oplus, 0_{\mathcal{V}}, \preceq)$ be the declared Appendix-M-compatible commutative quantale. Let P be one selected admissible verifier-side comparison route, represented by a finite necessary target sub-DAG containing every branch actually consumed by the target certificate, and let S_P be exactly the supported primitive metric discrepancies consumed on that route among the base, hunt, map, lift, score-conversion, re-entry, and tower-artifact transport namespaces. Because S_P is finite, aggregation consumes only the finite ordered commutative-monoid reduct of the quantale. After alias/refinement/aggregate/disjoint resolution, define

$$\mathfrak{d}_{\text{search}}(P) := \bigoplus_{e \in S_P} \delta_e.$$

The scalarization $\mathrm{val}_B : \mathcal{V} \to [0, \infty]$ is monotone, normalized by $\mathrm{val}_B(0_{\mathcal{V}}) = 0$, and subadditive. Unused alternative routes contribute no cost; every required branch of the selected route is included; the same primitive discrepancy is counted at most once. Nonmetric effects and non-scalar obligations are excluded from S_P and retain separate statuses. If another value system is used locally, Core consumption additionally requires a certified unit-preserving conversion into the declared Core quantale and a target-side profile-bound support record.

Definition 2.103 (Search ledger relation record)**.** A *search ledger relation record* binds every consumed local effect name to its canonical Appendix M slot, source and target profiles, artifact-backed upper bound, selected route, aggregation law, and relation to all overlapping names. A missing relation record makes the numerical consumption undefined.

Proposition 2.104 (Search-adjusted evidence must pass scalarized budget)**.** *A Core certificate consuming search-induced metric discrepancies may pass only when, for the selected admissible route P, the declared scalarization supplies a protected pre-threshold profile family $\mathfrak{B}_{\mathrm{search}}(P)$ such that*

$$\mathrm{val}_B\big(\mathfrak{d}_{\mathrm{search}}(P)\big) < \mathrm{gap}_\tau\big(\mathfrak{B}_{\mathrm{search}}(P)\big),$$

and the scalarized value is proved to upper-bound the actual composed bottleneck discrepancy consumed by the certificate. Every inference crossing hard deletion additionally satisfies the governing two-sided threshold-gap theorem; exact zero transport uses $2\varepsilon \le \tau$, with $2\varepsilon < \tau$ as the operational reserve.

Proof. The threshold-gap theorem applies to actual compared pre-threshold profiles and an actual discrepancy bound. A ledger expression is admissible only when its selected-route aggregate has that support and contains no duplicate primitive contribution. □

Proposition 2.105 (No double counting of search effects)**.** *The same primitive discrepancy may not be counted under multiple hunt, map, lift, bridge, interface, tower, storage, or execution names in one selected certificate route.*

Proof. Duplicating a primitive discrepancy changes the ledger value without changing the actual selected comparison route. The result no longer represents the intended upper bound semantics. □

Proposition 2.106 (Search budget cannot repair non-scalar obligations)**.** *No amount of search-budget slack repairs missing constructibility, detector applicability or completeness, representatives, higher-degree eligibility, actual tower morphisms, terminal tameness, finite-to-infinite reduction, Return, source authority, claim permission, manifest consistency, or artifact identity.*

Proof. These requirements are not metric discrepancies and do not inhabit the scalarized bottleneck value system. □

Remark 2.107 (Re-entry mode and Core status)**.** The only re-entry mode that may support Core consumption is *Core verified*, and only at the exact strength of the verifier-side checks actually performed.

V.8ter. Finite-to-infinite, Return, and Known-Theorem Recovery search

Definition 2.108 (Finite-to-infinite search candidate)**.** A *finite-to-infinite search candidate* proposes a theorem-controlled arrow from finite certificates or a finite family of bounded-window certificates to an infinite, tail, colimit, completion, or apex predicate. It records the finite object, infinite target, transition maps, cofinality or stabilization data, limit construction, apex agreement, exact statement, and artifact identity.

Definition 2.109 (Return search candidate)**.** A *Return search candidate* proposes a direction-specific theorem from an exact audited AK-side predicate to an exact external predicate. It records the source theorem or source semantics, purpose-faithful realization, all intermediate predicates, hypotheses, prohibited converse, and authoritative source binding.

Definition 2.110 (Known-Theorem Recovery search track)**.** A *Known-Theorem Recovery search track* separates:

(i) the frozen benchmark statement and locator;

(ii) the permitted precursor package;

(iii) source-to-AK realization inputs;

(iv) finite Core certificate;

(v) finite-to-infinite reduction, if required;

(vi) Return theorem;

(vii) independent exact-strength validation;

(viii) source-conclusion quarantine and non-circularity audit.

The immutable benchmark statement and locator may specify the exact target and the consequent of the proposed Return theorem. The benchmark proof, known truth value, answer table, target constants or exceptional bounds, conclusion-derived witnesses, and equivalent oracles are unavailable to Hunter, Mapper, and Lifter actions that construct items (iii)–(vi).

Proposition 2.111 (Finite plateau is not reduction)**.** *A finite plateau, repeated barcode, prefix agreement, numerical convergence, stable score, graph path, or stored apex does not establish eventual constancy, colimit identification, completion, apex agreement, or a finite-to-infinite reduction.*

Proof. Each infinite conclusion quantifies beyond the observed finite data or requires a categorical comparison not supplied by repetition alone. □

Proposition 2.112 (Return-chain search does not license conclusion)**.** *Finding every named node of a Projection–Return chain does not license the external conclusion unless every invoked arrow is proved at the required strength and the composed scopes match.*

Proof. A list of candidate nodes and arrows is a proof program, not the theorem composition. Any missing, weaker, reversed, or scope-mismatched arrow blocks the conclusion. □

Proposition 2.113 (Benchmark agreement is validation only)**.** *Agreement between a recovered conclusion and a quarantined benchmark validates strength only after the independent derivation is complete; it cannot supply a missing realization, certificate, reduction, or Return arrow.*

Proof. Using the benchmark earlier would make the recovery circular. At the final step it compares two already established statements and does not contribute to the derivation. □

Remark 2.114 (v20 reference recovery tracks)**.** The frozen v20 examples IW-KTR-GROWTH and NS-KTR-SERRIN-L6 may be used to test search schemas, source-binding fields, quarantine rules, and replay records. Search infrastructure does not re-prove their mathematics: the Iwasawa track remains exact only relative to its registered precursor package, and the Serrin track remains the one-way fixed-exponent conditional criterion with the analytic conclusion quarantined until Return.

V.9. Hunter / Mapper / Lifter failure modes

Definition 2.115 (Hunter failure)**.** A *Hunter failure* occurs when search produces candidates that are irrelevant, unsupported, misranked, duplicate, malformed, stale, outside the declared hunt space, or inconsistent with the hunt objective.

Definition 2.116 (Mapper failure)**.** A *Mapper failure* occurs when relations, edges, dependencies, status labels, or map structures are incorrect, ambiguous, untyped, cyclic when acyclicity is required, inconsistent with artifacts, or semantically inflated.

Definition 2.117 (Lifter failure)**.** A *Lifter failure* occurs when a candidate cannot be converted into the declared target type, fails constructibility, lacks representative compatibility, lacks eligibility, lacks required tower artifacts, has unledgered defects, or lacks required artifact references.

Definition 2.118 (Re-entry failure mode)**.** A *re-entry failure mode* is a failure occurring after lifting but before Core verification, including missing manifest fields, mismatched labels, invalid artifacts, missing repaired evaluations, missing tower artifacts, or unsupported ledger entries.

Proposition 2.119 (Search failure requires status downgrade)**.** *If a Hunter, Mapper, Lifter, or re-entry failure is detected, the affected object must be downgraded to an appropriate non-verified status until repaired and rechecked.*

Proof. A search failure undermines the reliability of the affected object or its workflow status. By Appendix U, detected failure modes require status downgrade until repair and rechecking. □

Remark 2.120 (Failure mode localization)**.** The distinction among Hunter, Mapper, Lifter, and re-entry failures helps locate the repair task: search objective, relation map, Core-readability conversion, or verifier-side submission.

V.10. Search records

Definition 2.121 (Search record)**.** A *search record* is a structured record of a search process, including hunt, map, lift, verification, re-entry, and audit status.

Definition 2.122 (Hunter record)**.** A *Hunter record* contains:

(i) hunt space;

(ii) hunt objective;

(iii) search constraints;

(iv) candidates found;

(v) scores or rankings, if used;

(vi) selection policy;

(vii) rejected or blocked candidates, if tracked;

(viii) detector stage, family, coverage, and no-go boundary when finite proof search is invoked;

(ix) source corpus, query, stopping policy, negative-search scope, and artifact references.

Definition 2.123 (Mapper record)**.** A *Mapper record* contains:

(i) mapped objects;

(ii) relation types;

(iii) edge semantics;

(iv) status labels;

(v) blocked routes;

(vi) unresolved gaps;

(vii) conflict-resolution notes;

(viii) path-composition status, missing-edge status, and higher-coherence obligations;

(ix) artifact references.

Definition 2.124 (Lifter record)**.** A *Lifter record* contains:

(i) lift input;

(ii) lift target;

(iii) transformation rule;

(iv) resulting lifted object;

(v) constructibility status;

(vi) representative and eligibility status, if invoked;

(vii) tower artifact status, if invoked;

(viii) metric, nonmetric, and non-scalar effects in separate fields;

(ix) detector, higher-edge, reduction, Bridge, and Return status when invoked;

(x) re-entry readiness status;

(xi) source bindings, semantic-diff records, and artifact references.

Definition 2.125 (Re-entry record)**.** A *re-entry record* contains:

(i) the re-entry candidate;

(ii) the re-entry package;

(iii) required verifier-side data;

(iv) missing verifier-side data, if any;

(v) Core verification status and exact strength;

(vi) typed failure routes and missing obligations, if any;

(vii) source bindings, dependency closure, and artifact references.

Proposition 2.126 (Search records support reproducibility)**.** *Search records support reproducibility by recording how candidates were found, mapped, lifted, and prepared for verification.*

Proof. Reproducibility requires reconstructing the workflow. Search records record the relevant inputs, outputs, actions, statuses, and artifact references. Therefore they support reconstruction. □

Remark 2.127 (Search record is not proof record)**.** A search record explains how a candidate was obtained. It does not by itself prove the candidate.

V.11. Search protocol manifest entries

Definition 2.128 (Search protocol manifest entry)**.** A *search protocol manifest entry* records:

(i) the protocol type: Hunter, Mapper, Lifter, re-entry, or hybrid;

(ii) the agent identity or class;

(iii) the search objective;

(iv) input artifacts;

(v) output artifacts;

(vi) output status labels;

(vii) scores, rankings, or confidence values, if present;

(viii) edge semantics, if mapping is used;

(ix) lift target, if lifting is used;

(x) verifier-side target, if re-entry is attempted;

(xi) failure modes, if detected;

(xii) repair history, if any;

(xiii) Core re-entry status.

Definition 2.129 (Hybrid search protocol)**.** A *hybrid search protocol* combines Hunter, Mapper, Lifter, re-entry, verification, or audit actions in a single workflow.

Proposition 2.130 (Hybrid protocol does not collapse role distinctions)**.** *A hybrid search protocol does not erase the semantic distinction among hunting, mapping, lifting, verification, re-entry, and audit.*

Proof. A hybrid workflow may combine actions operationally. However, each action has a distinct semantic role by Appendix U. Thus the distinctions must remain visible in the manifest. □

V.11bis. Artifact identity, drift, and replay limits

Definition 2.131 (Search artifact identity record)**.** A *search artifact identity record* contains the artifact identifier, content hash or immutable version, generation environment, input artifacts, output artifacts, status label, and repair ancestry.

Definition 2.132 (Stale search artifact)**.** A search artifact is *stale* when its source data, generator, normalization, dependency, artifact identity, environment, or declared scope has changed relative to the record that produced it.

Definition 2.133 (Replay-success status)**.** *Replay-success status* means that a declared search workflow was reconstructed under the stated environment and artifacts. It does not assert mathematical truth beyond the criteria replayed.

Proposition 2.134 (Stale artifact cannot be silently consumed)**.** *A stale search artifact cannot be silently reused as current verifier-side evidence or current search guidance without repair, revalidation, or explicit demotion.*

Proof. Staleness means that the artifact identity or dependency context has changed. Consuming it as current would erase repair ancestry and may mismatch the declared certificate scope. Therefore repair, revalidation, or demotion is required. □

Proposition 2.135 (Replay success is not proof)**.** *Replay success for a Hunter / Mapper / Lifter workflow does not prove the mathematical validity of the generated candidate.*

Proof. Replay reconstructs a workflow output under declared conditions. Mathematical validity requires the relevant proof or verifier-side checks. The former supports auditability, not theorem status. □

Remark 2.136 (Artifact identity is part of search safety)**.** Search protocols often produce many similar artifacts. Content identity, dependency identity, and repair ancestry prevent accidental reuse of a nearby but nonidentical candidate.

V.11ter. Source binding, semantic replay, and search reproducibility

Definition 2.137 (Search source-binding record)**.** A *search source-binding record* identifies the authoritative source artifact, exact statement locator, source role, permitted precursor use, prohibited conclusion use, content identity, and freshness policy. A retrieved passage or model-generated paraphrase is not an authoritative binding.

Definition 2.138 (Computational and semantic search replay)**.** A *computational search replay* reconstructs the declared Hunter / Mapper / Lifter outputs from pinned inputs and environments. A *semantic search replay* additionally checks that the reconstructed objects have the same statements, scopes, stages, edge meanings, claim axes, source bindings, and permitted uses as the certified records.

Proposition 2.139 (Computational replay is not semantic replay)**.** *Successful computational replay does not establish semantic identity, source faithfulness, non-circularity, or theorem validity without the corresponding semantic checks.*

Proof. The same process can reproducibly regenerate a mistranscribed theorem, mis-typed edge, contaminated prompt, or scope error. Operational reproducibility and semantic correctness are independent properties. □

Proposition 2.140 (Search corpus inclusion is not source authority)**.** *Inclusion of a theorem statement, benchmark conclusion, citation, or claim in a retrieval corpus, prompt, memory, map, or platform page does not authorize its use as theorem evidence or precursor input.*

Proof. Authority depends on immutable source binding, status, permitted use, and claim-use preflight, none of which follows from context inclusion. □

Remark 2.141 (Model and prompt provenance)**.** A reproducible AI-assisted search record pins, as applicable, the model or service identity, system and user instructions, retrieved context identities, tool calls, decoding or sampling policy, seeds where meaningful, post-processing, and artifact outputs. These fields support replay but remain non-verdict.

V.12. Search protocol invariants

Definition 2.142 (Search protocol invariant)**.** A *search protocol invariant* is a rule that must remain true throughout the Hunter / Mapper / Lifter workflow.

Definition 2.143 (Non-verdict invariant)**.** The *non-verdict invariant* states that no search-side object determines Core pass/reject before verified Core re-entry.

Definition 2.144 (Typed-edge invariant)**.** The *typed-edge invariant* states that every edge in a search map must have declared semantics.

Definition 2.145 (Lift-target invariant)**.** The *lift-target invariant* states that every lift must declare its target type before the lifted output is used.

Definition 2.146 (Re-entry invariant)**.** The *re-entry invariant* states that no lifted candidate may be treated as Core-certified until it passes the required verifier-side checks.

Proposition 2.147 (Search protocol invariants prevent claim inflation)**.** *The non-verdict, typed-edge, lift-target, and re-entry invariants prevent search artifacts from being silently promoted into Core claims.*

Proof. The non-verdict invariant blocks search-side acceptance. The typed-edge invariant blocks ambiguous map implications. The lift-target invariant blocks untyped conversion. The re-entry invariant blocks treating prepared candidates as verified certificates. Together, these rules prevent claim inflation. □

V.12bis. Stable-band and Type IV proxy discipline

Definition 2.148 (Search stable-band candidate)**.** A *search stable-band candidate* is a threshold interval, window family, or parameter region that search suggests may support stable verifier-side behavior. It is not a Core stable band unless actual diagnostic or threshold-gap evidence is supplied throughout the declared band, or a theorem reduces the band claim to certified finite data.

Definition 2.149 (Search Type IV proxy)**.** A *search Type IV proxy* is a search-side signal suggesting possible terminal-generic kernel or cokernel behavior in a tower. It does not define μ_{Collapse} or u_{Collapse}.

Definition 2.150 (Search-produced Type IV diagnostic candidate)**.** A *search-produced Type IV diagnostic candidate* is a record proposing a declared monitored scope and a candidate tower artifact of the form

$$\phi^{\mathrm{search}}_{i,W,\tau} : \mathrm{ColimProfile}_{i,W,\tau}(F_\bullet) \longrightarrow \mathrm{ApexProfile}_{i,W,\tau}(F_\infty).$$

It is not a Type IV diagnostic until the actual artifact, terminal-tameness evidence, and tgdim_W kernel/-cokernel computations are verified.

Proposition 2.151 (Stable-band candidate is not Core stable band)**.** *A search stable-band candidate is not a Core stable band by sampling, ranking, visual continuity, or agent agreement alone.*

Proof. A Core stable band is a verifier-side statement over the declared band. Search sampling or ranking supplies only candidate evidence. Universal diagnostic or threshold-gap evidence, or a theorem reducing the universal claim to finite data, is required. □

Proposition 2.152 (Search Type IV proxy is not diagnosis)**.** *A search Type IV proxy does not determine*

$$(\mu_{\mathrm{Collapse}},\ u_{\mathrm{Collapse}}) = (0, 0)$$

or its negation.

Proof. The Type IV diagnostic is computed from the kernel and cokernel of a declared tower comparison artifact on a terminal-tame monitored scope. A proxy supplies neither the artifact nor the terminal-generic computation by default. □

Remark 2.153 (Do not encode missing Type IV as zero)**.** If a search-produced tower artifact is missing, ill-typed, stale, or not terminal-tame, the corresponding Type IV status is undefined or not invoked, not $(0, 0)$.

V.12ter. Type IV, transient-defect, and finite-scope separation

Definition 2.154 (Search transient-defect candidate)**.** A *search transient-defect candidate* proposes a detector on a full kernel or cokernel defect profile at source stage kernel_defect or cokernel_defect. It is separate from the terminal-generic Type IV pair and carries its own complete detector obligations.

Definition 2.155 (Search Type IV scope record)**.** A *search Type IV scope record* fixes one window W, one threshold τ, a finite monitored degree set, an actual comparison-morphism candidate, semantic authority, terminal-tameness obligations, and one status among

$$\text{not\_invoked},\quad \text{undefined},\quad \text{certified\_zero},\quad \text{obstructed}.$$

No numerical pair is attached to the first two statuses. After the actual morphism and terminal-tameness obligations pass, define

$$\mu_{i,W,\tau} := \mathrm{tgdim}_W(\ker \phi_{i,W,\tau}), \qquad u_{i,W,\tau} := \mathrm{tgdim}_W(\mathrm{coker}\, \phi_{i,W,\tau}).$$

For the declared finite monitored degree set I_{mon} at that same (W, τ),

$$\mu_{\mathrm{Collapse}} := \sum_{i \in I_{\mathrm{mon}}} \mu_{i,W,\tau}, \qquad u_{\mathrm{Collapse}} := \sum_{i \in I_{\mathrm{mon}}} u_{i,W,\tau}.$$

No aggregate is formed across different windows or thresholds.

Proposition 2.156 (Terminal search zero is not full defect zero)**.** *Even after a valid terminal Type IV zero verdict, a search protocol may not infer certified absence of finite interior defects, long transients, full kernel or cokernel profile vanishing, comparison-morphism isomorphism, or external information preservation.*

Proof. The terminal-generic invariant measures a restricted asymptotic feature on the fixed monitored scope. The listed stronger conclusions require different profiles and additional completeness or algebraic theorems. □

Proposition 2.157 (Object similarity does not create Type IV morphism)**.** *Equal or close stored barcodes, matching Betti numbers, visual agreement, object isomorphism, or a bottleneck matching found by search does not construct the actual persistence morphism required for Type IV diagnostics.*

Proof. Type IV kernel and cokernel profiles are defined from an actual comparison morphism. Object-level similarity contains no canonical morphism by itself. □

V.13. Summary of Hunter / Mapper / Lifter protocols

Theorem 2.158 (Appendix V search-protocol package)**.** *Within the Search / Platform layer, the following package is available.*

(i) Search objects and search artifacts are non-verdict by default.

(ii) Hunters generate candidates from declared hunt spaces.

(iii) Hunter outputs are not Core certificates.

(iv) Mappers organize candidates into maps, diagrams, dependencies, validity landscapes, or classifications.

(v) Mapper outputs are navigation artifacts, not theorems.

(vi) Map edges must have declared semantics.

(vii) Lifters attempt to transform candidates into Core-readable objects, bridge candidates, certificate candidates, formal fragments, or proof-object candidates.

(viii) Lift outputs may include candidates for

$$\mathrm{Eval}_{i,W,\tau}, \qquad C_{\tau,W}F, \qquad \phi_{i,W,\tau},$$

but these are not verified merely by lifting.

(ix) Core-looking lift outputs are not Core verified until checked.

(x) Search lifecycle status records workflow progress, not mathematical validity.

(xi) Search scores are not proof.

(xii) Search-to-Core re-entry requires verifier-side manifest data.

(xiii) Re-entry candidates are not Core certificates until verified.

(xiv) Hunter, Mapper, Lifter, and re-entry failures require status downgrade.

(xv) Search records support reproducibility but do not prove validity.

(xvi) Hybrid search protocols must preserve semantic role distinctions.

(xvii) Search protocol invariants prevent claim inflation.

Proof. Item (i) is Proposition 2.19. Items (ii)–(iii) follow from the Hunter definitions and Proposition 2.26. Items (iv)–(vi) follow from the Mapper definitions, Proposition 2.40, and the typed-edge discipline. Items (vii)–(ix) follow from the Lifter definitions and Propositions 2.57 and 2.58. Item (x) is Proposition 2.77. Item (xi) is Proposition 2.87. Items (xii)–$(xiii)$ follow from the Search-to-Core re-entry definitions and Proposition 2.99. Item (xiv) is Proposition 2.119. Item (xv) follows from Proposition 2.126 and Remark 2.127. Item (xvi) is Proposition 2.130. Item $(xvii)$ is Proposition 2.147. □

V.13bis. v20 compatibility hardening package

Theorem 2.159 (Appendix V v20 compatibility hardening package)**.** *Within the Search / Platform layer, the inherited repair package remains available at v20.0.0 strength with the following constraints.*

(i) Search sovereignty prevents search-side records from overriding verifier-side Core requirements.

(ii) Search-to-verifier conversion requires a declared conversion record.

(iii) Map-edge verification status must be typed and evidence-backed.

(iv) Candidate implication edges do not establish implications.

(v) Lifting does not preserve mathematical content by default.

(vi) Search-induced defects must be typed before budget use.

(vii) Scores are not metric-ledger components without certified conversion.

(viii) Agent consensus is not verification.

(ix) Search-adjusted metric evidence must satisfy a scalarized threshold-gap budget.

(x) Search budget slack does not repair non-scalar obligations.

(xi) Stale artifacts cannot be silently consumed.

(xii) Replay success is not proof.

(xiii) Stable-band candidates are not Core stable bands.

(xiv) Search Type IV proxies are not Type IV diagnostics.

(xv) Missing or undefined Type IV data are not encoded as a zero diagnostic.

Proof. Items (i)–(ii) follow from Definitions 2.5 and 2.6, together with Propositions 2.8 and 2.9. Items (iii)–(iv) follow from Definitions 2.42 and 2.43, and Propositions 2.44 and 2.45. Items (v)–(vi) follow from Definitions 2.60, 2.61, and 2.62, and Propositions 2.63 and 2.64. Items (vii)–$(viii)$ follow from Definitions 2.89, 2.90, and 2.91, and Propositions 2.92 and 2.93. Items (ix)–(x) are Propositions 2.104 and 2.106. Items (xi)–(xii) are Propositions 2.134 and 2.135. Items $(xiii)$–(xv) follow from Definitions 2.148, 2.149, and 2.150, Propositions 2.151 and 2.152, and Remark 2.153. □

V.13ter. v20 finite-proof and Projection–Return search package

Theorem 2.160 (Appendix V v20 search extension package)**.** *Within the Search / Platform layer, the following v20 extension is available.*

(i) *Search objectives and outputs carry independent claim, conformance, and workflow axes.*

(ii) *Missing or ambiguous theorem-facing evidence is consumed at the weaker safe status* undefined.

(iii) *Finite-detector hunting fixes one source stage and separates certificate status from mathematical value.*

(iv) *Finite observation, zero-looking output, and failure to find a counterexample are not finite proofs.*

(v) *Mapper reachability is not theorem composition, and pairwise agreement is not higher coherence or descent.*

(vi) *Stage-aware lifting does not mutate detector stages.*

(vii) *Degree-n higher-edge candidates require the complete standard eligibility and edge package; the default direction is one-way.*

(viii) *Synthetic shifts, generated Return arrows, and target-shaped encodings do not create natural Problem Bridges.*

(ix) *Search failures follow typed routes; missing evidence is not converted into a mathematical obstruction.*

(x) *Only actual supported profile discrepancies on a selected path enter the Appendix-M-compatible quantale ledger.*

(xi) *Alternative routes are not cumulatively charged, and primitive discrepancies are not double counted.*

(xii) *Finite plateaus, stored apexes, and numerical convergence do not establish finite-to-infinite reduction or apex agreement.*

(xiii) *Known-Theorem Recovery keeps benchmark conclusions quarantined from realization, certification, reduction, and Return construction.*

(xiv) *Computational replay is distinct from semantic replay and source-authority verification.*

(xv) *Terminal Type IV, transient defect, full-interior defect, full defect zero, and isomorphism remain distinct.*

(xvi) *Part V local labels remain noncanonical until the final CR-v20 extraction after Part V and the Companion are frozen.*

Proof. Items (i)–(ii) follow from Definitions 2.11 and 2.12 and Propositions 2.13 and 2.14. Items (iii)–(iv) follow from Definitions 2.28, 2.29, and 2.30, and Propositions 2.31, 2.32, and 2.33. Item (v) follows from Propositions 2.49 and 2.50. Items (vi)–(viii) follow from Definitions 2.66, 2.67, and 2.68, and Propositions 2.69, 2.70, and 2.71. Item (ix) follows from Definition 2.79 and Propositions 2.81 and 2.82. Items (x)–(xi) follow from Definitions 2.61, 2.102, and 2.103, and Propositions 2.104 and 2.105. Items (xii)–(xiii) follow from Definitions 2.108, 2.109, and 2.110, and Propositions 2.111, 2.112, 2.113, and 2.83. Item (xiv) follows from Definitions 2.137 and 2.138 and Propositions 2.139 and 2.140. Item (xv) follows from Definitions 2.154 and 2.155 and Propositions 2.156 and 2.157. Item (xvi) is Remark 2.4. □

V.14. Explicit exclusions

The following are not asserted in Appendix V.

- No Hunter output is treated as proof.
- No Mapper output is treated as theorem.
- No Lifter output is treated as verified merely by lifting.
- No search score is treated as validity.
- No agent confidence value is treated as a Core verdict.
- No search artifact updates Core pass/reject.
- No search map edge is treated as implication unless typed and verified.
- No candidate stable band is treated as Core stable band without diagnostics.
- No candidate bridge is treated as bridge theorem.
- No candidate repaired evaluation record

  $$\mathrm{Eval}_{i,W,\tau}(F)$$

  is treated as verified merely by being produced.
- No candidate admissible representative

  $$C_{\tau,W}F$$

  is treated as verified merely by being produced.
- No candidate tower artifact

  $$\phi_{i,W,\tau}$$

  is treated as verified merely by being produced.
- No re-entry candidate is treated as Core certificate before verification.
- No search record is treated as proof record.
- No hybrid search workflow collapses proposer and verifier roles.
- No search protocol replaces B-Gate^{+}.
- No search protocol replaces the repaired topological condition

  $$\mathrm{Eval}_{n,W,\tau}(F) = 0.$$

- No search protocol replaces admissible representative records.
- No search protocol replaces monitored Type IV diagnostics.
- No search protocol replaces the tower comparison artifact

  $$\phi_{i,W,\tau}.$$

- No search protocol replaces the budget condition

  $$\mathrm{val}_B(\mathfrak{d}_{\mathrm{met}}) < \mathrm{gap}_\tau(\mathfrak{B}).$$

- No search-to-verifier conversion is achieved merely by relabeling.
- No candidate implication edge is treated as a proved implication.
- No map visualization, color, layout, or clustering is treated as logical evidence.
- No lift is treated as preservation without a lift preservation record.
- No search score is treated as a metric-ledger component without certified score-to-budget conversion.
- No agent consensus is treated as verification.
- No search-adjusted metric comparison bypasses

$$\mathrm{val}_B(\mathfrak{d}_{\mathrm{search}}) < \mathrm{gap}_\tau(\mathfrak{B}_{\mathrm{search}}).$$

- No search-budget slack repairs missing non-scalar obligations.
- No stale search artifact is silently reused as current evidence.
- No replay success is treated as proof.
- No search stable-band candidate is treated as a Core stable band.
- No search Type IV proxy is treated as a Type IV diagnostic.
- No missing, not-invoked, or undefined Type IV diagnostic is encoded as $(0, 0)$.
- No finite sample, zero-looking detector output, or finite search horizon is treated as a complete detector certificate.
- No detector candidate is moved among windowed_prethreshold, repaired_postthreshold, kernel_defect, and cokernel_defect without a proved stage-specific theorem.
- No failed search for a counterexample or retained bar is treated as an exhaustive zero theorem.
- No graph path, DAG reachability, or pairwise overlap agreement is treated as theorem composition, higher coherence, or descent.
- No degree-n higher-edge candidate is accepted without the complete eligibility, realization, extractor, H^{-n}-identification, and naturality package.
- No synthetic shift or target-shaped realization is treated as a natural PH_n-to-Ext^n bridge.
- No generated Return arrow or complete-looking Projection–Return diagram licenses an external conclusion without proved arrows.
- No finite plateau, repeated profile, numerical convergence, or stored apex establishes eventual constancy, colimit identification, completion, apex agreement, or finite-to-infinite reduction.
- No benchmark conclusion is admitted into a Known-Theorem Recovery realization, detector, reduction, or Return construction.
- No operational or computational replay is treated as semantic identity, source authority, noncircularity, or mathematical proof.
- No local search-effect name enters the metric ledger without actual profile-discrepancy support and an Appendix-M relation record.

- No alternative routes are cumulatively charged and no primitive discrepancy is counted twice under multiple Search, Bridge, Interface, Platform, or Execution names.
- No terminal Type IV zero is treated as transient-defect absence, full-interior defect absence, full defect zero, isomorphism, or external information preservation.
- No provisional Claim UID, local promotion receipt, or final-CR pass is generated during Appendix V construction.

V.15. Provenance and status

The material in this appendix depends on:

(i) the claim taxonomy and Minimal Manifest discipline of Chapter 1;

(ii) the Core sovereignty doctrine of Chapter 7;

(iii) the non-claim discipline, Core/interface boundary, and interface protocol of Chapter 8;

(iv) the Agent Semantics of Appendix U;

(v) the baseline claim-use preflight and repair-governance discipline of Appendix CR-v20;

(vi) the window-first metric and repaired-zero discipline of Appendix A;

(vii) the representative and stage-transport discipline of Appendix B;

(viii) the standard degree-n eligible realization regime of Appendix C;

(ix) the actual-morphism Type IV, terminal, transient-defect, and tower finite-to-infinite discipline of Appendix D;

(x) the coverage, atlas, overlap, Restart, higher-coherence, and local-to-global discipline of Appendix E, when invoked;

(xi) the atomic-status, typed-failure, and non-compensation discipline of Appendix F;

(xii) the canonical manifest and verifier discipline of Appendix G;

(xiii) the advisory and Technical Extension disciplines of Appendices H–N;

(xiv) the quantale ledger, selected-route, and no-double-counting discipline of Appendix M;

(xv) the frozen Problem Interface and Known-Theorem Recovery records of Part IV, only at their reviewed local strengths.

Its role is to define how search processes may discover, organize, and prepare candidates without converting search into proof.

Status labels for Appendix V. *Search / Platform definitions:* Definitions 2.15, 2.16, 2.17, 2.18, 2.21, 2.22, 2.23, 2.24, 2.25, 2.35, 2.36, 2.37, 2.38, 2.39, 2.52, 2.53, 2.54, 2.55, 2.56, 2.73, 2.74, 2.75, 2.84, 2.85, 2.86, 2.95, 2.96, 2.97, 2.98, 2.115, 2.116, 2.117, 2.118, 2.121, 2.122, 2.123, 2.124, 2.125, 2.128, 2.129, 2.142, 2.143, 2.144, 2.145, 2.146.
Search / Platform propositions/theorems: Propositions 2.19, 2.26, 2.40, 2.57, 2.58, 2.77, 2.87, 2.99, 2.119, 2.126, 2.130, 2.147, Theorem 2.158.
Operational cautions: Remarks 2.1, 2.2, 2.3, 2.20, 2.27, 2.41, 2.59, 2.78, 2.88, 2.100, 2.120, 2.127.
Inherited compatibility definitions: Definitions 2.5, 2.6, 2.7, 2.42, 2.43, 2.60, 2.61, 2.62, 2.89, 2.90, 2.91, 2.101, 2.102, 2.131, 2.132, 2.133, 2.148, 2.149, 2.150.

Inherited compatibility propositions/theorems: Propositions 2.8, 2.9, 2.44, 2.45, 2.63, 2.64, 2.92, 2.93, 2.104, 2.106, 2.134, 2.135, 2.151, 2.152, Theorem 2.159.
Inherited compatibility remarks: Remarks 2.10, 2.76, 2.46, 2.65, 2.94, 2.107, 2.136, 2.153.
v20 extension definitions: Definitions 2.11, 2.12, 2.28, 2.29, 2.30, 2.47, 2.48, 2.66, 2.67, 2.68, 2.79, 2.80, 2.103, 2.108, 2.109, 2.110, 2.137, 2.138, 2.154, 2.155.
v20 extension propositions/theorems: Propositions 2.13, 2.14, 2.31, 2.32, 2.33, 2.49, 2.50, 2.69, 2.70, 2.71, 2.81, 2.82, 2.83, 2.105, 2.111, 2.112, 2.113, 2.139, 2.140, 2.156, 2.157, Theorem 2.160.
v20 extension remarks: Remarks 2.4, 2.34, 2.51, 2.72, 2.114, 2.141.
Explicit exclusions: Section V.14.

3 Appendix W: Bridge Programs

W.1. Scope, dependency position, and program status

This appendix defines Bridge Programs for AK-HPDST v20.0.0. It belongs to the Search / Platform Appendices and does not enlarge the Core mathematical package of Chapters 1–8 or the Foundation Appendices. Its role is to decompose proposed cross-layer implications into typed, auditable obligations without converting a roadmap, graph, generated proof plan, or successful execution into theorem evidence.

The central rule is:

A Bridge Program specifies what must be proved; it is not the proof.

Fix a monitored degree i for every invoked Type IV scope and, independently, a degree n for every invoked higher obstruction edge. No Bridge Program, bridge draft, dependency graph, formalization plan, agent consensus, generated Return arrow, benchmark match, build success, or replay result may replace any required arrow or record in the v20 chain, including:

external source semantics and immutable source binding,

purpose-faithful realization into a Core-readable object,

$B_{n,W}(F) := \mathsf{W}_W \mathbf{P}_n(F), \qquad \mathrm{Eval}_{n,W,\tau}(F) := T_\tau^{\mathrm{bar}}\big(B_{n,W}(F)\big),$

a complete stage-correct finite detector certificate when invoked,

$C_{\tau,W}F,$ the complete degree-n eligibility and edge package when invoked,

$\phi_{i,W,\tau} : \mathrm{ColimProfile}_{i,W,\tau}(F_\bullet) \longrightarrow \mathrm{ApexProfile}_{i,W,\tau}(F_\infty),$

separate terminal Type IV, transient-defect, and full-interior records,

$\mathrm{val}_B(\mathfrak{d}_{\mathrm{met}}) < \mathrm{gap}_\tau(\mathfrak{B}),$

a finite-to-infinite reduction when the target is infinite,

a separately proved Return theorem licensing the external conclusion.

Remark 3.1 (Search / Platform status)**.** Bridge Programs coordinate proof construction across Search, Core, Technical Extension, Problem Interface, formalization, and audit layers. They create no new Core theorem, Interface theorem, Bridge theorem, or external-domain theorem merely by being complete as project records.

Remark 3.2 (Relation to Appendices U and V)**.** Appendix U defines agent semantics. Appendix V defines Hunter / Mapper / Lifter protocols. The present appendix defines how candidate bridges discovered, mapped, or lifted by those protocols are organized into auditable proof programs.

Remark 3.3 (Relation to verifier-side re-entry)**.** A Bridge Program may prepare a Core re-entry package or a Projection–Return chain. Every package remains candidate data until each invoked detector, representative, higher edge, tower artifact, metric comparison, reduction, Return, claim-use, and artifact obligation has been checked at its own type.

Remark 3.4 (v20 dependency and registration boundary)**.** This appendix consumes the canonical v20 Main and Foundation artifacts, frozen Appendices U and V, and CM-v20, TB-v20, Technical Extension H–N, and Problem Interface Part IV only at their reviewed or frozen local strengths and immutable artifact identities. The current CR-v20 baseline does not yet promote Part V. Accordingly, local W-labels are stable construction identifiers rather than canonical Claim UIDs; no provisional Claim UID, local release-gate pass, or promotion receipt is fabricated before Part V and the Companion are frozen and cross-audited.

W.1bis. Bridge Program sovereignty and verifier-side re-entry

Definition 3.5 (Bridge Program sovereignty)**.** *Bridge Program sovereignty* is the rule that a Bridge Program may organize proposed cross-layer implications but may not itself establish any Core verdict, interface theorem, external-domain theorem, or bridge theorem. Only the declared verifier-side objects, proof objects, discharged hypotheses, and registered claim-use permissions determine theorem-level consumption.

Definition 3.6 (Bridge verifier-side conversion record)**.** A *bridge verifier-side conversion record* converts one declared Bridge Program output into a verifier-side candidate or theorem-facing record. It declares:

(i) the source program artifact, workflow status, document role, evidence status, invocation role, and conformance status;

(ii) the exact target record type and the source and target semantics;

(iii) the conversion theorem, proof object, stage-transport theorem, valid import, or verified artifact supporting the conversion;

(iv) every hypothesis, dependency, non-scalar obligation, and discharge status;

(v) the detector source stage and separate certificate/value fields when a detector is consumed;

(vi) the degree-n eligibility and edge package when a higher obstruction is consumed;

(vii) the actual comparison morphism, fixed monitored scope, and terminal/transient status when tower semantics are consumed;

(viii) the runtime comparison mode and every supported primitive metric discrepancy on the selected admissible proof route;

(ix) finite-to-infinite and Return records when required;

(x) immutable source, artifact, environment, and semantic-replay identities;

(xi) the atomic result pass, reject, undefined, or clause-level not_invoked under an immutable scope exclusion;

(xii) the typed failure route and prohibited stronger readings.

A program output is not converted merely because all fields of the program record are populated.

Proposition 3.7 (Bridge Program sovereignty cannot be overridden)**.** *No Bridge Program label, roadmap, dependency graph, proof-first alignment, human approval, AI assessment, search score, or platform status overrides the verifier-side requirements for a Core certificate or bridge theorem.*

Proof. Bridge Programs are proof-management records. The Core and bridge-theorem layers consume declared proof data, verifier-side artifacts, discharged hypotheses, ledgers, comparison records, and claim-governance permissions. A program label or roadmap is not one of those consumed records. Therefore it cannot override the required verifier-side conditions. □

Remark 3.8 (Program meaning is proposer-side until converted)**.** A Bridge Program may be mathematically insightful and strategically central. Nevertheless, until its outputs are converted through a bridge verifier-side conversion record, they remain proposer-side or program-side data.

W.1ter. Claim axes, weaker-safe interpretation, and governance deferral

Definition 3.9 (Bridge claim-axis record)**.** A *bridge claim-axis record* keeps distinct the document role, evidence status, invocation role, conformance status, Bridge Program workflow status, and verifier atomic status. No value on one axis determines a stronger value on another axis.

Definition 3.10 (Bridge weaker-safe rule)**.** If a bridge statement, source binding, detector stage, direction, hypothesis, comparison mode, artifact identity, reduction, Return arrow, or permitted use is ambiguous or missing, the item is consumed only at the weaker safe interpretation. Missing required evidence yields undefined; it is not silently rewritten as pass, zero, not invoked, imported, proved, or a numbered mathematical obstruction.

Proposition 3.11 (Program progress cannot collapse claim axes)**.** *A favorable workflow status, completed roadmap, closed graph, successful build, or unanimous review cannot infer theorem evidence, claim conformance, verifier pass, or external validity.*

Proof. These fields answer different questions and have different evidence requirements. Program completion supplies project-state evidence only; theorem-facing promotion requires the independently typed proof and governance records. □

Proposition 3.12 (No local final-CR promotion)**.** *Before Part V and the Companion are frozen and the final CR-v20 extraction is completed, Appendix W may record local readiness but may not declare a canonical Claim UID, release-gate pass, or final promotion receipt.*

Proof. The final register must bind the frozen cross-appendix dependency closure and immutable artifacts. A local program record cannot certify that later closure in advance. □

W.2. Bridge statements

Definition 3.13 (Bridge statement)**.** A *bridge statement* is a typed directional implication between two declared semantic layers. It records a source predicate, target predicate, source class, target class, direction, hypotheses, naturality scope, artifact regime, permitted use, prohibited converse, and failure boundary. A slogan, graph edge, analogy, or target-shaped encoding is not a bridge statement at theorem strength.

Definition 3.14 (Core-facing bridge statement)**.** A *Core-facing bridge statement* has a source or target involving exact Core-readable data, for example a windowed pre-threshold profile, repaired evaluation, complete detector certificate, admissible representative, degree-n edge package, actual tower morphism, transient-defect certificate, finite-to-infinite reduction, or scalarized metric comparison. Naming one of these objects does not construct it.

Definition 3.15 (Problem-facing bridge statement)**.** A *problem-facing bridge statement* has a source or target in an external mathematical domain, such as arithmetic, PDE, geometric, categorical, formal-interpretation, or cryptographic semantics. It must distinguish the external-to-AK realization direction from the AK-to-external Return direction.

Definition 3.16 (Bridge theorem)**.** A *bridge theorem* is a bridge statement with a complete proof at the displayed strength, immutable source and artifact bindings, discharged hypotheses, dependency closure, direction-specific semantics, comparison policy, failure routes, and successful claim-use preflight. If an external conclusion is claimed, the theorem includes or consumes a separately proved Return theorem whose antecedent is exactly the audited AK-side conclusion.

Definition 3.17 (Bridge candidate)**.** A *bridge candidate* is a bridge statement proposed for possible proof but not yet established as a bridge theorem.

Definition 3.18 (Bridge draft)**.** A *bridge draft* is an informal, partial, agent-generated, exploratory, or schematic version of a bridge candidate.

Definition 3.19 (Bridge Program)**.** A *Bridge Program* is a structured project record whose purpose is to turn one or more bridge candidates into bridge theorems or to classify why they fail.

Proposition 3.20 (Bridge candidate is not bridge theorem)**.** *A bridge candidate is not a bridge theorem.*

Proof. A bridge candidate is a proposed implication. A bridge theorem requires a proof under declared hypotheses. Proposal and proof are distinct statuses by Appendix U. □

Proposition 3.21 (Bridge draft is not bridge candidate unless specified)**.** *A bridge draft is not a bridge candidate unless its statement, source semantics, target semantics, and intended status are explicitly specified.*

Proof. A draft may be informal, incomplete, ambiguous, or schematic. A bridge candidate requires a definite statement to be evaluated, mapped, decomposed, or proved. Thus a draft must be specified before it becomes a candidate. □

Remark 3.22 (Bridge terminology caution)**.** The word "bridge" should not be read as proof. Only a proved bridge theorem can support theorem-level transport across layers.

W.2bis. Bridge statement specificity and directionality

Definition 3.23 (Bridge statement specificity record)**.** A *bridge statement specificity record* records the exact source semantics, target semantics, direction of implication, hypotheses, allowed use, forbidden stronger reading, and failure behavior of a bridge statement. A bridge statement without such a record is treated as a draft or [Spec] bridge, not as theorem evidence.

Definition 3.24 (Bridge directionality record)**.** A *bridge directionality record* declares whether the intended implication is source-to-Core, Core-to-source, technical-to-Core, formal-to-informal, or bidirectional. A bidirectional bridge requires two separately proved implications unless a single theorem explicitly proves the equivalence at the declared strength.

Proposition 3.25 (Bridge direction is not reversible by name)**.** *A proved bridge in one direction does not establish the reverse bridge unless the reverse implication is separately proved or validly imported with its own hypotheses and artifacts.*

Proof. Bridge statements connect different semantic layers. The hypotheses and constructions required for one direction need not imply those required for the reverse direction. Thus direction reversal is an additional theorem obligation, not a naming convention. □

Proposition 3.26 (External theorem is not Core theorem by import name)**.** *An external theorem, even if mathematically correct in its source domain, is not a Core theorem or Core certificate unless a bridge theorem supplies the declared realization, interpretation, artifact, ledger, and claim-governance data required for Core consumption.*

Proof. External theorems and Core certificates have different source semantics and verification schemas. Core consumption requires the Core-readable records and claim-use preflight data. A theorem name in the external domain supplies neither by default. □

Remark 3.27 (No bridge theorem from slogan form)**.** A slogan such as "Core collapse implies the problem theorem" is not a bridge statement at theorem level. It becomes a bridge candidate only after the source semantics, target semantics, direction, hypotheses, proof obligations, and failure routes are fixed.

W.2ter. Projection–Certificate–Reduction–Return chain

Definition 3.28 (Complete bridge chain)**.** A *complete bridge chain* for an external conclusion is the ordered composition

$$\begin{aligned}&\text{external source} \longrightarrow \text{purpose-faithful realization}\\ &\longrightarrow \text{Core-readable profile} \longrightarrow \text{finite Core certificate}\\ &\longrightarrow \text{finite-to-infinite reduction, if required}\\ &\longrightarrow \text{higher obstruction transport, if invoked}\\ &\longrightarrow \text{Return theorem} \longrightarrow \text{external conclusion.}\end{aligned}$$

Every arrow has its own statement, hypotheses, artifacts, direction, status, and failure route.

Definition 3.29 (Bridge arrow record)**.** A *bridge arrow record* stores the exact domain and codomain predicates, source and target types, theorem or policy status, proof artifact, comparison mode, naturality and representative scope, dependencies, prohibited reverse reading, and atomic result.

Definition 3.30 (Return theorem record)**.** A *Return theorem record* identifies the exact audited AK-side antecedent and the exact external conclusion, together with source-domain hypotheses, proof, artifact identity, direction, and failure boundary. A realization theorem is not a Return theorem and does not imply one.

Proposition 3.31 (Named nodes do not compose themselves)**.** *The presence of every named node in a complete bridge chain does not license the external conclusion unless every invoked arrow is proved and the codomain of each arrow matches the domain of the next at the consumed strength.*

Proof. A list of objects is not a composable proof. A missing, weaker, reversed, or scope-mismatched arrow breaks the implication chain. □

Proposition 3.32 (Core certification is not Return)**.** *A valid finite Core certificate, including a valid higher-Ext discharge or Type IV record, does not by itself establish an external-domain conclusion.*

Proof. The Core certificate has AK-internal semantics. External meaning is supplied only by the separately proved direction-specific Return theorem. □

W.3. Bridge Program types

Definition 3.33 (Core-to-interface Bridge Program)**.** A *Core-to-interface Bridge Program* attempts to prove a statement of the form:

$$\text{Core certificate behavior} \Longrightarrow \text{problem-specific conclusion.}$$

Definition 3.34 (Interface-to-Core Bridge Program)**.** An *Interface-to-Core Bridge Program* attempts to prove a statement of the form:

$$\text{problem-specific or external structure} \Longrightarrow \text{Core-readable behavior.}$$

Definition 3.35 (Technical-to-Core Bridge Program)**.** A *Technical-to-Core Bridge Program* attempts to prove that technical-extension data, such as discretization, measurement, controlled commutation, Mirror/Transfer comparison, derived realization, or higher-categorical structure, safely re-enter the Core verifier.

Definition 3.36 (Search-to-proof Bridge Program)**.** A *Search-to-proof Bridge Program* attempts to convert search artifacts, candidate maps, lifted objects, bridge drafts, or advisory signals into proof objects, verified formal fragments, bridge theorems, or Core certificate candidates.

Definition 3.37 (Formal-to-informal Bridge Program)**.** A *Formal-to-informal Bridge Program* attempts to prove that a verified formal fragment or proof assistant theorem corresponds to the intended informal mathematical statement.

Definition 3.38 (Problem-to-proof-first Bridge Program)**.** A *Problem-to-proof-first Bridge Program* attempts to decompose a problem-specific theorem target into proof-first records, required bridge lemmas, Core certificate targets, analytic or arithmetic hypotheses, and proof object requirements.

Remark 3.39 (Bridge Program classes may overlap)**.** A single Bridge Program may combine several types. For example, a Navier–Stokes regularity bridge may require interface-to-Core realization, technical-to-Core discretization soundness, Core-to-interface regularity transport, and formal-to-informal interpretation.

W.3bis. Recovery, reduction, and successor-program classes

Definition 3.40 (Known-Theorem Recovery Bridge Program)**.** A *Known-Theorem Recovery Bridge Program* attempts to recover one frozen external theorem at exactly its source strength through an independent realization–certificate–reduction–Return chain. It separates the benchmark statement from the permitted precursor package. The immutable benchmark statement and locator may identify the target and exact Return consequent, while the benchmark proof, known truth value, answer table, target constants, exceptional bounds, conclusion-derived witnesses, and equivalent oracles are quarantined from every construction step.

Definition 3.41 (Finite-to-infinite Bridge Program)**.** A *finite-to-infinite Bridge Program* attempts to prove an exact reduction from finite certified data to an infinite, colimit, completion, tail, or apex statement. Finite observation, numerical convergence, repeated values, and a stored apex are not reduction theorems.

Definition 3.42 (Successor-release Bridge Program)**.** A *successor-release Bridge Program* is intentionally excluded from current theorem use and retained as a typed future target. Its current invocation status is not_invoked only under an immutable scope-exclusion policy; otherwise missing evidence is undefined.

Proposition 3.43 (Successor priority is not current evidence)**.** *A high-priority successor Bridge Program cannot discharge a v20.0.0 theorem, release, or Return obligation unless it is separately proved and promoted through the applicable frozen governance process.*

Proof. Priority records future importance, not present proof status. □

W.4. Bridge Program record

Definition 3.44 (Bridge Program record)**.** A *Bridge Program record* contains at least:

(i) a stable local program identifier and immutable artifact identity;

(ii) exact bridge statement, directionality, source and target semantics;

(iii) program class, document role, evidence status, invocation role, conformance status, and workflow status;

(iv) source bindings, benchmark and precursor policy, and source-conclusion quarantine when recovery is intended;

(v) hypotheses, necessary lemmas, dependency graph, and discharge statuses;

(vi) realization, coefficient, parameter, constructibility, detector-stage, representative, higher-edge, tower, transient, reduction, and Return obligations as invoked;

(vii) selected admissible proof route and comparison modes;

(viii) primitive metric discrepancies and non-scalar obligations in separate fields;

(ix) failure routes, permitted use, prohibited stronger readings, and non-circularity conditions;

(x) responsible agents and semantic-replay environment;

(xi) audit notes, repair ancestry, and final-CR promotion state.

Definition 3.45 (Bridge Program workflow status)**.** A Bridge Program has exactly one workflow status among

proposed, specified, mapped, decomposed, partially_proved,
verification_ready, locally_verified, blocked, failed, rejected, superseded.

These tokens report workflow progress only. The former phrases "verified as bridge theorem" and "imported theorem" are theorem-facing statuses and are therefore recorded on the independent evidence and invocation axes, not in this workflow enumeration.

Definition 3.46 (Bridge Program claim status)**.** The *claim status* of a Bridge Program output is determined independently from workflow progress and is chosen from the canonical evidence statuses

proved, conditional, operational, spec, deprecated, rejected, unknown.

Only proved, or conditional with every registered hypothesis discharged, may be consumed as theorem evidence.

Proposition 3.47 (Program status is not theorem status)**.** *No Bridge Program workflow status implies theorem status. Theorem-facing use requires an independent evidence status of* proved*, or* conditional *with every registered hypothesis discharged, together with the required invocation, conformance, artifact, and claim-use records.*

Proof. Workflow status records project progress, whereas theorem status records mathematical evidence and permitted use. □

Remark 3.48 (Status conservatism)**.** Bridge Program status should be conservative. A partially proved, specified, mapped, or decomposed program must not be described as a theorem.

W.4bis. Bridge Program status axes and promotion governance

Definition 3.49 (Bridge workflow status field)**.** The workflow status of Definition 3.45 does not replace Appendix U output status, Chapter 1 document role, CR-v20 evidence and invocation status, conformance status, or verifier atomic status. Every consumed record carries the relevant axes separately, and conflicts are resolved by the weaker safe interpretation.

Definition 3.50 (Bridge promotion-readiness record)**.** A *bridge promotion-readiness record* documents that a frozen bridge theorem or valid import appears ready for later CR-v20 promotion. It contains the exact statement, proof or import, discharged hypotheses, dependency closure, selected comparison path, metric and non-scalar obligations, source and artifact identities, semantic replay, permitted use, prohibited stronger readings, and repair ancestry. It is not a final promotion certificate while final CR-v20 extraction remains deferred. The legacy local label is retained for cross-artifact stability.

Proposition 3.51 (Workflow progress does not imply claim promotion)**.** *A Bridge Program becoming specified, mapped, decomposed, or partially proved does not promote its bridge statement to theorem evidence.*

Proof. Those statuses record project progress. Theorem evidence requires a bridge promotion certificate or a registered theorem-level source with all hypotheses discharged. Project progress alone supplies neither. □

Proposition 3.52 (Promotion readiness requires claim-use preflight)**.** *A bridge promotion-readiness record is invalid unless the item passes the applicable current CR-v20 baseline preflight and records every downstream unregistered dependency explicitly; final promotion remains pending until the final register is generated.*

Proof. Preflight controls admissible use at the current registered strength. It cannot manufacture final registration of downstream artifacts that the baseline intentionally leaves pending. □

Remark 3.53 (Bridge status vocabulary is two-axis)**.** Terms such as *specified*, *mapped*, and *decomposed* are workflow-status tokens. Terms such as *proved*, *conditional*, *operational*, *spec*, and *rejected* are claim-governance tokens. The two axes are recorded together but are not interchangeable.

W.5. Bridge hypotheses

Definition 3.54 (Bridge hypothesis)**.** A *bridge hypothesis* is one typed assumption required for the exact directional bridge statement. Typical hypotheses concern source-domain validity, purpose-faithful realization, coefficient passage, constructibility, window-first evaluation, detector applicability or completeness, representative existence, degree-n eligibility and naturality, actual tower morphisms, terminal tameness, transient scope, metric transport, coverage or coherence, finite-to-infinite reduction, formal interpretation, or Return. The program records the source, scope, proof obligation, dependencies, and forbidden stronger reading of each hypothesis.

Definition 3.55 (Bridge hypothesis status)**.** Each bridge hypothesis carries the canonical evidence status

proved, conditional, operational, spec, deprecated, rejected, unknown,

and a separate discharge mode such as local proof, valid import, verified artifact, operational policy, or not discharged. A discharge mode does not strengthen the evidence status beyond the exact theorem or artifact it supplies.

Definition 3.56 (Hypothesis discharge)**.** A *hypothesis discharge* binds a hypothesis to an exact proof, valid imported theorem, verified artifact, or admissible operational policy at the required source, scope, direction, and strength. A conditional hypothesis is discharged only when every registered subhypothesis is satisfied; a specification, unknown, rejected, or deprecated item cannot be consumed as theorem evidence.

Proposition 3.57 ([Spec] hypotheses prevent theorem-level bridge)**.** *A bridge statement depending on an unresolved* [Spec] *hypothesis is not a bridge theorem.*

Proof. A theorem requires all necessary hypotheses and implications to be proved or validly imported. An unresolved [Spec] hypothesis is not proved. Therefore the bridge statement cannot be theorem-level. □

Remark 3.58 (Spec is allowed but must be labeled)**.** A Bridge Program may contain [Spec] hypotheses. This is useful for research planning. However, they block theorem-level status until discharged.

W.5bis. v20 certificate, edge, tower, reduction, and Return obligations

Definition 3.59 (Finite detector bridge obligation)**.** A *finite detector bridge obligation* fixes exactly one source stage among

$$\begin{gathered}\mathrm{windowed\_prethreshold}, \quad \mathrm{repaired\_postthreshold},\\ \mathrm{kernel\_defect}, \quad \mathrm{cokernel\_defect}\end{gathered}$$

and separately records applicability, nonzero soundness, zero completeness, endpoint convention, birth/germ or stencil coverage, family membership, excess-bound provenance, certificate status, mathematical value, and artifact identity. The standard v20 finite detector is sourced at windowed_prethreshold; any other stage requires its own stage-specific theorem or explicit stage-transport theorem.

Definition 3.60 (Higher-edge bridge obligation)**.** A *higher-edge bridge obligation* in degree n requires the exact repaired degree-n profile, admissible representative, actual profile isomorphism, genuine realization or separately proved admissible replacement, realization-domain membership, amplitude, fixed extractor E_n, proof that $E_n(0) = 0$, artifact-backed H^{-n} identification, naturality scope, change compatibility, claim status, and non-circularity. Its default direction is

$$\mathrm{Eval}_{n,W,\tau}(F) = 0 \Longrightarrow \mathrm{Ext}^n(A_n(F), k) = 0.$$

A reverse direction requires a separately proved natural zero-reflection or faithfulness theorem.

Definition 3.61 (Tower and transient bridge obligation)**.** A *tower and transient bridge obligation* separates:

(i) the actual morphism $\phi_{i,W,\tau}$ on one fixed (W, τ) and finite monitored degree set;

(ii) terminal Type IV status not_invoked, undefined, certified_zero, or obstructed;

(iii) full-interior status not_checked, certified_absent, or certified_present;

(iv) long-transient status not_invoked, undefined, certified_absent, or certified_present;

(v) separately staged kernel- or cokernel-defect detector records.

Terminal zero is not full profile zero, full-interior absence, long-transient absence, isomorphism, apex agreement, or external information preservation.

Definition 3.62 (Reduction and Return bridge obligation)**.** A *reduction and Return bridge obligation* distinguishes finite observation, finite certificate, finite prefix, numerical convergence, Cauchy behavior, eventual constancy, colimit or homotopy-colimit identification, completion, apex agreement, finite-to-infinite reduction, and Return. Each invoked transition requires a theorem and immutable artifact at the required strength.

Proposition 3.63 (Stage names do not authorize detectors)**.** *Declaring a detector source stage does not establish that a detector theorem is valid at that stage.*

Proof. The stage is a type field. Applicability, soundness, completeness, and any stage transport are separate theorem obligations. □

Proposition 3.64 (Synthetic higher edge is not natural bridge)**.** *A shift or target-shaped derived object that yields a formal algebraic identity does not by itself establish a natural* PH_n*-to-*Ext^n *bridge, a purpose-faithful realization, or representative-independent external meaning.*

Proof. Synthetic construction supplies neither the required realization theorem nor the naturality and interpretation records. □

W.6. Bridge lemmas

Definition 3.65 (Bridge lemma)**.** A *bridge lemma* is a smaller claim required to prove a bridge theorem.

Definition 3.66 (Core-facing bridge lemma)**.** A *Core-facing bridge lemma* is a bridge lemma involving Core-readable data, such as repaired evaluation records, admissible representatives, eligible realizations, tower comparison artifacts, diagnostics, gates, or ledgers.

Definition 3.67 (Problem-facing bridge lemma)**.** A *problem-facing bridge lemma* is a bridge lemma involving external semantic content, such as arithmetic, PDE, categorical, geometric, cryptographic, or formal-interpretation claims.

Definition 3.68 (Bridge lemma graph)**.** A *bridge lemma graph* is a directed graph whose nodes are bridge lemmas and whose edges represent dependency relations among them.

Definition 3.69 (Discharge condition)**.** A *discharge condition* is a criterion under which a bridge lemma is considered proved, imported, verified, rejected, or blocked.

Proposition 3.70 (All necessary bridge lemmas must be discharged)**.** *A Bridge Program cannot yield a bridge theorem unless all necessary bridge lemmas are discharged.*

Proof. A bridge theorem depends on its required lemmas. If any necessary lemma is unresolved, the proof is incomplete. Therefore all necessary bridge lemmas must be discharged. □

Remark 3.71 (Bridge lemma graph links to certificate DAG)**.** The bridge lemma graph becomes part of the global certificate DAG described in the later validity-map and certificate-DAG appendix.

W.6bis. Bridge-graph path and coherence discipline

Definition 3.72 (Bridge proof path)**.** A *bridge proof path* is a selected composable sequence of theorem-backed arrows from the declared source predicate to the declared target predicate. Every edge records exact semantics and evidence; visual reachability alone is insufficient.

Definition 3.73 (Bridge graph negative edge)**.** A *bridge graph negative edge* records a proved contradiction, failed obligation, rejected implication, prohibited converse, or scope mismatch. A missing edge records absence of evidence and therefore yields undefined, not a proved negative statement.

Proposition 3.74 (Graph reachability is not theorem composition)**.** *A directed path in a bridge lemma graph does not establish the composed bridge unless all edges are theorem-backed, directionally compatible, and scope-matched, and all required side conditions are discharged.*

Proof. Graph reachability is combinatorial. Theorem composition additionally requires semantic and hypothesis compatibility at every interface. □

Proposition 3.75 (Pairwise arrows do not supply higher coherence)**.** *Pairwise bridge comparisons do not establish higher Cech coherence, descent, homotopy coherence, or local-to-global transport unless the corresponding higher compatibility theorem is supplied.*

Proof. Higher coherence imposes compatibility on triples and higher simplices that pairwise arrows alone do not encode. □

W.7. Core-facing bridge obligations

Definition 3.76 (Core-facing bridge obligation)**.** A *Core-facing bridge obligation* is one typed arrow connecting Bridge Program data to verifier-side Core data. As invoked, it may require a constructible one-parameter realization, window-first profile, repaired evaluation, complete stage-correct detector certificate, admissible representative, degree-n eligible edge package, actual Type IV morphism, transient-defect detector, selected-route metric record, finite-to-infinite reduction, gate verdict, and canonical manifest entry. No raw external invariant or program field replaces these objects.

Definition 3.77 (Core bridge payload)**.** A *Core bridge payload* is the collection of verifier-side Core data produced or required by a Bridge Program.

Proposition 3.78 (Core-facing bridge obligations do not verify themselves)**.** *A Bridge Program listing Core-facing obligations does not verify those obligations.*

Proof. Listing an obligation records what must be proved or checked. Verification requires actual verifier-side data and the declared checking process. Therefore the obligation list is not self-verifying. □

Remark 3.79 (Core bridge payload remains candidate until checked)**.** A Core bridge payload produced by a Bridge Program remains candidate data until checked under the Core protocol.

W.7bis. Core payload typing and comparison modes

Definition 3.80 (Bridge payload stage record)**.** A *bridge payload stage record* identifies whether each consumed profile is windowed pre-threshold, repaired post-threshold, kernel defect, or cokernel defect, and prohibits relabeling one stage as another.

Definition 3.81 (Bridge comparison-mode record)**.** Every consumed bridge comparison resolves to exactly one runtime mode:

exact, metric_gap_certified, not_compared.

The mode not_compared cannot discharge an invoked comparison obligation, and exact equality is not a positive-radius robustness certificate.

Proposition 3.82 (Object similarity does not manufacture Type IV morphism)**.** *Equal or close barcodes, equal Betti numbers, common storage format, graph alignment, or agent consensus does not construct the actual persistence morphism required for Type IV diagnostics.*

Proof. Type IV kernel and cokernel are defined from a specified morphism, not from object-level similarity data. □

Proposition 3.83 (Exact comparison is not robustness)**.** *An exact bridge comparison does not supply threshold-gap-safe perturbation transport unless a separate metric discrepancy bound and two-sided threshold-gap certificate are present.*

Proof. Exact equality is a zero-radius statement; metric robustness is a neighborhood statement with additional hypotheses. □

W.8. Bridge defects and ledger integration

Definition 3.84 (Typed bridge effect)**.** A *typed bridge effect* is exactly one of:

(i) a metric discrepancy supported as an upper bound on an actual profile discrepancy;

(ii) a theorem-controlled nonmetric mismatch;

(iii) a non-scalar obligation such as realization, interpretation, eligibility, actual morphism, reduction, Return, claim permission, or artifact identity;

(iv) an advisory or workflow quantity with no verifier-side force.

Only the first class may enter the metric ledger.

Definition 3.85 (Bridge metric-ledger component)**.** A *bridge metric-ledger component* $\delta_e \in \mathcal{V}$ is admissible only when it is attached to an actual declared pair of windowed pre-threshold profiles and supported by

$$d_B\left(B_e^{\mathrm{src}}, B_e^{\mathrm{tgt}}\right) \le \mathrm{val}_B(\delta_e).$$

The component must be consumed on the selected bridge proof route at the same scope. Each local name declares its Appendix M relation as exactly one of

$$\text{alias}, \quad \text{refinement}, \quad \text{aggregate}, \quad \text{disjoint}.$$

A generic symbol δ^{bridge} is shorthand only for a declared aggregate of identified primitive components; it is not a free numerical allowance.

Definition 3.86 (Selected-route bridge metric value)**.** Let P be the selected admissible bridge proof route, represented by a finite necessary target sub-DAG containing every branch consumed by the target theorem, and let S_P be the finite set of supported primitive metric discrepancies consumed on that route. In the finite ordered commutative-monoid reduct of the declared Appendix-M-compatible commutative quantale $(\mathcal{V}, \oplus, 0_{\mathcal{V}}, \preceq)$, define

$$\mathfrak{d}_{\mathrm{bridge}}(P) := \bigoplus_{e \in S_P} \delta_e.$$

The scalarization $\mathrm{val}_B : \mathcal{V} \to [0, \infty]$ is monotone, normalized by $\mathrm{val}_B(0_{\mathcal{V}}) = 0$, and subadditive. Alternative routes not used by P are not added, while every required branch of P is included. If another value system is used locally, Core consumption additionally requires a certified unit-preserving conversion into the declared Core quantale and a target-side profile-bound support record.

Definition 3.87 (Bridge effect status)**.** Every bridge effect records its type and atomic status. A metric discrepancy may be certified, failed, or undefined; a non-scalar clause may be pass, reject, undefined, or not_invoked under an immutable exclusion. The words "zero", "bounded", and "ledgered" do not replace these typed statuses.

Proposition 3.88 (Only supported metric discrepancies enter the ledger)**.** *A bridge effect affects the metric budget only when it is a supported metric discrepancy included on the selected route. Theorem-controlled nonmetric mismatches and non-scalar obligations remain separate and cannot be hidden in a numerical component.*

Proof. The metric ledger bounds actual profile discrepancies. Existence, semantic, governance, and interpretation obligations are differently typed propositions and have no default scalar encoding. □

Proposition 3.89 (Selected-route bridge budget must pass)**.** *If the selected admissible bridge proof route consumes metric discrepancies, certification requires a protected family of the actual windowed pre-threshold profiles $\mathfrak{B}_P$ such that*

$$\mathrm{val}_B\left(\mathfrak{d}_{\mathrm{bridge}}(P)\right) < \mathrm{gap}_\tau(\mathfrak{B}_P),$$

together with evidence that the scalarized value upper-bounds every actual bottleneck discrepancy consumed on P. Any exact-zero transport additionally consumes $2\varepsilon \le \tau$, with $2\varepsilon < \tau$ as the operational reserve.

Proof. This is the v20 threshold-gap-safe metric transport rule applied to the actual selected comparison route. □

Remark 3.90 (Semantic mismatch is not numerical by default)**.** A changed semantic interpretation, weakened hypothesis, strengthened conclusion, continuum-to-discrete replacement, coefficient loss, or formal-to-informal mismatch is a nonmetric or non-scalar obligation unless a separate theorem converts the exact mismatch into an actual profile discrepancy bound. It may not be repaired by assigning an arbitrary number.

W.8bis. Bridge defect namespace, non-compensation, and Type IV artifact discipline

Definition 3.91 (Primitive bridge discrepancy)**.** A *primitive bridge discrepancy* is one supported metric comparison on the selected proof route. Local names such as

$$\delta^{\text{bridge-real}}, \quad \delta^{\text{bridge-disc}}, \quad \delta^{\text{bridge-transport}}, \quad \Delta_{\phi,\text{bridge}}$$

are admissible only when their actual profile pair, scope, theorem, scalarization, and Appendix M relation are declared. The tokens $\delta^{\text{bridge-interp}}$ and $\delta^{\text{bridge-formal}}$ are nonmetric by default and enter the metric namespace only after such a conversion theorem is proved.

Definition 3.92 (Bridge non-scalar obligation)**.** A *bridge non-scalar obligation* includes source authority, purpose-faithful realization, coefficient passage, constructibility, detector applicability and completeness, representative existence, higher-edge eligibility and naturality, actual comparison morphism, terminal tameness, transient scope, finite-to-infinite reduction, Return, claim permission, environment compatibility, semantic replay, and artifact identity.

Definition 3.93 (Bridge-produced Type IV candidate)**.** A *bridge-produced Type IV candidate* proposes an actual morphism

$$\phi^{\text{bridge}}_{i,W,\tau} : \text{ColimProfile}_{i,W,\tau}(F_\bullet) \longrightarrow \text{ApexProfile}_{i,W,\tau}(F_\infty)$$

on one fixed (W, τ) and finite monitored degree set. After source, target, kernel, and cokernel terminal tameness is certified, the degreewise terminal diagnostics are

$$\mu_{i,W,\tau} := \text{tgdim}_W(\ker \phi^{\text{bridge}}_{i,W,\tau}), \qquad u_{i,W,\tau} := \text{tgdim}_W(\text{coker}\, \phi^{\text{bridge}}_{i,W,\tau}).$$

Any aggregate $(\mu_{\text{Collapse}},\ u_{\text{Collapse}})$ is formed only over the declared finite monitored degree set at that same (W, τ). The record carries separate transient and full-interior fields.

Proposition 3.94 (Bridge numerical slack does not repair non-scalar obligations)**.** *No strict metric reserve, however large, discharges a missing or failed bridge non-scalar obligation.*

Proof. The obligations have different logical types and are not summands of the metric quantale. □

Proposition 3.95 (Bridge Type IV candidate is not terminal or full-defect zero)**.** *A proposed comparison, stable-looking tower, equal barcode, or terminal-zero candidate does not establish certified Type IV zero without the actual morphism and terminal-tame kernel/cokernel computation; even certified terminal zero does not establish transient absence, full-interior absence, full defect zero, isomorphism, apex agreement, or external information preservation.*

Proof. The listed conclusions consume distinct objects and theorems. □

Remark 3.96 (Bridge ledger registration and no double counting)**.** Every local bridge metric name remains noncanonical until its alias, refinement, aggregate, or disjoint relation to the gate-scoped ledger is declared. The same primitive discrepancy may not be charged again as search, lift, realization, bridge, storage, replay, or execution cost, and alternative paths are not cumulatively summed.

W.8ter. Selected-path and no-double-counting calculus

Definition 3.97 (Bridge ledger relation record)**.** A *bridge ledger relation record* binds each local metric component to its primitive discrepancy identity, selected route, source and target profiles, and one relation token: alias, refinement, aggregate, or disjoint.

Proposition 3.98 (No bridge double counting)**.** *A primitive discrepancy may contribute at most once to* $\mathfrak{d}_{\mathrm{bridge}}(P)$*, and costs belonging to unused alternative proof routes are excluded.*

Proof. The ledger represents the composed discrepancy of one proof route, not the inventory of all candidate routes or all local names for the same error. □

Proposition 3.99 (Alternative route costs are not sequential)**.** *Costs from mutually alternative bridge routes may be added only when a theorem-facing proof actually composes those routes sequentially on the same route.*

Proof. An alternative route is not an additional perturbation of the selected object unless it is executed and consumed in sequence. □

W.9. Bridge Program and Core sovereignty

Theorem 3.100 (Bridge Programs do not update Core verdicts)**.** *A Bridge Program does not update Core pass/reject unless it produces verifier-side data that are checked by the Core verifier.*

Proof. A Bridge Program is a search and proof-management object. By Core sovereignty, Core verdicts are determined only by verifier-side gates, diagnostics, repaired evaluation records, representative and eligibility records when invoked, tower artifacts when invoked, ledgers, and manifest data. Thus the program itself does not update Core verdicts. □

Remark 3.101 (Bridge Program as coordination layer)**.** A Bridge Program coordinates proof work. It does not bypass proof work.

W.10. Bridge Program failure modes

Definition 3.102 (Bridge Program failure)**.** A *Bridge Program failure* occurs when a Bridge Program cannot establish its intended bridge theorem under the declared hypotheses.

Definition 3.103 (Hypothesis failure)**.** *Hypothesis failure* occurs when a required bridge hypothesis is false, unsupported, unavailable, or marked as failed.

Definition 3.104 (Lemma failure)**.** *Lemma failure* occurs when a required bridge lemma cannot be proved, imported, verified, or discharged.

Definition 3.105 (Realization failure)**.** *Realization failure* occurs when the source object cannot be realized into the target layer in the required form.

Definition 3.106 (Core payload failure)**.** *Core payload failure* occurs when the Bridge Program cannot produce required verifier-side Core data, such as:

$$\mathrm{Eval}_{i,W,\tau}(F), \qquad C_{\tau,W}F, \qquad \phi_{i,W,\tau},$$

or the required diagnostic, gate, or ledger records.

Definition 3.107 (Transport failure)**.** *Transport failure* occurs when the intended implication between source and target semantics cannot be established.

Definition 3.108 (Ledger failure)**.** *Ledger failure* occurs when a consumed primitive metric discrepancy lacks a valid profile-bound support record, cannot be converted into the declared Core quantale, is double counted, or causes the selected-route inequality to fail:

$$\mathrm{val}_B\big(\mathfrak{d}_{\mathrm{bridge}}(P)\big) \geq \mathrm{gap}_\tau(\mathfrak{B}_P).$$

Missing metric evidence is undefined; an evidenced false required inequality is reject.

Definition 3.109 (Interpretation failure)**.** *Interpretation failure* occurs when a formally verified or Core verified statement cannot be validly interpreted as the intended external or problem-specific statement.

Definition 3.110 (Promotion failure)**.** *Promotion failure* occurs when a Bridge Program cannot satisfy the evidence required to promote a bridge draft, bridge candidate, or [Spec] bridge to theorem-level status.

Proposition 3.111 (Bridge failure blocks theorem-level transport)**.** *If a necessary Bridge Program fails, then the intended theorem-level transport is blocked.*

Proof. The intended transport depends on the bridge theorem. If the Bridge Program cannot establish that theorem, the transport implication is unavailable. □

Remark 3.112 (Failure is informative)**.** Bridge failure should be recorded precisely. It identifies whether the obstruction is in hypotheses, lemmas, realization, Core payload, transport, ledger, interpretation, or promotion.

W.10bis. Typed failure-route calculus

Definition 3.113 (Bridge failure-route record)**.** A *bridge failure-route record* distinguishes at least:

(i) missing evidence, mapped to undefined;

(ii) certified repaired-profile nonvanishing, mapped to the applicable topological reject route;

(iii) certified detector applicability or completeness failure, mapped to the applicable operational reject route;

(iv) valid nonzero eligible higher obstruction, mapped to the categorical reject route;

(v) valid nonzero terminal Type IV diagnostic, mapped to Type IV reject;

(vi) certified retained transient defect, mapped to transient_defect, not renamed Type IV;

(vii) absent Interface, reduction, or Return arrow, leaving the external conclusion unlicensed and undefined;

(viii) governance, identity, promotion, provenance, or repair-ancestry failure, mapped to the governance reject route;

(ix) use of a non-claim or specification as theorem evidence, mapped to a non-verdict or governance failure.

Proposition 3.114 (Missing bridge evidence is not a mathematical counterexample)**.** *Failure to construct, find, retrieve, formalize, or verify a required bridge object does not establish that the mathematical bridge is false.*

Proof. Absence of evidence and evidence of falsity are different atomic states. □

Proposition 3.115 (Program success is noncompensating)**.** *Success on one bridge obligation cannot compensate for a missing or failed independent detector, eligibility, actual-morphism, reduction, Return, source-binding, or governance obligation.*

Proof. Each obligation is consumed at its own type; the verifier has no cross-type compensation rule. □

W.11. Bridge Program lifecycle

Definition 3.116 (Bridge Program lifecycle)**.** The *Bridge Program lifecycle* is:

$$\text{propose} \longrightarrow \text{specify} \longrightarrow \text{decompose} \longrightarrow \text{prove or import lemmas}$$
$$\longrightarrow \text{verify} \longrightarrow \text{audit} \longrightarrow \text{promote or reject.}$$

Definition 3.117 (Promotion)**.** *Promotion* is a governance act changing the admissible theorem-facing use of a frozen bridge statement. Local workflow advancement, successful verification, or release readiness is not canonical promotion until the governing CR snapshot binds the exact source artifact and dependencies.

Definition 3.118 (Promotion condition)**.** A *promotion condition* requires the exact statement, independent claim axes, complete proof or valid import, discharged hypotheses and lemmas, selected-route comparison and ledger records, non-scalar obligations, source and artifact identities, semantic replay, failure routes, permitted use, prohibited stronger readings, repair ancestry, and claim-use preflight. For final v20 promotion it additionally requires frozen Part V and Companion dependency closure.

Definition 3.119 (Promotion record)**.** A *promotion record* records the prior and new theorem-facing statuses, exact evidence consumed, governing register snapshot, approving verifier or governance authority, immutable artifact references, repair ancestry, and limitations. Appendix W itself creates no final promotion record during construction.

Proposition 3.120 (No silent promotion)**.** *A bridge candidate or Bridge Program may not be silently promoted to theorem status.*

Proof. Promotion changes claim status. Claim status changes must be explicit, justified, and audit-readable. Silent promotion would violate the claim taxonomy and non-claim discipline. □

Remark 3.121 (Promotion is rare)**.** Most Bridge Programs remain research programs or [Spec] interfaces for long periods. This is acceptable and should be recorded honestly.

W.11bis. Imported theorem and formal-interpretation discipline

Definition 3.122 (Valid imported bridge theorem)**.** A *valid imported bridge theorem* is an external or formal theorem imported into a Bridge Program with source identity, exact statement, hypotheses, proof location or formal environment, interpretation theorem when required, permitted use, forbidden stronger reading, and CR-v20-compatible claim-use metadata.

Definition 3.123 (Formal interpretation bridge obligation)**.** A *formal interpretation bridge obligation* is the obligation to prove that a verified formal fragment states and proves the intended informal bridge statement under the declared semantics and hypotheses.

Proposition 3.124 (Imported theorem is scoped to its imported statement)**.** *A validly imported theorem may be used only at the exact strength, scope, hypotheses, and semantics recorded in its import record.*

Proof. Import does not rewrite a theorem into a stronger or differently typed statement. The admissible use is bounded by the source statement, proof environment, hypotheses, interpretation record, and permitted-use metadata. □

Proposition 3.125 (Formal proof does not eliminate interpretation burden)**.** *A verified formal fragment does not prove the intended informal bridge unless a formal interpretation bridge obligation is discharged.*

Proof. The proof assistant checks the formal statement in its formal environment. The intended informal bridge may have different semantics, hypotheses, or scope. An interpretation theorem is required to connect them. □

Remark 3.126 (Import is not inflation)**.** A valid import is a controlled theorem-use mechanism, not a way to inflate an external theorem, formal theorem, or toy theorem into an unrestricted bridge. The imported item remains scoped to its recorded statement and dependencies.

W.11ter. Source binding and Known-Theorem Recovery quarantine

Definition 3.127 (Recovery source-binding package)**.** A *recovery source-binding package* contains the authoritative source identity, exact benchmark statement and locator, permitted precursor statements, prohibited benchmark-conclusion use, source class, hypotheses, content-addressed local binding, and final independent-validation policy.

Definition 3.128 (Benchmark quarantine record)**.** A *benchmark quarantine record* proves that the benchmark proof, known truth value, conclusion-derived witness, answer table, target constants, exceptional bounds, or equivalent oracle was not used to construct the realization, detector, finite certificate, reduction, higher edge, or Return proof. The benchmark statement itself may identify the target and exact Return consequent.

Proposition 3.129 (Benchmark agreement is validation, not derivation)**.** *Agreement of an independently recovered result with the quarantined benchmark may validate exact strength only after the full derivation is complete; it cannot fill a missing arrow or hypothesis.*

Proof. Earlier use would be circular; later comparison adds no derivational premise. □

Proposition 3.130 (Source theorem import cannot contaminate realization)**.** *A theorem imported for the final Return or independent validation may not be used as a realization input when the program advertises independent Known-Theorem Recovery.*

Proof. The recovered predicate would then encode the conclusion it claims to recover. □

W.12. Bridge Programs and proof-first records

Definition 3.131 (Proof-first alignment)**.** A Bridge Program is *proof-first aligned* if its bridge statement, hypotheses, lemmas, failure modes, Core payload, and promotion conditions are derived from a declared proof-first record.

Definition 3.132 (Proof-first dependency)**.** A *proof-first dependency* is a dependency between a Bridge Program and a proof-first record, target theorem template, proof object schema, or theorem-level route.

Proposition 3.133 (Proof-first alignment prevents search inflation)**.** *Proof-first alignment reduces the risk that search artifacts are mistaken for theorem-level bridges.*

Proof. A proof-first record specifies the target theorem and required bridges before search outputs are evaluated. Thus search artifacts are measured against predefined proof obligations rather than being inflated into claims after discovery. □

Remark 3.134 (Relation to Appendix NS-C)**.** Appendix NS-C is a major example of proof-first alignment. Its Navier–Stokes proof-first program decomposes the PDE theorem into bridge obligations.

W.12bis. v20 reference programs and non-invoked successors

Remark 3.135 (Iwasawa exact-growth recovery)**.** The frozen program IW-KTR-GROWTH is a reference exact-strength Known-Theorem Recovery of the Iwasawa 1959 Theorem 4 characteristic-growth clause, relative to its registered precursor package. It is not an AK-only reconstruction of the full module-structure theory, an Iwasawa main conjecture, a class-number theorem without the separate arithmetic Interface, a Type IV theorem, or information-losslessness result.

Remark 3.136 (Serrin fixed-exponent recovery)**.** The frozen program NS-KTR-SERRIN-L6 is a reference exact one-way recovery of the fixed strict-subcritical implication

$$u \in L^6(0,T;L^6(\mathbb{R}^3)) \Longrightarrow u \text{ is internally regular on } \mathbb{R}^3 \times (0,T].$$

Its finite witness includes an analytic proof artifact certifying the norm bound. It is not a new analytic proof, an unconditional regularity theorem, a Type IV result, or a Clay-level result.

Remark 3.137 (Fukaya and normalization successor boundary)**.** For v20.0.0 the concrete problem-facing Fukaya/HMS Bridge Program and concrete external AGI-N normalization bridge are not_invoked under explicit scope-exclusion policies. The bridge schemas remain available as successor targets, but no external theorem, normalization success, build success, or replay success is promoted from them.

W.13. Bridge Program manifest entries

Definition 3.138 (Bridge Program manifest entry)**.** A *Bridge Program manifest entry* records the program identity and claim axes; exact directional statement; source and target semantics; source bindings and quarantine policy; workflow status; hypotheses and lemmas; detector-stage and finite-certificate records; representative and higher-edge records; actual Type IV and transient records; selected admissible proof route and comparison modes; primitive metric components and non-scalar obligations; finite-to-infinite and Return records; proof-first dependencies; responsible agents; immutable artifacts and environments; semantic-replay status; typed failures; repair ancestry; and final-CR promotion state.

Proposition 3.139 (Bridge manifest supports audit)**.** *A Bridge Program manifest entry supports audit by recording the bridge statement, dependencies, hypotheses, proof status, Core-facing obligations, defects, and artifacts.*

Proof. Auditing requires reconstruction of what was claimed, what was assumed, what was proved, what remains blocked, and what artifacts support the record. The manifest entry supplies these data. □

Remark 3.140 (Manifest does not prove bridge)**.** A Bridge Program manifest records the state of the program. It does not prove the bridge unless it references a complete proof or verified proof object.

W.13bis. Artifact identity and semantic replay

Definition 3.141 (Bridge semantic replay)**.** A *bridge computational replay* reconstructs program outputs from pinned artifacts and environments. A *bridge semantic replay* additionally checks the identity and meaning of every source statement, arrow, hypothesis, profile stage, comparison mode, claim axis, reduction, Return conclusion, and permitted use.

Definition 3.142 (Bridge artifact-drift record)**.** A *bridge artifact-drift record* is opened when a source, theorem statement, program artifact, proof environment, model, prompt, retrieved context, code, dependency, or manifest differs from the identity consumed by the certified program. It determines whether replay, semantic diff, demotion, re-verification, or a new claim is required.

Proposition 3.143 (Computational replay is not semantic identity)**.** *Successful reconstruction, build, formal check, or execution does not establish semantic identity, source faithfulness, non-circularity, or external theorem validity without the corresponding semantic replay and interpretation records.*

Proof. A reproducible process can reproduce a stale source, mistranscribed theorem, scope error, circular realization, or wrong interpretation. □

Proposition 3.144 (Artifact drift blocks silent theorem use)**.** *A drifted bridge artifact cannot be silently consumed at the status of the prior artifact.*

Proof. The prior verification was relative to a different immutable object or environment. □

W.14. Bridge Program invariants

Definition 3.145 (Bridge Program invariant)**.** A *Bridge Program invariant* is a rule that must remain true throughout the Bridge Program lifecycle.

Definition 3.146 (Non-theorem invariant)**.** The *non-theorem invariant* states that a Bridge Program is not theorem-level until a bridge theorem is proved or validly imported.

Definition 3.147 (Hypothesis-label invariant)**.** The *hypothesis-label invariant* states that every required bridge hypothesis must have an explicit status label.

Definition 3.148 (Lemma-discharge invariant)**.** The *lemma-discharge invariant* states that theorem-level bridge status requires discharge of all necessary bridge lemmas.

Definition 3.149 (Core-payload invariant)**.** The *Core-payload invariant* states that any Core-facing bridge payload remains candidate data until checked by the declared Core verifier.

Definition 3.150 (No-silent-promotion invariant)**.** The *no-silent-promotion invariant* states that every promotion in bridge status must be explicit, justified, and audit-readable.

Proposition 3.151 (Bridge Program invariants prevent bridge inflation)**.** *The Bridge Program invariants prevent bridge drafts, bridge candidates, and Bridge Programs from being silently inflated into theorem-level bridge claims.*

Proof. The non-theorem invariant blocks program-to-theorem conflation. The hypothesis-label invariant exposes unproved assumptions. The lemma-discharge invariant blocks incomplete proof graphs. The Core-payload invariant blocks unchecked Core-facing data. The no-silent-promotion invariant blocks unsupported claim upgrades. Together, they prevent bridge inflation. □

W.14bis. v20 Bridge Program invariants

Definition 3.152 (Projection–Return completeness invariant)**.** The *Projection–Return completeness invariant* requires every external conclusion to have a complete, directionally composable chain from source semantics through realization, Core certification, any required reduction and higher edge, and Return.

Definition 3.153 (Source-quarantine invariant)**.** The *source-quarantine invariant* prevents a Known-Theorem Recovery benchmark conclusion from entering the independent construction path.

Definition 3.154 (Typed-ledger invariant)**.** The *typed-ledger invariant* permits only supported metric discrepancies on the selected admissible route to enter the quantale ledger and forbids double counting and numerical repair of non-scalar obligations.

Definition 3.155 (Terminal-transient separation invariant)**.** The *terminal–transient separation invariant* keeps terminal Type IV, transient defect, full-interior defect, full defect zero, apex agreement, and isomorphism as distinct predicates.

Proposition 3.156 (v20 invariants prevent cross-layer inflation)**.** *The Projection–Return, source-quarantine, typed-ledger, and terminal–transient invariants prevent a complete-looking Bridge Program from being consumed as a stronger cross-layer theorem than its evidence supports.*

Proof. Each invariant blocks a separate inflation route: missing arrows, circular benchmark use, metric/nonmetric type confusion, and tower-diagnostic overreading. □

W.15. Summary of Bridge Programs

Theorem 3.157 (Appendix W Bridge Program package)**.** *Within the Search / Platform layer:*

(i) Bridge Programs are typed proof programs, not bridge theorems.

(ii) Bridge statements are directional and require exact source and target semantics.

(iii) Workflow progress, graph closure, manifest completeness, and build success do not promote claim axes.

(iv) *External conclusions require a complete Projection–Certificate–Reduction–Return chain.*

(v) *Core-facing obligations include stage-correct finite detectors, degree-n edges, actual Type IV morphisms, transient records, and finite-to-infinite reductions as invoked.*

(vi) *Graph reachability is not theorem composition, and pairwise arrows are not higher coherence.*

(vii) *Only supported primitive metric discrepancies on the selected admissible proof route enter the Appendix-M-compatible quantale ledger.*

(viii) *Alternative paths and aliases are not double counted; scalar slack does not repair non-scalar obligations.*

(ix) *Missing evidence remains undefined and does not constitute a mathematical counterexample.*

(x) *Known-Theorem Recovery quarantines the benchmark conclusion from the construction path.*

(xi) *Imported and formal theorems remain scoped to exact statements and interpretation records.*

(xii) *Terminal Type IV zero does not imply transient absence, full defect zero, isomorphism, apex agreement, or external information preservation.*

(xiii) *Computational replay is distinct from semantic replay.*

(xiv) *No canonical final-CR promotion occurs locally before the full downstream dependency closure is frozen.*

Proof. Items (i)–(iii) follow from Definitions 3.5, 3.13, 3.45, and 3.9, and Propositions 3.20 and 3.11. Item (iv) follows from Definition 3.28 and Propositions 3.31 and 3.32. Items (v)–(vi) follow from Definitions 3.59, 3.60, 3.61, and 3.62, and Propositions 3.74 and 3.75. Items (vii)–(viii) follow from Definitions 3.85, 3.86, and 3.97, and Propositions 3.89, 3.98, and 3.94. Item (ix) follows from Definition 3.113 and Proposition 3.114. Item (x) follows from Definitions 3.40, 3.127, and 3.128 and Proposition 3.129. Item (xi) follows from Definition 3.122 and Propositions 3.124 and 3.125. Item (xii) follows from Definition 3.61 and Proposition 3.95. Item (xiii) follows from Definition 3.141 and Proposition 3.143. Item (xiv) is Proposition 3.12. □

Theorem 3.158 (Appendix W v20 compatibility hardening package)**.** *The inherited Bridge Program sovereignty, directional statement, status-axis, import, formal-interpretation, noncompensation, Type IV artifact, ledger registration, and proof-first disciplines remain available at their repaired strengths, now subject to the v20 finite-certificate, higher-edge, reduction, Return, source-quarantine, selected-route, and final-CR deferral rules of this appendix.*

Proof. The inherited items are preserved by their original local definitions and propositions, while the v20 restrictions are supplied by Sections W.1ter, W.2ter, W.5bis, W.6bis, W.8ter, W.10bis, W.11ter, W.13bis, and W.14bis. No inherited statement is strengthened by removing a hypothesis. □

W.15bis. v20 Bridge Program extension package

Theorem 3.159 (Appendix W v20 Projection–Return extension)**.** *The following additional v20 constraints hold:*

(i) *claim axes and workflow axes remain independent;*

(ii) *finite detector stage declarations require stage-specific theorem support;*

(iii) *higher-n transport is one-way by default and synthetic shifts do not create natural bridges;*

(iv) *actual-morphism Type IV is separated from transient and full-interior defects;*

(v) *finite observation and numerical convergence do not establish finite-to-infinite reduction;*

(vi) *every external conclusion requires a Return theorem;*

(vii) *only supported selected-route metric discrepancies enter the finite monoid reduct of the declared quantale;*

(viii) *benchmark conclusions remain quarantined in Known-Theorem Recovery;*

(ix) *computational replay does not replace semantic replay;*

(x) *final CR-v20 promotion is deferred until the downstream artifact set is frozen.*

Proof. The items follow respectively from Sections W.1ter, W.5bis, W.5bis, W.5bis and W.7bis, W.2ter and W.5bis, W.2ter, W.8–W.8ter, W.11ter, W.13bis, and Remark 3.4. □

W.16. Explicit exclusions

The following are not asserted in Appendix W.

- No Bridge Program is treated as a bridge theorem.
- No Bridge Program sovereignty label is treated as verifier-side evidence.
- No bridge verifier-side conversion record is accepted without result status, failure route, and immutable artifact references.
- No bridge direction is reversed without a separately proved reverse bridge.
- No external theorem is treated as a Core theorem by name alone.
- No workflow status is treated as claim-governance status.
- No bridge promotion is accepted without a promotion certificate and claim-use preflight.
- No scalarized bridge budget repairs missing source theorem validity, interpretation, eligibility, tower artifact identity, terminal tameness, or claim-use permission.
- No Bridge Program Type IV candidate is treated as certified zero without an actual tower comparison artifact and terminal-generic kernel/cokernel computation.
- No formal proof assistant result is interpreted as the intended informal bridge without an interpretation theorem.
- No bridge draft is treated as a specified bridge candidate.
- No bridge candidate is treated as proved.
- No [Spec] bridge is treated as theorem-level.
- No unresolved bridge hypothesis is treated as discharged.
- No partial bridge lemma graph is treated as complete proof.
- No Bridge Program status is treated as theorem status without proof.
- No Core-facing obligation is treated as verified by being listed.
- No Core bridge payload is treated as Core verified without Core verification.

- No bridge defect is ignored when used in certification.
- No bridge-adjusted budget is bypassed.
- No Bridge Program updates Core pass/reject by itself.
- No failed Bridge Program supports theorem-level transport.
- No search-generated bridge is promoted silently.
- No proof-first record is replaced by a Bridge Program.
- No Bridge Program replaces B-Gate^{+}.
- No Bridge Program replaces the repaired topological condition

$$\mathrm{Eval}_{n,W,\tau}(F) = 0.$$

- No Bridge Program replaces admissible representative records

$$C_{\tau,W}F.$$

- No Bridge Program replaces monitored Type IV diagnostics.
- No Bridge Program replaces tower comparison artifacts

$$\phi_{i,W,\tau}.$$

- No Bridge Program replaces the budget condition

$$\mathrm{val}_B(\mathfrak{d}_{\mathrm{met}}) < \mathrm{gap}_\tau(\mathfrak{B}).$$

- No finite observation, plateau, repeated barcode, numerical convergence, or stored apex is treated as a finite-to-infinite reduction.
- No stage name authorizes detector soundness or completeness without the stage-specific theorem package.
- No synthetic shift or target-shaped encoding is treated as a natural higher-n bridge.
- No Core certificate is treated as an external theorem without the separately proved Return arrow.
- No semantic, interpretation, governance, realization, eligibility, actual-morphism, reduction, Return, or artifact obligation is hidden in a numerical ledger component.
- No alternative bridge routes are cumulatively charged unless they are actually composed on the selected admissible proof route.
- No primitive discrepancy is counted under multiple search, lift, bridge, storage, replay, or execution aliases.
- No benchmark conclusion or conclusion-derived witness is used to construct an advertised Known-Theorem Recovery chain.
- No successful build, formal check, execution, or computational replay substitutes for semantic replay or source authority.
- No terminal Type IV zero is read as transient absence, full-interior absence, isomorphism, apex agreement, or external information preservation.

- No concrete Fukaya/HMS or external AGI-N bridge is asserted in v20.0.0; those tracks are not invoked under explicit scope exclusions.
- No local Part V record fabricates a canonical Claim UID, final promotion receipt, or release-gate pass.

W.17. Provenance and status

The material in this appendix depends on:

(i) Chapters 1 and 7–8 for claim typing, verifier sovereignty, finite-certificate, Problem-Bridge, Known-Theorem Recovery, and Return architecture;

(ii) the current CR-v20 baseline for registered Main/Foundation claim use and final-promotion deferral;

(iii) Appendices U and V for agent and search conversion semantics;

(iv) Appendix A for window-first hard deletion, two-sided threshold-gap transport, and repaired-zero separation;

(v) Appendix B for finite detector stages, completeness, representatives, and stage-transport discipline;

(vi) Appendix C for the standard degree-n eligible higher-edge package;

(vii) Appendix D for actual-morphism Type IV, transient defects, and tower finite-to-infinite reduction;

(viii) Appendix E for coverage, atlas, overlap, Restart, higher coherence, and local-to-global transport when invoked;

(ix) Appendix F for atomic status and noncompensation;

(x) Appendix G for canonical manifest and verifier discipline;

(xi) CM-v20 and TB-v20 at frozen local strengths;

(xii) Appendix M for the declared commutative quantale, scalarization, selected-route support, and no-double-counting discipline;

(xiii) Problem Interface Part IV for purpose-faithful realization, source quarantine, finite-to-infinite reduction, Return, Iwasawa and Serrin recovery boundaries, and the not-invoked Fukaya/AGI-N successor decisions;

(xiv) Appendix NS-C for proof-first decomposition at its frozen local strength.

Its role is to define how proposed cross-layer implications are managed as auditable proof programs without being confused with proved theorems.

Status labels for Appendix W. *Search / Platform definitions:* Definitions 3.13, 3.14, 3.15, 3.16, 3.17, 3.18, 3.19, 3.33, 3.34, 3.35, 3.36, 3.37, 3.38, 3.44, 3.45, 3.46, 3.54, 3.55, 3.56, 3.65, 3.66, 3.67, 3.68, 3.69, 3.76, 3.77, 3.84, 3.85, 3.86, 3.87, 3.102, 3.103, 3.104, 3.105, 3.106, 3.107, 3.108, 3.109, 3.110, 3.116, 3.117, 3.118, 3.119, 3.131, 3.132, 3.138, 3.145, 3.146, 3.147, 3.148, 3.149, 3.150.
Search / Platform propositions/theorems: Propositions 3.20, 3.21, 3.47, 3.57, 3.70, 3.78, 3.88, 3.89, 3.111, 3.120, 3.133, 3.139, 3.151, Theorems 3.100, 3.157.
Operational cautions: Remarks 3.1, 3.2, 3.3, 3.22, 3.39, 3.48, 3.58, 3.71, 3.79, 3.90, 3.101, 3.112, 3.121, 3.134, 3.140.
Inherited compatibility definitions: Definitions 3.5, 3.6, 3.23, 3.24, 3.49, 3.50, 3.91, 3.92, 3.93, 3.122, 3.123.

Inherited compatibility propositions/theorems: Theorem 3.158, Propositions 3.7, 3.25, 3.26, 3.51, 3.52, 3.94, 3.95, 3.124, 3.125.
Inherited compatibility remarks: Remarks 3.8, 3.27, 3.53, 3.96, 3.126.
v20 extension definitions: Definitions 3.9, 3.10, 3.28, 3.29, 3.30, 3.40, 3.41, 3.42, 3.59, 3.60, 3.61, 3.62, 3.72, 3.73, 3.80, 3.81, 3.97, 3.113, 3.127, 3.128, 3.141, 3.142, 3.152, 3.153, 3.154, 3.155.
v20 extension propositions/theorems: Propositions 3.11, 3.12, 3.31, 3.32, 3.43, 3.63, 3.64, 3.74, 3.75, 3.82, 3.83, 3.98, 3.99, 3.114, 3.115, 3.129, 3.130, 3.143, 3.144, 3.156, Theorem 3.159.
v20 extension remarks: Remarks 3.4, 3.135, 3.136, 3.137.
Explicit exclusions: Section W.16.

4 Appendix X: Validity Map and Global Certificate DAG

X.1. Scope, dependency position, and map status

This appendix defines the Validity Map and the Global Certificate DAG for AK-HPDST v20.0.0. It belongs to the Search / Platform Appendices and does not enlarge the Core mathematical package of Chapters 1–8, the Foundation Appendices, or the frozen Problem Interface results. Its role is to expose target-local proof dependencies, status boundaries, artifact identities, failure routes, and admissible assembly paths without turning graph structure into mathematical evidence.

The two central rules are:

A Validity Map is navigation, not a theorem.

A Certificate DAG is a typed proof skeleton, not proof by existence.

Fix a monitored degree i for every invoked Type IV scope and, independently, an invoked higher-edge degree n. No node, edge, graph path, topological ordering, centrality score, shortest path, generated dependency chain, successful build, replay, source retrieval, or release dashboard may replace any required verifier-side object or arrow, including:

$B_{n,W}(F) := \mathsf{W}_W \mathbf{P}_n(F)$, $\quad \mathrm{Eval}_{n,W,\tau}(F) := T_\tau^{\mathrm{bar}}\big(B_{n,W}(F)\big)$,
a complete detector record at exactly one declared source stage when invoked,
$C_{\tau,W}F$, the complete degree-n realization and edge package when invoked,
$\phi_{i,W,\tau} : \mathrm{ColimProfile}_{i,W,\tau}(F_\bullet) \longrightarrow \mathrm{ApexProfile}_{i,W,\tau}(F_\infty)$,
separate terminal Type IV, long-transient, and full-interior records,
$\mathrm{val}_B(\mathfrak{d}_{\mathrm{met}}) < \mathrm{gap}_\tau(\mathfrak{B})$,
a finite-to-infinite reduction whenever finite data support an infinite target,
a separately proved Return or soundness-out theorem licensing an external conclusion.

Remark 4.1 (Search / Platform status)**.** Validity Maps and Certificate DAGs are platform-level organization devices. They support navigation, target-local closure, audit, dependency tracking, proof planning, status control, release governance, and reproducibility. They create no theorem, certificate, bridge, reduction, Return statement, or external conclusion merely by being complete as graph records.

Remark 4.2 (Relation to Appendices U, V, and W)**.** Appendix U fixes agent semantics, Appendix V fixes Hunter / Mapper / Lifter search protocols, and Appendix W fixes Bridge Programs. The present appendix assembles their outputs only as typed nodes and edges. Producer identity, agent agreement, map placement, or Bridge Program completion never determines the discharge strength of a consumed node or edge.

Remark 4.3 (Map and DAG are governance structures)**.** A Validity Map governs broad status and navigation. A target Certificate DAG governs a declared necessary dependency subgraph. Neither governs truth unless the referenced proof, verification, artifact, metric, detector, reduction, interpretation, source-binding, and Return data are complete at the exact consumed strength.

Remark 4.4 (v20 dependency and registration boundary)**.** This appendix consumes the canonical v20 Main and Foundation artifacts, frozen Appendices U–W, CM-v20, TB-v20, Technical Extension H–N, and Problem Interface Part IV only at their reviewed or frozen local strengths and immutable artifact identities. The normative final CR-v20 is intentionally deferred until Part V, the Companion, and the cross-appendix dependency closure are frozen. Accordingly, local X-labels are stable construction identifiers rather than canonical Claim UIDs; no provisional Claim UID, local release-gate pass, promotion receipt, or final registry row is fabricated in this appendix.

X.1bis. Map and DAG sovereignty

Definition 4.5 (Map-DAG sovereignty)**.** *Map-DAG sovereignty* is the rule that Validity Maps, Certificate DAGs, graph layouts, path searches, dependency visualizations, and certificate skeletons remain governance and navigation objects until their referenced nodes and edges are independently discharged by the appropriate verifier-side protocol. They may organize proof obligations, but they do not change the mathematical status of any node, edge, bridge, certificate, or external-domain claim by placement alone.

Definition 4.6 (Verifier-side projection of a DAG)**.** For a target Certificate DAG D, the *verifier-side projection* $\pi_{\mathrm{ver}}(D)$ is the subrecord consisting only of discharged verifier-side nodes, discharged required edges, verified bridge or interpretation nodes, ledger records with certified scalarization, diagnostic records, gate verdicts, artifact identities, and manifest data consumed by the target certificate. Map layout, advisory regions, search scores, aesthetic graph positions, and proposer-side comments are omitted.

Proposition 4.7 (Map placement does not establish discharge)**.** *Placing a node or edge in a Validity Map or Certificate DAG does not discharge that node or edge.*

Proof. Placement is a platform action. Discharge requires the evidence specified by the node or edge type: proof, verification, bridge theorem, interpretation theorem, ledger record, diagnostic computation, artifact identity check, or manifest audit. A platform action supplies none of these by default. □

Proposition 4.8 (Verifier-side projection controls target use)**.** *A target certificate may consume a Certificate DAG only through its verifier-side projection. Changing map layout, display order, path highlighting, or advisory metadata without changing $\pi_{\mathrm{ver}}(D)$ does not change the target verdict.*

Proof. The target verdict is determined by discharged verifier-side obligations and their audit-readable records. Layout and advisory metadata are outside those obligations. Therefore only the verifier-side projection can affect target use. □

Remark 4.9 (Graph structure is not semantic strength)**.** A short path, central node, dense cluster, or visually prominent subgraph may be useful for navigation. It is not stronger theorem evidence than a peripheral node unless the corresponding discharge records are stronger.

X.1ter. Graph domains and proof-chain semantics

Definition 4.10 (Proof-dependency graph)**.** A *proof-dependency graph* is the directed graph whose consumed edges assert necessary logical, verifier-side, metric, detector, bridge, interpretation, reduction, Return, or artifact dependencies for a fixed target. Only this graph is required to be acyclic for ordinary DAG certification.

Definition 4.11 (Governance and provenance graph)**.** A *governance and provenance graph* records status history, repair ancestry, supersession, review relations, storage locations, source-binding history, and release governance. Its edges are not proof-dependency edges and are omitted from the verifier-side proof projection unless a target explicitly consumes the corresponding artifact-identity or claim-use record.

Definition 4.12 (Typed proof-chain path)**.** A *typed proof-chain path* is an ordered list of discharged nodes and edges

$$N_0 \xrightarrow{e_1} N_1 \xrightarrow{e_2} \cdots \xrightarrow{e_r} N_r$$

such that every codomain equals the next domain at the declared semantic type, every direction is preserved, every required hypothesis is discharged, and every edge has a permitted discharge mode. A graph path that lacks these conditions is only a navigation path.

Proposition 4.13 (Graph reachability is not theorem composition)**.** *Reachability of a target node from a source node in a map or DAG does not establish the composite implication from source to target.*

Proof. Graph reachability requires only a sequence of drawn edges. Theorem composition additionally requires compatible source and target semantics, correct directions, discharged hypotheses, valid edge modes, artifact identity, and non-circularity. A path missing any of these data is not a composable proof chain. □

Remark 4.14 (No universal all-purpose graph)**.** The proof-dependency graph, governance graph, provenance graph, search map, and release graph may share identifiers, but their edges are not interchangeable. Keeping the graph domains separate prevents a review, citation, storage, or supersession edge from being consumed as mathematical implication.

X.2. Validity Map

Definition 4.15 (Validity Map)**.** A *Validity Map* is a structured map of claims, candidates, bridges, certificate fragments, proof objects, artifacts, failure modes, and proof statuses in an AK workflow. It records where claims stand relative to verification, audit, bridge discharge, and theorem-level readiness.

Definition 4.16 (Validity Map node)**.** A *Validity Map node* is an entry representing one of:

(i) a Core claim;

(ii) a Core-facing object;

(iii) a repaired evaluation record;

(iv) an admissible representative record;

(v) an eligible realization record;

(vi) a tower comparison artifact;

(vii) a finite observation record;

(viii) a finite detector certificate with one declared profile stage;

(ix) a detector certificate-status record and a separate detector mathematical-value record;

(x) a higher obstruction edge record in a declared degree n;

(xi) a finite-to-infinite reduction record;

(xii) a Return or soundness-out theorem;

(xiii) a diagnostic result;

(xiv) a terminal Type IV record;

(xv) a long-transient defect record;

(xvi) a full-interior defect record;

(xvii) a ledger component;

(xviii) an interface claim;

(xix) a bridge draft;

(xx) a bridge candidate;

(xxi) a Bridge Program;

(xxii) a bridge theorem;

(xxiii) a search artifact;

(xxiv) a formal fragment;

(xxv) a verified formal fragment;

(xxvi) a certificate candidate;

(xxvii) a verified certificate;

(xxviii) a proof-first record;

(xxix) a proof object;

(xxx) an immutable source-binding or precursor-binding record;

(xxxi) a Known-Theorem Recovery track;

(xxxii) a benchmark-quarantine or source-conclusion-quarantine record;

(xxxiii) an operational replay or semantic replay record;

(xxxiv) a failure mode;

(xxxv) a non-claim;

(xxxvi) a [Spec] statement.

Definition 4.17 (Core-facing map node)**.** A *Core-facing map node* is a Validity Map node involving verifier-side Core data, including:

$$\mathrm{Eval}_{i,W,\tau}(F), \qquad C_{\tau,W}F, \qquad \phi_{i,W,\tau}, \qquad (\mu_{\mathrm{Collapse}},\ u_{\mathrm{Collapse}}) = (0,0),$$
$$\mathrm{val}_B(\mathfrak{d}_{\mathrm{met}}) < \mathrm{gap}_\tau(\mathfrak{B}),$$

or the gate verdicts and manifest entries needed to support them.

Definition 4.18 (Validity Map edge)**.** A *Validity Map edge* is a typed relation between two Validity Map nodes. Permitted edge types include:

(i) depends-on;

(ii) supports;

(iii) refines;

(iv) blocks;

(v) contradicts;

(vi) specializes;

(vii) generalizes;

(viii) imports;

(ix) discharges;

(x) suggests;

(xi) cites-artifact;

(xii) verifies;

(xiii) audits;

(xiv) realizes;

(xv) lifts-to;

(xvi) normalizes-to;

(xvii) interprets-as;

(xviii) ledger-contributes-to;

(xix) detects-at-stage;

(xx) realizes-into;

(xxi) extracts-higher-edge;

(xxii) reduces-finite-to-infinite;

(xxiii) returns-to-source;

(xxiv) binds-source;

(xxv) validates-benchmark;

(xxvi) bridge-transports-to;

(xxvii) formalizes;

(xxviii) deprecates;

(xxix) supersedes.

Proposition 4.19 (Validity Map edges must be typed)**.** *Every edge in a Validity Map must have a declared edge type.*

Proof. Untyped edges are ambiguous. They may be misread as implication, dependency, evidence, analogy, artifact reference, theorem, or verified transport. Typed edges prevent claim inflation and support audit. □

Remark 4.20 (Suggests is not implies)**.** An edge labeled *suggests* is advisory. It must not be read as logical implication.

X.2bis. Edge-mode discipline and status-axis separation

Definition 4.21 (Edge discharge mode)**.** Every consumed edge in a target Certificate DAG has exactly one *edge discharge mode*:

$$\begin{gathered}\text{proved},\quad \text{verified\_artifact},\quad \text{imported\_theorem},\quad \text{operational\_rule},\\ \text{specification\_target},\quad \text{advisory\_only},\quad \text{failed},\quad \text{undefined}.\end{gathered}$$

Only the first four modes can discharge a necessary edge, and only at their declared scope. The mode operational_rule discharges only operational or governance-typed edges such as labeling, serialization, preflight, routing, or manifest-conformance obligations; it does not discharge theorem, bridge-theorem, diagnostic, metric, interpretation, or formal-proof edges.

Definition 4.22 (Validity status axis)**.** The validity statuses of Section X.3 are map-local status labels. They do not replace:

(i) the agent-output status labels of Appendix U;

(ii) the workflow status fields of Appendix V;

(iii) the Bridge Program status fields of Appendix W;

(iv) the Chapter 1 document-role taxonomy;

(v) the current CR-v20 baseline evidence status and invocation role.

When a node is consumed by a certificate, the stronger governance axes must also be present.

Definition 4.23 (Node consumption profile)**.** A *node consumption profile* records, for each consumed node, its map-local validity status, Appendix U agent-output label when applicable, Chapter 1 document role, CR-v20 baseline evidence status, CR-v20 baseline invocation role, conformance status, permitted use, forbidden stronger reading, and artifact identity.

Proposition 4.24 (Validity status does not replace claim governance)**.** *A Validity Map status label cannot by itself authorize theorem-level use of a node.*

Proof. The map-local status records navigation and workflow state. Theorem-level use is governed by the Chapter 1 taxonomy, the current CR-v20 baseline, source proof strength, discharged hypotheses, permitted use, and artifact identity. Those data are independent of the map-local label. □

Remark 4.25 (Typical status projections)**.** Typical projections are: *search artifact* remains proposer-side; *bridge candidate* remains non-theorem until a bridge theorem record exists; *Core certificate candidate* remains non-verdict until the Core verifier accepts it; *Core verified* must be backed by the corresponding verifier-side record and CR-v20 baseline permitted-use field. These projections are not bidirectional definitions.

X.2ter. Problem-Bridge and Return-chain nodes

Definition 4.26 (Problem-Bridge chain node family)**.** For an external target, the DAG records separately typed nodes for the chain

$$\begin{aligned}&\text{external source statement} \longrightarrow \text{purpose-faithful realization}\\ &\longrightarrow \text{Core-readable filtered or persistence data} \longrightarrow \text{finite Core certificate}\\ &\longrightarrow \text{finite-to-infinite reduction, when required} \longrightarrow \text{higher obstruction edge, when invoked}\\ &\longrightarrow \text{Return or soundness-out theorem} \longrightarrow \text{external mathematical conclusion}.\end{aligned}$$

An unused arrow is recorded as not_invoked only under an immutable target-scope exclusion policy; a required but missing arrow is undefined.

Definition 4.27 (Problem-Bridge chain edge)**.** A *Problem-Bridge chain edge* is one declared arrow in Definition 4.26. It records direction, exact source and target type, hypotheses, proof or valid import, artifacts, permitted use, forbidden stronger reading, and failure route. No edge inherits the proof strength of a neighboring edge.

Definition 4.28 (Return-complete target sub-DAG)**.** A target sub-DAG is *Return-complete* if every necessary arrow from the external source statement through the AK-side certificate, every required reduction and higher edge, and the final Return theorem has result status pass at the exact target strength. A Core-certified node without the final Return arrow is not Return-complete.

Proposition 4.29 (Core certification is not external conclusion)**.** *A discharged Core certificate node does not discharge an external-domain target node unless the necessary reduction and Return edges are separately discharged.*

Proof. The Core node and the external target have different semantics. Finite Core evidence may also have a smaller scope than an infinite, asymptotic, analytic, tower-level, or global target. The reduction and Return theorems supply the missing scope and semantic transport; without them the target edge remains undefined. □

Remark 4.30 (Generated Return arrows are non-verdict)**.** A generated label, visually completed path, theorem-shaped text, or successful replay cannot create a Return theorem. The DAG may expose a missing Return node, but it may not fill that node by graph completion.

X.3. Validity statuses

Definition 4.31 (Validity status)**.** A *validity status* is a label attached to a node describing its current verification state. Permitted statuses include:

(i) search artifact;

(ii) candidate;

(iii) draft;

(iv) advisory;

(v) [Spec];

(vi) operational policy;

(vii) interface specification;

(viii) bridge draft;

(ix) bridge candidate;

(x) Bridge Program;

(xi) partially proved;

(xii) blocked;

(xiii) failed;

(xiv) rejected;

(xv) formal fragment;

(xvi) verified formal fragment;

(xvii) Core certificate candidate;

(xviii) Core-facing candidate;

(xix) Core verified;

(xx) interface theorem;

(xxi) bridge theorem;

(xxii) imported theorem;

(xxiii) proof-first record;

(xxiv) proof object candidate;

(xxv) theorem-level proof object;

(xxvi) non-claim;

(xxvii) deprecated;

(xxviii) superseded.

Definition 4.32 (Status transition)**.** A *status transition* is a recorded change from one validity status to another.

Definition 4.33 (Status transition rule)**.** A *status transition rule* specifies what evidence, verification, audit, proof, import, discharge, or demotion condition is required before a status transition is allowed.

Definition 4.34 (Promotion transition)**.** A *promotion transition* is a status transition to a stronger claim status. Examples include:

$$\text{bridge candidate} \longrightarrow \text{bridge theorem},$$

$$\text{Core certificate candidate} \longrightarrow \text{Core verified},$$

$$\text{proof object candidate} \longrightarrow \text{theorem-level proof object}.$$

Definition 4.35 (Demotion transition)**.** A *demotion transition* is a status transition to a weaker or safer claim status due to failure, artifact drift, missing dependencies, invalid proof, or supersession.

Proposition 4.36 (Status transitions require evidence)**.** *A node may not be promoted to a stronger validity status unless the required transition evidence is recorded.*

Proof. Validity status affects how a node may be used in proof planning, certification, bridge transport, and theorem-level claims. Promotion without evidence would allow unsupported claim inflation. Therefore transition evidence is required. □

Remark 4.37 (Demotion is allowed)**.** Nodes may be demoted when failures, gaps, artifact drift, interpretation errors, or dependency invalidation are discovered. Demotion protects the integrity of the map.

X.3bis. Promotion, demotion, and transition certificates

Definition 4.38 (Map-DAG promotion certificate)**.** A *Map-DAG promotion certificate* is an audit-readable record authorizing a stronger status transition for a node or edge. It contains the prior status, proposed new status, exact source claim or object, evidence supplied, discharged hypotheses, comparison-mode policy when relevant, CR-v20 baseline claim-use preflight result, required artifacts, result status

$$\mathrm{pass}, \qquad \mathrm{reject}, \qquad \mathrm{undefined},$$

and a typed failure route.

Definition 4.39 (Transition result status)**.** A status transition has exactly one result status: pass, reject, or $\mathrm{undefined}$. Missing evidence, stale artifacts, unresolved dependencies, or incompatible permitted use produce $\mathrm{undefined}$, not pass and not a numerical obstruction.

Proposition 4.40 (Promotion requires preflight)**.** *A node or edge cannot be promoted to theorem-level, bridge-theorem-level, Core verified, or target-discharge status unless its Map-DAG promotion certificate passes claim-use preflight and artifact identity checks.*

Proof. The promoted status changes how the node or edge may be consumed. The current CR-v20 baseline requires claim identity, source binding, evidence status, invocation role, hypotheses, permitted use, failure routes, and repair ancestry to be checked before consumption. Without that preflight and artifact identity, the stronger status is unsupported. □

Proposition 4.41 (Demotion is non-retroactive)**.** *Demotion of a node, edge, or subgraph creates a new recorded state and does not mutate the historical artifact or erase the prior provenance.*

Proof. The v20 repair and provenance discipline is immutable. A demotion changes the current admissible use of an item, but the previous record remains part of repair ancestry and audit history. □

Remark 4.42 (Undefined is not a failed theorem)**.** A missing discharge certificate yields $\mathrm{undefined}$ for the transition. It does not prove the negation of the target theorem; it only blocks the attempted consumption route.

X.3ter. Local discharge and final-CR deferral

Definition 4.43 (Local discharge record)**.** A *local discharge record* states that one node or edge has passed its declared mathematical, operational, or artifact check inside the frozen local document set. It is not a canonical Claim Register entry, release-gate pass, or global promotion receipt.

Definition 4.44 (Final-CR extraction boundary)**.** The *final-CR extraction boundary* is reached only after Part V, the Companion, and the cross-document label, artifact, dependency, and provenance closures are frozen and audited. Only then may stable local labels be mapped to canonical Claim UIDs and final release-registry rows.

Proposition 4.45 (Local discharge does not imply release promotion)**.** *A local node or edge discharge does not promote the item into final CR-v20 or establish a release-level gate verdict.*

Proof. Local discharge checks one typed obligation in the current artifact set. Final promotion additionally requires global identity, dependency closure, cross-document consistency, release governance, and canonical registry extraction. Those later obligations are not supplied by a local DAG state. □

Remark 4.46 (No provisional registry fabrication)**.** A temporary JSON row, spreadsheet entry, dashboard badge, generated UID, or path-level pass token remains non-normative until final extraction. The DAG records pending promotion rather than inventing canonical registration.

X.4. Global Certificate DAG

Definition 4.47 (Certificate DAG)**.** A *Certificate DAG* is a directed acyclic graph whose nodes are certificate-relevant objects and whose edges record typed dependency relations among them.

Definition 4.48 (Global Certificate DAG)**.** The *Global Certificate DAG* is the certificate-level DAG that organizes all Core, interface, bridge, technical-extension, proof-first, formalization, ledger, artifact, and platform dependencies relevant to a target proof object or target certificate.

Definition 4.49 (DAG node)**.** A *DAG node* may represent:

(i) a definition;

(ii) a lemma;

(iii) a theorem;

(iv) a bridge statement;

(v) a Bridge Program;

(vi) a bridge theorem;

(vii) a repaired evaluation record;

(viii) an admissible representative record;

(ix) an eligibility record;

(x) a tower comparison artifact;

(xi) a finite detector certificate and its separate certificate/value fields;

(xii) a higher obstruction edge record;

(xiii) a finite-to-infinite reduction record;

(xiv) a Return theorem record;

(xv) a diagnostic result;

(xvi) a terminal Type IV record;

(xvii) a long-transient or full-interior defect record;

(xviii) a certificate fragment;

(xix) a ledger component;

(xx) a gate verdict;

(xxi) a formal fragment;

(xxii) a proof object;

(xxiii) an artifact reference;

(xxiv) a failure condition;

(xxv) an imported theorem;

(xxvi) an interpretation theorem;

(xxvii) an immutable source-binding or precursor-binding record;

(xxviii) a Known-Theorem Recovery track;

(xxix) a benchmark-quarantine record;

(xxx) an operational or semantic replay record.

Definition 4.50 (Core DAG node)**.** A *Core DAG node* is a DAG node representing verifier-side Core content, including:

$$\begin{aligned} &\mathrm{Eval}_{i,W,\tau}(F), \qquad C_{\tau,W}F, \qquad \phi_{i,W,\tau}, \qquad B\text{-Gate}^{+}, \\ &(\mu_{\mathrm{Collapse}},\ u_{\mathrm{Collapse}}) = (0,0), \qquad \mathrm{val}_B(\mathfrak{d}_{\mathrm{met}}) < \mathrm{gap}_\tau(\mathfrak{B}). \end{aligned}$$

Definition 4.51 (DAG edge)**.** A *DAG edge* is a typed dependency from one node to another. It must declare whether it represents logical dependency, artifact dependency, verification dependency, ledger dependency, diagnostic dependency, bridge dependency, interpretation dependency, formalization dependency, advisory dependency, or audit dependency. The DAG-edge namespace is distinct from the broader Validity Map edge namespace of Definition 4.18; a map edge may record navigation or classification, while a consumed DAG edge must satisfy the target-scope discharge discipline.

Proposition 4.52 (DAG existence is not proof)**.** *The existence of a Certificate DAG does not prove the target claim.*

Proof. A DAG records dependencies and statuses. The target claim is proved only if the required nodes are proved or verified and all necessary dependency edges are discharged according to their semantics. The graph structure alone does not establish those conditions. □

Remark 4.53 (DAG is proof skeleton)**.** A Certificate DAG is best understood as a proof skeleton or audit skeleton. It becomes proof-supporting only when its necessary nodes and edges are sufficiently discharged.

X.4bis. Target scope and necessary-edge closure

Definition 4.54 (Target certificate scope)**.** A *target certificate scope* declares the target node, required sub-DAG, necessary nodes, optional nodes, necessary edges, optional edges, accepted edge discharge modes, ledger aggregation policy, diagnostic scope, bridge and interpretation dependencies, and artifact identity policy for one certificate or theorem-level proof object.

Definition 4.55 (Necessary-edge closure)**.** A target sub-DAG has *necessary-edge closure* if every necessary edge has a declared type, source, target, discharge mode, discharge certificate, artifact references when required, and result status pass.

Proposition 4.56 (Optional edges cannot discharge necessary edges)**.** *An optional advisory, explanatory, layout, or citation edge cannot discharge a necessary logical, bridge, diagnostic, ledger, interpretation, or formalization edge.*

Proof. Optional edges are outside the declared necessary dependency set for the target certificate. A necessary edge imposes a specific proof or verification obligation. Changing or adding optional structure does not satisfy that obligation. □

Remark 4.57 (Target-local closure)**.** Closure is target-local. A global map may contain many unresolved or failed nodes while one target sub-DAG is fully discharged, provided the unresolved nodes are not necessary for that target.

X.4ter. Composability and closure grades

Definition 4.58 (Edge composability record)**.** An *edge composability record* declares that two consecutive consumed edges have matching semantic type, parameter scope, monitored degree or higher-edge degree, window, threshold, endpoint convention, artifact identity, and direction. For metric transport it also records the runtime mode

$$\mathrm{exact}, \qquad \mathrm{metric\_gap\_certified}, \qquad \mathrm{not\_compared}.$$

The mode $\mathrm{not\_compared}$ cannot discharge an invoked comparison edge.

Definition 4.59 (Target closure grade)**.** A target sub-DAG has exactly one closure grade:

(i) $\mathrm{navigation\_only}$;

(ii) $\mathrm{manifest\_complete}$;

(iii) $\mathrm{locally\_discharged}$;

(iv) $\mathrm{return\_complete}$;

(v) $\mathrm{release\_promotion\_pending}$;

(vi) $\mathrm{release\_promoted}$;

(vii) $\mathrm{blocked}$;

(viii) $\mathrm{undefined}$.

These are target-workflow grades, not substitutes for the atomic mathematical statuses of the consumed nodes and edges.

Proposition 4.60 (Local closure is not global closure)**.** *A locally discharged target sub-DAG does not imply that the whole Validity Map, another target sub-DAG, or the release registry is closed.*

Proof. Necessary-edge closure is defined relative to one target scope. Other targets may consume additional nodes, stronger directions, different windows or degrees, finite-to-infinite reductions, or Return theorems. Release promotion also consumes global governance data. Hence closure does not propagate beyond the declared target. □

Remark 4.61 (Shortest-path fallacy)**.** A shortest or lowest-cost graph path is not automatically the strongest or admissible proof path. Proof-path selection is constrained by type compatibility, theorem strength, direction, artifact identity, and permitted use before any optimization criterion is applied.

X.5. Acyclicity

Definition 4.62 (DAG acyclicity condition)**.** A Certificate DAG satisfies the *acyclicity condition* if it has no directed cycles among ordinary dependency edges.

Definition 4.63 (Dependency cycle)**.** A *dependency cycle* is a directed cycle in which a node depends, directly or indirectly, on itself.

Definition 4.64 (Justified cyclic proof principle)**.** A *justified cyclic proof principle* is a separately stated fixed-point, mutual-induction, coinduction, guarded recursion, or circular-proof principle that permits a controlled cyclic dependency.

Proposition 4.65 (Dependency cycles block DAG certification)**.** *A dependency cycle blocks ordinary DAG certification unless the cycle is replaced by a separately justified fixed-point, mutual-induction, coinductive, guarded, or otherwise accepted proof principle.*

Proof. Ordinary DAG-based certification requires acyclic dependency order. A cycle prevents topological ordering of proof obligations. It can be allowed only if replaced by a justified proof principle that explicitly handles cyclic dependence. □

Remark 4.66 (No hidden circular proof)**.** The DAG discipline is designed to expose circular reasoning. If a circular structure is mathematically valid, its justification must be explicit.

Proposition 4.67 (Provenance cycles do not justify proof cycles)**.** *A cycle in review, citation, storage, supersession, or repair-ancestry metadata does not justify a cycle in the ordinary proof-dependency graph.*

Proof. Governance and provenance edges record history or organization, not logical dependence. A proof cycle requires its own accepted cyclic proof principle and cannot borrow justification from metadata connectivity. □

X.6. Edge semantics

Definition 4.68 (Logical edge)**.** A *logical edge*

$$A \to B$$

means that B logically depends on A.

Definition 4.69 (Artifact edge)**.** An *artifact edge*

$$A \to B$$

means that B references or requires artifact A.

Definition 4.70 (Ledger edge)**.** A *ledger edge*

$$A \to B$$

means that ledger component A contributes to the budget or audit condition of B.

Definition 4.71 (Diagnostic edge)**.** A *diagnostic edge*

$$A \to B$$

means that diagnostic result A is required for certificate node B.

Definition 4.72 (Bridge edge)**.** A *bridge edge*

$$A \to B$$

means that bridge theorem or bridge candidate A is used to connect the semantics of B to another layer. Its status must specify whether A is theorem-level, [Spec], candidate, failed, or imported.

Definition 4.73 (Interpretation edge)**.** An *interpretation edge*

$$A \to B$$

means that formal, Core, or platform-side statement A is interpreted as supporting informal or problem-specific claim B. Such an edge requires an interpretation theorem or explicit [Spec] status.

Definition 4.74 (Formalization edge)**.** A *formalization edge*

$$A \to B$$

means that informal statement A is represented by formal fragment or formal theorem B.

Definition 4.75 (Verification edge)**.** A *verification edge*

$$A \to B$$

means that verifier-side check or verified record A is required to establish the declared verification status of node B. It must record the verification authority, authority scope, environment when applicable, result status, and typed failure route. A verification edge is not interchangeable with an artifact, audit, advisory, or layout edge.

Definition 4.76 (Audit edge)**.** An *audit edge*

$$A \to B$$

means that audit record A supports the reconstructibility, provenance, manifest completeness, artifact identity, or replay-readiness of node B. An audit edge supports certification governance only at its declared audit scope; it does not by itself prove the mathematical theorem represented by B.

Definition 4.77 (Advisory edge)**.** An *advisory edge*

$$A \to B$$

means that A suggests, motivates, or prioritizes B, but does not logically support B.

Proposition 4.78 (Advisory edges are non-proof edges)**.** *Advisory edges do not discharge proof obligations.*

Proof. An advisory edge records motivation or navigation. It is not a logical, verification, audit, diagnostic, ledger, verified bridge, interpretation, or formal proof dependency. Therefore it cannot discharge a proof obligation. □

Remark 4.79 (Edge typing prevents misuse)**.** A single visual arrow can be misleading. The typed edge system prevents advisory, artifact, formalization, interpretation, bridge, and logical relations from being conflated.

X.6bis. Edge compatibility and misrouting control

Definition 4.80 (Edge compatibility record)**.** An *edge compatibility record* states that the source node type, target node type, edge type, discharge mode, status labels, and permitted use are mutually compatible for the intended consumption. For example, an advisory edge is compatible with prioritization, but not with theorem discharge.

Definition 4.81 (Edge misrouting)**.** *Edge misrouting* occurs when an edge is consumed as if it had a stronger or different semantic type than its declared type, for example using *suggests* as *proves*, artifact citation as theorem evidence, formalization as interpretation, or Bridge Program placement as bridge theorem discharge.

Proposition 4.82 (Misrouted edges block target discharge)**.** *If a necessary edge is misrouted, the target node is not discharged until the edge is corrected or replaced by an edge with the required type and discharge certificate.*

Proof. The target consumes the edge at a specific semantic strength. A misrouted edge lacks that strength. Therefore the required dependency remains undischarged. □

Remark 4.83 (Visual arrows are never enough)**.** The same drawn arrow can mean citation, motivation, proof, obstruction, dependency, interpretation, or transport. Only the declared edge type and discharge mode determine admissible use.

X.6ter. v20 proof-chain edge families

Definition 4.84 (Realization edge)**.** A *realization edge*

$$A_{\mathrm{ext}} \longrightarrow F$$

asserts that declared external data are mapped into a Core-readable filtered or persistence object with the required purpose-faithfulness, parameter, filtration, and artifact guarantees. A target-shaped encoding without a proved source-faithfulness theorem is not a discharged realization edge.

Definition 4.85 (Finite-detector edge)**.** A *finite-detector edge* consumes a complete detector record at exactly one source stage:

$$\begin{gathered}\mathrm{windowed\_prethreshold}, \quad \mathrm{repaired\_postthreshold},\\ \mathrm{kernel\_defect}, \quad \mathrm{cokernel\_defect}.\end{gathered}$$

The edge records the exact source profile, endpoint convention, nonzero soundness, zero completeness, coverage or characterization, family scope, excess provenance, non-adaptivity, artifact identity, certificate status, and mathematical value. The standard v20 finite detector is proved at $\mathrm{windowed\_prethreshold}$; another stage requires its own theorem or explicit stage transport.

Definition 4.86 (Higher-edge DAG edge)**.** A *higher-edge DAG edge* records a degree-n implication

$$\mathrm{Eval}_{n,W,\tau}(F) = 0 \Longrightarrow \mathrm{Ext}^n(A_n(F), k) = 0$$

supported by the complete n-eligible realization and edge package. The reverse direction is a separate edge requiring a natural zero-reflection or faithfulness theorem on the declared profile class.

Definition 4.87 (Finite-to-infinite reduction edge)**.** A *finite-to-infinite reduction edge* is a theorem-backed edge from certified finite data to an infinite, asymptotic, tower-level, analytic, completion, colimit, homotopy-colimit, or apex statement. It records the exact finite premise, infinite conclusion, convergence or stabilization theorem, scope, artifacts, and failure route.

Definition 4.88 (Return edge)**.** A *Return edge* is a proved AK-to-external implication licensing the exact external conclusion from the exact AK-side source node. It is directional and does not arise from realization, graph symmetry, or theorem-name matching.

Definition 4.89 (Source-binding edge)**.** A *source-binding edge* connects a source theorem node to an immutable statement artifact and exact locator. It records the source statement, hypotheses, source authority, permitted use, forbidden stronger reading, and source-artifact identity. Source retrieval or citation metadata alone are not theorem discharge.

Definition 4.90 (Replay edge)**.** A *replay edge* records reconstruction of a declared operational workflow or semantic object under a declared environment. It must declare whether the replay is byte-level, execution-level, operational, semantic, formal, or mathematical. A weaker replay mode does not discharge a stronger edge mode.

Proposition 4.91 (Proof-chain edge families are non-interchangeable)**.** *Realization, detector, higher-edge, reduction, Return, source-binding, and replay edges cannot substitute for one another merely because they occur on the same graph path.*

Proof. Each edge has a different domain, codomain, semantic direction, and discharge criterion. A complete path requires every invoked transformation at its own type. Substituting one edge family for another leaves the corresponding obligation undischarged. □

Remark 4.92 (Synthetic shift is not a higher bridge)**.** A shifted detector value may satisfy a tautological algebraic identity with an Ext^n-group. Unless a natural realization identifies it with the independently meaningful object $A_n(F)$, the corresponding node is a toy or synthetic node, not the principal higher-edge node.

X.7. Node discharge

Definition 4.93 (Discharged node)**.** A node is *discharged* if its required verification, proof, artifact, diagnostic, ledger, bridge, interpretation, formalization, or audit condition has been satisfied.

Definition 4.94 (Undischarged node)**.** A node is *undischarged* if at least one necessary condition remains unresolved.

Definition 4.95 (Discharge certificate)**.** A *discharge certificate* is a record explaining why a node is considered discharged. It must include:

(i) node identifier;

(ii) node type;

(iii) required conditions;

(iv) evidence supplied;

(v) verifier or verifying agent, if any;

(vi) verification environment, if any;

(vii) artifact references;

(viii) status label after discharge;

(ix) limitations or residual [Spec] assumptions, if any.

Definition 4.96 (Core node discharge)**.** A *Core node discharge* is the discharge of a Core DAG node through the relevant verifier-side data, such as repaired evaluation records, admissible representatives, eligibility records, tower comparison artifacts, monitored diagnostics, gate verdicts, and ledger checks.

Proposition 4.97 (Target node requires all necessary dependencies discharged)**.** *A target node in the Global Certificate DAG may be considered discharged only if all necessary dependency nodes and dependency edges are discharged.*

Proof. The target depends on its necessary dependency structure. If any necessary dependency remains unresolved, the target proof is incomplete. Therefore all necessary dependencies must be discharged. □

Proposition 4.98 (Core node discharge requires Core verifier data)**.** *A Core DAG node cannot be discharged merely by map status, agent assertion, or DAG placement. It requires the relevant Core verifier data.*

Proof. Core DAG nodes represent Core-facing objects or verdicts. By Core sovereignty and Appendix U, such nodes require verifier-side support. Map placement, agent assertion, or dependency layout does not supply that support. □

Remark 4.99 (Optional support is different)**.** Optional advisory or explanatory dependencies need not be discharged for theorem-level status unless explicitly marked as necessary.

X.7bis. Specialized v20 node discharge

Definition 4.100 (Detector node discharge)**.** A detector node is discharged only when its source stage, source profile, endpoint convention, soundness, completeness, coverage or characterization, family scope, excess provenance, non-adaptivity, artifact identity, certificate status, and mathematical value are all present and compatible with the consumed predicate. A finite observation, empty output, zero-looking sample, or successful computation is insufficient.

Definition 4.101 (Higher-edge node discharge)**.** A higher-edge node in degree n is discharged only when the exact repaired profile, representative, actual profile isomorphism, genuine realization or admissible replacement, amplitude, fixed extractor E_n, condition $E_n(0) = 0$, artifact-backed H^{-n}-identification, naturality scope, change compatibility, claim status, and non-circularity are verified.

Definition 4.102 (Reduction node discharge)**.** A finite-to-infinite reduction node is discharged only by a theorem at the exact finite premise and infinite target scope. Finite prefixes, plateaux, repeated barcodes, numerical convergence, Cauchy evidence, stored apex objects, graph paths, or replay success do not discharge the node unless the reduction theorem explicitly consumes them.

Definition 4.103 (Return node discharge)**.** A Return node is discharged only by a proved or validly imported soundness-out theorem whose source is the exact AK-side node and whose target is the exact external conclusion. The node records every external hypothesis and every prohibition on stronger reading.

Proposition 4.104 (Missing specialized discharge is undefined)**.** *If a required detector, higher-edge, reduction, or Return discharge record is missing or ill-typed, the corresponding necessary node or edge has result status* undefined.

Proof. The absence of a required record establishes neither the predicate nor its negation. It blocks the attempted route at the missing type. The weaker-safe atomic status is therefore undefined. □

Remark 4.105 (Finite observation versus finite proof)**.** The DAG may store finite observations for exploration and replay. Only a node carrying the complete theorem, typing, coverage, provenance, and artifact package is a finite proof-certificate node.

X.8. Global certificate assembly

Definition 4.106 (Global certificate assembly)**.** *Global certificate assembly* is the process of collecting discharged nodes, verified edges, ledger records, diagnostics, bridge theorems, interpretation theorems, and artifacts into a target certificate record or theorem-level proof object.

Definition 4.107 (Assembly policy)**.** An *assembly policy* declares:

(i) target node;

(ii) required dependency subgraph;

(iii) necessary and optional dependencies;

(iv) discharge conditions;

(v) ledger aggregation policy;

(vi) diagnostic inclusion policy;

(vii) bridge inclusion policy;

(viii) interpretation inclusion policy;

(ix) artifact completeness policy;

(x) audit requirements;

(xi) failure and demotion rules.

Definition 4.108 (Certificate assembly payload)**.** A *certificate assembly payload* is the collection of records assembled for the target certificate, including repaired evaluation records, representative records, eligibility records, tower artifacts, diagnostics, ledgers, bridge theorem references, artifact references, and manifest entries.

Proposition 4.109 (Global assembly requires manifest completeness)**.** *Global certificate assembly is admissible only if the resulting certificate record satisfies the Minimal Manifest Axiom.*

Proof. A global certificate is still a certificate record. By Appendix G, any certificate record must include the required declarations, repaired evaluation records, threshold and collapse policies, ledger policy, diagnostics, gate verdicts, and artifact references. Therefore manifest completeness is required. □

Remark 4.110 (Assembly is not invention)**.** Assembly collects and organizes already verified, proved, declared, or explicitly status-labeled components. It does not invent missing proof obligations.

X.8bis. Assembly budget and non-compensation

Definition 4.111 (Assembly ledger record)**.** Fix one selected admissible proof route P inside the target sub-DAG, where P is the finite necessary sub-DAG containing every branch consumed by the target certificate and need not be a single linear path. An *assembly ledger record* declares the active Appendix-M-compatible commutative quantale

$$(\mathcal{V}, \oplus, 0_{\mathcal{V}}, \preceq)$$

and uses only its finite ordered commutative-monoid reduct on the finite selected component set S_P:

$$\mathfrak{d}_{\mathrm{asm}}(P) := \bigoplus_{e \in S_P} \delta_e.$$

Every included δ_e must upper-bound an actual discrepancy between a declared pair of windowed pre-threshold profiles,

$$d_B\left(B_e^{\mathrm{src}}, B_e^{\mathrm{tgt}}\right) \le \mathrm{val}_B(\delta_e),$$

at the same scope consumed by the target. The record declares the protected family $\mathfrak{B}_P$, the bottleneck scalarization $\mathrm{val}_B : \mathcal{V} \to [0, \infty]$, with $\mathrm{val}_B(0_{\mathcal{V}}) = 0$, monotonicity, and subadditivity, and the strict comparison

$$\mathrm{val}_B\big(\mathfrak{d}_{\mathrm{asm}}(P)\big) < \mathrm{gap}_\tau(\mathfrak{B}_P).$$

If a different value system is used upstream, Core consumption requires a certified unit-preserving conversion into the declared quantale and a target-side profile-bound support record.

Definition 4.112 (Map-DAG assembly defect)**.** A *Map-DAG assembly defect* is classified before any ledger use as exactly one of:

(i) a primitive metric discrepancy with an actual profile bound;

(ii) an alias of an already registered metric discrepancy;

(iii) a refinement or aggregate with explicit primitive support;

(iv) a non-scalar obligation;

(v) an advisory or workflow quantity.

Local names such as

$$\delta^{\text{X-edge}}, \quad \delta^{\text{X-asm}}, \quad \delta^{\text{X-art}}, \quad \delta^{\text{X-interp}}, \quad \Delta_{\phi,\mathrm{DAG}}$$

enter $\mathfrak{d}_{\mathrm{asm}}(P)$ only in the first three cases and only after an Appendix-M relation

$$\mathrm{alias}, \qquad \mathrm{refinement}, \qquad \mathrm{aggregate}, \qquad \mathrm{disjoint}$$

has been declared.

Definition 4.113 (Assembly no-double-counting record)**.** An *assembly no-double-counting record* identifies the primitive discrepancy underlying every selected ledger component and proves that no primitive discrepancy is counted twice under Search-, Lifter-, Bridge-, storage-, replay-, or Map-DAG-local names. Unused alternative routes are omitted from the selected-route aggregate, while every required branch of the selected route is included.

Proposition 4.114 (Selected-route assembly is not all-route accumulation)**.** *The metric budget for a target is computed on the selected admissible proof route, not by summing every metric label appearing anywhere in the Validity Map.*

Proof. Alternative routes are not simultaneously consumed by the target certificate. Adding their discrepancies would not represent the actual comparison chain and could double-count shared primitive errors. Only supported components on the selected route enter the target ledger. □

Proposition 4.115 (Assembly budget cannot repair non-scalar obligations)**.** *No scalarized assembly budget can repair a missing proof, missing interpretation theorem, missing bridge theorem, missing Type IV artifact, missing terminal-tameness evidence, failed claim-use preflight, or artifact identity failure.*

Proof. These obligations are non-scalar. They concern proof, semantics, artifact identity, status, or typed morphism existence rather than bottleneck discrepancy. Ledger slack can bound metric discrepancies only when the required discrepancy theorem and scalarization record exist. □

Remark 4.116 (Map-DAG ledger catalog registration)**.** The symbols $\delta^{\text{X-edge}}$, $\delta^{\text{X-asm}}$, $\delta^{\text{X-art}}$, $\delta^{\text{X-interp}}$, and $\Delta_{\phi,\text{DAG}}$ remain local until related to the Search- and Bridge-layer symbols of Appendices V and W. A semantic mismatch, missing Return theorem, missing reduction, source-authority defect, or artifact-identity failure is non-scalar and must not be hidden inside a numerical component. Undeclared overlap is inadmissible because it can double-count or conceal a primitive discrepancy.

X.9. Validity Map versus Certificate DAG

Definition 4.117 (Map-DAG distinction)**.** The *Validity Map* is a broad navigation and status structure. The *Global Certificate DAG* is a proof-dependency structure for a specific target certificate or proof object.

Proposition 4.118 (Validity Map is broader than Certificate DAG)**.** *A Validity Map may contain nodes and edges that are not part of the Global Certificate DAG for a given target.*

Proof. A Validity Map includes candidates, rejected routes, advisory relations, search artifacts, failed bridge programs, deprecated nodes, and [Spec] statements. A Certificate DAG includes the dependencies relevant to a specific certificate target. Thus the map is broader. □

Definition 4.119 (Target sub-DAG)**.** A *target sub-DAG* is the dependency subgraph of the Global Certificate DAG required to support a specific target node.

Remark 4.120 (Do not over-certify the map)**.** Only the target certificate sub-DAG requires discharge for a given certificate. The entire Validity Map need not be fully discharged.

X.9bis. Pairwise edges and higher coherence

Definition 4.121 (Pairwise compatibility subgraph)**.** A *pairwise compatibility subgraph* records discharged pairwise overlap, comparison, replacement, or transport edges among declared local objects. It does not by itself contain triple-overlap, simplex, descent, homotopy-coherence, or global assembly data.

Definition 4.122 (Higher-coherence node)**.** A *higher-coherence node* records a separately proved compatibility condition on triples, higher simplices, descent data, homotopies, or another declared coherence structure required by the target.

Proposition 4.123 (Pairwise closure is not higher coherence)**.** *Discharge of every pairwise edge in a finite subgraph does not discharge a required higher-coherence or descent node.*

Proof. Pairwise compatibility constrains two objects at a time. Higher coherence imposes additional conditions on composites, triples, simplices, or descent data. Those conditions are not logical consequences of pairwise discharge without a separate theorem. □

Remark 4.124 (Graph completion does not create descent)**.** A complete underlying graph is a combinatorial property. It is not a proof of categorical, homotopical, sheaf-theoretic, or persistence-theoretic descent.

X.10. Failure and demotion

Definition 4.125 (Map failure)**.** A *Map failure* occurs when a Validity Map node, edge, status label, artifact reference, or transition record is incorrect, ambiguous, stale, unsupported, or inconsistent.

Definition 4.126 (DAG failure)**.** A *DAG failure* occurs when a Certificate DAG contains an unresolved necessary dependency, invalid edge, undischarged required node, forbidden cycle, inconsistent status, missing artifact, failed bridge, invalid interpretation edge, or unsupported Core node.

Definition 4.127 (Demotion event)**.** A *demotion event* is a recorded downgrade of a node, edge, subgraph, map region, or certificate status due to discovered failure, artifact drift, unresolved dependency, invalid interpretation, or bridge failure.

Definition 4.128 (Subgraph quarantine)**.** *Subgraph quarantine* is the temporary isolation of a map or DAG subgraph whose status is uncertain, inconsistent, stale, or under review.

Proposition 4.129 (DAG failure blocks target certification)**.** *A DAG failure in a necessary dependency subgraph blocks target certification until repaired.*

Proof. A necessary dependency subgraph is required for the target certificate. If it contains failure, the target certificate is unsupported. Repair, replacement, demotion, or removal of the failed dependency is required. □

Remark 4.130 (Demotion protects integrity)**.** Demotion is not a failure of the platform. It is a safety mechanism that prevents stale, invalid, or over-promoted claims from remaining active at too strong a status.

X.10bis. Typed failure routing

Definition 4.131 (Map-DAG atomic result)**.** Each invoked node and necessary edge has exactly one atomic result:

$$\mathrm{pass}, \qquad \mathrm{reject}, \qquad \mathrm{undefined},$$

or $\mathrm{not\_invoked}$ when an immutable target-scope policy excludes the obligation. Workflow labels such as $\mathrm{pending}$, $\mathrm{blocked}$, or $\mathrm{under\_review}$ do not replace the atomic result.

Definition 4.132 (Undefined dependency)**.** An *undefined dependency* is a required node or edge whose identity, type, theorem, artifact, scope, or discharge evidence is missing, stale, incompatible, or not reconstructible. It is not a proof of falsity.

Definition 4.133 (Rejected dependency)**.** A *rejected dependency* is an invoked node or edge for which the declared verifier-side predicate is evidenced false or the consumed theorem conditions are evidenced not to hold.

Proposition 4.134 (Missing evidence is not obstruction value)**.** *An undefined dependency is not encoded as a numerical defect, zero diagnostic, failed theorem, or numbered Type I–V obstruction.*

Proof. Missing evidence does not determine a mathematical value. Numerical defects and typed obstruction classes require their own source objects and evidence. The correct result is $\mathrm{undefined}$, together with a failure route naming the missing obligation. □

Remark 4.135 (Not invoked requires scope policy)**.** The token $\mathrm{not\_invoked}$ is admissible only when a frozen target-scope policy proves that the node or edge is unnecessary. It is not a synonym for missing, inconvenient, unimplemented, or failed.

X.11. Artifact consistency

Definition 4.136 (Artifact consistency)**.** A node or edge has *artifact consistency* if every referenced artifact exists, has the declared identity, has the declared version, and supports the claim or relation for which it is cited.

Definition 4.137 (Artifact drift)**.** *Artifact drift* occurs when a referenced artifact changes, disappears, is superseded, becomes inconsistent, or no longer supports the node or edge that cites it.

Definition 4.138 (Artifact identity record)**.** An *artifact identity record* is a record of an artifact's identifier, version, hash or equivalent integrity marker, source, status, and supported nodes or edges.

Definition 4.139 (Artifact support relation)**.** An *artifact support relation* declares which claim, node, edge, diagnostic, ledger, proof, or certificate component an artifact supports.

Proposition 4.140 (Artifact drift requires review)**.** *If artifact drift is detected in a necessary dependency, the affected node or edge must be reviewed and may need demotion.*

Proof. Artifact drift may invalidate the evidence supporting a node or edge. Until reviewed, the dependency is not audit-stable. Thus review and possible demotion are required. □

Remark 4.141 (Hashing and versioning)**.** Hashing, versioning, immutable identifiers, and proof-store references are platform mechanisms for controlling artifact drift. They are developed further in the execution and reproducibility appendices.

X.11bis. Artifact identity and replay limits

Definition 4.142 (Artifact identity preflight)**.** An *artifact identity preflight* checks that every necessary artifact has the declared identifier, version, hash or integrity marker, source, status, authority scope, and support relation before a node or edge is consumed.

Definition 4.143 (Replay-success node)**.** A *replay-success node* records that a declared workflow replayed under a declared environment. It establishes reproducibility of the replayed workflow at that scope, not mathematical truth beyond the replayed criteria.

Proposition 4.144 (Replay success is not proof by itself)**.** *A replay-success node does not prove a theorem unless the replayed workflow includes the required proof or verifier-side checks for that theorem.*

Proof. Replay verifies reproducibility of an execution or reconstruction. Mathematical proof requires the relevant theorem evidence, bridge evidence, diagnostic evidence, ledger comparison, and artifact checks. A replay may include such checks, but replay status alone is not identical to theorem proof. □

Proposition 4.145 (Artifact identity failure blocks necessary dependencies)**.** *If a necessary node or edge fails artifact identity preflight, the target dependency remains undischarged.*

Proof. The dependency is supported by a particular artifact identity. If that identity is missing, stale, mismatched, or unsupported, the claimed support cannot be reconstructed. Thus the dependency cannot be consumed. □

Remark 4.146 (Storage is not support)**.** A platform may store an artifact without the artifact supporting the claim for which it is cited. The support relation must be declared and checked separately.

X.11ter. Operational, semantic, and mathematical replay

Definition 4.147 (Operational replay node)**.** An *operational replay node* records that code, tools, prompts, models, configurations, seeds, dependencies, and artifacts reproduced a declared output under a declared replay boundary.

Definition 4.148 (Semantic replay node)**.** A *semantic replay node* records that the replayed output was reconstructed as the same declared mathematical or interface object under an explicit interpretation and semantic-diff protocol. It is stronger than byte or execution replay but is not automatically a theorem proof.

Definition 4.149 (Mathematical reconstruction node)**.** A *mathematical reconstruction node* records independent reconstruction of the consumed proof, verifier-side certificate, bridge theorem, reduction, or Return theorem at the declared hypotheses and scope.

Proposition 4.150 (Replay modes are strictly separated)**.** *Operational replay does not imply semantic replay, and semantic replay does not imply mathematical reconstruction.*

Proof. Operational replay concerns execution reproducibility. Semantic replay additionally concerns object identity and interpretation. Mathematical reconstruction additionally checks the proof obligations and theorem strength. Each implication requires extra evidence and cannot be inferred from the weaker mode. □

Remark 4.151 (Model and prompt provenance)**.** When AI or stochastic tools participate, replay provenance records the model or tool identity, prompt or task, system instructions when relevant, retrieval corpus, parameters, random seeds when available, post-processing, and immutable input/output artifacts. These fields support reconstruction but do not convert model output into proof.

X.12. Bridge Programs inside the DAG

Definition 4.152 (Bridge Program DAG node)**.** A *Bridge Program DAG node* is a DAG node representing a Bridge Program, bridge candidate, bridge theorem, bridge lemma graph, or bridge discharge record.

Definition 4.153 (Bridge theorem discharge edge)**.** A *bridge theorem discharge edge* is an edge showing that a bridge theorem discharges a bridge dependency required by a target node.

Definition 4.154 (Bridge Program blocked edge)**.** A *Bridge Program blocked edge* is an edge showing that an unresolved, failed, or [Spec] Bridge Program blocks theorem-level transport.

Proposition 4.155 (Bridge Program node does not discharge bridge dependency by existence)**.** *A Bridge Program DAG node does not discharge a bridge dependency unless it is supported by a bridge theorem, valid import, or verified discharge record.*

Proof. By Appendix W, a Bridge Program is not a bridge theorem. Therefore its presence in the DAG records a proof program or dependency, but does not discharge the dependency by itself. □

Remark 4.156 (Bridge Program placement)**.** Bridge Programs are useful DAG nodes because they expose what must be proved. Their status must remain distinct from bridge theorem nodes.

X.12bis. Bridge and Type IV nodes inside the DAG

Definition 4.157 (DAG Type IV artifact node)**.** A *DAG Type IV artifact node* records an actual comparison artifact

$$\phi_{i,W,\tau} : \mathrm{ColimProfile}_{i,W,\tau}(F_\bullet) \longrightarrow \mathrm{ApexProfile}_{i,W,\tau}(F_\infty)$$

together with source and target identities, monitored scope, terminal-tameness evidence, and diagnostic computation.

Definition 4.158 (DAG Type IV zero node)**.** A *DAG Type IV zero node* is a discharged diagnostic node asserting certified zero Type IV status for the declared monitored scope. It may be used only when the actual artifact node exists, terminal-tameness is verified, and

$$\mu_{\mathrm{Collapse}} = \sum_{i \in I_{\mathrm{mon}}} \mathrm{tgdim}_W \ker(\phi_{i,W,\tau}) = 0,$$
$$u_{\mathrm{Collapse}} = \sum_{i \in I_{\mathrm{mon}}} \mathrm{tgdim}_W \operatorname{coker}(\phi_{i,W,\tau}) = 0.$$

The node must point to the full typed Type IV diagnostic record of Chapter 5. Whenever the interior-defect field is part of the invoked schema, it carries one declared status:

$$\mathrm{not\_checked}, \qquad \mathrm{certified\_absent}, \qquad \mathrm{certified\_present}.$$

A zero node is therefore a pointer to a complete diagnostic record, not a reduced replacement schema.

Definition 4.159 (DAG long-transient node)**.** A *DAG long-transient node* records the separately invoked status of a complete detector on the full kernel or cokernel defect profile over the declared long-transient scope. Its status is exactly one of

$$\mathrm{not\_invoked}, \quad \mathrm{undefined}, \quad \mathrm{certified\_absent}, \quad \mathrm{certified\_present}.$$

It is not inferred from terminal-generic dimensions.

Definition 4.160 (DAG full-interior defect node)**.** A *DAG full-interior defect node* records the status

$$\mathrm{not\_checked}, \qquad \mathrm{certified\_absent}, \qquad \mathrm{certified\_present}$$

for the invoked full-interior kernel or cokernel predicate. It is distinct from both terminal Type IV and long-transient status.

Proposition 4.161 (Terminal zero does not discharge transient or interior nodes)**.** *A discharged Type IV zero node does not discharge a necessary long-transient or full-interior defect node.*

Proof. Terminal Type IV uses terminal-generic dimensions of the kernel and cokernel. Long-transient and full-interior predicates inspect different portions of the defect profiles and require their own complete detector records. Terminal zero therefore supplies no automatic discharge of those nodes. □

Proposition 4.162 (Bridge Program path is not bridge theorem path)**.** *A path through Bridge Program nodes does not discharge a bridge dependency unless the path contains a discharged bridge theorem, valid import, or verified bridge-discharge record at the required semantic direction.*

Proof. Appendix W separates Bridge Programs from bridge theorems. A path through programs records planning and dependencies. It does not establish the theorem-level implication without a discharged bridge theorem or equivalent verified record. □

Proposition 4.163 (Type IV placement does not imply zero)**.** *Placing a Type IV artifact candidate, proxy, or diagnostic placeholder in a map or DAG does not imply certified zero Type IV status.*

Proof. Certified zero Type IV status requires an actual comparison artifact, terminal-tameness, monitored scope, and terminal-generic kernel and cokernel computations. A placeholder or candidate node lacks those data by default. □

Remark 4.164 (Undefined Type IV is not zero in the DAG)**.** A missing or ill-typed Type IV artifact node has status $\mathrm{undefined}$ or $\mathrm{not\_invoked}$; the DAG must not encode it as a zero diagnostic node.

X.12ter. Finite-to-infinite state separation

Definition 4.165 (Finite-to-infinite state family)**.** The DAG distinguishes the following node types:

(i) finite observation;

(ii) finite proof certificate;

(iii) certified finite prefix;

(iv) numerical convergence evidence;

(v) Cauchy or compactness record;

(vi) eventual-constancy record;

(vii) colimit, homotopy-colimit, completion, or apex object;

(viii) apex-agreement theorem;

(ix) finite-to-infinite reduction theorem.

No node in this list is identified with another by naming, storage, or graph proximity.

Definition 4.166 (Apex-agreement edge)**.** An *apex-agreement edge* is a theorem-backed edge identifying the declared infinite construction with the declared apex semantics at the required scope. A stored apex object, an eventually constant finite diagram, or a zero terminal defect does not create this edge automatically.

Proposition 4.167 (Finite prefix is not reduction)**.** *A certified finite prefix, plateau, repeated profile, or numerical convergence node does not discharge a finite-to-infinite reduction node.*

Proof. The finite node records data on a bounded or finite scope. A reduction theorem proves that those data control the declared infinite target. Without that theorem, future births, nonuniform behavior, completion effects, or apex mismatch remain possible. □

Remark 4.168 (Right-unbounded no-go boundary)**.** On a right-unbounded scope with unbounded possible birth locations, the DAG must display the additional stabilization, finite-birth, global-bound, termination, or reduction structure that defeats the finite-detector no-go theorem. A finite stored prefix alone is not such structure.

X.12quater. Known-Theorem Recovery sub-DAGs

Definition 4.169 (Known-Theorem Recovery sub-DAG)**.** A *Known-Theorem Recovery sub-DAG* is a Return-complete target sub-DAG whose external target is a frozen known theorem statement at an exact registered strength. It contains separate nodes for source binding, precursor inputs, purpose-faithful realization, Core certificate, reduction when required, Return, and independent benchmark validation.

Definition 4.170 (Recovery quarantine cut)**.** A *recovery quarantine cut* prevents the benchmark proof, known truth value, answer table, target constants, exceptional bounds, conclusion-derived witness, equivalent oracle, or independent validation artifact from flowing into the realization, detector, certificate, reduction, higher-edge proof, or Return proof. The immutable benchmark statement and locator may identify the exact target and Return consequent. Only registered precursor hypotheses and independently permitted source data may cross into the construction side.

Definition 4.171 (Recovery validation edge)**.** A *recovery validation edge* compares the independently reconstructed external conclusion with the frozen benchmark statement at exact strength. It is a terminal validation edge and is not a source of proof data for upstream nodes.

Proposition 4.172 (DAG representation does not regenerate recovery proof)**.** *Encoding a known-theorem recovery track in the Global Certificate DAG does not regenerate, strengthen, or independently prove the recovered theorem.*

Proof. The DAG records the already proved or imported arrows and their dependencies. The mathematical strength comes from the source-bound realization, certificate, reduction, Return, and validation theorems. Graph representation contributes organization and audit only. □

Remark 4.173 (v20 reference recovery tracks)**.** Track IW-KTR-GROWTH is represented only as the exact characteristic-growth-clause recovery relative to its registered precursor package. Track NS-KTR-SERRIN-L6 is represented only as the fixed strict-subcritical criterion-level recovery. No Iwasawa main conjecture, class-number specialization, unconditional Navier–Stokes regularity, Clay-level result, concrete Fukaya/HMS Bridge, or concrete external AGI-N Bridge is inferred from their DAG presence.

X.13. Formalization and interpretation inside the DAG

Definition 4.174 (Formalization DAG node)**.** A *formalization DAG node* is a node representing a formal statement, formal fragment, proof assistant file, verified formal theorem, or formal environment.

Definition 4.175 (Interpretation theorem node)**.** An *interpretation theorem node* is a node representing a theorem that connects a verified formal statement to the intended informal or problem-specific claim.

Proposition 4.176 (Verified formal node does not imply intended claim without interpretation)**.** *A verified formal node does not discharge an intended informal or problem-specific claim unless the required interpretation edge is discharged.*

Proof. A proof assistant verifies the formal statement in its formal environment. The intended informal claim may have different semantics, hypotheses, or scope. An interpretation theorem is required to connect them. □

Remark 4.177 (Formalization mismatch tracking)**.** Formalization mismatch should appear in the Validity Map and Certificate DAG as either an undischarged interpretation edge, a blocked bridge, or a demotion event.

X.14. Validity Map manifest entries

Definition 4.178 (Validity Map manifest entry)**.** A *Validity Map manifest entry* records:

(i) map identifier;

(ii) node set;

(iii) edge set;

(iv) node status labels;

(v) edge types;

(vi) Core-facing nodes;

(vii) finite-detector, higher-edge, reduction, Return, and Type IV nodes;

(viii) Bridge Program and Known-Theorem Recovery nodes;

(ix) source-binding, quarantine, formalization, interpretation, and replay nodes;

(x) artifact references;

(xi) status transition history;

(xii) demotion events;

(xiii) quarantined subgraphs, if any;

(xiv) audit notes.

Definition 4.179 (Certificate DAG manifest entry)**.** A *Certificate DAG manifest entry* records:

(i) DAG identifier;

(ii) target node;

(iii) required dependency subgraph;

(iv) node discharge statuses;

(v) edge discharge statuses;

(vi) Core node discharge records;

(vii) ledger dependencies;

(viii) diagnostic dependencies;

(ix) bridge and realization dependencies;

(x) finite-detector and higher-edge dependencies;

(xi) finite-to-infinite, apex-agreement, and Return dependencies;

(xii) source-binding, quarantine, interpretation, formalization, and replay dependencies;

(xiii) artifact references and artifact-identity preflight;

(xiv) selected admissible proof route and no-double-counting ledger record;

(xv) assembly policy;

(xvi) local discharge status, final-CR promotion status, and audit result.

Proposition 4.180 (Map and DAG manifests support audit)**.** *Validity Map and Certificate DAG manifest entries support audit by recording structure, status, dependencies, artifacts, discharge conditions, and demotion history.*

Proof. Audit requires reconstructing what was used, how it was connected, which dependencies were necessary, and whether those dependencies were discharged. The map and DAG manifests record these data. □

Remark 4.181 (Manifest is not proof)**.** A map or DAG manifest records structure and status. It does not prove the target unless the referenced proof, verification, interpretation, and artifact data are complete.

X.15. Map and DAG invariants

Definition 4.182 (Map-DAG invariant)**.** A *Map-DAG invariant* is a rule that must remain true across Validity Map and Certificate DAG updates.

Definition 4.183 (Typed-edge invariant)**.** The *typed-edge invariant* states that every map or DAG edge must have declared semantics.

Definition 4.184 (Evidence-promotion invariant)**.** The *evidence-promotion invariant* states that no node may be promoted to a stronger status without recorded evidence.

Definition 4.185 (Core-sovereignty invariant)**.** The *Core-sovereignty invariant* states that no map or DAG structure may replace Core verifier conditions.

Definition 4.186 (Bridge-status invariant)**.** The *bridge-status invariant* states that bridge drafts, bridge candidates, Bridge Programs, and bridge theorem nodes must remain status-distinct.

Definition 4.187 (Artifact-consistency invariant)**.** The *artifact-consistency invariant* states that necessary nodes and edges must maintain artifact consistency or be reviewed and possibly demoted.

Definition 4.188 (Reduction-Return invariant)**.** The *reduction-Return invariant* states that no infinite or external target is discharged from a finite Core node unless every required finite-to-infinite and Return edge is separately discharged.

Definition 4.189 (Detector-stage invariant)**.** The *detector-stage invariant* states that every detector node retains exactly one source stage and that no map update, lift, normalization, or edge rerouting silently changes that stage.

Definition 4.190 (No-double-counting invariant)**.** The *no-double-counting invariant* states that each primitive metric discrepancy occurs at most once in the selected-route ledger aggregate, regardless of the number of local aliases or graph layers that reference it.

Definition 4.191 (Source-quarantine invariant)**.** The *source-quarantine invariant* states that a benchmark conclusion or conclusion-derived witness never flows upstream into a recovery construction unless the target theorem explicitly permits that datum as an independent hypothesis.

Definition 4.192 (Final-registration invariant)**.** The *final-registration invariant* states that local graph discharge does not create canonical Claim UIDs or final release rows before the final-CR extraction boundary.

Proposition 4.193 (Map-DAG invariants prevent proof inflation)**.** *The typed-edge, evidence-promotion, Core-sovereignty, bridge-status, artifact-consistency, reduction-Return, detector-stage, no-double-counting, source-quarantine, and final-registration invariants prevent navigation structures from being inflated into proof or release promotion.*

Proof. Typed edges prevent ambiguous arrows. Evidence-promotion blocks unsupported status upgrades. Core sovereignty blocks map/DAG replacement of verifier-side conditions. Bridge-status separation blocks Bridge Program inflation. Artifact consistency blocks stale dependency support. Reduction-Return separation blocks finite or internal evidence from being read as an infinite external theorem. Detector-stage preservation blocks semantic relabeling. No-double-counting preserves metric meaning. Source quarantine blocks circular recovery. Final-registration separation blocks local graph status from becoming release governance. Together, these invariants preserve proof and registry integrity. □

X.15bis. v20 compatibility Map-DAG hardening package

Theorem 4.194 (Appendix X v20 compatibility map-DAG hardening package)**.** *Within the Search / Platform layer, the inherited map-DAG hardening package remains available at its repaired v20 strength. In addition:*

(i) proof-dependency, governance, provenance, search, and release graphs are type-distinct;

(ii) graph reachability is not theorem composition;

(iii) target use is controlled by a verifier-side target projection and necessary-edge closure;

(iv) a Problem-Bridge target separates realization, finite certificate, reduction, higher edge, Return, and external conclusion;

(v) a Core-certified node is not an external conclusion;

(vi) detector nodes preserve exactly one source stage and separate certificate status from mathematical value;

(vii) higher-edge nodes require the complete degree-n eligible realization and edge package;

(viii) finite-to-infinite nodes distinguish finite observations, certificates, prefixes, convergence, stabilization, apex semantics, apex agreement, and reduction;

(ix) terminal Type IV, long-transient, and full-interior defect nodes are separate;

(x) pairwise graph closure does not create higher coherence or descent;

(xi) selected-route metric assembly uses actual profile discrepancy bounds in the finite monoid reduct of the declared quantale;

(xii) local aliases declare alias, refinement, aggregate, *or* disjoint *relations and satisfy no-double-counting;*

(xiii) scalarized budget slack does not repair a missing theorem, morphism, artifact, source binding, reduction, Return, or claim-use permission;

(xiv) operational replay, semantic replay, and mathematical reconstruction are separate node types;

(xv) Known-Theorem Recovery sub-DAGs preserve source, precursor, and benchmark quarantine;

(xvi) local discharge does not create canonical Claim UIDs or release promotion before final-CR extraction;

(xvii) missing evidence yields undefined*;* not_invoked *requires a frozen scope-exclusion policy;*

(xviii) stable legacy local labels remain construction anchors until final cross-document extraction.

Proof. The graph-domain and composition claims follow from Definitions 4.10, 4.11, and 4.12, and Proposition 4.13. Problem-Bridge separation follows from Definitions 4.26–4.28 and Proposition 4.29. Detector, higher-edge, reduction, and Return discharge follow from Definitions 4.85–4.88 and 4.100–4.103. Type IV and finite-to-infinite separation follow from Definitions 4.157, 4.158, 4.159, 4.160, and 4.165, together with Propositions 4.161 and 4.167. The selected-route ledger and noncompensation claims follow from Definitions 4.111–4.113 and Propositions 4.114 and 4.115. Replay and recovery claims follow from Definitions 4.147–4.149 and 4.169–4.171, and Propositions 4.150 and 4.172. Final registration and typed failure routing follow from Definitions 4.43, 4.44, 4.131, and 4.132, and Propositions 4.45 and 4.134. □

X.16. Summary of Validity Map and Global Certificate DAG

Theorem 4.195 (Appendix X v20 map-DAG package)**.** *Within the Search / Platform layer, the following package is available.*

(i) A Validity Map is navigation and status control, not theorem evidence.

(ii) A target Certificate DAG is a typed proof and audit skeleton, not proof by graph existence.

(iii) Proof-dependency edges are separated from search, governance, provenance, storage, replay, and release edges.

(iv) Every consumed node and edge carries compatible mathematical, workflow, evidence, invocation, conformance, artifact, and atomic-status data.

(v) Graph reachability, short paths, dense clusters, and complete pairwise graphs do not establish composable theorem chains or higher coherence.

(vi) A target sub-DAG is discharged only through target-local necessary-edge closure.

(vii) A Problem-Bridge target separately discharges realization, Core readability, finite certification, every required finite-to-infinite reduction, every invoked higher edge, Return, and the external conclusion.

(viii) Detector nodes preserve exactly one source stage and require separate soundness, completeness, coverage, certificate-status, and mathematical-value records.

(ix) Higher obstruction nodes are degree-n and use the standard one-way edge by default; reverse zero reflection is a separate theorem edge.

(x) Type IV nodes require an actual comparison morphism and separate terminal, long-transient, and full-interior statuses.

(xi) Finite observations, certificates, prefixes, convergence records, eventual constancy, colimit or apex objects, apex agreement, and finite-to-infinite reduction are distinct nodes.

(xii) Global assembly uses only supported primitive metric discrepancies on the selected admissible proof route and satisfies no-double-counting.

(xiii) The scalarized target ledger must upper-bound actual windowed pre-threshold bottleneck discrepancies and remain below the declared threshold gap.

(xiv) Metric slack cannot repair non-scalar theorem, morphism, interpretation, source, artifact, reduction, Return, or governance obligations.

(xv) Operational replay, semantic replay, formal checking, and mathematical reconstruction are distinct discharge modes.

(xvi) Known-Theorem Recovery sub-DAGs preserve source binding, precursor separation, benchmark quarantine, exact-strength Return, and terminal validation.

(xvii) Missing evidence is undefined*; evidenced false conditions are* reject*;* not_invoked *requires a frozen exclusion policy.*

(xviii) Local discharge and manifest completeness do not create final CR-v20 promotion.

(xix) Artifact drift, semantic mismatch, invalid interpretation, source leakage, or dependency failure triggers quarantine, demotion, or route rejection without mutating historical provenance.

(xx) Map and DAG manifests support independent reconstruction but do not themselves prove any mathematical target.

Proof. Items (i)–(vi) follow from the sovereignty, graph-domain, edge-typing, target-scope, and necessary-edge closure results of Sections X.1bis–X.6bis. Items (vii)–(xi) follow from the Problem-Bridge, detector, higher-edge, Type IV, and finite-to-infinite definitions and discharge propositions of Sections X.2ter, X.6ter, X.7bis, X.12bis, and X.12ter. Items (xii)–(xiv) follow from the selected-route assembly ledger, no-double-counting, threshold-gap, and noncompensation results of Section X.8bis. Items (xv)–(xvi) follow from the replay and Known-Theorem Recovery packages of Sections X.11ter and X.12quater. Items $(xvii)$–(xix) follow from the typed failure, local-discharge, promotion-deferral, quarantine, demotion, and artifact-consistency disciplines. Item (xx) follows from Proposition 4.180 and Remark 4.181. □

X.17. Explicit exclusions

The following are not asserted in Appendix X.

- No Validity Map, Certificate DAG, graph path, layout, centrality score, shortest path, or complete subgraph is treated as theorem evidence by existence.
- No search, governance, provenance, storage, replay, supersession, or release edge is silently consumed as a proof-dependency edge.
- No untyped, misrouted, advisory, optional, citation, or artifact edge discharges a necessary logical, detector, metric, bridge, reduction, Return, interpretation, or formal-proof obligation.
- No graph reachability statement is treated as a composable implication without compatible source and target types, directions, hypotheses, artifacts, and discharge modes.
- No pairwise-complete graph is treated as higher coherence, descent, or global assembly.
- No dependency cycle is accepted without an explicit valid cyclic proof principle.
- No node or edge is promoted without a transition certificate, claim-use preflight, artifact-identity check, and exact-strength evidence.
- No local discharge, manifest-complete graph, dashboard pass, generated UID, JSON row, or replay result is treated as final CR-v20 promotion.
- No Core-facing node is treated as Core verified merely by appearing in a map or DAG.
- No map or DAG node replaces B-Gate^{+}, an admissible representative, a required eligibility record, an actual tower comparison morphism, a typed diagnostic, or the metric threshold-gap condition.
- No finite detector node is accepted without exactly one source stage and separate soundness, completeness, coverage, certificate-status, mathematical-value, provenance, and artifact fields.
- No detector theorem is silently transferred among the four declared source stages; every nonstandard stage use requires its own theorem or explicit stage transport.
- No finite observation, empty result, zero-looking output, sampled plateau, repeated barcode, or successful computation is treated as a finite proof certificate by finiteness alone.
- No higher-edge node is discharged by a synthetic shift, target-shaped encoding, object-name match, or isolated Ext^{n}-calculation without the standard n-eligible realization and edge package.

- No reverse implication
$$\mathrm{Ext}^n(A_n(F), k) = 0 \Longrightarrow \mathrm{Eval}_{n,W,\tau}(F) = 0$$
is inferred without a separately proved natural zero-reflection or faithfulness theorem.
- No Type IV candidate, proxy, barcode match, object equality, Betti-number match, or stored representative creates the actual morphism
$$\phi_{i,W,\tau} : \mathrm{ColimProfile}_{i,W,\tau}(F_\bullet) \longrightarrow \mathrm{ApexProfile}_{i,W,\tau}(F_\infty).$$
- No missing, undefined, or not-invoked Type IV record is encoded as $(\mu_{\mathrm{Collapse}},\ u_{\mathrm{Collapse}}) = (0, 0)$.
- No terminal Type IV zero node discharges a long-transient, full-interior, full-defect-zero, isomorphism, or apex-agreement node.
- No finite prefix, numerical convergence record, Cauchy evidence, eventual plateau, stored colimit, completion, homotopy-colimit, or apex object is treated as a finite-to-infinite reduction theorem.
- No Core certificate, higher-edge result, reduction node, or visually completed path is treated as an external-domain conclusion without a separately discharged Return theorem.
- No realization edge is reversed by graph symmetry or used as a Return edge.
- No selected-route metric ledger contains an unsupported advisory quantity, semantic mismatch, missing theorem, artifact failure, source-authority defect, reduction gap, or Return gap.
- No primitive metric discrepancy is counted more than once under Search-, Lifter-, Bridge-, storage-, replay-, or Map-DAG-local aliases.
- No unused alternative path is added to the target metric budget.
- No metric slack repairs a missing proof, morphism, interpretation, source binding, terminal tameness, claim-use permission, artifact identity, finite-to-infinite reduction, or Return theorem.
- No operational replay implies semantic replay, and no semantic replay implies mathematical reconstruction.
- No proof-assistant success, build success, serialization success, or semantic-diff success proves a stronger informal theorem without the corresponding interpretation and proof records.
- No source citation, retrieved passage, theorem name, or benchmark statement is consumed without immutable source binding and exact permitted use.
- No benchmark conclusion, answer table, conclusion-derived witness, or validation artifact flows upstream into a Known-Theorem Recovery construction.
- No DAG representation of IW-KTR-GROWTH strengthens it beyond the registered characteristic-growth clause relative to its precursor package.
- No DAG representation of NS-KTR-SERRIN-L6 yields a new analytic proof, unconditional regularity, or Clay-level conclusion.
- No concrete Fukaya/HMS or external AGI-N Bridge is inferred from a candidate, schema, graph node, or platform record when its invocation status is $\mathrm{not\_invoked}$.
- No artifact storage event is treated as an artifact support relation, and no stale artifact is silently consumed.
- No missing evidence is converted into a numerical obstruction, failed theorem, certified zero, or numbered Type I–V defect; its atomic result is $\mathrm{undefined}$.
- No $\mathrm{not\_invoked}$ status is assigned without an immutable target-scope exclusion policy.

X.18. Provenance and status

The material in this appendix depends on:

(i) the v20 claim taxonomy, finite-certificate contract, higher-edge contract, Problem-Bridge chain, and manifest discipline of Chapters 1–8;

(ii) the current CR-v20 baseline for claim-use governance, with final promotion deferred;

(iii) Appendix A for window-first repaired evaluation, hard deletion, exact transport, and two-sided threshold-gap safety;

(iv) Appendix B for representatives, detector stages, completeness, and stage transport;

(v) Appendix C for the degree-n eligible realization and higher-edge package;

(vi) Appendix D for actual-morphism Type IV, terminal, transient, full-interior, and finite-to-infinite tower discipline;

(vii) Appendix E for coverage, atlas, overlap, Restart, higher coherence, and local-to-global assembly when invoked;

(viii) Appendix F for atomic status, typed failure routing, and noncompensation;

(ix) Appendix G for canonical manifest and verifier discipline;

(x) Appendices U, V, and W for agent semantics, search protocols, and Bridge Programs;

(xi) Appendix M for the commutative-quantale ledger, scalarization, profile support, and no-double-counting discipline;

(xii) Technical Extension Appendices H–N for advisory, discretization, controlled comparison, ledger, and enhancement re-entry at frozen local strength;

(xiii) Problem Interface Part IV for realization, reduction, Return, source-binding, and Known-Theorem Recovery examples;

(xiv) Appendix NS-C for target-first proof-route decomposition and residual-cut discipline;

(xv) the frozen source-binding registry and manifest for Track IW-KTR-GROWTH;

(xvi) the frozen source-binding registry and manifest for Track NS-KTR-SERRIN-L6.

Its role is to provide target-local map and DAG structures for proof planning, typed dependency tracking, Bridge and Return governance, certificate assembly, recovery quarantine, replay audit, and release-promotion deferral without confusing navigation, reproducibility, or registration with proof.

Status-label census for Appendix X. The following register contains every claim-bearing local label in this appendix exactly once. The section anchor app:X is excluded from the claim census. Local labels remain stable construction identifiers until final CR-v20 extraction.
Search / Platform and v20 definitions: Definitions 4.5, 4.6, 4.10, 4.11, 4.12, 4.15, 4.16, 4.17, 4.18, 4.21, 4.22, 4.23, 4.26, 4.27, 4.28, 4.31, 4.32, 4.33, 4.34, 4.35, 4.38, 4.39, 4.43, 4.44, 4.47, 4.48, 4.49, 4.50, 4.51, 4.54, 4.55, 4.58, 4.59, 4.62, 4.63, 4.64, 4.68, 4.69, 4.70, 4.71, 4.72, 4.73, 4.74, 4.75, 4.76, 4.77, 4.80, 4.81, 4.84, 4.85, 4.86, 4.87, 4.88, 4.89, 4.90, 4.93, 4.94, 4.95, 4.96, 4.100, 4.101, 4.102, 4.103, 4.106, 4.107, 4.108, 4.111, 4.112, 4.113, 4.117, 4.119, 4.121, 4.122, 4.125, 4.126, 4.127, 4.128, 4.131, 4.132, 4.133, 4.136, 4.137, 4.138, 4.139, 4.142, 4.143, 4.147, 4.148, 4.149, 4.152, 4.153, 4.154, 4.157, 4.158, 4.159, 4.160, 4.165, 4.166, 4.169, 4.170, 4.171, 4.174, 4.175, 4.178, 4.179, 4.182, 4.183, 4.184, 4.185, 4.186, 4.187, 4.188, 4.189, 4.190, 4.191, 4.192.

Search / Platform and v20 propositions: Propositions 4.7, 4.8, 4.13, 4.19, 4.24, 4.29, 4.36, 4.40, 4.41, 4.45, 4.52, 4.56, 4.60, 4.65, 4.67, 4.78, 4.82, 4.91, 4.97, 4.98, 4.104, 4.109, 4.114, 4.115, 4.118, 4.123, 4.129, 4.134, 4.140, 4.144, 4.145, 4.150, 4.155, 4.161, 4.162, 4.163, 4.167, 4.172, 4.176, 4.180, 4.193.
Search / Platform and v20 theorems: Theorems 4.194, 4.195.
Operational cautions and boundary remarks: Remarks 4.1, 4.2, 4.3, 4.4, 4.9, 4.14, 4.20, 4.25, 4.30, 4.37, 4.42, 4.46, 4.53, 4.57, 4.61, 4.66, 4.79, 4.83, 4.92, 4.99, 4.105, 4.110, 4.116, 4.120, 4.124, 4.130, 4.135, 4.141, 4.146, 4.151, 4.156, 4.164, 4.168, 4.173, 4.177, 4.181.
Explicit exclusions: Section X.17.

5 Appendix Y: AI Platform and Proof Store

Y.1. Scope, dependency position, and platform status

This appendix defines the AI Platform and Proof Store layer of the AK framework. It belongs to the Search / Platform Appendices and consumes the reviewed local v20.0.0 strength of Appendices U–X, the Foundation and Technical Extension appendices, and the Problem Interface appendices. It does not enlarge the Core mathematical package of Chapters 1–8, prove an external theorem, or create final Claim Register authority.

The central rule of this appendix is:

Storage is not certification.

A Proof Store may store, index, version, retrieve, compare, and audit proof-related objects. However, the fact that an object is stored does not imply that it is true, verified, Core-admissible, or theorem-level.

In particular, no platform entry, stored proof object, AI-generated artifact, claim registry item, version tag, proof-store identifier, hash, query result, or storage event may replace:

$$B\text{-Gate}^{+}, \qquad \mathrm{Eval}_{n,W,\tau}(F) = 0, \qquad C_{\tau,W}F, \qquad \phi_{i,W,\tau},$$
$$(\mu_{\mathrm{Collapse}},\ u_{\mathrm{Collapse}}) = (0,0), \qquad \mathrm{val}_B(\mathfrak{d}_{\mathrm{met}}) < \mathrm{gap}_{\tau}(\mathfrak{B}),$$

a required finite-to-infinite reduction and a discharged Return theorem,

together with final claim-use permission.

Remark 5.1 (Search / Platform status)**.** This appendix concerns infrastructure for managing proof objects, artifacts, claims, agent outputs, verification records, audit records, and replay references. It does not create mathematical validity.

Remark 5.2 (Relation to Appendices U–X)**.** Appendix U defines agent semantics. Appendix V defines search protocols. Appendix W defines Bridge Programs. Appendix X defines Validity Maps and Certificate DAGs. The present appendix defines the storage, registry, lifecycle, and platform layer supporting those structures.

Remark 5.3 (Storage supports governance)**.** The platform layer is not a proof engine by default. It preserves identities, histories, statuses, and dependencies so that proof engines, auditors, and human reviewers can reconstruct what has been claimed and checked.

Y.1bis. Platform sovereignty and verifier-side conversion

Definition 5.4 (Platform sovereignty)**.** *Platform sovereignty* is the rule that platform storage, indexing, retrieval, versioning, hashing, synchronization, dashboard display, access control, and lifecycle management do not alter the mathematical status of an object. A platform record affects theorem use only through a declared verifier-side record, governance record, or audit record at its stated scope.

Definition 5.5 (Platform-to-verifier conversion record)**.** A *platform-to-verifier conversion record* is a record that attempts to use a platform object as verifier-side evidence. It must contain:

(i) the source platform entry identifier and immutable content identity;

(ii) the target verifier-side record type;

(iii) the scope, window, threshold, monitored degree, and comparison mode, when relevant;

(iv) the conversion theorem, verified computation, or proof assistant artifact supporting the conversion;

(v) the authority scope and environment of the conversion check;

(vi) all non-scalar obligations consumed by the target record;

(vii) the resulting status, exactly one of *pass*, *reject*, or *undefined*;

(viii) the typed failure route when the result is reject or undefined;

(ix) artifact references and repair ancestry.

It is the platform-specialized instance of the verifier-side conversion discipline of Appendix U.

Proposition 5.6 (Platform record does not override verifier-side status)**.** *No platform record, by being stored, indexed, displayed, versioned, hashed, queried, synchronized, or access-approved, overrides a verifier-side status.*

Proof. The listed operations are platform actions. They affect availability, identity, visibility, routing, or governance metadata. They do not by themselves establish repaired evaluation, representative compatibility, eligibility, tower artifact validity, diagnostic vanishing, metric budget admissibility, or bridge soundness. Therefore they cannot override verifier-side status. □

Proposition 5.7 (Verifier-side use requires conversion)**.** *A platform object may be consumed as verifier-side evidence only through a platform-to-verifier conversion record with result pass.*

Proof. A platform object is initially storage-side or workflow-side data. Verifier-side consumption requires typed reconstruction of the target record and its obligations. Definition 5.5 supplies exactly that boundary. Without it, the object remains non-verdict platform data. □

Remark 5.8 (Platform noninterference)**.** Two certificate records with the same verifier-side projection have the same Core status even if their storage locations, query rankings, dashboard views, access histories, or Proof Store identifiers differ.

Y.1ter. v20 dependency boundary and final-CR deferral

Definition 5.9 (Platform dependency boundary)**.** The *platform dependency boundary* is the rule that the Proof Store may preserve and index reviewed local objects from the v20 document family, but may consume each object only at its frozen local statement, role, evidence status, invocation status, and artifact identity. A later registry row, dashboard label, or store migration does not strengthen the consumed theorem.

Definition 5.10 (Final-CR deferral record)**.** A *final-CR deferral record* states that final Claim Register promotion, final Claim UID assignment, release-gate pass, and normative machine-readable registry generation remain deferred until Part V, the Companion, and the cross-document dependency closure are frozen and audited. Local labels and immutable artifact identities remain the stable references during this interval.

Definition 5.11 (Local registry row)**.** A *local registry row* is a non-final platform record used for navigation, dependency tracking, audit preparation, or candidate export before final Claim Register promotion. It must carry the token

$$\mathrm{final\_cr\_status{=}deferred}$$

and may not present a provisional identifier as a final Claim UID.

Proposition 5.12 (Local storage is not release promotion)**.** *Storing a locally reviewed theorem, certificate, manifest, recovery track, or platform record does not promote it into the final CR-v20 release registry.*

Proof. Local review, artifact storage, target-sub-DAG closure, final Claim Register promotion, and release publication are distinct governance actions. The final-CR deferral record withholds the latter actions until the declared dependency closure is frozen. Therefore storage or local discharge cannot perform release promotion. □

Proposition 5.13 (No provisional Claim UID authority)**.** *A generated JSON row, CSV row, database key, URI, graph identifier, or platform UUID created before final-CR promotion is not a final Claim UID and does not authorize theorem-level consumption.*

Proof. A platform identifier establishes record identity only within the declared platform namespace. Final Claim UID authority additionally requires the final CR extraction, dependency closure, claim-use review, and release-governance action. Those conditions are not created by identifier generation. □

Remark 5.14 (Reviewed local strength)**.** Throughout this appendix, "stored", "registered", "verified", or "reviewed" is read at the exact local strength of the referenced artifact. No stored field silently imports a stronger Main, Companion, external-domain, or release-level conclusion.

Y.2. AI Platform

Definition 5.15 (AI Platform)**.** An *AI Platform* is a software, data, and workflow environment supporting agent-assisted search, proof management, artifact storage, verification, audit, replay, and reproducibility for the AK framework.

Definition 5.16 (Platform component)**.** A *platform component* is a subsystem of the AI Platform. Examples include:

(i) agent interface;

(ii) search engine;

(iii) proof assistant interface;

(iv) persistence-computation engine;

(v) artifact store;

(vi) proof store;

(vii) claim registry;

(viii) validity map service;

(ix) certificate DAG service;

(x) ledger engine;

(xi) diagnostic engine;

(xii) replay engine;

(xiii) audit dashboard;

(xiv) access-control service.

Definition 5.17 (Platform action)**.** A *platform action* is an operation performed by the AI Platform or one of its components. Examples include:

$$\text{create, store, retrieve, version, hash, verify,}$$
$$\text{audit, replay, promote, demote, reject, supersede.}$$

Definition 5.18 (Platform authority scope)**.** A *platform authority scope* is the declared scope within which a platform component or action is authorized to operate.

Proposition 5.19 (Platform action is scoped)**.** *A platform action establishes only the effect declared by its action type and authority scope.*

Proof. Different platform actions have different meanings. Storing an object records it. Hashing identifies content. Verification checks declared criteria. Audit checks reconstructibility. Promotion changes status only under an authority rule. Therefore each action is scoped to its declared semantics. □

Remark 5.20 (Platform semantics must be explicit)**.** The same interface button or workflow step may hide multiple platform actions. For auditability, the semantic action must be recorded explicitly.

Y.2bis. Platform trust boundary and action-role separation

Definition 5.21 (Platform trust boundary)**.** The *platform trust boundary* separates untrusted or proposer-side input from verifier-side, governance-side, and source-authoritative records. Uploaded text, retrieved snippets, prompts, generated code, model memory, external metadata, and imported sidecars remain untrusted for theorem use until the applicable identity, adequacy, source-binding, and conversion checks pass.

Definition 5.22 (Platform action-role record)**.** A *platform action-role record* declares whether an action is storage, indexing, retrieval, transformation, computation, verification, audit, governance, promotion, replay, or publication. An action performed by one service does not inherit the authority of another role merely because both actions occur in one workflow.

Definition 5.23 (Untrusted payload quarantine)**.** *Untrusted payload quarantine* is the isolation of content whose instructions, claims, locators, status tokens, or artifact references have not passed the applicable source and authority checks. Quarantined content may be searched or reviewed but may not mutate verifier criteria, status mappings, source bindings, or promotion policy.

Proposition 5.24 (Combined interface does not combine authority)**.** *A single user interface or agent workflow that stores, retrieves, computes, verifies, and promotes objects does not thereby give every action the union of those authority scopes.*

Proof. Authority is attached to the declared action, criteria, environment, and governance rule, not to the visual interface or process identity. Hence co-location of actions does not collapse their semantic scopes. □

Remark 5.25 (Prompt and source separation)**.** Text occurring in a prompt, retrieved context, comment, filename, or imported metadata is not source authority. A source statement is consumed only through its immutable source-binding record and permitted-use field.

Y.3. Proof Store

Definition 5.26 (Proof Store)**.** A *Proof Store* is a repository for proof objects, certificate records, formal fragments, Bridge Programs, Validity Map entries, Certificate DAG entries, artifact references, ledger records, diagnostic records, and associated metadata.

Definition 5.27 (Proof Store entry)**.** A *Proof Store entry* is a stored item in the Proof Store. It must include:

(i) entry identifier;

(ii) object type;

(iii) content or content reference;

(iv) status label;

(v) version identifier;

(vi) provenance metadata;

(vii) dependency references, if any;

(viii) artifact references;

(ix) audit metadata;

(x) authority and access metadata.

Definition 5.28 (Stored object)**.** A *stored object* is any object contained in or referenced by the Proof Store.

Definition 5.29 (Stored Core-facing object)**.** A *stored Core-facing object* is a stored object representing or referencing Core-relevant verifier-side material, such as:

$$\mathrm{Eval}_{i,W,\tau}(F), \qquad C_{\tau,W}F, \qquad \phi_{i,W,\tau}, \qquad B\text{-Gate}^{+},$$
$$(\mu_{\mathrm{Collapse}},\ u_{\mathrm{Collapse}}), \qquad \mathrm{val}_B(\mathfrak{d}_{\mathrm{met}}) < \mathrm{gap}_\tau(\mathfrak{B}).$$

Definition 5.30 (Verified Core-facing object)**.** A *verified Core-facing object* is a stored Core-facing object whose relevant Core verifier-side conditions have been checked and whose verification record is stored and audit-readable.

Proposition 5.31 (Stored object is not certified by storage)**.** *A stored object is not certified merely by being stored in the Proof Store.*

Proof. Storage records availability and identity. Certification requires verifier-side conditions, proof data, diagnostics, ledgers, gate verdicts, and audit-readable manifests. Therefore storage alone does not certify an object. □

Proposition 5.32 (Stored Core-facing object is not verified by storage)**.** *A stored Core-facing object is not Core verified merely by being stored.*

Proof. A stored Core-facing object may represent a repaired evaluation record, representative record, tower artifact, diagnostic, or ledger condition. Core verification requires checking the corresponding verifier-side criteria. Storage records the object but does not perform or imply that check. □

Remark 5.33 (Storage supports, but does not replace, verification)**.** The Proof Store is essential for reproducibility and audit. Its function is to preserve and organize evidence, not to create validity automatically.

Y.3bis. Verifier projection and evidence-bundle closure

Definition 5.34 (Verifier-side projection of a store entry)**.** For a Proof Store entry E, the *verifier-side projection* $\pi_{\mathrm{ver}}(E)$ is the subrecord containing only the typed objects, discharged obligations, authority-scoped verification records, actual artifacts, source bindings, metric-ledger records, diagnostics, and claim-use permissions consumed by the target verifier. Storage location, query rank, display label, mirror count, user popularity, and explanatory annotations are omitted.

Definition 5.35 (Evidence bundle)**.** An *evidence bundle* is a finite, immutable collection of target-relevant records with a declared target scope, necessary dependency closure, artifact identities, status axes, and replay boundary. It is not complete merely because all files are present; every necessary typed dependency must also be discharged.

Definition 5.36 (Bundle closure status)**.** An evidence bundle has exactly one atomic closure result

$$\mathrm{pass}, \qquad \mathrm{reject}, \qquad \mathrm{undefined}.$$

Missing required records, unresolved source authority, stale artifacts, unproved Return, or absent finite-to-infinite reduction yield $\mathrm{undefined}$; an evidenced false invoked condition yields reject.

Proposition 5.37 (Store co-location is not proof composition)**.** *Placing proof fragments, theorem statements, artifacts, and dependency rows in one Proof Store entry, folder, archive, or evidence bundle does not compose them into a proof.*

Proof. Proof composition requires type-compatible statements, matching hypotheses and scopes, discharged dependency edges, and valid interpretation or Return arrows. Storage co-location supplies none of those relations by itself. □

Proposition 5.38 (Verifier projection controls theorem use)**.** *Changing platform-only fields while preserving $\pi_{\mathrm{ver}}(E)$ does not change the theorem-level status supported by E.*

Proof. The target verifier consumes only the verifier-side projection. Fields outside that projection affect navigation or operations, not the discharged mathematical obligations. □

Y.4. Object classes

Definition 5.39 (Proof Store object class)**.** A *Proof Store object class* is a declared type of object stored in the Proof Store. Standard classes include:

(i) search artifact;

(ii) candidate;

(iii) bridge draft;

(iv) bridge candidate;

(v) Bridge Program;

(vi) bridge theorem;

(vii) formal fragment;

(viii) verified formal fragment;

(ix) certificate candidate;

(x) Core certificate;

(xi) Core-facing object;

(xii) verified Core-facing object;

(xiii) interface theorem;

(xiv) proof-first record;

(xv) proof object;

(xvi) theorem-level proof object;

(xvii) ledger record;

(xviii) diagnostic record;

(xix) artifact reference;

(xx) interpretation theorem;

(xxi) non-claim;

(xxii) rejected object;

(xxiii) deprecated object;

(xxiv) superseded object.

Definition 5.40 (Object status)**.** An *object status* is a validity or lifecycle label attached to a Proof Store object. It must be compatible with the status taxonomy of Appendices U, V, W, and X.

Definition 5.41 (Status authority)**.** A *status authority* is the declared agent, tool, procedure, or governance rule authorized to assign or update a particular object status.

Proposition 5.42 (Object class does not imply validity)**.** *The object class of a stored item does not by itself imply validity.*

Proof. An object class describes what kind of object is stored. Validity depends on proof, verification, status, discharge, and audit records. Therefore class and validity are distinct. □

Remark 5.43 (Class/status separation)**.** A formal fragment may be unverified or verified. A Core-facing object may be stored but unchecked. A certificate candidate may pass or fail. A bridge theorem may later be superseded. Thus object class and object status must be tracked separately.

Y.4bis. Status axes and token compatibility

Definition 5.44 (Proof Store status axis)**.** The Proof Store status axis is a platform-local lifecycle and validity field. It does not replace:

(i) the agent-output status labels of Appendix U;

(ii) the search workflow statuses of Appendix V;

(iii) the Bridge Program statuses of Appendix W;

(iv) the map and DAG statuses of Appendix X;

(v) the Chapter 1 claim taxonomy;

(vi) the current CR-v20 evidence status, invocation role, conformance status, or claim-use preflight result.

Definition 5.45 (Status compatibility record)**.** A *status compatibility record* declares how a Proof Store object class and platform status relate to the relevant Appendix U label, Chapter 1 document role, and current CR-v20 governance status. It must state whether the platform status is lifecycle-only, theorem-facing, audit-facing, or non-verdict.

Proposition 5.46 (Platform status is not claim-governance status)**.** *A platform status label does not by itself determine a claim's theorem evidence status, invocation role, or permitted use.*

Proof. Platform status records workflow and storage state. Claim-governance status is controlled by Chapter 1 and the current CR-v20 baseline and depends on proof evidence, hypotheses, dependencies, permitted use, and failure routing. The axes are therefore distinct unless a status compatibility record and claim-use preflight explicitly connect them. □

Remark 5.47 (Machine-readable token discipline)**.** Human-readable labels such as "Core verified" or "audit-ready" require machine-readable canonical tokens in the registry. Display punctuation, spacing, or capitalization is not a status proof.

Y.4ter. Canonical status axes and weaker-safe resolution

Definition 5.48 (Canonical platform status tuple)**.** A theorem-facing Proof Store entry records, without conflation:

(i) object class;

(ii) platform lifecycle status;

(iii) document role;

(iv) evidence status;

(v) invocation role;

(vi) conformance status;

(vii) atomic result status;

(viii) claim-use permission;

(ix) artifact identity status;

(x) final-CR status.

Definition 5.49 (Weaker-safe status resolution)**.** *Weaker-safe status resolution* assigns the weakest status compatible with all authoritative fields when platform, document, claim-governance, artifact, and verifier records disagree. No display label or later metadata update may override an authoritative weaker status without a promotion certificate.

Proposition 5.50 (Missing is not not-invoked)**.** *A missing verifier, artifact, source-binding, reduction, Return, or interpretation record maps to* $\mathrm{undefined}$*, not* $\mathrm{not\_invoked}$.

Proof. The token $\mathrm{not\_invoked}$ requires a positive immutable scope-exclusion policy. Mere absence does not establish that the obligation lies outside the target scope. □

Remark 5.51 (Status strings are not authorities)**.** A string such as $\mathrm{verified}$, passed, final, or $\mathrm{theorem}$ has no theorem-facing force unless its authority, criteria, target scope, and supporting records are independently validated.

Y.5. Claim Registry

Definition 5.52 (Claim Registry)**.** A *Claim Registry* is a platform component that stores mathematical, operational, interface, bridge, platform, and non-claim statements together with status labels, dependencies, proof records, and artifact references.

Definition 5.53 (Registered claim)**.** A *registered claim* is a claim recorded in the Claim Registry. It must include:

(i) claim identifier;

(ii) claim text;

(iii) claim type;

(iv) status label;

(v) dependencies;

(vi) supporting artifacts;

(vii) responsible agents or roles;

(viii) verification or rejection record, if any;

(ix) demotion or supersession record, if any.

Definition 5.54 (Claim type)**.** A *claim type* may be:

(i) Core theorem;

(ii) Core lemma;

(iii) Core certificate claim;

(iv) operational policy;

(v) interface theorem;

(vi) bridge draft;

(vii) bridge candidate;

(viii) bridge theorem;

(ix) [Spec];

(x) search artifact;

(xi) platform claim;

(xii) non-claim;

(xiii) rejected claim.

Proposition 5.55 (Registration is not validation)**.** *A registered claim is not validated merely by being registered.*

Proof. Registration records the claim and its metadata. Validation requires proof or verification under the declared status criteria. Therefore registration is not validation. □

Remark 5.56 (Registry prevents orphan claims)**.** The Claim Registry helps prevent unsupported or forgotten claims by requiring status labels, dependencies, and artifact references.

Y.5bis. Claim-use preflight and registration sovereignty

Definition 5.57 (Platform claim-use preflight)**.** A *platform claim-use preflight* is the platform-side execution of the current CR-v20 claim-use checks before a registered claim is consumed. It checks at least identity, source binding, document role, evidence status, invocation role, conformance status, hypotheses, dependencies, comparison policy, artifact requirements, permitted use, forbidden stronger reading, repair ancestry, and typed failure route.

Proposition 5.58 (Claim Registry entry is not permitted use)**.** *The mere presence of a claim in the Claim Registry does not authorize its use as theorem evidence.*

Proof. A registry entry records metadata and governance fields. Theorem-evidence use additionally requires that the entry's role, status, hypotheses, dependencies, source binding, comparison policy, and permitted-use field pass preflight. Therefore registration alone is insufficient. □

Remark 5.59 (Rejected and non-claim entries remain useful)**.** The platform may store rejected claims, deprecated claims, explicit non-claims, and failed bridge attempts. Their stored presence supports audit and avoids repetition; it does not promote them.

Y.5ter. Local Claim Registry and final register separation

Definition 5.60 (Local Claim Registry)**.** The *Local Claim Registry* is the platform registry used during document construction and review. It preserves local labels, statement hashes, roles, dependencies, and review status, but is non-normative relative to the deferred final CR-v20 release registry.

Definition 5.61 (Register-equivalence record)**.** A *register-equivalence record* compares a human-readable claim table, TeX source, PDF rendering, JSON sidecar, CSV export, graph database row, and final CR entry. It specifies which fields are byte-derived, semantically preserved, normalized, omitted, or authority-bearing.

Proposition 5.62 (Machine-readable export is not normative by serialization)**.** *An export of the Local Claim Registry is not the final Claim Register. This applies to JSON, YAML, CSV, database, and graph formats. Machine readability and schema validity do not confer final registry authority.*

Proof. Serialization validates representation against a schema. Normative final status requires the final extraction, cross-document closure, claim-use review, and release-governance act. These are semantic and governance obligations, not serialization properties. □

Remark 5.63 (Final register extraction)**.** Final register extraction is performed only after the document family is frozen. Until then, local labels and immutable content identities control references; generated final-looking identifiers remain non-final.

Y.6. Artifact Store

Definition 5.64 (Artifact Store)**.** An *Artifact Store* is a repository for files, datasets, code, logs, formal proof scripts, numerical outputs, barcodes, diagrams, manifests, ledger files, diagnostic files, replay records, and other artifacts referenced by proof objects or certificates.

Definition 5.65 (Artifact entry)**.** An *artifact entry* contains:

(i) artifact identifier;

(ii) artifact type;

(iii) content address or location;

(iv) hash or checksum, if used;

(v) version identifier;

(vi) creator or source;

(vii) associated proof-store entries;

(viii) supported claim, node, or edge references;

(ix) license or access constraints, if relevant;

(x) audit notes.

Definition 5.66 (Artifact binding)**.** An *artifact binding* is a declared relation between a proof-store object and an artifact entry.

Definition 5.67 (Artifact adequacy)**.** *Artifact adequacy* is the property that an artifact supports the specific claim, computation, proof, diagnostic, ledger, or certificate condition for which it is cited.

Proposition 5.68 (Artifact availability is not artifact adequacy)**.** *The availability of an artifact does not imply that it adequately supports the claim that cites it.*

Proof. Availability means the artifact can be retrieved. Adequacy means it supports the relevant claim, proof, computation, diagnostic, ledger, or certificate condition. These are distinct properties. □

Remark 5.69 (Artifact adequacy is audit matter)**.** Whether an artifact adequately supports a claim must be checked during verification or audit.

Y.6bis. Artifact identity, adequacy, and drift preflight

Definition 5.70 (Artifact preflight)**.** An *artifact preflight* checks that an artifact is present, identity-matched, version-correct, readable in the declared environment, adequate for the cited claim, and not superseded for the intended use.

Definition 5.71 (Artifact adequacy failure)**.** An *artifact adequacy failure* occurs when an available artifact does not support the specific proof, computation, diagnostic, ledger, bridge, interpretation, or audit claim for which it is cited.

Proposition 5.72 (Available artifact with failed adequacy is non-evidence)**.** *An artifact that is available but fails adequacy preflight cannot be consumed as supporting evidence for the cited claim.*

Proof. Evidence use requires not only retrieval but also a support relation between artifact and claim. Adequacy failure destroys that support relation. Thus the artifact remains available data but not evidence for that use. □

Remark 5.73 (Artifact drift is semantic drift when adequacy changes)**.** A byte-identical artifact can become inadequate if the cited claim, scope, theorem target, or verifier criterion changes. Conversely, a content change creates a new artifact identity even when the informal description is similar.

Y.6ter. Source binding and Known-Theorem Recovery quarantine

Definition 5.74 (Recovery source-binding package)**.** A *recovery source-binding package* stores, as distinct immutable artifacts:

(i) authoritative source identity;

(ii) exact source statement and locator;

(iii) permitted recovered clause;

(iv) registered precursor package;

(v) purpose-faithful realization record;

(vi) finite witness and analytic or arithmetic proof artifact;

(vii) finite-to-infinite reduction, when required;

(viii) Return record;

(ix) benchmark comparison record;

(x) forbidden stronger readings.

Definition 5.75 (Source-conclusion quarantine)**.** *Source-conclusion quarantine* permits the immutable benchmark statement and locator to identify the exact target and the consequent of the proposed Return theorem. It prevents the benchmark proof, known truth value, answer table, target constants, exceptional bounds, conclusion-derived witnesses, equivalent oracles, and validation artifacts from entering the realization, certificate, reduction, higher-edge proof, or Return proof. The quarantined benchmark is consumed only for terminal independent comparison after the construction is frozen.

Definition 5.76 (Reference recovery tracks)**.** The v20.0.0 Proof Store may index the reviewed local tracks

$$\text{IW-KTR-GROWTH}, \qquad \text{NS-KTR-SERRIN-L6}$$

only at their registered exact strengths. The first records the characteristic-growth clause of Iwasawa's 1959 Theorem 4 relative to its precursor package; the second records the fixed strict-subcritical Serrin implication at $L_t^6 L_x^6$. Neither record supplies the Iwasawa main conjecture, a class-number specialization, unconditional Navier–Stokes regularity, or a Clay-problem result.

Proposition 5.77 (Source storage does not recover theorem)**.** *Storing an authoritative source, source statement, or source-binding record does not by itself establish a Known-Theorem Recovery track.*

Proof. Recovery additionally requires the registered precursor input, purpose-faithful realization, finite certificate, any necessary reduction, Return, and exact-strength comparison. Source availability supplies only one component of that chain. □

Proposition 5.78 (Benchmark leakage invalidates recovery use)**.** *If quarantined benchmark-conclusion information is consumed in constructing a required upstream witness without an independently justified theorem permitting that use, the affected recovery route is not admissible as an independent recovery certificate.*

Proof. The route would become circular or conclusion-conditioned rather than a reconstruction from the registered source-side inputs. Therefore it cannot certify independent recovery at the declared strength. □

Remark 5.79 (Non-invoked successor interfaces)**.** Concrete Fukaya/HMS and external AGI-N bridge tracks remain $\mathrm{not\_invoked}$ in v20.0.0 unless a later immutable scope record activates a fully typed external bridge. Platform storage of schemas or candidate nodes does not activate them.

Y.7. Versioning and immutability

Definition 5.80 (Version identifier)**.** A *version identifier* is a label or content-derived identifier distinguishing one version of an object or artifact from another.

Definition 5.81 (Immutable object)**.** An *immutable object* is an object whose proof-relevant content is not changed after storage. Any modification creates a new version.

Definition 5.82 (Mutable record)**.** A *mutable record* is a platform record whose metadata, status, or pointers may change over time while preserving history.

Definition 5.83 (Version lineage)**.** A *version lineage* is the recorded history of object versions and their relationships.

Definition 5.84 (Status inheritance)**.** *Status inheritance* is the attempted transfer of a status label from one version of an object to another.

Proposition 5.85 (Version change may require re-verification)**.** *If a stored object's proof-relevant content changes, any verification or audit status depending on the old content does not automatically transfer to the new version.*

Proof. Verification applies to the object content that was checked. A changed version may have different definitions, proofs, dependencies, or artifacts. Therefore status transfer requires justification or re-verification. □

Proposition 5.86 (Status inheritance requires justification)**.** *Status inheritance from one version to another is admissible only if a preservation argument, equivalence check, or re-verification record justifies it.*

Proof. A status label is attached to a verified or reviewed object under declared conditions. If the object changes, those conditions may no longer hold. Thus inheritance requires explicit justification. □

Remark 5.87 (Immutable content, mutable metadata)**.** A robust Proof Store should treat proof-relevant content as immutable while allowing metadata and status records to evolve with full history.

Y.8. Hashing and content identity

Definition 5.88 (Content hash)**.** A *content hash* is a deterministic identifier derived from the content of an object or artifact.

Definition 5.89 (Hash binding)**.** A *hash binding* is a relation between an object identifier and a content hash.

Definition 5.90 (Hash mismatch)**.** A *hash mismatch* occurs when retrieved content does not match the declared hash.

Definition 5.91 (Content identity)**.** *Content identity* means that the retrieved content agrees with the declared stored content according to the platform identity policy.

Proposition 5.92 (Hash match proves identity, not validity)**.** *A content hash match supports content identity, but it does not prove mathematical validity.*

Proof. A hash can show that the retrieved content is the same as the stored or referenced content. It does not establish that the content is true, verified, sufficient, or theorem-level. □

Proposition 5.93 (Hash mismatch invalidates artifact binding until resolved)**.** *A hash mismatch invalidates the affected artifact binding until the mismatch is resolved.*

Proof. If the content no longer matches the declared identity, the proof-store object may be referencing the wrong artifact. The binding is therefore not audit-stable until resolved. □

Remark 5.94 (Hashing controls drift)**.** Hashing is a practical mechanism for controlling artifact drift, but it must be paired with adequacy checks.

Y.8bis. Identity witnesses and semantic preservation

Definition 5.95 (Identity witness)**.** An *identity witness* is a hash match, signature verification, content-addressed reference, immutable storage pointer, or equivalent mechanism establishing that retrieved content is the declared content.

Definition 5.96 (Semantic preservation record)**.** A *semantic preservation record* justifies that a transformation, migration, format conversion, normalization, compression, export, import, or rendering preserves the proof-relevant content needed by a claim.

Proposition 5.97 (Identity witness is not semantic preservation)**.** *An identity witness for one artifact does not prove semantic preservation across a transformed or migrated artifact.*

Proof. Identity confirms sameness of one declared content object. Semantic preservation compares two objects across a transformation. The first supplies no theorem that the transformation preserved the proof-relevant statement. □

Remark 5.98 (PDF, source, and machine-readable sidecars)**.** A PDF, source file, JSON sidecar, CSV register, or rendered image may be stored as an artifact. Equivalence among them is not automatic; it requires a semantic preservation or register-equivalence record.

Y.8ter. Identity levels and cross-format equivalence

Definition 5.99 (Byte identity)**.** *Byte identity* is equality under the declared byte-level or content-hash policy. It proves that two retrieved payloads are the same stored payload under that policy.

Definition 5.100 (Semantic identity)**.** *Semantic identity* is a proved or verified preservation relation between possibly different representations at the exact theorem-relevant scope. It may require parsing, normalization, interpretation, register equivalence, or a transformation-preservation theorem.

Definition 5.101 (Authority identity)**.** *Authority identity* is the verified relation between an artifact and the source, verifier, governance authority, or release authority whose status is being consumed. Byte or semantic identity does not create authority identity by itself.

Proposition 5.102 (Identity levels do not collapse)**.** *Byte identity, semantic identity, and authority identity are pairwise non-interchangeable without an explicit connecting record.*

Proof. Byte identity concerns payload sameness, semantic identity concerns theorem-relevant meaning, and authority identity concerns admissible attribution and permitted use. Each answers a different question. □

Remark 5.103 (Rendered artifact caution)**.** A compiled PDF may be byte-different from its TeX source while semantically faithful; a byte-identical PDF may become governance-inadequate when its target claim changes. Accordingly, rendered artifacts require both identity and adequacy records for theorem-facing use.

Y.9. Proof object lifecycle

Definition 5.104 (Proof object lifecycle)**.** The *proof object lifecycle* is the sequence of states through which a proof-related object may pass:

$$\text{created} \longrightarrow \text{stored} \longrightarrow \text{labeled} \longrightarrow \text{linked} \longrightarrow \text{verified} \longrightarrow \text{audited} \longrightarrow \text{promoted or rejected.}$$

Definition 5.105 (Lifecycle event)**.** A *lifecycle event* is a recorded state change or action in the proof object lifecycle.

Definition 5.106 (Lifecycle log)**.** A *lifecycle log* is an ordered record of lifecycle events for a stored object.

Definition 5.107 (Lifecycle authority)**.** A *lifecycle authority* is the agent, tool, or governance rule authorized to perform a lifecycle event.

Proposition 5.108 (Lifecycle progress is not validity monotonicity)**.** *Progress through the proof object lifecycle does not monotonically imply increasing mathematical validity.*

Proof. An object may be stored, linked, and audited but later rejected due to a discovered gap. Lifecycle progress records workflow state, not mathematical truth. □

Remark 5.109 (Lifecycle logs support accountability)**.** Lifecycle logs support accountability by showing when and why status changes occurred.

Y.9bis. Problem-Bridge chain storage and target-local closure

Definition 5.110 (Problem-Bridge store chain)**.** A *Problem-Bridge store chain* stores separately:

$$\begin{aligned}&\text{external source} \longrightarrow \text{purpose-faithful realization} \longrightarrow \text{Core-readable profile}\\ &\longrightarrow \text{finite Core certificate} \longrightarrow \text{finite-to-infinite reduction, if required}\\ &\longrightarrow \text{higher edge, if invoked} \longrightarrow \text{Return theorem} \longrightarrow \text{external conclusion.}\end{aligned}$$

Each arrow has its own type, direction, hypotheses, discharge record, artifact identity, and invocation status.

Definition 5.111 (Target-local store closure)**.** A stored target has *target-local closure* only when every necessary entry and dependency edge in its selected admissible proof route, including all necessary branches, is present, identity-matched, and discharged. Unrelated failed or unresolved store entries do not block the target; omitted necessary branches do.

Proposition 5.112 (Stored chain skeleton is not complete chain)**.** *A collection of correctly named Problem-Bridge entries does not establish the external conclusion unless every required arrow is discharged at the required direction and strength.*

Proof. Node presence does not establish edge validity. The conclusion depends on the typed realization, certificate, reduction, edge, and Return implications, not on their filenames or adjacency in storage. □

Proposition 5.113 (Stored finite certificate is not infinite conclusion)**.** *A stored finite observation, finite certificate, finite prefix, numerical plateau, Cauchy-looking sequence, or replayed finite computation does not establish colimit, completion, apex agreement, or an external infinite conclusion without the required reduction theorem.*

Proof. Finite evidence and finite-to-infinite transport are distinct obligations. The latter requires its own compactness, eventual-constancy, comparison, or reduction argument at the declared target. □

Y.10. Promotion, demotion, and rejection

Definition 5.114 (Promotion event)**.** A *promotion event* is a recorded status change to a stronger status, such as from candidate to verified fragment, from bridge candidate to bridge theorem, or from Core certificate candidate to Core verified.

Definition 5.115 (Demotion event)**.** A *demotion event* is a recorded status change to a weaker status, such as from verified to blocked, superseded, rejected, deprecated, or incomplete.

Definition 5.116 (Rejection event)**.** A *rejection event* is a recorded status change indicating that an object or claim has failed the relevant criteria.

Definition 5.117 (Promotion evidence)**.** *Promotion evidence* is the proof, verification record, import record, audit record, discharge certificate, or authority record required to justify promotion.

Proposition 5.118 (Promotion requires recorded evidence)**.** *A promotion event is admissible only if the required evidence for the stronger status is recorded.*

Proof. Promotion changes how an object may be used in proof planning, certification, and theorem-level claims. Without evidence, promotion would enable claim inflation. Therefore evidence is required. □

Proposition 5.119 (Demotion preserves audit integrity)**.** *Demotion preserves audit integrity by preventing known-defective or unsupported objects from retaining stronger statuses.*

Proof. If a defect is discovered, the previous status may no longer be justified. Demotion records this change and prevents continued misuse. □

Remark 5.120 (Rejection is useful information)**.** Rejected objects should usually remain stored with their rejection reason. They prevent repeated work and help map failure regions.

Y.10bis. Promotion certificates and non-retroactive repair

Definition 5.121 (Platform promotion certificate)**.** A *platform promotion certificate* is a record authorizing a stronger platform or claim-facing status. It must contain the prior status, target status, evidence, authority scope, current CR-v20 preflight result when a claim is consumed, artifact identities, result status pass/reject/undefined, and failure route when applicable.

Definition 5.122 (Non-retroactive platform repair)**.** *Non-retroactive platform repair* is the rule that repairing a stored object, artifact binding, status label, or registry entry creates a new revision or repair record rather than mutating the historical verdict of the prior record.

Proposition 5.123 (Promotion certificate is required for theorem-facing promotion)**.** *A stored item may not be promoted to theorem-facing, Core verified, bridge theorem, interface theorem, or theorem-level proof-object status without a platform promotion certificate.*

Proof. Theorem-facing promotion changes permitted use. It therefore requires evidence, authority, preflight, artifact identity, and failure routing. These are exactly the fields of a platform promotion certificate. □

Proposition 5.124 (Repair does not rewrite prior verdicts)**.** *A platform repair does not retroactively convert a prior failed, undefined, stale, or superseded record into a passing one.*

Proof. Historical audit requires that prior records remain reconstructible at their original content and status. A repair supplies a new record or revision with its own evidence. Thus the previous verdict is not mutated. □

Y.11. AI output management

Definition 5.125 (AI output record)**.** An *AI output record* is a Proof Store or Artifact Store entry recording an AI-generated output and associated metadata.

Definition 5.126 (AI output metadata)**.** AI output metadata may include:

(i) model or agent identifier;

(ii) prompt or task description;

(iii) output content;

(iv) confidence or ranking, if supplied;

(v) output status label;

(vi) human review status;

(vii) verification status;

(viii) artifact references;

(ix) supersession or rejection history, if any.

Definition 5.127 (AI-generated Core-facing record)**.** An *AI-generated Core-facing record* is an AI output claiming to produce or describe Core-facing material, such as:

$$\mathrm{Eval}_{i,W,\tau}(F), \qquad C_{\tau,W}F, \qquad \phi_{i,W,\tau}, \qquad (\mu_{\mathrm{Collapse}},\, u_{\mathrm{Collapse}}), \qquad \mathrm{val}_B(\mathfrak{d}_{\mathrm{met}}) < \mathrm{gap}_\tau(\mathfrak{B}).$$

Proposition 5.128 (AI output must be status-labeled)**.** *AI output used in the AK workflow must be assigned an explicit status label.*

Proof. By Appendix U, unlabeled agent output is non-verdict by default. To use AI output responsibly in workflow management, its status must be declared. □

Proposition 5.129 (AI storage does not validate AI output)**.** *Storing AI output in the Proof Store or Artifact Store does not validate it.*

Proof. Storage preserves the output and metadata. Validation requires proof, verification, or audit appropriate to the output's intended status. □

Proposition 5.130 (AI-generated Core-facing records require Core verification)**.** *AI-generated Core-facing records cannot support Core certification unless checked under the declared Core verifier-side protocol.*

Proof. AI generation is proposer-side production. Core certification requires verifier-side checks. Therefore AI-generated Core-facing records remain candidates until verified. □

Remark 5.131 (AI outputs are traceable candidates)**.** AI outputs should be treated as traceable candidates, drafts, summaries, bridge sketches, or search artifacts unless and until verified.

Y.11bis. AI retrieval, consensus, and context records

Definition 5.132 (AI context record)**.** An *AI context record* stores the retrieval context, prompt context, model context, selected artifacts, summaries, memory items, or platform snippets used to generate an AI output.

Definition 5.133 (AI consensus record)**.** An *AI consensus record* records agreement among multiple AI agents, model runs, prompts, reviewers, or ranking procedures.

Proposition 5.134 (Retrieved context is not evidence by inclusion)**.** *Including a source, snippet, context item, or memory record in an AI context record does not make it theorem evidence.*

Proof. Context inclusion records what the model could use or did use. Theorem evidence requires verified source binding, admissible claim use, and the appropriate verifier-side or bridge record. Thus inclusion alone is non-verdict. □

Proposition 5.135 (AI consensus is not proof)**.** *AI consensus, ensemble agreement, or repeated generation of the same claim is not proof.*

Proof. Consensus is a proposer-side or review-side signal. It does not supply the mathematical proof, formal verification, Core record, bridge theorem, or artifact adequacy needed for theorem status. □

Remark 5.136 (Context retention supports audit)**.** Retaining AI context is useful because it lets auditors reconstruct how an output was generated. It does not validate the output.

Y.11ter. Model, prompt, and retrieval provenance

Definition 5.137 (AI execution provenance record)**.** An *AI execution provenance record* may include model family and version, system and task instructions, prompt template, retrieved artifact identities and locator ranges, tool calls, decoding configuration, random seed when meaningful, post-processing, human edits, and output hash. Unknown or unavailable fields are recorded as unknown rather than reconstructed from memory.

Definition 5.138 (Source-instruction injection event)**.** A *source-instruction injection event* occurs when untrusted source content attempts to alter verifier rules, promotion policy, source authority, artifact selection, or platform behavior outside its permitted data role.

Proposition 5.139 (AI provenance enables audit, not proof)**.** *A complete AI execution provenance record supports reconstruction and failure analysis but does not prove the mathematical correctness of the output.*

Proof. Provenance records how an output was generated. Correctness requires proof, verification, source adequacy, and typed bridge or Core records. □

Proposition 5.140 (Injected instructions are non-authoritative)**.** *Instructions embedded in retrieved mathematical sources, filenames, metadata, comments, or generated artifacts do not modify the AK verifier or governance policy unless they are separately authenticated and adopted by the appropriate authority.*

Proof. Retrieved content is data within the platform trust boundary. Verifier and governance rules are controlled by declared authorities, not by untrusted payload text. □

Y.12. Verification records

Definition 5.141 (Verification record)**.** A *verification record* is a Proof Store entry recording the result of a verification action. It must include:

(i) verified object identifier;

(ii) verification criteria;

(iii) verification authority;

(iv) verification result;

(v) environment or configuration;

(vi) artifact references;

(vii) timestamp or version reference;

(viii) failure log, if verification failed;

(ix) scope limitations, if any.

Definition 5.142 (Verification result)**.** A *verification result* is one of:

(i) pass;

(ii) fail;

(iii) inconclusive;

(iv) not applicable;

(v) blocked;

(vi) superseded.

Definition 5.143 (Core verification record)**.** A *Core verification record* is a verification record for a Core-facing object or Core certificate condition. It must specify which Core condition was checked, such as repaired evaluation, admissible representative, eligibility, tower artifact, diagnostic, gate verdict, or ledger budget.

Proposition 5.144 (Verification result is scoped to criteria)**.** *A verification result establishes only what was checked under the declared criteria.*

Proof. Verification criteria define the property being checked. A pass under one criterion does not imply pass under another. Thus the result is scoped. □

Remark 5.145 (Verification record must include environment)**.** For computational or formal verification, the environment and configuration may affect reproducibility. They should be recorded.

Y.12bis. Verification-result mapping and scope control

Definition 5.146 (Verification-to-atomic-status mapping)**.** A verification record maps to the canonical atomic status calculus as follows:

(i) pass maps to pass within the declared criteria;

(ii) fail maps to reject within the declared criteria;

(iii) inconclusive and blocked map to undefined;

(iv) not applicable maps to not_invoked only when a declared scope-exclusion policy positively justifies that the criterion is not invoked; absent such a declaration, it maps to undefined for theorem-facing use;

(v) superseded is unavailable for current theorem evidence unless revalidated.

Definition 5.147 (Verification scope overrun)**.** A *verification scope overrun* occurs when a verification result is used for a statement, theorem target, environment, version, artifact, comparison mode, or authority scope beyond what was checked.

Proposition 5.148 (Verification scope overrun invalidates the use)**.** *A verification result used outside its declared scope is not valid evidence for that use.*

Proof. Verification establishes only the checked criterion in the declared environment and scope. Using it elsewhere changes the claim without supplying a new check. The use is therefore unsupported. □

Remark 5.149 (Formal and computational pass are scoped)**.** A proof-assistant pass, parser pass, build pass, replay pass, or computation pass establishes only the property that the corresponding authority is declared to check.

Y.12ter. Specialized v20 verifier records

Definition 5.150 (Stored finite-detector record)**.** A *stored finite-detector record* preserves separately:

(i) exactly one source stage, chosen from

$$\begin{array}{cc} \text{windowed\_prethreshold}, & \text{repaired\_postthreshold}, \\ \text{kernel\_defect}, & \text{cokernel\_defect}; \end{array}$$

(ii) certificate status;

(iii) mathematical value;

(iv) soundness and completeness theorems;

(v) coverage and endpoint policy;

(vi) family, mesh, and non-adaptivity data;

(vii) excess provenance and artifact identity.

The standard v20 finite detector is sourced at windowed_prethreshold; any other stage requires its own stage-specific theorem or explicit stage-transport theorem.

Definition 5.151 (Stored higher-edge record)**.** A *stored higher-edge record* for degree n contains the exact repaired degree-n profile, representative, actual profile isomorphism, genuine realization or admissible replacement, domain and amplitude records, fixed extractor E_n with $E_n(0) = 0$, artifact-backed H^{-n} identification, naturality scope, change compatibility, and non-circularity data. Its default implication is only

$$\mathrm{Eval}_{n,W,\tau}(F) = 0 \Longrightarrow \mathrm{Ext}^n(A_n(F), k) = 0.$$

The reverse direction requires a separately stored natural zero-reflection or faithfulness theorem.

Definition 5.152 (Stored Type IV record)**.** A *stored Type IV record* is based on an actual persistence morphism

$$\phi_{i,W,\tau} : \mathrm{ColimProfile}_{i,W,\tau}(F_\bullet) \longrightarrow \mathrm{ApexProfile}_{i,W,\tau}(F_\infty)$$

and records fixed (W, τ), a finite monitored degree set, terminal-tameness evidence, and degreewise

$$\mu_{i,W,\tau} = \mathrm{tgdim}_W \ker \phi_{i,W,\tau}, \qquad u_{i,W,\tau} = \mathrm{tgdim}_W \operatorname{coker} \phi_{i,W,\tau}.$$

For one fixed (W, τ) and one finite monitored degree set I_{mon}, the canonical aggregate is

$$\mu_{\mathrm{Collapse}} := \sum_{i \in I_{\mathrm{mon}}} \mu_{i,W,\tau}, \qquad u_{\mathrm{Collapse}} := \sum_{i \in I_{\mathrm{mon}}} u_{i,W,\tau}.$$

No aggregation across distinct windows or thresholds is permitted. Its terminal status is exactly one of not_invoked, undefined, certified_zero, or obstructed; no numerical pair is stored for the first two statuses.

Definition 5.153 (Stored transient-defect record)**.** A *stored transient-defect record* separates terminal Type IV status from full-interior status

$$\mathrm{not\_checked}, \quad \mathrm{certified\_absent}, \quad \mathrm{certified\_present}$$

and long-transient status

$$\mathrm{not\_invoked}, \quad \mathrm{undefined}, \quad \mathrm{certified\_absent}, \quad \mathrm{certified\_present}.$$

Kernel and cokernel detector records retain their original detector stages and are not relabeled as terminal diagnostics.

Proposition 5.154 (Zero-looking stored output is not certified zero)**.** *An empty array, zero scalar, green dashboard, no-hit query, successful build, or agent statement "no defect found" does not establish detector zero or Type IV certified zero without the complete typed verifier record.*

Proof. The visual or serialized value does not establish source stage, certificate validity, completeness, actual morphism identity, terminal tameness, or diagnostic scope. Those fields determine the mathematical meaning of zero. □

Proposition 5.155 (Terminal zero does not close transient or apex nodes)**.** *A stored terminal Type IV certified-zero record does not by itself establish full-interior defect absence, long-transient defect absence, isomorphism, apex agreement, or external information preservation.*

Proof. Terminal-generic kernel and cokernel vanishings are weaker than global profile isomorphism and finite-to-infinite or external Return statements. The stronger conclusions require their own records and theorems. □

Proposition 5.156 (Detector stage is immutable under storage operations)**.** *Versioning, migration, serialization, indexing, or dashboard relabeling does not change the source stage of a stored detector record. A stage change requires a new typed detector record and the applicable stage-transport theorem.*

Proof. The detector theorem is scoped to its declared source stage. Platform operations can change representation or metadata but cannot prove that the detector predicate is preserved at another mathematical stage. □

Proposition 5.157 (Stored higher-edge reverse requires a separate theorem)**.** *The presence of a stored implication*

$$\mathrm{Eval}_{n,W,\tau}(F) = 0 \Longrightarrow \mathrm{Ext}^n(A_n(F), k) = 0$$

does not authorize the reverse implication without a separately verified zero-reflection or faithfulness record.

Proof. Storage preserves the direction and strength of the imported implication. Reversing it is an additional theorem obligation. □

Proposition 5.158 (Object similarity does not create a Type IV morphism)**.** *Stored barcode equality, matching Betti data, equal serialized representatives, or visually identical diagrams do not create the actual morphism $\phi_{i,W,\tau}$ required by the Type IV record.*

Proof. An actual persistence morphism includes compatible maps and source–target typing. Object-level similarity supplies no such map by itself. □

Y.13. Audit records

Definition 5.159 (Audit record)**.** An *audit record* is a Proof Store entry recording whether a proof object, certificate, artifact set, platform record, map, DAG, or lifecycle history is independently reconstructible.

Definition 5.160 (Audit result)**.** An *audit result* is one of:

(i) audit-pass;

(ii) audit-fail;

(iii) audit-incomplete;

(iv) audit-blocked;

(v) audit-superseded.

Definition 5.161 (Audit scope)**.** The *audit scope* specifies which objects, artifacts, dependencies, ledgers, diagnostics, maps, DAGs, execution records, and proof-store entries were checked.

Proposition 5.162 (Audit pass is not mathematical proof by itself)**.** *An audit-pass result shows reconstructibility within the declared audit scope, but does not by itself prove the underlying mathematical claim.*

Proof. Audit checks reconstructibility and record completeness. Mathematical proof requires the relevant logical or verifier-side conditions to be satisfied. Thus audit pass and proof are distinct. □

Remark 5.163 (Audit is necessary but not sufficient)**.** For high-integrity proof infrastructure, auditability is necessary. It is not sufficient unless the audited proof content is also valid.

Y.13bis. Audit completeness and theorem incompleteness

Definition 5.164 (Audit completeness record)**.** An *audit completeness record* states which artifacts, dependencies, ledger entries, diagnostic records, verification records, environment records, and status transitions were inspected.

Proposition 5.165 (Audit completeness is not theorem completeness)**.** *An audit-complete platform record does not imply that the underlying theorem has been proved.*

Proof. Audit completeness concerns reconstructibility and record coverage. Theorem completeness concerns mathematical discharge of the target obligations. The former can hold while a required theorem, bridge, interpretation, or Core condition remains unproved. □

Remark 5.166 (Audit can certify incompleteness)**.** A successful audit may correctly certify that a record is incomplete, blocked, or undefined. This is a valid audit outcome, not a proof failure of the audit system.

Y.13ter. Operational, semantic, and mathematical replay

Definition 5.167 (Operational replay record)**.** An *operational replay record* establishes that a declared sequence of platform actions reproduced the declared outputs under a declared environment and replay boundary.

Definition 5.168 (Semantic replay record)**.** A *semantic replay record* verifies that the replayed outputs preserve the theorem-relevant statements, scopes, statuses, source bindings, dependency directions, and artifact support relations consumed by a target.

Definition 5.169 (Mathematical reconstruction record)**.** A *mathematical reconstruction record* provides the proofs or verifier-side checks needed to reconstruct the target mathematical conclusion from the declared premises. It may consume operational and semantic replay records, but is not identified with either.

Proposition 5.170 (Replay hierarchy is strict)**.** *In general,*

$$\textit{operational replay} \not\Rightarrow \textit{semantic replay} \not\Rightarrow \textit{mathematical reconstruction}.$$

Each implication requires an additional preservation or proof record.

Proof. Operational replay concerns reproduced execution; semantic replay concerns preserved meaning; mathematical reconstruction concerns discharged proof obligations. These are different predicates with different evidence. □

Remark 5.171 (Build success scope)**.** Compilation, parsing, schema validation, rendering, and byte-for-byte reproduction are valuable operational checks. They prove only their declared modes unless interpretation and mathematical verification are also included.

Y.14. Proof Store queries

Definition 5.172 (Proof Store query)**.** A *Proof Store query* is a request to retrieve, filter, compare, rank, or inspect stored objects.

Definition 5.173 (Query result)**.** A *query result* is the list, set, graph, table, or summary returned by a Proof Store query.

Definition 5.174 (Status-filtered query)**.** A *status-filtered query* retrieves objects satisfying declared status constraints.

Definition 5.175 (Dependency query)**.** A *dependency query* retrieves dependency relations, map edges, DAG edges, proof-store links, or artifact bindings relevant to one or more objects.

Proposition 5.176 (Query result is retrieval, not verification)**.** *A Proof Store query result does not verify the returned objects.*

Proof. A query retrieves or organizes existing records. Verification requires checking declared criteria. Retrieval is not checking. □

Remark 5.177 (Search bias)**.** Query ranking may bias what users inspect. It should not bias what is accepted as valid.

Y.14bis. Query freshness, completeness, and negative-result discipline

Definition 5.178 (Query provenance record)**.** A *query provenance record* stores the queried corpus identity, query expression, filters, ranking policy, pagination or truncation policy, retrieval time or version, returned artifact identities, and unresolved retrieval failures.

Definition 5.179 (Negative query result)**.** A *negative query result* is a query response reporting no matching stored record under the declared corpus, query, filters, and retrieval limits.

Proposition 5.180 (Negative query does not prove nonexistence)**.** *A negative query result does not prove that a theorem, source statement, artifact, or dependency does not exist unless a separately justified completeness theorem covers the searched domain and query procedure.*

Proof. A query can miss content because of indexing gaps, stale mirrors, filters, synonyms, pagination, parsing errors, or corpus incompleteness. Nonexistence is stronger than failure to retrieve. □

Remark 5.181 (Query ranking is advisory)**.** Ranking and relevance scores may order review work but do not alter source authority, theorem strength, or permitted use.

Y.15. Platform security and access

Definition 5.182 (Access policy)**.** An *access policy* specifies who or what may read, write, modify, verify, audit, promote, demote, reject, supersede, or delete platform objects.

Definition 5.183 (Write authority)**.** A *write authority* is permission to create or modify Proof Store or Artifact Store entries.

Definition 5.184 (Promotion authority)**.** A *promotion authority* is permission to change an object's status to a stronger status.

Definition 5.185 (Demotion authority)**.** A *demotion authority* is permission to change an object's status to a weaker or safer status.

Definition 5.186 (Verification authority record)**.** A *verification authority record* identifies the agent, tool, or procedure authorized to perform a verification action and its authority scope.

Proposition 5.187 (Write authority is not promotion authority)**.** *Write authority does not imply promotion authority.*

Proof. Writing or modifying records is operational access. Promotion changes validity status. These permissions have different risk profiles and must be separately controlled. □

Remark 5.188 (Access control protects claim status)**.** Access control is not merely cybersecurity. It protects the integrity of mathematical status labels and proof records.

Y.15bis. Security, deletion, and immutable audit history

Definition 5.189 (Proof-relevant tombstone)**.** A *proof-relevant tombstone* is an immutable record that a proof-store object or artifact was withdrawn, deleted from active storage, access-restricted, or superseded. It preserves identity, prior status, dependency impact, authority, reason, and replacement pointer without preserving prohibited payload content when policy forbids retention.

Definition 5.190 (Dependency revocation event)**.** A *dependency revocation event* records that an artifact, theorem import, bridge, verifier environment, or authority record may no longer be consumed by specified targets. Affected target closures are recomputed or marked $\mathrm{undefined}$ until repaired.

Proposition 5.191 (Deletion does not erase dependency history)**.** *Removing an active artifact does not erase the historical fact that prior certificates depended on it.*

Proof. Audit and non-retroactive repair require reconstruction of the prior dependency state. A tombstone preserves that history while allowing current use to be revoked. □

Remark 5.192 (Security event is not mathematical negation)**.** Access revocation, quarantine, or deletion can block current evidence use without proving the negation of the underlying mathematical statement. The resulting theorem-facing status is normally $\mathrm{undefined}$ unless an independent rejecting proof exists.

Y.16. Platform manifest entries

Definition 5.193 (Platform manifest entry)**.** A *platform manifest entry* records:

(i) platform component;

(ii) action performed;

(iii) agent or process identifier;

(iv) input objects;

(v) output objects;

(vi) object classes;

(vii) object statuses;

(viii) version identifiers;

(ix) hashes or content identities, if used;

(x) verification or audit records, if any;

(xi) access and authority scope;

(xii) artifact references;

(xiii) lifecycle event identifier.

Definition 5.194 (Proof Store manifest)**.** A *Proof Store manifest* is a collection of platform manifest entries sufficient to reconstruct the lifecycle of a proof object, certificate record, map node, DAG node, bridge program, or stored artifact.

Proposition 5.195 (Platform manifests support reproducibility)**.** *Platform manifests support reproducibility by recording actions, versions, identities, statuses, authorities, and artifact references.*

Proof. Reproducibility requires reconstructing not only the content but also the workflow and status history. Platform manifests record these elements. □

Remark 5.196 (Manifest still needs execution schema)**.** The execution-level schema for replay, environment capture, run configuration, and computational reproducibility is specified in Appendix Z.

Y.16bis. Machine-readable sidecars and schema authority

Definition 5.197 (Platform sidecar)**.** A *platform sidecar* is a JSON, YAML, CSV, database, graph, signature, hash-list, or manifest artifact associated with a human-readable or formal proof object.

Definition 5.198 (Sidecar authority record)**.** A *sidecar authority record* declares whether a sidecar is descriptive, operational, verifier-consumed, audit-consumed, or normative, and identifies the preservation and synchronization checks connecting it to the represented object.

Proposition 5.199 (Schema validity is not semantic equivalence)**.** *A sidecar that validates against its schema is not thereby semantically equivalent to the TeX, PDF, proof assistant object, source theorem, or final registry entry it purports to represent.*

Proof. Schema validation checks permitted fields and syntax. Semantic equivalence additionally requires content correspondence, status preservation, source and artifact identity, and authority mapping. □

Remark 5.200 (Generated sidecars before final CR)**.** Before final-CR promotion, generated registry sidecars are candidate operational artifacts only. They may support census and audit preparation but may not declare themselves the final normative register.

Y.17. Proof Store and DAG synchronization

Definition 5.201 (Map-store synchronization)**.** *Map-store synchronization* is the process of keeping Validity Map nodes and Proof Store entries consistent in identifiers, statuses, artifacts, and lifecycle history.

Definition 5.202 (DAG-store synchronization)**.** *DAG-store synchronization* is the process of keeping Certificate DAG nodes, edges, discharge records, and Proof Store entries mutually consistent.

Definition 5.203 (Synchronization failure)**.** A *synchronization failure* occurs when a Proof Store entry, Validity Map node, Certificate DAG node, artifact binding, or status record becomes inconsistent with its counterpart.

Proposition 5.204 (Synchronization failure requires review)**.** *A synchronization failure affecting a necessary proof dependency requires review and may require demotion or quarantine.*

Proof. If the Proof Store and map/DAG records disagree, an auditor cannot reliably reconstruct dependency status. A necessary dependency with inconsistent records is not audit-stable until reviewed. □

Remark 5.205 (Synchronization is governance, not proof)**.** Synchronization keeps records coherent. It does not by itself verify the mathematical content of those records.

Y.17bis. Store synchronization, mirrors, and ledger defects

Definition 5.206 (Mirror store)**.** A *mirror store* is a replicated or synchronized copy of Proof Store or Artifact Store content.

Definition 5.207 (Platform synchronization defect)**.** A *platform synchronization defect* is a discrepancy between store, map, DAG, registry, artifact, or mirror records that affects identity, status, dependency, or permitted use. It is denoted

$$\delta^{Y\text{-}sync}.$$

Definition 5.208 (Platform identity defect)**.** A *platform identity defect* is an unresolved inconsistency in hash binding, version binding, artifact identity, or object identity. It is denoted

$$\delta^{Y\text{-}id}.$$

Definition 5.209 (Platform adequacy defect)**.** A *platform adequacy defect* is a discrepancy in artifact adequacy, source binding, or the support relation consumed by a proof or audit route. It is denoted

$$\delta^{Y\text{-}adequacy}.$$

Proposition 5.210 (Mirroring does not multiply evidence)**.** *Replicating an artifact or proof-store entry across mirror stores does not create independent mathematical evidence.*

Proof. Mirroring copies or synchronizes content. It improves availability and redundancy, but it does not create a new proof, verifier-side record, bridge theorem, or independent artifact adequacy argument. □

Remark 5.211 (Platform defect catalog registration)**.** The symbols $\delta^{Y\text{-}sync}$, $\delta^{Y\text{-}id}$, and $\delta^{Y\text{-}adequacy}$ are local platform-layer defect namespaces until Appendix M registers their relation to any gate-scoped $\mathfrak{d}_{\mathrm{met}}$, local alias, refinement, aggregate, or disjointness convention. The identity and adequacy components are typically status-ledger or qualitative-governance components, not metric-admissible components; identity or adequacy failure routes to undefined or reject and is not repaired by scalar metric slack unless Appendix M and the local certificate record explicitly supply a metric-admissible conversion theorem. They must not be double-counted with artifact, DAG, search, or bridge defects from Appendices V–X.

Y.17ter. Platform defects, selected proof routes, and no-double-counting

Definition 5.212 (Platform metric component)**.** A *platform metric component* is a platform-induced discrepancy $\delta_e \in \mathcal{V}$ for which actual windowed pre-threshold profiles $B_e^{\mathrm{src}}, B_e^{\mathrm{tgt}}$ and evidence

$$d_B\left(B_e^{\mathrm{src}}, B_e^{\mathrm{tgt}}\right) \le \mathrm{val}_B(\delta_e)$$

are declared. The scalarization

$$\mathrm{val}_B : (\mathcal{V}, \oplus, 0_{\mathcal{V}}, \preceq) \longrightarrow [0, \infty]$$

is monotone, satisfies $\mathrm{val}_B(0_{\mathcal{V}}) = 0$, and is subadditive:

$$\mathrm{val}_B(q \oplus q') \le \mathrm{val}_B(q) + \mathrm{val}_B(q').$$

Definition 5.213 (Selected platform proof route)**.** A *selected platform proof route P* is the finite admissible target sub-DAG, including all necessary branches, whose records are actually consumed by one target certificate. Its metric ledger is

$$\mathfrak{d}_Y(P) := \bigoplus_{e \in S_P} \delta_e,$$

where S_P contains each consumed primitive metric discrepancy exactly once.

Definition 5.214 (Platform ledger relation record)**.** A *platform ledger relation record* declares each local platform defect name to be exactly one of

$$\mathrm{alias}, \qquad \mathrm{refinement}, \qquad \mathrm{aggregate}, \qquad \mathrm{disjoint}$$

relative to Appendix M and to related Search, Bridge, DAG, artifact, execution, or replay defects.

Proposition 5.215 (Only actual profile discrepancies enter the metric ledger)**.** *Synchronization failure, identity failure, source-authority failure, adequacy failure, missing proof, missing Return, missing reduction, status conflict, or semantic-replay failure does not enter $\mathfrak{d}_Y(P)$ unless a specific theorem converts it to an actual profile-discrepancy upper bound at the consumed scope.*

Proof. These failures are normally non-scalar existence, authority, semantic, or governance obligations. The metric ledger contains only elements with certified profile-bound support. □

Proposition 5.216 (Mirrors and aliases are counted once)**.** *Replicas, mirrors, aliases, repeated query hits, and multiple platform labels for the same primitive discrepancy do not create additional ledger contributions or independent evidence.*

Proof. They refer to the same primitive content or discrepancy unless a disjointness theorem proves otherwise. Counting each representation would double-count one cause. □

Remark 5.217 (Platform threshold-gap use)**.** When platform-induced metric discrepancies are consumed, the target must verify

$$\mathrm{val}_B\big(\mathfrak{d}_Y(P)\big) < \mathrm{gap}_\tau(\mathfrak{B}_P),$$

with componentwise profile-bound support. Exact zero transport uses $2\varepsilon \le \tau$; the operational reserve uses $2\varepsilon < \tau$. No numerical margin repairs a non-scalar platform obligation.

Definition 5.218 (Higher-coherence synchronization record)**.** A *higher-coherence synchronization record* supplies the compatibility data required to pass from pairwise store or mirror agreement to a multi-object descent, atlas, or higher-coherence claim.

Proposition 5.219 (Pairwise synchronization is not higher coherence)**.** *Pairwise agreement among Proof Store, Artifact Store, Claim Registry, DAG service, and mirror stores does not establish higher coherence, global descent, or local-to-global compatibility without a higher-coherence synchronization record.*

Proof. Pairwise equalities or compatibility checks need not satisfy triple-overlap, cocycle, associativity, or descent conditions. Those are additional obligations. □

Y.18. Summary of AI Platform and Proof Store

Theorem 5.220 (Appendix Y platform-store package)**.** *Within the Search / Platform layer, the following package is available.*

(i) The AI Platform supports search, storage, verification, audit, replay, and reproducibility workflows.

(ii) Platform actions are scoped.

(iii) A Proof Store stores proof-related objects but does not certify them by storage.

(iv) Stored Core-facing objects are not Core verified merely by storage.

(v) Object class and object status must be separated.

(vi) Registration is not validation.

(vii) Artifact availability is not artifact adequacy.

(viii) Version changes may require re-verification.

(ix) Status inheritance requires justification.

(x) Hash matches support identity, not validity.

(xi) Hash mismatches invalidate affected artifact bindings until resolved.

(xii) Lifecycle progress is not validity monotonicity.

(xiii) Promotion requires recorded evidence.

(xiv) AI output must be status-labeled, and AI storage does not validate it.

(xv) AI-generated Core-facing records require Core verification.

(xvi) *Verification results are scoped to declared criteria.*

(xvii) *Audit pass is not mathematical proof by itself.*

(xviii) *Query results are retrieval, not verification.*

(xix) *Write authority is not promotion authority.*

(xx) *Platform manifests support reproducibility but do not themselves prove mathematical claims.*

(xxi) *Map-store and DAG-store synchronization failures require review when they affect necessary dependencies.*

Proof. Items (*i*)–(*ii*) follow from the AI Platform and platform action definitions and Proposition 5.19. Item (*iii*) is Proposition 5.31. Item (*iv*) is Proposition 5.32. Item (*v*) follows from object class and status definitions and Proposition 5.42. Item (*vi*) is Proposition 5.55. Item (*vii*) is Proposition 5.68. Item (*viii*) is Proposition 5.85. Item (*ix*) is Proposition 5.86. Items (*x*)–(*xi*) are Propositions 5.92 and 5.93. Item (*xii*) is Proposition 5.108. Item (*xiii*) is Proposition 5.118. Item (*xiv*) follows from Propositions 5.128 and 5.129. Item (*xv*) is Proposition 5.130. Item (*xvi*) is Proposition 5.144. Item (*xvii*) is Proposition 5.162. Item (*xviii*) is Proposition 5.176. Item (*xix*) is Proposition 5.187. Item (*xx*) follows from Proposition 5.195. Item (*xxi*) is Proposition 5.204.

□

Y.18bis. v20 platform compatibility and hardening package

Theorem 5.221 (Appendix Y v20.0 platform compatibility and hardening package)**.** *Within the Search / Platform layer, the following additional v20 compatibility and hardening package is available.*

(i) *Platform sovereignty prevents storage, indexing, retrieval, hashing, synchronization, display, or access approval from changing verifier-side status.*

(ii) *Platform-to-verifier use requires a conversion record with result pass.*

(iii) *Proof Store status is distinct from Appendix U labels, Chapter 1 roles, and current CR-v20 claim-governance status.*

(iv) *Platform claim use requires current CR-v20 preflight.*

(v) *Available artifacts must still pass adequacy preflight.*

(vi) *Identity witnesses do not prove semantic preservation across transformed artifacts.*

(vii) *Theorem-facing promotion requires a platform promotion certificate.*

(viii) *Repairs are non-retroactive and create new records or revisions.*

(ix) *AI context inclusion is not theorem evidence.*

(x) *AI consensus is not proof.*

(xi) *Verification results map to pass, reject, undefined, or non-verdict only within their declared criteria.*

(xii) *Verification scope overrun invalidates the attempted use.*

(xiii) *Audit completeness is not theorem completeness.*

(xiv) *Mirror stores do not multiply evidence.*

(xv) *Platform synchronization, identity, and adequacy defects require Appendix M catalog registration before global budget consumption.*

Proof. Items (*i*)–(*ii*) are Definition 5.4 and Propositions 5.6 and 5.7. Item (*iii*) is Definition 5.44 and Proposition 5.46. Item (*iv*) is Definition 5.57 and Proposition 5.58. Item (*v*) is Definition 5.70 and Proposition 5.72. Item (*vi*) is Proposition 5.97. Items (*vii*)–(*viii*) are Propositions 5.123 and 5.124. Items (*ix*)–(*x*) are Propositions 5.134 and 5.135. Items (*xi*)–(*xii*) are Definition 5.146 and Proposition 5.148. Item (*xiii*) is Proposition 5.165. Item (*xiv*) is Proposition 5.210. Item (*xv*) is Remark 5.211. □

Y.18ter. v20 Proof Store extension package

Theorem 5.222 (Appendix Y v20 Proof Store extension package)**.** *The v20 platform layer additionally enforces the following.*

(i) *Final Claim Register promotion remains deferred until the document family and dependency closure are frozen.*

(ii) *Platform identifiers and machine-readable exports are not final Claim UIDs or normative registry authority by themselves.*

(iii) *Untrusted payloads cannot alter verifier or governance policy.*

(iv) *Store co-location does not compose a proof; theorem use is controlled by the verifier-side projection.*

(v) *Byte identity, semantic identity, and authority identity remain distinct.*

(vi) *Problem-Bridge source, realization, certificate, reduction, higher edge, Return, and conclusion records remain separate.*

(vii) *Known-Theorem Recovery uses immutable source binding and conclusion quarantine.*

(viii) *Finite certificates do not establish infinite or external conclusions without the required reduction and Return records.*

(ix) *Detector, higher-edge, Type IV, transient, and full-interior records retain their canonical v20 types.*

(x) *Operational replay, semantic replay, and mathematical reconstruction remain distinct.*

(xi) *Negative retrieval does not prove nonexistence without a completeness theorem.*

(xii) *Proof-relevant deletion preserves immutable dependency history through tombstones or equivalent audit records.*

(xiii) *Only certified actual profile discrepancies enter the selected-route metric ledger.*

(xiv) *Mirrors, aliases, and repeated representations do not multiply evidence or metric defects.*

Proof. Items (*i*)–(*ii*) follow from Definitions 5.10, 5.11, and Propositions 5.12, 5.13. Items (*iii*)–(*iv*) follow from Definitions 5.21, 5.23, 5.34, and Proposition 5.37. Item (*v*) is Proposition 5.102. Items (*vi*)–(*viii*) follow from Definitions 5.110, 5.74, 5.75, and Propositions 5.77, 5.113. Item (*ix*) follows from Definitions 5.150, 5.151, 5.152, and 5.153. Item (*x*) is Proposition 5.170. Item (*xi*) is Proposition 5.180. Item (*xii*) follows from Definition 5.189 and Proposition 5.191. Items (*xiii*)–(*xiv*) follow from Definitions 5.212, 5.213, 5.214, and Propositions 5.215, 5.216. □

Y.19. Explicit exclusions

The following are not asserted in Appendix Y.

- No stored object is certified by storage alone.
- No stored Core-facing object is Core verified by storage alone.
- No registered claim is validated by registration alone.
- No artifact is adequate merely because it is available.
- No hash match proves mathematical validity.
- No version tag proves theorem status.
- No lifecycle progress proves increasing validity.
- No AI output is validated by being stored.
- No AI confidence value is treated as proof.
- No AI-generated Core-facing record is accepted without Core verification.
- No query result is treated as verification.
- No audit pass is treated as mathematical proof by itself.
- No write authority is treated as promotion authority.
- No promotion is allowed without recorded evidence.
- No status inheritance is allowed without justification.
- No Proof Store entry replaces B-Gate^{+}.
- No Proof Store entry replaces the repaired topological condition

$$\mathrm{Eval}_{n,W,\tau}(F) = 0.$$

- No Proof Store entry replaces admissible representative records

$$C_{\tau,W}F.$$

- No Proof Store entry replaces monitored Type IV diagnostics.
- No Proof Store entry replaces tower comparison artifacts

$$\phi_{i,W,\tau}.$$

- No Proof Store entry replaces the budget condition

$$\mathrm{val}_B(\mathfrak{d}_{\mathrm{met}}) < \mathrm{gap}_\tau(\mathfrak{B}).$$

- No platform storage event turns a Bridge Program into a bridge theorem.
- No platform record turns a Validity Map into a theorem.
- No map-store or DAG-store synchronization event proves the synchronized mathematical content.

- No platform query, retrieval context, dashboard view, or AI context record is theorem evidence by inclusion alone.
- No AI consensus record is treated as proof.
- No content hash, signature, or immutable pointer proves semantic preservation across transformed artifacts.
- No mirror store or replicated artifact creates independent mathematical evidence.
- No platform repair retroactively rewrites an old verdict.
- No operational access permission discharges theorem-facing obligations.
- No platform identifier, generated UID, JSON row, CSV row, or database key is treated as a final Claim UID before final-CR promotion.
- No local registry, sidecar, or schema-valid export is treated as the normative final Claim Register by serialization alone.
- No prompt, retrieved snippet, filename, metadata field, or imported instruction is treated as source or governance authority.
- No co-located collection of proof fragments is treated as a composed proof without typed dependency discharge.
- No byte identity is treated as semantic or authority identity without the required connecting record.
- No stored Problem-Bridge chain skeleton is treated as a completed realization–certificate–reduction–Return chain.
- No finite observation, finite prefix, numerical plateau, stored apex, or replayed finite computation is treated as finite-to-infinite reduction.
- No benchmark conclusion or conclusion-derived witness is allowed to leak into an upstream Known-Theorem Recovery construction.
- No zero-looking stored value is treated as a certified detector or Type IV zero record.
- No terminal Type IV zero is treated as full-interior, long-transient, isomorphism, apex-agreement, or external information-preservation evidence.
- No operational replay is treated as semantic replay or mathematical reconstruction by status inheritance.
- No negative query result is treated as proof of nonexistence without a search-completeness theorem.
- No mirror, replica, alias, duplicate query hit, or repeated label multiplies mathematical evidence or metric defect.
- No identity, adequacy, source-authority, status, proof, reduction, Return, or replay failure is repaired by metric slack without a certified profile-bound conversion theorem.

Y.20. Provenance and status

The material in this appendix depends on:

(i) the Agent Semantics of Appendix U;

(ii) the Hunter / Mapper / Lifter protocols of Appendix V;

(iii) the Bridge Programs of Appendix W;

(iv) the Validity Map and Global Certificate DAG of Appendix X;

(v) the claim taxonomy and manifest discipline of Chapter 1;

(vi) the current CR-v20 local claim-use and repair-governance baseline, with final CR deferred;

(vii) the window-first, threshold-gap, and repaired-zero discipline of Appendix A;

(viii) the detector-stage and stage-transport discipline of Appendix B, including completeness and representative control;

(ix) the degree-n eligible-realization and higher-edge discipline of Appendix C;

(x) the actual-morphism Type IV, transient-defect, and tower finite-to-infinite discipline of Appendix D;

(xi) the coverage, atlas, overlap, Restart, higher-coherence, and local-to-global discipline of Appendix E, when invoked;

(xii) the atomic-status and non-compensation discipline of Appendix F;

(xiii) the canonical manifest and verifier discipline of Appendix G;

(xiv) the Core sovereignty doctrine of Chapter 7;

(xv) the non-claim, interface-boundary, Problem-Bridge, and Return discipline of Chapter 8;

(xvi) the ledger catalog and no-double-counting discipline of Appendix M;

(xvii) the frozen local Problem Interface and Known-Theorem Recovery artifacts of Part IV;

(xviii) the execution and reproducibility schema of Appendix Z when consumed, without treating this forward dependency as already discharged.

Its role is to provide the storage, registry, lifecycle, synchronization, and platform infrastructure required for auditable proof management without confusing storage with certification.

Status labels for Appendix Y. *Search / Platform definitions:* Definitions 5.15, 5.16, 5.17, 5.18, 5.26, 5.27, 5.28, 5.29, 5.30, 5.39, 5.40, 5.41, 5.52, 5.53, 5.54, 5.64, 5.65, 5.66, 5.67, 5.80, 5.81, 5.82, 5.83, 5.84, 5.88, 5.89, 5.90, 5.91, 5.104, 5.105, 5.106, 5.107, 5.114, 5.115, 5.116, 5.117, 5.125, 5.126, 5.127, 5.141, 5.142, 5.143, 5.159, 5.160, 5.161, 5.172, 5.173, 5.174, 5.175, 5.182, 5.183, 5.184, 5.185, 5.186, 5.193, 5.194, 5.201, 5.202, 5.203.
Additional v20.0 compatibility-hardening definitions: Definitions 5.4, 5.5, 5.44, 5.45, 5.57, 5.70, 5.71, 5.95, 5.96, 5.121, 5.122, 5.132, 5.133, 5.146, 5.147, 5.164, 5.206, 5.207, 5.208, 5.209.
Search / Platform propositions/theorems: Propositions 5.19, 5.31, 5.32, 5.42, 5.55, 5.68, 5.85, 5.86, 5.92, 5.93, 5.108, 5.118, 5.119, 5.128, 5.129, 5.130, 5.144, 5.162, 5.176, 5.187, 5.195, 5.204, Theorem 5.220.
Additional v20.0 compatibility-hardening propositions/theorems: Propositions 5.6, 5.7, 5.46, 5.58, 5.72, 5.97, 5.123, 5.124, 5.134, 5.135, 5.148, 5.165, 5.210, Theorem 5.221.

Operational cautions: Remarks 5.1, 5.2, 5.3, 5.20, 5.33, 5.43, 5.56, 5.69, 5.87, 5.94, 5.109, 5.120, 5.131, 5.145, 5.163, 5.177, 5.188, 5.196, 5.205.
Additional v20.0 compatibility-hardening operational cautions: Remarks 5.8, 5.47, 5.59, 5.73, 5.98, 5.136, 5.149, 5.166, 5.211.
Additional v20.0 Proof Store extension definitions: Definitions 5.137, 5.101, 5.36, 5.99, 5.48, 5.190, 5.35, 5.10, 5.218, 5.60, 5.11, 5.169, 5.179, 5.167, 5.22, 5.9, 5.214, 5.212, 5.197, 5.21, 5.110, 5.189, 5.178, 5.74, 5.76, 5.61, 5.213, 5.100, 5.168, 5.198, 5.75, 5.138, 5.150, 5.151, 5.153, 5.152, 5.111, 5.23, 5.34, 5.49.
Additional v20.0 Proof Store extension propositions/theorems: Propositions 5.139, 5.78, 5.112, 5.37, 5.24, 5.191, 5.156, 5.62, 5.113, 5.157, 5.102, 5.140, 5.12, 5.216, 5.50, 5.180, 5.13, 5.158, 5.215, 5.219, 5.170, 5.199, 5.77, 5.155, 5.38, 5.154.
Theorems 5.222.
Additional v20.0 Proof Store extension cautions: Remarks 5.171, 5.63, 5.200, 5.79, 5.217, 5.25, 5.181, 5.103, 5.14, 5.192, 5.51.
Explicit exclusions: Section Y.19.

6 Appendix Z: Execution and Reproducibility Schema

Z.1. Scope and reproducibility status

This appendix defines the execution and reproducibility schema for the AK framework. It belongs to the Search / Platform Appendices. It does not enlarge the Core mathematical package of Chapters 1–8.

The central rule of this appendix is:

Reproducibility supports audit; it is not proof by itself.

A reproducible execution record may allow another auditor to reconstruct a run, recompute artifacts, verify hashes, inspect diagnostics, replay ledger aggregation, replay gate checks, and inspect platform actions. However, reproducibility alone does not establish mathematical validity.

In particular, no execution record, replay log, run configuration, environment hash, container image, output file, reproducibility report, replay-pass result, generated sidecar, stored apex, finite plateau, or successful build may replace:

$$B\text{-Gate}^{+}, \qquad \mathrm{Eval}_{n,W,\tau}(F) = 0, \qquad C_{\tau,W}F, \qquad \mathrm{Ext}^{n}(A_n(F), k),$$
$$\phi_{i,W,\tau}, \qquad (\mu_{\mathrm{Collapse}},\ u_{\mathrm{Collapse}}) = (0,0), \qquad \mathrm{val}_B(\mathfrak{d}_{\mathrm{met}}) < \mathrm{gap}_{\tau}(\mathfrak{B}),$$
$$\text{a required finite-to-infinite reduction,} \qquad \text{a separately discharged Return theorem.}$$

Remark 6.1 (Search / Platform status)**.** This appendix specifies how runs, artifacts, ledgers, manifests, diagnostics, gate records, and environments should be recorded so that results are reconstructible. It does not create new theorems.

Remark 6.2 (Relation to Appendix Y)**.** Appendix Y defines the AI Platform and Proof Store. The present appendix defines the execution-level schema by which platform entries become replayable and audit-readable.

Remark 6.3 (Final platform appendix)**.** This is the final Search / Platform Appendix. Its role is to close the operational infrastructure around the AK Core while preserving the distinction between execution, reproducibility, auditability, Core certification, and theorem-level proof.

Z.2. Execution run

Definition 6.4 (Execution run)**.** An *execution run* is a recorded instance of an AK workflow, computation, verification, audit, replay, search, lifting, diagnostic calculation, ledger calculation, gate check, or certificate assembly.

Definition 6.5 (Run identifier)**.** A *run identifier* is a unique label assigned to an execution run.

Definition 6.6 (Run scope)**.** The *run scope* specifies what the execution run is intended to do. Examples include:

(i) compute persistence artifacts;

(ii) compute repaired evaluations

$$\mathrm{Eval}_{i,W,\tau}(F);$$

(iii) construct or check admissible representative records

$$C_{\tau,W}F;$$

(iv) compute or check tower comparison artifacts

$$\phi_{i,W,\tau};$$

(v) compute monitored Type IV diagnostics;

(vi) evaluate ledger budgets;

(vii) check B-Gate^{+};

(viii) replay a certificate;

(ix) verify a formal fragment;

(x) assemble a Certificate DAG;

(xi) reproduce a search result;

(xii) audit artifact completeness.

Definition 6.7 (Run authority)**.** A *run authority* is the agent, tool, platform component, or verification procedure authorized to execute or validate a run within its declared scope.

Proposition 6.8 (Run scope limits interpretation)**.** *An execution run establishes only what lies within its declared run scope.*

Proof. A run is configured to perform a declared task. Outputs outside that task are not checked by the run unless explicitly included. Therefore interpretation is limited by the run scope. □

Remark 6.9 (Run success is scoped)**.** A successful run may mean that a script executed, a file was generated, a diagnostic was computed, a gate clause was checked, or a proof assistant accepted a formal fragment. These meanings are distinct and must be recorded separately.

Z.3. Execution schema

Definition 6.10 (Execution schema)**.** An *execution schema* is a structured specification of the inputs, parameters, environment, tools, artifacts, commands, outputs, logs, and verification criteria of an execution run.

Definition 6.11 (run.yaml)**.** A run.yaml file is a human-readable execution schema file describing an AK execution run. It should contain:

(i) run identifier;

(ii) run scope;

(iii) run authority;

(iv) input artifacts;

(v) output artifacts;

(vi) commands or workflow steps;

(vii) environment references;

(viii) parameter values;

(ix) random seeds, if applicable;

(x) repaired evaluation configuration, if applicable;

(xi) representative configuration, if applicable;

(xii) tower artifact configuration, if applicable;

(xiii) ledger configuration;

(xiv) diagnostic configuration;

(xv) gate configuration;

(xvi) verification criteria;

(xvii) expected outputs;

(xviii) artifact hashes or hash policy;

(xix) replay instructions.

Definition 6.12 (Execution parameter)**.** An *execution parameter* is a value controlling an execution run. Examples include:

$$\tau, \qquad W, \qquad i, \qquad \beta, \qquad M(\tau), \qquad s, \qquad \text{grid size}, \qquad \text{random seed}.$$

Definition 6.13 (Parameter binding)**.** A *parameter binding* is a recorded assignment of a value to an execution parameter within a run.

Proposition 6.14 (Execution schema is required for replayability)**.** *A run is replayable only if its execution schema records sufficient information to reconstruct the run.*

Proof. Replay requires reconstructing inputs, parameters, commands, environment, and expected outputs. If these data are missing, the run cannot be reproduced. Therefore a sufficient execution schema is required. □

Remark 6.15 (Schema completeness is task-relative)**.** A schema sufficient for recomputing barcodes may not be sufficient for replaying a proof assistant verification or a Certificate DAG assembly. Schema completeness depends on the run scope.

Z.4. Artifact manifest

Definition 6.16 (Artifact manifest)**.** An *artifact manifest* is a structured list of all artifacts used or produced by an execution run.

Definition 6.17 (Input artifact)**.** An *input artifact* is an artifact required by an execution run.

Definition 6.18 (Output artifact)**.** An *output artifact* is an artifact produced by an execution run.

Definition 6.19 (Intermediate artifact)**.** An *intermediate artifact* is an artifact produced during a run and consumed by later steps of the same or dependent runs.

Definition 6.20 (Artifact manifest entry)**.** An *artifact manifest entry* records:

(i) artifact identifier;

(ii) artifact role: input, output, intermediate, log, reference, or replay artifact;

(iii) artifact type;

(iv) storage location or content address;

(v) version identifier;

(vi) hash or checksum, if used;

(vii) producing run, if applicable;

(viii) consuming runs, if known;

(ix) adequacy notes;

(x) audit notes.

Proposition 6.21 (Artifact manifest supports audit but does not prove adequacy)**.** *An artifact manifest supports auditability, but it does not by itself prove that the artifacts are adequate for the claims that cite them.*

Proof. The manifest records artifact existence, identity, and role. Adequacy requires checking that each artifact supports the relevant claim, computation, diagnostic, ledger entry, gate clause, or certificate condition. Thus manifest presence and artifact adequacy are distinct. □

Remark 6.22 (Completeness versus adequacy)**.** A manifest may be complete but still inadequate if the listed artifacts do not support the intended claims. Both completeness and adequacy must be considered during audit.

Z.5. Environment capture

Definition 6.23 (Execution environment)**.** An *execution environment* is the collection of software, hardware, operating system, dependencies, libraries, proof assistant versions, computation engines, container images, and configuration settings used in an execution run.

Definition 6.24 (Environment capture)**.** *Environment capture* is the process of recording the execution environment sufficiently to support replay.

Definition 6.25 (Environment identifier)**.** An *environment identifier* is a label, hash, container digest, dependency lockfile, proof assistant environment record, or platform record identifying an execution environment.

Definition 6.26 (Environment compatibility)**.** *Environment compatibility* is a declared relation specifying when a different environment may be accepted as replay-equivalent to the original environment.

Proposition 6.27 (Environment capture is required for computational replay)**.** *Computational replay requires environment capture sufficient for the declared run scope.*

Proof. Different environments may produce different outputs due to dependency versions, numerical libraries, proof assistant versions, or system configuration. Replay requires controlling or recording these dependencies. Therefore environment capture is required. □

Remark 6.28 (Environment equality is stronger than needed in some cases)**.** Exact environment equality may be unnecessary for purely formal or deterministic checks if semantic compatibility is proved. However, absent such a proof, environment capture should be conservative.

Z.6. Determinism and randomness

Definition 6.29 (Deterministic run)**.** A run is *deterministic* if the same inputs, parameters, environment, and commands produce the same outputs under the declared replay equivalence.

Definition 6.30 (Randomized run)**.** A run is *randomized* if it depends on random seeds, sampling, stochastic optimization, randomized search, probabilistic algorithms, or nondeterministic execution.

Definition 6.31 (Random seed policy)**.** A *random seed policy* specifies how random seeds are chosen, recorded, replayed, and varied.

Definition 6.32 (Nondeterminism record)**.** A *nondeterminism record* records sources of nondeterminism in a run, including parallel execution, hardware nondeterminism, randomized algorithms, sampling procedures, and external service calls.

Proposition 6.33 (Randomized runs require seed or distribution records)**.** *A randomized run is reproducible only if the seed, sampling policy, nondeterminism record, or probability distribution is recorded sufficiently for the run scope.*

Proof. Without seed, distribution, or nondeterminism records, the random choices cannot be reconstructed or statistically characterized. Therefore reproducibility requires recording them. □

Remark 6.34 (Statistical reproducibility)**.** For randomized search, exact replay may be less important than statistical reproducibility. If so, the statistical criterion must be declared.

Z.7. Repaired evaluation execution files

Definition 6.35 (eval.json)**.** An eval.json file is a structured record of repaired evaluation computations:

$$\mathrm{Eval}_{i,W,\tau}(F) = T_{\tau}^{\mathrm{bar}}\big(\mathsf{W}_W \mathbf{P}_i(F)\big).$$

Definition 6.36 (Evaluation execution entry)**.** An *evaluation execution entry* records:

(i) input object identifier;

(ii) monitored degree i;

(iii) window W;

(iv) threshold τ;

(v) persistence extraction method;

(vi) barcode policy;

(vii) window restriction policy;

(viii) threshold deletion policy;

(ix) resulting repaired profile;

(x) vanishing or nonvanishing result, if checked;

(xi) artifact references.

Proposition 6.37 (Repaired evaluation replay requires evaluation records)**.** *A repaired evaluation is replayable only if the evaluation execution record contains sufficient data to reconstruct*

$$\mathrm{Eval}_{i,W,\tau}(F).$$

Proof. The repaired evaluation depends on the input object, persistence extraction, degree, window, threshold, barcode policy, and threshold deletion policy. Without these records, an auditor cannot reconstruct the evaluation. □

Remark 6.38 (Evaluation summary is not enough)**.** A statement such as

$$\mathrm{Eval}_{n,W,\tau}(F) = 0$$

is not audit-readable unless the supporting evaluation computation or proof can be reconstructed.

Z.8. Ledger execution files

Definition 6.39 (ledger.json)**.** A ledger.json file is a structured record of ledger components, aggregation semantics, gap values, and budget verdicts pass, reject, undefined, or not_invoked used in an execution run.

Definition 6.40 (Ledger execution entry)**.** A *ledger execution entry* records:

(i) ledger semantics;

(ii) component identifiers;

(iii) component values or bounds;

(iv) component sources;

(v) aggregation law;

(vi) declared metric ledger value $\mathfrak{d}_{\mathrm{met}}$;

(vii) scalarized budget $\mathrm{val}_B(\mathfrak{d}_{\mathrm{met}})$;

(viii) protected-profile clearance value $\mathrm{gap}_\tau(\mathfrak{B})$;

(ix) strict comparison convention;

(x) budget verdict pass, reject, undefined, or not_invoked;

(xi) artifact references.

Proposition 6.41 (Ledger replay requires ledger execution records)**.** *A budget comparison is replayable only if the ledger execution record contains sufficient component and aggregation data.*

Proof. Budget replay requires recomputing or checking

$$\mathrm{val}_B(\mathfrak{d}_{\mathrm{met}}) < \mathrm{gap}_\tau(\mathfrak{B}).$$

Without component values, aggregation semantics, and gap values, the comparison cannot be reconstructed. □

Remark 6.42 (Ledger file is not budget pass by itself)**.** The existence of ledger.json does not imply budget pass. The recorded comparison must actually satisfy the declared strict inequality.

Z.9. Diagnostic execution files

Definition 6.43 (diagnostics.json)**.** A diagnostics.json file is a structured record of diagnostic computations, including declared tower comparison artifacts, kernel and cokernel defect data, terminal-tame dimension measurements, interior-defect status fields, and Type IV verdicts.

Definition 6.44 (Diagnostic execution entry)**.** A *diagnostic execution entry* records:

(i) diagnostic type;

(ii) monitored degree;

(iii) window;

(iv) threshold;

(v) tower comparison artifact

$$\phi_{i,W,\tau};$$

(vi) source profile

$$\mathrm{ColimProfile}_{i,W,\tau}(F_\bullet);$$

(vii) target profile

$$\mathrm{ApexProfile}_{i,W,\tau}(F_\infty);$$

(viii) kernel defect object;

(ix) cokernel defect object;

(x) tgdim_W-kernel and cokernel dimensions;

(xi) values of μ_{Collapse} and u_{Collapse}, if applicable;

(xii) diagnostic verdict;

(xiii) artifact references.

Proposition 6.45 (Diagnostic replay requires diagnostic execution records)**.** *A monitored Type IV diagnostic is replayable only if the diagnostic execution record contains sufficient data to reconstruct the tower comparison artifact and defect calculations.*

Proof. Type IV diagnostics are defined by kernel and cokernel defect objects of the declared tower comparison artifact

$$\phi_{i,W,\tau}.$$

Replay requires reconstructing this artifact and its defect dimensions. Therefore the diagnostic execution record must include the required data. □

Remark 6.46 (Diagnostic summary is not enough)**.** A line saying

$$(\mu_{\mathrm{Collapse}},\ u_{\mathrm{Collapse}}) = (0, 0)$$

is not enough for audit unless the supporting diagnostic computation can be reconstructed.

Z.10. Gate execution files

Definition 6.47 (gate.json)**.** A gate.json file is a structured record of Core gate checks, including B-Gate$^+$, admissibility conditions, representative requirements, diagnostics, ledger conditions, and gate verdicts.

Definition 6.48 (Gate execution entry)**.** A *gate execution entry* records:

(i) gate identifier;

(ii) gate definition;

(iii) input object;

(iv) window and threshold;

(v) checked clauses;

(vi) repaired evaluation clause, if used;

(vii) representative clause, if used;

(viii) eligibility clause, if used;

(ix) tower diagnostic clause, if used;

(x) ledger clause, if used;

(xi) clause-level results;

(xii) final gate verdict;

(xiii) verifying authority;

(xiv) artifact references.

Proposition 6.49 (Gate verdict requires clause-level support)**.** *A gate verdict is audit-readable only if its required clauses are recorded with sufficient support.*

Proof. A gate verdict is determined by its clauses. Without clause-level records, an auditor cannot reconstruct why the verdict was assigned. Therefore clause-level support is required. □

Remark 6.50 (Gate file is not self-certifying)**.** The existence of gate.json does not by itself prove that the gate passed. Its contents must support the verdict.

Z.11. Replay

Definition 6.51 (Replay)**.** A *replay* is an attempt to reproduce an execution run from its recorded schema, environment, inputs, parameters, commands, and artifacts.

Definition 6.52 (Replay result)**.** A *replay result* is one of:

(i) replay-pass;

(ii) replay-fail;

(iii) replay-incomplete;

(iv) replay-divergent;

(v) replay-not-applicable;

(vi) replay-blocked.

Definition 6.53 (Replay equivalence)**.** *Replay equivalence* is a declared relation specifying when replay output is considered equivalent to the original output. It may require exact equality, hash equality, numerical tolerance, proof assistant acceptance, diagnostic equality, ledger equality, gate verdict equality, or semantic equivalence.

Definition 6.54 (Replay witness)**.** A *replay witness* is an artifact, log, hash, proof assistant output, diagnostic output, ledger output, or gate output supporting a replay result.

Proposition 6.55 (Replay pass is not theorem proof by itself)**.** *A replay-pass result shows reproducibility under the declared replay criterion, but it does not by itself prove the underlying mathematical claim.*

Proof. Replay checks whether a run can be reproduced. Mathematical proof requires the relevant verifier-side or logical conditions. Reproduction and proof are distinct. □

Remark 6.56 (Replay criterion must be declared)**.** For numerical runs, exact equality may be too strict or too weak depending on the task. For formal runs, proof assistant acceptance may be appropriate. For Core diagnostics, diagnostic equality or certified semantic equivalence may be required. The replay equivalence criterion must be declared.

Z.12. Reproducibility levels

Definition 6.57 (Reproducibility level)**.** A *reproducibility level* classifies how much of an execution run can be reconstructed.

Definition 6.58 (Level R0)**.** *Level R0* means that the result is stated but no sufficient execution schema or artifacts are available.

Definition 6.59 (Level R1)**.** *Level R1* means that artifacts are referenced, but replay is not possible from the record.

Definition 6.60 (Level R2)**.** *Level R2* means that inputs, parameters, and outputs are recorded, but the environment or commands are incomplete.

Definition 6.61 (Level R3)**.** *Level R3* means that the run can be replayed in a compatible environment with declared tolerances.

Definition 6.62 (Level R4)**.** *Level R4* means that the run can be replayed exactly or content-identically using captured environment, artifacts, commands, and hashes.

Definition 6.63 (Level R5)**.** *Level R5* means that the result is reproducible and independently audited, with all required proof-store, artifact-store, evaluation, ledger, diagnostic, gate, map, DAG, and replay records linked.

Proposition 6.64 (Higher reproducibility level does not imply theorem validity)**.** *A higher reproducibility level improves auditability but does not by itself imply theorem validity.*

Proof. Reproducibility concerns reconstruction of runs and artifacts. Theorem validity concerns the correctness of mathematical claims. A reproducible invalid computation remains invalid. □

Remark 6.65 (Preferred level)**.** For high-stakes Core certificates, Level R5 is preferred. However, even R5 must be paired with valid proof content.

Z.13. Formal verification replay

Definition 6.66 (Formal verification replay)**.** A *formal verification replay* is a replay in which proof assistant code or formal fragments are checked again under a recorded proof assistant environment.

Definition 6.67 (Formal environment)**.** A *formal environment* records:

(i) proof assistant name and version;

(ii) libraries and versions;

(iii) axioms used;

(iv) definitions imported;

(v) build commands;

(vi) compilation options;

(vii) formal artifact hashes.

Definition 6.68 (Formal replay artifact)**.** A *formal replay artifact* is a proof assistant output, build log, checked theorem object, compiled file, or formal verification report produced during formal replay.

Proposition 6.69 (Formal replay is scoped to formal statements)**.** *Formal verification replay establishes only the checked formal statements in the recorded formal environment.*

Proof. A proof assistant checks formal statements relative to its environment. It does not automatically prove informal interpretations or external problem-specific claims. Those require interpretation theorems or interface bridges. □

Remark 6.70 (Formal replay and interpretation)**.** A formally replayed theorem may still need an interpretation layer to connect it to the intended AK or problem-specific claim.

Z.14. Numerical reproducibility

Definition 6.71 (Numerical reproducibility)**.** *Numerical reproducibility* is the ability to reproduce numerical outputs within declared tolerances, methods, and environments.

Definition 6.72 (Numerical tolerance)**.** A *numerical tolerance* is a declared acceptable discrepancy between original and replayed numerical outputs.

Definition 6.73 (Numerical stability record)**.** A *numerical stability record* documents sensitivity of numerical outputs to grid, time step, precision, algorithmic choices, and perturbations.

Definition 6.74 (Continuum interpretation record)**.** A *continuum interpretation record* documents the bridge, theorem, or [Spec] status by which numerical or discretized outputs are interpreted as statements about a continuum object.

Proposition 6.75 (Numerical reproducibility is not numerical validity)**.** *Numerical reproducibility does not by itself establish numerical validity or mathematical proof.*

Proof. A numerical computation may be reproducible and still approximate the wrong object, use an unstable method, or fail to justify continuum interpretation. Numerical validity requires additional error, stability, and soundness analysis. □

Remark 6.76 (Relation to Appendices I and J)**.** Appendices I and J provide discretization and measurement stability disciplines. This appendix records the execution schema needed to replay such analyses.

Z.15. Reproducibility manifest

Definition 6.77 (Reproducibility manifest)**.** A *reproducibility manifest* is a top-level record connecting run schema, artifact manifest, environment capture, repaired evaluation records, ledger records, diagnostic records, gate records, replay results, audit records, and proof-store links.

Definition 6.78 (Reproducibility manifest entry)**.** A *reproducibility manifest entry* records:

(i) run identifier;

(ii) run scope;

(iii) $\mathrm{run.yaml}$ reference;

(iv) artifact manifest reference;

(v) environment reference;

(vi) $\mathrm{eval.json}$ reference, if applicable;

(vii) $\mathrm{ledger.json}$ reference, if applicable;

(viii) $\mathrm{diagnostics.json}$ reference, if applicable;

(ix) $\mathrm{gate.json}$ reference, if applicable;

(x) replay result;

(xi) reproducibility level;

(xii) audit result;

(xiii) proof-store links;

(xiv) Validity Map or Certificate DAG links, if applicable.

Proposition 6.79 (Reproducibility manifest supports audit)**.** *A reproducibility manifest supports audit by linking the records needed to reconstruct and inspect an execution run.*

Proof. Audit requires access to the run schema, artifacts, environment, repaired evaluations, ledger, diagnostics, gates, replay results, and proof-store records. The reproducibility manifest links these records. □

Remark 6.80 (Manifest is not proof)**.** A reproducibility manifest may make a run audit-readable. It does not by itself establish that the mathematical conclusion is valid.

Z.16. Reproducibility failure modes

Definition 6.81 (Reproducibility failure)**.** A *reproducibility failure* occurs when an execution run cannot be reconstructed, replayed, or audited at the claimed reproducibility level.

Definition 6.82 (Missing artifact failure)**.** A *missing artifact failure* occurs when a required artifact is absent or inaccessible.

Definition 6.83 (Environment failure)**.** An *environment failure* occurs when the recorded environment is insufficient, unavailable, or incompatible with replay.

Definition 6.84 (Parameter failure)**.** A *parameter failure* occurs when required execution parameters are missing, ambiguous, or inconsistent.

Definition 6.85 (Replay divergence)**.** *Replay divergence* occurs when replayed outputs fail the declared replay equivalence criterion.

Definition 6.86 (Manifest inconsistency)**.** *Manifest inconsistency* occurs when run schema, artifact manifest, evaluation records, ledger records, diagnostic records, gate records, or proof-store entries disagree.

Definition 6.87 (Evaluation replay failure)**.** An *evaluation replay failure* occurs when

$$\mathrm{Eval}_{i,W,\tau}(F)$$

cannot be reconstructed or does not satisfy the declared replay equivalence.

Definition 6.88 (Diagnostic replay failure)**.** A *diagnostic replay failure* occurs when monitored diagnostic values or tower artifact computations cannot be reconstructed or do not satisfy the declared replay equivalence.

Proposition 6.89 (Reproducibility failure requires status review)**.** *If reproducibility failure affects a necessary dependency, the corresponding certificate or proof object requires status review and possible demotion.*

Proof. A necessary dependency must be audit-readable and reconstructible at the claimed level. If reproducibility fails, the support for that dependency is weakened. Therefore status review and possible demotion are required. □

Remark 6.90 (Reproducibility failure is not always mathematical failure)**.** A reproducibility failure may indicate missing infrastructure rather than false mathematics. However, until repaired, the affected proof object is not audit-stable.

Z.17. Execution-to-certificate boundary

Definition 6.91 (Execution-to-certificate boundary)**.** The *execution-to-certificate boundary* is the boundary between producing execution artifacts and establishing certificate status.

Definition 6.92 (Execution-supported certificate)**.** An *execution-supported certificate* is a certificate whose computational, diagnostic, ledger, gate, or artifact-dependent components are supported by adequate execution and reproducibility records.

Proposition 6.93 (Execution success does not imply certificate success)**.** *Successful execution of a workflow does not imply that the resulting object is a valid certificate.*

Proof. Execution success may mean that commands ran and outputs were generated. Certificate success requires that the outputs satisfy the Core gate, repaired evaluation, diagnostic, ledger, manifest, and audit conditions. These are distinct. □

Proposition 6.94 (Certificate success requires execution support when computation is used)**.** *If a certificate relies on computed artifacts, then those computations must be supported by adequate execution and reproducibility records.*

Proof. Computed artifacts must be reconstructible and auditable when used as certificate evidence. Execution and reproducibility records provide the required support. □

Remark 6.95 (Boundary discipline)**.** Execution infrastructure should make certificate checking easier. It should never be confused with certificate checking itself.

Z.1bis. v20 dependency boundary and release position

Definition 6.96 (v20 execution dependency boundary)**.** The *v20 execution dependency boundary* is the rule that Appendix Z may serialize, replay, and audit only the mathematical strength already available in the invoked Core, Foundation, Technical Extension, Problem Interface, and Search / Platform records. It does not create a detector theorem, higher edge, tower comparison morphism, finite-to-infinite reduction, Return theorem, source authority, or final Claim UID by execution.

Definition 6.97 (Execution verifier projection)**.** For an execution bundle E, the *execution verifier projection*

$$\pi^{Z}_{\mathrm{ver}}(E)$$

is the subrecord containing only the identity-matched inputs, theorem-relevant parameters, verifier-side outputs, discharged conversion records, certified metric bounds, non-scalar obligation statuses, source and artifact bindings, and typed result statuses consumed by the target. Logs, dashboards, path names, cache locations, UI labels, query order, and advisory metadata are excluded unless a target theorem explicitly consumes them.

Proposition 6.98 (Local replay does not promote a release claim)**.** *A local execution, replay, verification, or audit pass does not promote an item into the final CR-v20 release registry.*

Proof. Local execution and replay establish only the declared local criteria. Final CR-v20 promotion additionally requires frozen Part V and Companion artifacts, cross-document dependency closure, claim-use review, final registry extraction, and release authority. Those conditions are not consequences of local replay. □

Remark 6.99 (Final CR-v20 deferral)**.** Until Part V, the Companion, and the cross-appendix dependency closure are frozen, generated JSON, CSV, graph rows, UUIDs, receipt objects, and release dashboards are non-normative local sidecars. They must not be represented as canonical Claim UIDs, final promotion receipts, or release-gate passes.

Remark 6.100 (Reference recovery tracks)**.** The reference Known-Theorem Recovery tracks are IW-KTR-GROWTH and NS-KTR-SERRIN-L6 at their frozen local source strength. No Iwasawa main conjecture, class-number formula, unconditional Navier–Stokes regularity, Clay-level result, concrete Fukaya/HMS bridge, or concrete external AGI-N normalization bridge is asserted here. The latter two successor families remain $\mathrm{not\_invoked}$ in v20.0.0.

Z.2bis. Execution trust boundary, identity, and authority separation

Definition 6.101 (Execution trust boundary)**.** The *execution trust boundary* separates executable instructions and verifier-approved configuration from untrusted prompts, retrieved text, filenames, metadata, comments, model output, external service responses, and embedded source instructions. Untrusted content may be stored as data but may not mutate the run scope, detector stage, target statement, authority scope, permitted use, or promotion status.

Definition 6.102 (v20 run identity record)**.** A *v20 run identity record* contains the run identifier, content digest of the execution schema, exact input artifact digests, command or workflow digest, environment identity, tool and library versions, parameter bindings, randomness record, authority scope, target claim or node, and repair ancestry. A mutable path, basename, display title, or timestamp is not a sufficient run identity.

Definition 6.103 (Execution authority separation)**.** Execution, verification, audit, approval, promotion, demotion, and publication authorities are distinct fields. A single agent, interface, or service account may hold several fields operationally, but one authority is not inferred from another.

Proposition 6.104 (Run name is not run identity)**.** *Equality of run names, filenames, job labels, or dashboard entries does not establish equality of execution runs.*

Proof. Run identity depends on proof-relevant content, inputs, environment, parameters, commands, and authority scope. A mutable display label does not determine those data. □

Proposition 6.105 (Execution authority is not compositional by interface)**.** *Combining execution, storage, verification, and promotion controls in one interface does not compose their authority scopes.*

Proof. Each authority has a distinct governance meaning and evidentiary consequence. Interface colocation supplies no theorem or governance rule identifying the scopes. □

Remark 6.106 (Untrusted instruction noninterference)**.** A source document or retrieved artifact may contain text that resembles a command, status token, detector stage, or promotion instruction. It remains source content unless a separately authorized execution policy imports it as an instruction.

Z.3bis. Canonical run envelope and independent status axes

Definition 6.107 (Canonical v20 run envelope)**.** A *canonical v20 run envelope* extends the execution schema with:

(i) target statement, target node, and exact permitted use;

(ii) source and derivative artifact roles;

(iii) source authority and source-binding references;

(iv) all independent status axes;

(v) detector source stage, when a finite detector is invoked;

(vi) monitored degrees and invoked higher-edge degree;

(vii) comparison modes and selected admissible proof route;

(viii) actual Type IV morphism identity, when invoked;

(ix) transient, finite-to-infinite, apex-agreement, and Return obligations;

(x) metric components, non-scalar obligations, and no-double-counting relations;

(xi) operational replay, semantic replay, and mathematical-reconstruction targets;

(xii) final-CR promotion state and typed failure routes.

Definition 6.108 (Run status-axis record)**.** A *run status-axis record* stores independently:

document role, evidence status, invocation role, conformance status,
workflow status, execution status, verification result, audit result, atomic result.

No axis is inferred solely from another axis.

Definition 6.109 (Execution scope-exclusion policy)**.** An *execution scope-exclusion policy* is an immutable declaration, fixed before the relevant run, proving that a clause is outside the target scope. Only such a policy may justify $\mathrm{not\_invoked}$. A missing, skipped, unavailable, or unimplemented required clause is $\mathrm{undefined}$.

Definition 6.110 (Typed execution failure route)**.** A *typed execution failure route* records the failed or missing obligation, affected target, atomic status, scope, artifacts, repair action, and whether the route is mathematical, metric, artifact, source-authority, environment, replay, governance, or implementation failure.

Proposition 6.111 (Missing execution evidence is undefined)**.** *Missing execution evidence for an invoked theorem-facing obligation yields* undefined*, not* pass*,* not_invoked*, or a zero numerical value.*

Proof. The obligation is invoked but not established. There is neither positive evidence nor an immutable scope exclusion. The safe atomic result is therefore undefined. □

Proposition 6.112 (Execution status axes do not collapse)**.** *An executed, replayed, stored, audited, or manifest-complete run need not be mathematically passing, and a mathematically valid theorem need not have a high reproducibility grade.*

Proof. The statuses answer different questions: whether a process ran, whether it replayed, whether records are complete, and whether a mathematical obligation is discharged. None is definitionally identical to another. □

Z.4bis. Artifact roles and transformation preservation

Definition 6.113 (Execution artifact-role record)**.** An *execution artifact-role record* classifies each artifact as authoritative source, local source-statement binding, precursor input, executable input, generated output, derived summary, rendered derivative, manifest, audit record, replay witness, benchmark statement, or independent validation artifact. One artifact may carry several roles only when the roles and permitted uses are separately declared.

Definition 6.114 (Source-derivative binding)**.** A *source-derivative binding* connects an authoritative or locally bound source artifact to a transcription, normalization, serialization, rendering, extracted statement, or machine-readable sidecar and records the exact transformation and claim-relative authority.

Definition 6.115 (Execution transformation-preservation record)**.** A *transformation-preservation record* verifies that a declared transformation preserves the theorem-relevant semantics used by the target. Covered transformations include migration, normalization, compilation, rendering, export, import, compression, parsing, extraction, and format conversion. The record includes a semantic diff and a failure route.

Proposition 6.116 (Rendered derivatives are not source authority)**.** *A PDF, screenshot, page image, generated summary, parsed table, JSON sidecar, or compiled artifact is not authoritative source evidence merely because it is derived from an authoritative source.*

Proof. Derivation may change content, omit context, alter notation, or introduce parser and rendering errors. Source authority transfers only through a declared preservation and source-binding record. □

Proposition 6.117 (Hash equality is not transformation preservation)**.** *A hash equality identifies content under one identity policy but does not prove that a transformation between distinct artifacts preserves mathematical meaning.*

Proof. Hash equality is an identity statement. Preservation is a relational semantic statement between source and target artifacts. The latter requires a separate theorem, verified comparison, or semantic-diff record. □

Z.6bis. Randomness, external services, and nondeterministic replay

Definition 6.118 (Randomness envelope)**.** A *randomness envelope* records all pseudo-random generators, seeds, seed derivation, sampling distributions, stopping rules, retry policies, model temperatures, decoding parameters, parallel scheduling, and hardware nondeterminism consumed by a run.

Definition 6.119 (External-service snapshot)**.** An *external-service snapshot* records the service identity, API or model version, request payload digest, response digest, time and region when relevant, retry sequence, tool policy, and any unavailable server-side state. If the server-side state cannot be frozen, exact replay must not be claimed.

Proposition 6.120 (A seed is not a complete determinism witness)**.** *Recording a random seed does not establish deterministic replay when unrecorded libraries, hardware kernels, concurrency, external services, retry policies, or nondeterministic algorithms affect the output.*

Proof. The seed controls only the random choices governed by the specified generator and execution path. Other nondeterministic sources remain unconstrained. □

Proposition 6.121 (Statistical replay is not instance proof)**.** *A statistically reproducible distribution of outputs does not prove that a particular sampled output satisfies a theorem-facing obligation.*

Proof. Distributional agreement concerns population behavior under a statistical criterion. Theorem-facing use of one output requires the relevant instance-level verifier or proof conditions. □

Z.7bis. Stage-aware detector execution records

Definition 6.122 (Finite-detector execution record)**.** A *finite-detector execution record* declares exactly one source stage:

$$\mathrm{windowed\_prethreshold}, \qquad \mathrm{repaired\_postthreshold},$$
$$\mathrm{kernel\_defect}, \qquad \mathrm{cokernel\_defect}.$$

It records the predicate, finite stencil or witness family, endpoint and coverage conventions, non-adaptivity policy, soundness theorem, completeness theorem when claimed, stage-specific theorem or stage-transport theorem, artifact identity, certificate status, and mathematical value. The standard v20 finite detector is sourced at $\mathrm{windowed\_prethreshold}$; another stage requires its own theorem or explicit transport.

Definition 6.123 (Detector certificate-value record)**.** A *detector certificate-value record* stores certificate status and mathematical value as independent fields. Certificate status may be pass, reject, or $\mathrm{undefined}$; the mathematical value belongs to the detector's declared codomain and is present only when well typed and computed.

Proposition 6.124 (Detector stage is immutable during replay)**.** *A replay or conversion may not reinterpret a detector output at a different source stage without a declared stage-transport theorem and new record.*

Proof. Detector soundness and completeness are stage-relative. Changing the source stage changes the proposition established by the computation. □

Proposition 6.125 (Zero-looking output is not certified zero)**.** *An empty list, zero scalar, zero array, absent hit, or visually empty plot is not a certified zero detector value unless the execution record is well typed and the declared detector certificate passes.*

Proof. A zero-looking representation may result from missing data, truncation, parsing failure, unsupported stage, or a failed computation. Certification requires the corresponding typed record and verifier-side theorem. □

Z.7ter. Degree-n higher-edge execution records

Definition 6.126 (Higher-edge execution record)**.** A *higher-edge execution record* for degree n stores the actual repaired profile, admissible representative or genuine eligible realization, profile isomorphism, amplitude condition, fixed extractor E_n with $E_n(0) = 0$, artifact-backed identification of H^{-n}, naturality scope, change compatibility, non-circularity, and result for

$$\mathrm{Eval}_{n,W,\tau}(F) = 0 \Longrightarrow \mathrm{Ext}^n(A_n(F), k) = 0.$$

Proposition 6.127 (The default higher edge is one-way)**.** *A replayed higher-edge package establishes only the recorded direction from repaired vanishing to* Ext^n*-vanishing unless an independent natural zero-reflection or faithfulness theorem supplies the reverse direction.*

Proof. The condition $E_n(0) = 0$ supports the forward zero transport. It does not imply that $E_n(X) = 0$ forces $X = 0$. □

Proposition 6.128 (Synthetic shifts are not principal higher edges)**.** *A stored shift, reindexing, target-shaped complex, or encoding that reproduces a desired cohomological degree is not a principal higher edge without the genuine realization, profile, naturality, and non-circularity records.*

Proof. A syntactic target shape does not establish that the external or Core object is realized by that construction at the declared semantics. □

Z.8bis. Selected-sub-DAG ledger execution

Definition 6.129 (Selected admissible execution route)**.** A *selected admissible execution route* P is a finite necessary target sub-DAG, including every required branch, through which the target certificate actually consumes artifacts and comparisons. It is not required to be a single linear path. Unused alternative routes are excluded.

Definition 6.130 (Execution metric component)**.** An *execution metric component* $\delta_e \in \mathcal{V}$ is admissible only when attached to an actual declared pair of windowed pre-threshold profiles

$$B_e^{\mathrm{src}}, \qquad B_e^{\mathrm{tgt}}$$

and supported by

$$d_B\left(B_e^{\mathrm{src}}, B_e^{\mathrm{tgt}}\right) \le \mathrm{val}_B(\delta_e).$$

Definition 6.131 (Execution ledger-relation record)**.** An *execution ledger-relation record* declares every local metric name to be an alias, refinement, aggregate, or disjoint contribution relative to the active Appendix-M-compatible catalog. It identifies the primitive support and prohibits duplicate consumption.

Definition 6.132 (Execution scalarization record)**.** An *execution scalarization record* fixes the Appendix-M-compatible commutative quantale

$$(\mathcal{V}, \oplus, 0_{\mathcal{V}}, \preceq)$$

and a monotone subadditive map

$$\mathrm{val}_B : \mathcal{V} \longrightarrow [0, \infty], \qquad \mathrm{val}_B(0_{\mathcal{V}}) = 0, \qquad \mathrm{val}_B(q \oplus q') \le \mathrm{val}_B(q) + \mathrm{val}_B(q').$$

For a finite route P, aggregation consumes the finite ordered commutative-monoid reduct:

$$\mathfrak{d}_Z(P) := \bigoplus_{e \in S_P} \delta_e.$$

Proposition 6.133 (Primitive discrepancies are counted once)**.** *A primitive metric discrepancy consumed by several aliases, mirrors, manifests, replay records, or graph edges contributes at most once to* $\mathfrak{d}_Z(P)$.

Proof. The metric budget represents actual profile discrepancy, not the number of storage or workflow references to that discrepancy. The ledger-relation record identifies aliases and aggregates and removes duplication. □

Proposition 6.134 (Unused alternative routes are not summed)**.** *A metric component appearing only on an unused alternative proof or execution route is not added to the target route budget.*

Proof. The target certificate consumes the selected necessary sub-DAG, not every candidate route present in the global map. □

Proposition 6.135 (Non-scalar obligations are not execution budget)**.** *Source authority, detector applicability, representative existence, higher-edge eligibility, actual morphism existence, terminal tameness, finite-to-infinite reduction, Return, artifact identity, semantic replay, and claim permission are not repaired by metric budget slack.*

Proof. These obligations concern theorem existence, typing, semantics, identity, or governance rather than a bounded bottleneck discrepancy. □

Z.9bis. Complete Type IV and transient execution records

Definition 6.136 (v20 Type IV execution record)**.** A *v20 Type IV execution record* stores the actual persistence morphism

$$\phi_{i,W,\tau} : \mathrm{ColimProfile}_{i,W,\tau}(F_\bullet) \longrightarrow \mathrm{ApexProfile}_{i,W,\tau}(F_\infty),$$

its semantic authority and provenance, source and target identities, transition and apex agreement obligations, terminal-tameness support, kernel and cokernel profiles, and degreewise values

$$\mu_{i,W,\tau} := \mathrm{tgdim}_W \ker(\phi_{i,W,\tau}), \qquad u_{i,W,\tau} := \mathrm{tgdim}_W \operatorname{coker}(\phi_{i,W,\tau}).$$

Any aggregate $(\mu_{\mathrm{Collapse}},\ u_{\mathrm{Collapse}})$ is taken only over a fixed finite monitored-degree set at the same (W,τ). This record refines Definition 6.177; when both records are present, the present v20 record controls theorem-facing consumption, while the earlier record remains a compatibility projection.

Definition 6.137 (Transient execution record)**.** A *transient execution record* separates:

(i) terminal Type IV status;

(ii) full-interior defect status: $\mathrm{not\_checked}$, $\mathrm{certified\_absent}$, or $\mathrm{certified\_present}$;

(iii) long-transient status, chosen from

$$\mathrm{not\_invoked}, \quad \mathrm{undefined}, \quad \mathrm{certified\_absent}, \quad \mathrm{certified\_present};$$

(iv) full profile isomorphism and apex-agreement status.

These fields are not inferred from one another.

Proposition 6.138 (Terminal zero is not full defect zero)**.** *Certified terminal Type IV zero does not establish absence of transient or full-interior defects, global profile isomorphism, or apex agreement.*

Proof. Terminal-generic kernel and cokernel dimensions discard bounded interior and transient information and do not prove the stronger morphism properties. □

Proposition 6.139 (Missing Type IV data are not zero)**.** *A missing, ill-typed, stale, or non-terminal-tame Type IV artifact yields* $\mathrm{undefined}$*, or* $\mathrm{not\_invoked}$ *under a prior scope exclusion, and never a numerical zero pair.*

Proof. The numerical pair is defined only after the actual morphism and terminal-generic computations are available. □

Z.11bis. Operational, semantic, and mathematical replay

Definition 6.140 (Operational replay record)**.** An *operational replay record* states that commands executed in the declared environment and produced outputs equivalent under an operational criterion such as byte equality, hash equality, parser acceptance, or numerical tolerance.

Definition 6.141 (Semantic replay record)**.** A *semantic replay record* verifies that the replay preserves the theorem-relevant statements, source bindings, profile stages, parameters, scopes, directions, status axes, dependency relations, permitted uses, and artifact support relations consumed by the target.

Definition 6.142 (Mathematical reconstruction record)**.** A *mathematical reconstruction record* independently reconstructs the invoked proof, verifier-side certificate, bridge theorem, reduction, Return theorem, or formal interpretation at the declared hypotheses and strength.

Proposition 6.143 (The replay hierarchy is strict)**.** *Operational replay does not imply semantic replay, and semantic replay does not imply mathematical reconstruction.*

Proof. The three records check increasingly strong and differently typed obligations: execution agreement, preservation of meaning, and independent mathematical discharge. □

Proposition 6.144 (Build pass is not semantic pass)**.** *Compilation, parser acceptance, proof-assistant build success, PDF rendering, or byte-identical regeneration does not establish semantic replay beyond the exact property checked.*

Proof. A successful tool run may preserve syntax while changing statement meaning, source authority, scope, or interpretation. □

Z.12bis. Reproducibility vectors and limits of R-levels

Definition 6.145 (Reproducibility vector)**.** A *reproducibility vector* records separately:

$$(R_{\mathrm{op}}, R_{\mathrm{sem}}, R_{\mathrm{math}}, R_{\mathrm{src}}, R_{\mathrm{env}}, R_{\mathrm{audit}}),$$

representing operational replay, semantic replay, mathematical reconstruction, source binding, environment reconstruction, and independent audit. The scalar R0–R5 level is an operational summary and does not replace this vector.

Proposition 6.146 (R5 is not semantic or theorem completeness)**.** *Level R5 does not by itself imply semantic preservation, source faithfulness, Return completeness, or theorem validity.*

Proof. R5 records linked replay and audit infrastructure. The omitted obligations are separately typed mathematical and semantic conditions. □

Z.14bis. Numerical evidence and threshold-gap execution

Definition 6.147 (Numerical evidence record)**.** A *numerical evidence record* stores the exact mathematical quantity approximated, discretization and measurement maps, precision and rounding policy, interval or rigorous error bounds when claimed, grid and time-step studies, implementation validation, and the theorem connecting the computation to the target.

Definition 6.148 (Threshold-gap execution record)**.** A *threshold-gap execution record* stores the actual protected pre-threshold profile family, two-sided clearance around τ, every metric component on the selected route, the scalarized bound ε, and separately:

$$2\varepsilon \le \tau \quad \text{for exact zero transport,} \qquad 2\varepsilon < \tau \quad \text{for strict operational reserve.}$$

Proposition 6.149 (Numerical tolerance is not a metric certificate)**.** *A replay tolerance or empirical numerical difference is not a metric-ledger certificate unless it is proved to upper-bound the actual bottleneck discrepancy of the declared profile pair.*

Proof. A numerical tolerance is an execution acceptance criterion. The Core metric record requires a theorem-supported profile discrepancy bound. □

Proposition 6.150 (Zero transport and reserve are distinct)**.** *The non-strict condition* $2\varepsilon \leq \tau$ *used for exact zero transport and the strict reserve* $2\varepsilon < \tau$ *are not interchangeable execution verdicts.*

Proof. They support different theorem and operational policies at the boundary case $2\varepsilon = \tau$. □

Z.15bis. Problem-Bridge execution bundle

Definition 6.151 (Problem-Bridge execution bundle)**.** A *Problem-Bridge execution bundle* records separate artifacts and discharge statuses for:

$$\begin{aligned}&\text{external source statement} \longrightarrow \text{purpose-faithful realization}\\ &\longrightarrow \text{Core-readable profile} \longrightarrow \text{finite Core certificate}\\ &\longrightarrow \text{finite-to-infinite reduction, when required}\\ &\longrightarrow \text{degree-}n\text{ higher edge, when invoked}\\ &\longrightarrow \text{Return theorem} \longrightarrow \text{external conclusion.}\end{aligned}$$

No arrow is inferred from execution adjacency or file order.

Definition 6.152 (Finite-to-infinite execution record)**.** A *finite-to-infinite execution record* separates finite observation, finite proof certificate, certified finite prefix, numerical convergence, Cauchy or compactness evidence, eventual constancy, colimit, homotopy-colimit, completion, apex semantics, apex agreement, and the exact reduction theorem connecting the finite premise to the infinite target.

Definition 6.153 (Return execution record)**.** A *Return execution record* stores the exact audited AK-side antecedent, exact external conclusion, source-domain hypotheses, direction, proof or valid import, source and artifact identities, non-circularity, permitted use, and failure boundary. A realization theorem is not a Return theorem.

Proposition 6.154 (Finite prefixes are not reductions)**.** *A finite plateau, repeated barcode, stable finite prefix, numerical convergence, stored apex, or replayed finite computation does not establish a finite-to-infinite reduction.*

Proof. These records describe finite or numerical behavior. The reduction is a theorem from an exact finite premise to an exact infinite target. □

Proposition 6.155 (Realization is not Return)**.** *A purpose-faithful realization from an external source into AK data does not establish the reverse implication from AK-side conclusions to the external theorem.*

Proof. The arrows have opposite direction and may require different hypotheses and constructions. □

Proposition 6.156 (External conclusions require Return)**.** *An execution bundle may support an external theorem only when every required intermediate arrow and the exact Return theorem are discharged.*

Proof. Core pass has internal AK semantics. Only the Return theorem licenses the declared external-domain conclusion. □

Z.15ter. Known-Theorem Recovery execution and quarantine

Definition 6.157 (Recovery execution bundle)**.** A *recovery execution bundle* is a Problem-Bridge execution bundle for a frozen known theorem at exact source strength. It stores authoritative source identity, exact benchmark statement and locator, permitted precursor package, construction artifacts, independent validation artifacts, and final comparison.

Definition 6.158 (Execution benchmark-quarantine record)**.** An *execution benchmark-quarantine record* proves that the benchmark proof, known truth value, answer table, target constants, exceptional bounds, conclusion-derived witnesses, and equivalent oracles did not enter the realization, detector, finite certificate, reduction, higher edge, or Return proof. The benchmark statement itself may identify the target and exact Return consequent.

Proposition 6.159 (Benchmark leakage invalidates independent recovery)**.** *If quarantined conclusion information is used to construct a required upstream witness without an independently justified theorem licensing that datum as a hypothesis, the route is not an independent Known-Theorem Recovery certificate.*

Proof. The advertised recovery would be circular: the target answer would participate in constructing the evidence claimed to recover it. □

Remark 6.160 (Iwasawa recovery boundary)**.** IW-KTR-GROWTH is limited to the frozen Iwasawa 1959 Theorem 4 characteristic-growth clause relative to its registered precursor package. No main conjecture, class-number theorem, general tower information-preservation theorem, or external analytic continuation claim is derived.

Remark 6.161 (Serrin recovery boundary)**.** NS-KTR-SERRIN-L6 is limited to the frozen one-way strict-subcritical Serrin implication for the registered $L_t^6 L_x^6$ witness. It supplies no new analytic proof, no unconditional global regularity, and no Clay-level result.

Z.16bis. Repair replay, staleness, and immutable history

Definition 6.162 (Repair-replay record)**.** A *repair-replay record* links the prior immutable run, typed failure, repair patch, new artifact identities, affected dependencies, re-verification scope, and new result. It does not mutate the prior verdict.

Definition 6.163 (Stale run record)**.** A run is *stale* when its source, dependency, environment, schema, authority, artifact, target statement, permitted use, or verifier criterion has changed relative to the recorded run.

Proposition 6.164 (Execution repair is non-retroactive)**.** *A repaired replay creates a new run or revision and does not convert the historical failed, undefined, or superseded run into a passing one.*

Proof. Auditability requires the historical state and its failure to remain reconstructible. The repair has new content, evidence, and identity. □

Proposition 6.165 (Stale runs are not current evidence)**.** *A stale run cannot be silently consumed as current theorem-facing evidence.*

Proof. The conditions under which it was verified no longer match the current target or environment. Revalidation or explicit demotion is required. □

Z.17bis. v20.0.0 execution hardening and compatibility

Definition 6.166 (Execution sovereignty)**.** *Execution sovereignty* is the rule that execution records, replay logs, environment captures, and reproducibility manifests may support audit but may not override verifier-side Core conditions, bridge-theorem requirements, interpretation theorems, or claim-use permissions.

Definition 6.167 (Execution-to-verifier conversion record)**.** An *execution-to-verifier conversion record* is a record converting an execution output into verifier-side data for a declared use. It must include:

(i) source run identifier;

(ii) source execution schema and artifact manifest;

(iii) target verifier-side condition;

(iv) authority scope;

(v) conversion rule;

(vi) result status pass, reject, or undefined;

(vii) typed failure route when the result is reject or undefined;

(viii) immutable artifact references;

(ix) repair ancestry, if any;

(x) claim-use preflight record when the converted data are used in a theorem-facing or Core-facing claim.

Definition 6.168 (Execution status axis)**.** The *execution status axis* records operational outcomes such as executed, failed, replayed, divergent, blocked, or superseded. It is an additional workflow field and does not replace Appendix U agent-output status labels, Appendix Y Proof Store statuses, Chapter 1 claim taxonomy, the current CR-v20 local claim-use status, or Core verifier verdicts.

Definition 6.169 (Execution claim-use preflight)**.** An *execution claim-use preflight* is the check that an execution output is permitted to be used for the specific target claim, target gate, target diagnostic, target ledger comparison, target bridge, or target proof object. It records the intended use, required status, available evidence, missing evidence, and result pass, reject, or undefined.

Definition 6.170 (Run identity witness)**.** A *run identity witness* is a content hash, environment digest, command transcript, signed log, or equivalent record showing that a replayed or referenced run is the intended run. It witnesses identity, not mathematical validity.

Definition 6.171 (Execution semantic preservation record)**.** An *execution semantic preservation record* states why replacing one environment, implementation, file format, numerical backend, proof-assistant version, or replay tolerance by another preserves the verifier-relevant semantics of the run.

Definition 6.172 (Replay atomic status mapping)**.** *Replay atomic status mapping* maps replay results to theorem-facing atomic statuses. A replay-pass may support reconstructibility only within its declared replay criterion. A replay-fail or replay-divergent result routes to reject or undefined for the affected execution support. A replay-incomplete or replay-blocked result maps to undefined for the affected execution support until the missing replay data or blocking condition is repaired and rechecked. A replay-not-applicable result maps to not_invoked only when a declared scope-exclusion policy shows that the replay criterion is irrelevant; without such a policy, theorem-facing use is undefined.

Definition 6.173 (Execution scope overrun)**.** *Execution scope overrun* occurs when a run result, replay result, formal replay, numerical replay, environment capture, or reproducibility manifest is used beyond its declared run scope or authority scope.

Definition 6.174 (Execution adequacy defect)**.** An *execution adequacy defect*, denoted locally by $\delta^{Z\text{-}adequacy}$, records a failure or uncertainty in whether execution records are adequate for the claim that cites them. It is typically a status-ledger or qualitative-governance component, not a metric-admissible bottleneck component.

Definition 6.175 (Environment compatibility defect)**.** An *environment compatibility defect*, denoted locally by $\delta^{Z\text{-}env}$, records a failure, uncertainty, or unproved substitution between the original environment and a replay environment. It cannot be repaired by metric budget slack unless Appendix M registers a metric-admissible interpretation with supporting bounds.

Definition 6.176 (Replay divergence defect)**.** A *replay divergence defect*, denoted locally by $\delta^{Z\text{-}replay}$, records divergence between original and replayed outputs under the declared replay equivalence. For Core-facing use, unresolved replay divergence routes to reject or undefined rather than to a silent tolerance waiver.

Definition 6.177 (Execution Type IV replay record)**.** An *execution Type IV replay record* is a replay record for Type IV diagnostics containing the declared tower comparison artifact

$$\begin{gathered}\phi_{i,W,\tau}: \\ \mathrm{ColimProfile}_{i,W,\tau}(F_\bullet) \\ \longrightarrow \\ \mathrm{ApexProfile}_{i,W,\tau}(F_\infty).\end{gathered}$$

Besides the displayed artifact, the record includes:

(i) terminal-tameness support;

(ii) kernel and cokernel defect objects;

(iii) tgdim_W computations;

(iv) the interior-defect status field;

(v) the four-state diagnostic result.

Proposition 6.178 (Execution records do not override verifier conditions)**.** *No execution record, replay log, environment capture, or reproducibility manifest overrides a missing or failed verifier-side condition.*

Proof. Execution artifacts are infrastructure records. Verifier-side conditions are the declared mathematical and Core-facing criteria for certification. By execution sovereignty, infrastructure support cannot replace the criteria it is meant to support. □

Proposition 6.179 (Verifier-side use requires conversion)**.** *An execution output may be used as verifier-side evidence only through an execution-to-verifier conversion record with an appropriate pass status for the intended use.*

Proof. The same output file may be a log, a diagnostic support artifact, a ledger component source, or merely a failed run artifact. The conversion record declares which use is intended and whether the evidence supports that use. Without it, the output remains execution-side data. □

Proposition 6.180 (Execution status is not claim status)**.** *Execution status does not determine mathematical claim status.*

Proof. Execution status records workflow outcomes such as execution, replay, divergence, or blockage. Claim status requires proof, verification, import, discharge, or explicit non-claim governance. The two axes are distinct. □

Proposition 6.181 (Run identity is not semantic preservation)**.** *A run identity witness establishes identity under the declared identity policy, but it does not establish semantic preservation across environments, formats, or interpretations.*

Proof. Identity records that the referenced content or run is the intended one. Semantic preservation is a statement about meaning under replacement or interpretation. The latter requires a separate preservation record or theorem. □

Proposition 6.182 (Replay pass remains scoped)**.** *A replay-pass result supports only the declared replay criterion and does not discharge theorem, bridge, interpretation, diagnostic, or metric obligations outside that criterion.*

Proof. Replay equivalence is declared by the run scope and replay criterion. A pass under one criterion does not prove obligations not checked by that criterion. □

Proposition 6.183 (Replay-not-applicable requires scope exclusion)**.** *A replay-not-applicable result may be treated as* not_invoked *only under a declared scope-exclusion policy; otherwise theorem-facing use is* undefined*.*

Proof. A clause is not safely omitted merely because a replay report says it was not applicable. The omission must be justified by the scope policy. Without such justification, the missing check remains undefined for theorem-facing use. □

Proposition 6.184 (Execution scope overrun invalidates use)**.** *If an execution output is used beyond its declared scope, that use is invalid until a new scope, conversion record, or verification record justifies it.*

Proof. The run authority and execution schema define what the run checked or produced. Use outside that scope lacks authorization and verifier-side interpretation. □

Proposition 6.185 (Execution defects are not automatically metric defects)**.** *The symbols* $\delta^{Z\text{-}adequacy}$*,* $\delta^{Z\text{-}env}$*, and* $\delta^{Z\text{-}replay}$ *are not automatically metric-admissible components of* $\mathfrak{d}_{\mathrm{met}}$*.*

Proof. Adequacy, environment compatibility, and replay divergence may be qualitative or governance failures. Metric budget slack can repair only declared metric-admissible deviations with supporting bounds and Appendix M registration. □

Proposition 6.186 (Type IV replay summary is insufficient)**.** *A replayed line reporting*

$$(\mu_{\mathrm{Collapse}},\ u_{\mathrm{Collapse}}) = (0, 0)$$

is insufficient for Type IV certification without the full execution Type IV replay record.

Proof. Type IV certification depends on the declared tower comparison artifact, terminal-tameness, kernel and cokernel defect objects, dimension calculations, and interior-defect status. A numerical pair alone does not provide those data. □

Theorem 6.187 (Appendix Z v20.0.0 execution hardening and compatibility package)**.** *The v20.0.0 execution hardening and compatibility layer supplies the following safeguards.*

(i) Execution sovereignty separates infrastructure records from verifier conditions.

(ii) Execution-to-verifier conversion records are required for theorem-facing use.

(iii) Execution status is distinct from claim status.

(iv) Claim-use preflight is required when execution outputs support claims.

(v) Run identity witnesses do not establish semantic preservation.

(vi) *Replay pass is scoped to the declared replay criterion.*

(vii) *Replay-not-applicable requires scope exclusion before it may be treated as not invoked.*

(viii) *Execution scope overrun invalidates the unsupported use.*

(ix) *Execution adequacy, environment compatibility, and replay divergence defects are not automatically metric defects.*

(x) *Type IV replay requires a full typed replay record, not just a zero pair.*

Proof. Items (i)–(ii) are Propositions 6.178 and 6.179. Item (iii) is Proposition 6.180. Item (iv) follows from Definition 6.169. Item (v) is Proposition 6.181. Item (vi) is Proposition 6.182. Item (vii) is Proposition 6.183. Item $(viii)$ is Proposition 6.184. Item (ix) is Proposition 6.185. Item (x) is Proposition 6.186. □

Remark 6.188 (Execution noninterference)**.** Execution infrastructure may preserve, replay, and expose evidence. It does not add theorem-level force beyond the checked mathematical or verifier-side content.

Remark 6.189 (Status token discipline)**.** Execution tokens such as replay-pass, replay-fail, and replay-not-applicable should be translated into the governing pass/reject/undefined/not-invoked vocabulary only through an explicit mapping record.

Remark 6.190 (Execution defect catalog registration)**.** The local symbols $\delta^{Z\text{-}adequacy}$, $\delta^{Z\text{-}env}$, and $\delta^{Z\text{-}replay}$ require Appendix M classification before global use. The same requirement applies to the local Search, Bridge, Map–DAG, and Proof Store namespaces declared in Appendices V–Y. The Part V cross-audit must classify every surface symbol as an alias, refinement, aggregate, disjoint contribution, or non-metric status obligation relative to the gate-scoped ledger; it must not assume that every surface name becomes an independent metric row. Without this classification, double counting, hidden overlap, or illicit metricization of qualitative failures is possible.

Remark 6.191 (PDF, image, and source sidecars)**.** A rendered PDF, page image, or screenshot is not automatically equivalent to the source artifact used for verification. If execution depends on source files, sidecar source identity and source-to-render preservation records should be stored.

Z.17ter. Part V closure grades and final-CR handoff

Definition 6.192 (Part V closure grade)**.** A *Part V closure grade* records the strongest attained stage:

$$\begin{aligned}&\mathrm{appendix\_local\_complete} \prec \mathrm{partV\_crosschecked}\\ &\prec \mathrm{companion\_crosschecked}\\ &\prec \mathrm{dependency\_closure\_frozen}\\ &\prec \mathrm{final\_cr\_promotion\_pending}\\ &\prec \mathrm{release\_promoted}.\end{aligned}$$

A blocking or undefined necessary dependency overrides the positive grade until repaired.

Definition 6.193 (Release handoff record)**.** A *release handoff record* identifies the frozen U–Z artifacts, Companion artifact, statement and label inventories, dependency closure, source bindings, selected proof routes, replay and audit records, unresolved non-claims, successor candidates, and the exact tasks delegated to final CR extraction.

Definition 6.194 (Final CR extraction boundary)**.** The *final CR extraction boundary* separates local stable labels and reviewed-local claim records from final canonical Claim UIDs, promotion receipts, release registry rows, and release-gate verdicts. Only the final extraction process may cross this boundary.

Proposition 6.195 (Local closure is not final promotion)**.** *Appendix-local or Part-V-local closure does not imply final CR promotion.*

Proof. Final promotion additionally consumes the Companion, global dependency closure, cross-document claim audit, and release authority. □

Proposition 6.196 (Generated sidecars are not normative registries)**.** *A generated JSON, CSV, SQLite row, graph node, UUID, dashboard entry, or receipt-shaped artifact is not a normative final registry item before the final CR extraction boundary is crossed.*

Proof. Serialization and identifier generation are platform actions. Canonical registry authority belongs to the final extraction and release-governance process. □

Theorem 6.197 (v20.0.0 final execution-closure package)**.** *The v20.0.0 execution layer supplies an auditable closure package in which:*

(i) execution, verification, audit, semantic replay, and mathematical reconstruction remain distinct;

(ii) detector stage, higher-edge degree, Type IV scope, transient scope, and finite-to-infinite scope are explicit;

(iii) selected-sub-DAG metric aggregation is typed and free of duplicate primitive discrepancies;

(iv) non-scalar obligations are not repaired by numerical slack;

(v) every external conclusion requires a complete Problem-Bridge and Return chain;

(vi) Known-Theorem Recovery preserves source and benchmark quarantine;

(vii) repairs and demotions are immutable and non-retroactive;

(viii) Part V local closure remains distinct from final CR-v20 release promotion.

Proof. Items (i)–(ii) follow from the run status-axis, detector, higher-edge, Type IV, replay, and Problem-Bridge records. Items (iii)–(iv) follow from the selected execution route and ledger records. Items (v)–(vi) follow from the Problem-Bridge and recovery-quarantine records. Item (vii) follows from the repair-replay and stale-run discipline. Item $(viii)$ follows from the closure-grade and final-CR extraction boundary. □

Remark 6.198 (Companion handoff)**.** Appendix Z closes Part V but does not close the full publication artifact. The next normative stage is the v20 Companion cross-audit, followed by whole-corpus dependency closure and final CR-v20 extraction.

Remark 6.199 (Part V freeze sequence)**.** The intended freeze sequence is:

$$\begin{gathered} U \to V \to W \to X \to Y \to Z \\ \to \text{Part V cross-audit} \to \text{Companion} \to \text{global cross-audit} \\ \to \text{final CR-v20}. \end{gathered}$$

This sequence is governance ordering, not a mathematical implication.

Z.18. Closure of Part V

Theorem 6.200 (Part V closure principle)**.** *The Search / Platform Appendices U–Z provide agent, search, bridge-program, map-DAG, proof-store, and execution infrastructure around the AK Core. They do not enlarge the Core mathematical package and do not replace:*

Core gates, *window-first repaired evaluations*, *stage-correct finite detector certificates*, *admissible representatives*, *degree-n eligible higher edges*, *actual Type IV morphisms*, *terminal and transient diagnostics*, *finite-to-infinite reductions*, *Return theorems*, *source authority*, *artifact identity*, *the selected-route scalarized audit budget*.

Part V closure is an infrastructure and governance closure, not final CR-v20 promotion or an external theorem.

Proof. Appendix U separates proposer-side and verifier-side agent semantics. Appendix V makes search protocols candidate-generating rather than certifying. Appendix W makes Bridge Programs proof-management objects rather than bridge theorems. Appendix X makes Validity Maps and Certificate DAGs navigation and dependency structures rather than proof by existence. Appendix Y makes Proof Store storage non-certifying. The present appendix makes execution and reproducibility audit-supporting rather than proof-generating and adds execution-to-verifier conversion safeguards. Together, these appendices preserve Core sovereignty in the sense of Chapter 7. □

Remark 6.201 (Meaning of Part V closure)**.** Part V closes the infrastructure layer. It explains how agents, search, bridges, maps, proof stores, and execution records may support proof work without becoming proof merely by being present.

Z.19. Final non-confusion principle

Theorem 6.202 (Final non-confusion principle)**.** *Across the AK framework, the following distinctions are mandatory:*

search eqverification, *storage eqcertification*, *execution eqproof*,
operational replay eqsemantic replay eqmathematical reconstruction,
finite observation eqfinite certificate eqfinite-to-infinite reduction,
realization eqReturn, *terminal zero eqfull defect zero*,
metric slack eqnon-scalar discharge, *local closure eqfinal CR promotion*,
Core pass eqexternal theorem without a complete bridge, [Spec] *eqtheorem*.

Proof. The first distinction follows from Appendices U and V. The second follows from Appendix Y. The third and fourth follow from the present appendix. The fifth follows from the Problem Interface discipline of Chapter 8 and the soundness-bridge requirements of the Problem Interface Appendices. The sixth follows from the non-claim and boundary discipline of Chapter 8. □

Remark 6.203 (Final role of Appendix Z)**.** Appendix Z is the final guard against infrastructure inflation. It ensures that even a fully reproducible, well-stored, agent-assisted, platform-managed, search-discovered, DAG-organized result must still pass the relevant mathematical and verifier-side conditions before it can be treated as proof.

Z.20. Summary of execution and reproducibility schema

Theorem 6.204 (Appendix Z execution-reproducibility package)**.** *Within the Search / Platform layer, the following package is available.*

(i) *Execution runs must have declared scope.*

(ii) *Execution schemas such as* run.yaml *are required for replayability.*

(iii) *Artifact manifests support audit but do not prove artifact adequacy.*

(iv) *Environment capture is required for computational replay.*

(v) *Randomized runs require seed, distribution, or nondeterminism records.*

(vi) *Repaired evaluation replay requires evaluation execution records.*

(vii) *Ledger replay requires ledger execution records.*

(viii) *Diagnostic replay requires diagnostic execution records based on declared tower comparison artifacts.*

(ix) *Gate verdicts require clause-level support.*

(x) *Replay pass is not theorem proof by itself.*

(xi) *Higher reproducibility level improves auditability but does not imply theorem validity.*

(xii) *Formal verification replay is scoped to formal statements.*

(xiii) *Numerical reproducibility is not numerical validity.*

(xiv) *Reproducibility manifests support audit but do not prove mathematical claims.*

(xv) *Reproducibility failure requires status review when necessary dependencies are affected.*

(xvi) *Execution success does not imply certificate success.*

(xvii) *Certificates relying on computed artifacts require adequate execution support.*

(xviii) *v20.0.0 execution hardening requires conversion records, claim-use preflight, scoped replay interpretation, and full Type IV replay records.*

(xix) *Part V infrastructure does not enlarge the AK Core.*

(xx) *The final non-confusion principle separates search, storage, execution, reproducibility, verification, Core pass, external theorem status, and* [Spec] *status.*

Proof. Item (i) follows from the definitions of execution run and run scope. Item (ii) is Proposition 6.14. Item (iii) is Proposition 6.21. Item (iv) is Proposition 6.27. Item (v) is Proposition 6.33. Item (vi) is Proposition 6.37. Item (vii) is Proposition 6.41. Item $(viii)$ is Proposition 6.45. Item (ix) is Proposition 6.49. Item (x) is Proposition 6.55. Item (xi) is Proposition 6.64. Item (xii) is Proposition 6.69. Item $(xiii)$ is Proposition 6.75. Item (xiv) follows from Proposition 6.79 and Remark 6.80. Item (xv) is Proposition 6.89. Item (xvi) is Proposition 6.93. Item $(xvii)$ is Proposition 6.94. Item $(xviii)$ is Theorem 6.187. Item (xix) is Theorem 6.200. Item (xx) is Theorem 6.202. □

Z.21. Explicit exclusions

The following are not asserted in Appendix Z.

- No execution run is treated as proof by itself.
- No run.yaml file is treated as certificate by itself.

- No artifact manifest proves artifact adequacy.
- No environment hash proves mathematical validity.
- No random seed proves correctness.
- No $\mathrm{eval.json}$ file implies repaired evaluation correctness by existence alone.
- No $\mathrm{ledger.json}$ file implies budget pass by existence alone.
- No $\mathrm{diagnostics.json}$ file implies Type IV cleanliness by existence alone.
- No $\mathrm{gate.json}$ file implies gate pass by existence alone.
- No replay-pass result proves a theorem by itself.
- No high reproducibility level implies theorem validity.
- No formal replay proves informal interpretation without an interpretation theorem.
- No numerical reproducibility proves continuum validity.
- No reproducibility manifest proves a mathematical claim by itself.
- No execution success implies certificate success.
- No reproducibility infrastructure replaces B-Gate^{+}.
- No reproducibility infrastructure replaces the repaired topological condition

$$\mathrm{Eval}_{n,W,\tau}(F) = 0.$$

- No reproducibility infrastructure replaces admissible representative records

$$C_{\tau,W}F.$$

- No reproducibility infrastructure replaces monitored Type IV diagnostics.
- No reproducibility infrastructure replaces tower comparison artifacts

$$\phi_{i,W,\tau}.$$

- No reproducibility infrastructure replaces the budget condition

$$\mathrm{val}_B(\mathfrak{d}_{\mathrm{met}}) < \mathrm{gap}_\tau(\mathfrak{B}).$$

- No Search / Platform appendix upgrades a [Spec] statement into a theorem.
- No Search / Platform appendix converts Core pass into an external theorem without a proved bridge.
- No execution output is used as verifier-side evidence without an execution-to-verifier conversion record.
- No replay-not-applicable result is treated as not invoked without a declared scope-exclusion policy.
- No run identity witness proves semantic preservation.
- No Type IV replay zero pair replaces the full typed diagnostic replay record.
- No detector output is replayed at a different source stage without an explicit stage-transport theorem.

- No zero-looking execution output is treated as a certified detector zero.
- No stored synthetic shift or target-shaped complex is treated as a principal higher edge.
- No terminal Type IV zero is treated as absence of transient or full-interior defects, profile isomorphism, or apex agreement.
- No finite plateau, repeated barcode, numerical convergence, or stored apex is treated as a finite-to-infinite reduction.
- No realization theorem is treated as a Return theorem.
- No external conclusion is certified without a complete direction-specific Return record.
- No benchmark proof, answer, conclusion-derived witness, or equivalent oracle leaks into an advertised independent recovery construction.
- No mirror, alias, manifest, graph edge, or replay reference multiplies mathematical evidence or metric discrepancy.
- No operational replay is treated as semantic replay or mathematical reconstruction by status inheritance.
- No local sidecar, generated UID, receipt-shaped object, or dashboard entry is treated as the final CR-v20 registry.
- No Part V closure grade is treated as final publication closure before Companion and global dependency review.

Z.22. Provenance and status

The material in this appendix depends on:

(i) the Agent Semantics of Appendix U;

(ii) the Hunter / Mapper / Lifter protocols of Appendix V;

(iii) the Bridge Programs of Appendix W;

(iv) the Validity Map and Global Certificate DAG of Appendix X;

(v) the AI Platform and Proof Store of Appendix Y;

(vi) the claim taxonomy and Minimal Manifest discipline of Chapter 1;

(vii) the window-first, threshold-gap, and repaired-zero discipline of Appendix A;

(viii) the finite-detector, representative, and stage-transport discipline of Appendix B;

(ix) the degree-n eligible realization and higher-edge discipline of Appendix C;

(x) the actual-morphism Type IV, transient-defect, and tower finite-to-infinite discipline of Appendix D;

(xi) the coverage, overlap, Restart, and local-to-global discipline of Appendix E, when invoked;

(xii) the atomic-status and non-compensation discipline of Appendix F;

(xiii) the canonical manifest and verifier schema of Appendix G;

(xiv) the Appendix-M-compatible commutative quantale, scalarization, selected-route support, and no-double-counting discipline;

(xv) the Core sovereignty and typed-audit doctrine of Chapter 7;

(xvi) the non-claim, boundary, Problem-Bridge, finite-to-infinite, Known-Theorem Recovery, and Return protocol of Chapter 8;

(xvii) the frozen local Problem Interface and recovery artifacts of Part IV;

(xviii) the current CR-v20 local claim-use and status-transition baseline, with final CR extraction deferred;

(xix) the deferred v20 Companion and final whole-corpus dependency closure.

Its role is to close the Search / Platform layer by specifying how execution records, artifacts, environments, repaired evaluations, ledgers, diagnostics, gates, and replay results are made reproducible without being mistaken for proofs.

Status labels for Appendix Z. *Search / Platform definitions:* Definitions 6.4, 6.5, 6.6, 6.7, 6.10, 6.11, 6.12, 6.13, 6.16, 6.17, 6.18, 6.19, 6.20, 6.23, 6.24, 6.25, 6.26, 6.29, 6.30, 6.31, 6.32, 6.35, 6.36, 6.39, 6.40, 6.43, 6.44, 6.47, 6.48, 6.51, 6.52, 6.53, 6.54, 6.57, 6.58, 6.59, 6.60, 6.61, 6.62, 6.63, 6.66, 6.67, 6.68, 6.71, 6.72, 6.73, 6.74, 6.77, 6.78, 6.81, 6.82, 6.83, 6.84, 6.85, 6.86, 6.87, 6.88, 6.91, 6.92.

Search / Platform propositions/theorems: Propositions 6.8, 6.14, 6.21, 6.27, 6.33, 6.37, 6.41, 6.45, 6.49, 6.55, 6.64, 6.69, 6.75, 6.79, 6.89, 6.93, 6.94, Theorem 6.204.

Operational cautions: Remarks 6.1, 6.2, 6.3, 6.9, 6.15, 6.22, 6.28, 6.34, 6.38, 6.42, 6.46, 6.50, 6.56, 6.65, 6.70, 6.76, 6.80, 6.90, 6.95, 6.201, 6.203.

v20.0.0 compatibility-hardening definitions: Definitions 6.166, 6.167, 6.168, 6.169, 6.170, 6.171, 6.172, 6.173, 6.174, 6.175, 6.176, 6.177.

v20.0.0 compatibility-hardening propositions/theorems: Propositions 6.178, 6.179, 6.180, 6.181, 6.182, 6.183, 6.184, 6.185, 6.186, Theorem 6.187.

v20.0.0 compatibility-hardening operational cautions: Remarks 6.188, 6.189, 6.190, 6.191.

v20.0.0 final-strengthening definitions: Definitions 6.96, 6.97, 6.101, 6.102, 6.103, 6.107, 6.108, 6.109, 6.110, 6.113, 6.114, 6.115, 6.118, 6.119, 6.122, 6.123, 6.126, 6.129, 6.130, 6.131, 6.132, 6.136, 6.137, 6.140, 6.141, 6.142, 6.145, 6.147, 6.148, 6.151, 6.152, 6.153, 6.157, 6.158, 6.162, 6.163, 6.192, 6.193, 6.194.

v20.0.0 final-strengthening propositions: Propositions 6.98, 6.104, 6.105, 6.111, 6.112, 6.116, 6.117, 6.120, 6.121, 6.124, 6.125, 6.127, 6.128, 6.133, 6.134, 6.135, 6.138, 6.139, 6.143, 6.144, 6.146, 6.149, 6.150, 6.154, 6.155, 6.156, 6.159, 6.164, 6.165, 6.195, 6.196.

v20.0.0 final-strengthening theorems: Theorems 6.197.

v20.0.0 final-strengthening operational cautions: Remarks 6.99, 6.100, 6.106, 6.160, 6.161, 6.198, 6.199.

Explicit exclusions: Section Z.21.

Part closure: Theorem 6.200.

Final non-confusion principle: Theorem 6.202.